

DevLab\_ICM20948  
1.0.2

Generated on Tue Jun 16 2026 17:30:42 for DevLab\_ICM20948 by Doxygen 1.9.8

Tue Jun 16 2026 17:30:42



|                                                 |    |
|-------------------------------------------------|----|
| 1 DevLab ICM20948 Arduino Library               | 1  |
| 1.1 Features                                    | 1  |
| 1.2 Connections / Wiring                        | 2  |
| 1.3 I <sup>2</sup> C Connection                 | 2  |
| 1.3.1 I <sup>2</sup> C Notes                    | 2  |
| 1.4 SPI Connection                              | 2  |
| 1.5 Installation                                | 2  |
| 1.5.1 Arduino Library Manager                   | 2  |
| 1.5.2 Manual Installation                       | 2  |
| 1.6 Library Overview                            | 3  |
| 1.6.1 Initialize Sensor                         | 3  |
| 1.6.2 Enable Sensors                            | 3  |
| 1.7 Reading Accelerometer                       | 3  |
| 1.8 Reading Gyroscope                           | 3  |
| 1.9 Reading Temperature                         | 3  |
| 1.10 Reading Magnetometer                       | 3  |
| 1.11 Setting Accelerometer Scale                | 3  |
| 1.11.1 Setting Gyroscope Scale                  | 4  |
| 1.11.2 Setting Sample Rate                      | 4  |
| 1.11.3 DLPF Configuration                       | 4  |
| 1.11.4 Averaging Configuration                  | 4  |
| 1.12 Applications                               | 4  |
| 1.13 License                                    | 4  |
| 2 Hierarchical Index                            | 5  |
| 2.1 Class Hierarchy                             | 5  |
| 3 Class Index                                   | 7  |
| 3.1 Class List                                  | 7  |
| 4 File Index                                    | 9  |
| 4.1 File List                                   | 9  |
| 5 Class Documentation                           | 11 |
| 5.1 BusIO_7Semi< Bus > Class Template Reference | 11 |
| 5.1.1 Detailed Description                      | 11 |
| 5.1.2 Constructor & Destructor Documentation    | 11 |
| 5.1.2.1 BusIO_7Semi()                           | 11 |
| 5.1.3 Member Function Documentation             | 12 |
| 5.1.3.1 read() [1/3]                            | 12 |
| 5.1.3.2 read() [2/3]                            | 12 |
| 5.1.3.3 read() [3/3]                            | 13 |
| 5.1.3.4 readBit() [1/2]                         | 13 |
| 5.1.3.5 readBit() [2/2]                         | 14 |
| 5.1.3.6 readBits() [1/2]                        | 14 |

|                                              |    |
|----------------------------------------------|----|
| 5.1.3.7 readBits() [2/2]                     | 15 |
| 5.1.3.8 write() [1/3]                        | 15 |
| 5.1.3.9 write() [2/3]                        | 16 |
| 5.1.3.10 write() [3/3]                       | 16 |
| 5.1.3.11 writeBit() [1/2]                    | 17 |
| 5.1.3.12 writeBit() [2/2]                    | 18 |
| 5.1.3.13 writeBits() [1/2]                   | 19 |
| 5.1.3.14 writeBits() [2/2]                   | 19 |
| 5.2 configDLPF Struct Reference              | 20 |
| 5.2.1 Detailed Description                   | 20 |
| 5.2.2 Member Data Documentation              | 20 |
| 5.2.2.1 div                                  | 20 |
| 5.2.2.2 dlpf_idx                             | 21 |
| 5.2.2.3 nbw_hz                               | 21 |
| 5.2.2.4 nombre                               | 21 |
| 5.2.2.5 odr_hz                               | 21 |
| 5.3 DevLab_ICM20948 Class Reference          | 21 |
| 5.3.1 Detailed Description                   | 22 |
| 5.3.2 Constructor & Destructor Documentation | 22 |
| 5.3.2.1 DevLab_ICM20948()                    | 22 |
| 5.3.3 Member Function Documentation          | 23 |
| 5.3.3.1 applyBasicDefaults()                 | 23 |
| 5.3.3.2 auxConfigSlave()                     | 23 |
| 5.3.3.3 auxMasterEnable()                    | 24 |
| 5.3.3.4 auxRead12bit()                       | 24 |
| 5.3.3.5 auxReadByte()                        | 25 |
| 5.3.3.6 auxReadResponse()                    | 26 |
| 5.3.3.7 auxReadSensorData()                  | 26 |
| 5.3.3.8 auxWriteByte()                       | 27 |
| 5.3.3.9 auxWriteCommand()                    | 28 |
| 5.3.3.10 beginI2C()                          | 28 |
| 5.3.3.11 beginSPI()                          | 29 |
| 5.3.3.12 checkIntStatus()                    | 30 |
| 5.3.3.13 getAccelAveraging()                 | 30 |
| 5.3.3.14 getAccelDLPF()                      | 31 |
| 5.3.3.15 getAccelOffset()                    | 32 |
| 5.3.3.16 getAccelSampleRate()                | 32 |
| 5.3.3.17 getAccelScale()                     | 33 |
| 5.3.3.18 getClock()                          | 33 |
| 5.3.3.19 getDLPF()                           | 33 |
| 5.3.3.20 getGyroOffset()                     | 34 |
| 5.3.3.21 getGyroSampleRate()                 | 35 |
| 5.3.3.22 getGyroScale()                      | 35 |

|          |                                                           |    |
|----------|-----------------------------------------------------------|----|
| 5.3.3.23 | <a href="#">getLowPower()</a>                             | 35 |
| 5.3.3.24 | <a href="#">getSensors()</a>                              | 36 |
| 5.3.3.25 | <a href="#">initMag()</a>                                 | 37 |
| 5.3.3.26 | <a href="#">intEnableConfig()</a>                         | 37 |
| 5.3.3.27 | <a href="#">intInit()</a>                                 | 38 |
| 5.3.3.28 | <a href="#">readAccel()</a>                               | 39 |
| 5.3.3.29 | <a href="#">readGyro()</a>                                | 40 |
| 5.3.3.30 | <a href="#">readMag()</a>                                 | 40 |
| 5.3.3.31 | <a href="#">readTemperature()</a>                         | 41 |
| 5.3.3.32 | <a href="#">readWhoAmI()</a>                              | 42 |
| 5.3.3.33 | <a href="#">selectBank()</a>                              | 43 |
| 5.3.3.34 | <a href="#">selfTestAccel()</a>                           | 43 |
| 5.3.3.35 | <a href="#">selfTestGyro()</a>                            | 43 |
| 5.3.3.36 | <a href="#">setAccelAveraging()</a>                       | 44 |
| 5.3.3.37 | <a href="#">setAccelDivRate()</a>                         | 45 |
| 5.3.3.38 | <a href="#">setAccelDLPF()</a>                            | 45 |
| 5.3.3.39 | <a href="#">setAccelOffset()</a>                          | 46 |
| 5.3.3.40 | <a href="#">setAccelSampleRate()</a>                      | 47 |
| 5.3.3.41 | <a href="#">setAccelScale()</a>                           | 48 |
| 5.3.3.42 | <a href="#">setClock()</a>                                | 48 |
| 5.3.3.43 | <a href="#">setDLPF()</a>                                 | 49 |
| 5.3.3.44 | <a href="#">setGyroAveraging()</a>                        | 49 |
| 5.3.3.45 | <a href="#">setGyroDivRate()</a>                          | 50 |
| 5.3.3.46 | <a href="#">setGyroOffset()</a>                           | 50 |
| 5.3.3.47 | <a href="#">setGyroSampleRate()</a>                       | 51 |
| 5.3.3.48 | <a href="#">setGyroScale()</a>                            | 51 |
| 5.3.3.49 | <a href="#">setLowPower()</a>                             | 52 |
| 5.3.3.50 | <a href="#">setMagOpMode()</a>                            | 53 |
| 5.3.3.51 | <a href="#">setSensors()</a>                              | 53 |
| 5.3.3.52 | <a href="#">sleep()</a>                                   | 54 |
| 5.3.3.53 | <a href="#">softReset()</a>                               | 55 |
| 5.4      | <a href="#">I2C_Interface Class Reference</a>             | 55 |
| 5.4.1    | <a href="#">Detailed Description</a>                      | 56 |
| 5.4.2    | <a href="#">Member Function Documentation</a>             | 57 |
| 5.4.2.1  | <a href="#">beginI2C()</a>                                | 57 |
| 5.4.2.2  | <a href="#">beginSPI()</a>                                | 57 |
| 5.4.2.3  | <a href="#">read()</a>                                    | 57 |
| 5.4.2.4  | <a href="#">write()</a>                                   | 58 |
| 5.4.3    | <a href="#">Member Data Documentation</a>                 | 58 |
| 5.4.3.1  | <a href="#">address</a>                                   | 58 |
| 5.4.3.2  | <a href="#">i2c</a>                                       | 58 |
| 5.5      | <a href="#">ICM20948_IntEnableConfig Struct Reference</a> | 58 |
| 5.5.1    | <a href="#">Detailed Description</a>                      | 59 |

|                                                        |    |
|--------------------------------------------------------|----|
| 5.5.2 Member Data Documentation . . . . .              | 59 |
| 5.5.2.1 dmpInt1En . . . . .                            | 59 |
| 5.5.2.2 fifoOvfEn . . . . .                            | 59 |
| 5.5.2.3 fifoWmEn . . . . .                             | 59 |
| 5.5.2.4 i2cMstIntEn . . . . .                          | 59 |
| 5.5.2.5 pllRdyEn . . . . .                             | 59 |
| 5.5.2.6 rawDataRdyEn . . . . .                         | 59 |
| 5.5.2.7 wofEn . . . . .                                | 59 |
| 5.5.2.8 womIntEn . . . . .                             | 59 |
| 5.6 ICM20948_IntPinConfig Struct Reference . . . . .   | 59 |
| 5.6.1 Detailed Description . . . . .                   | 60 |
| 5.6.2 Member Data Documentation . . . . .              | 60 |
| 5.6.2.1 activeLevel . . . . .                          | 60 |
| 5.6.2.2 bypassEn . . . . .                             | 60 |
| 5.6.2.3 clearMode . . . . .                            | 60 |
| 5.6.2.4 driveMode . . . . .                            | 60 |
| 5.6.2.5 fsyncActLevel . . . . .                        | 60 |
| 5.6.2.6 fsyncIntEn . . . . .                           | 60 |
| 5.6.2.7 latchMode . . . . .                            | 60 |
| 5.7 ICM20948_IntStatus Struct Reference . . . . .      | 61 |
| 5.7.1 Detailed Description . . . . .                   | 61 |
| 5.7.2 Member Data Documentation . . . . .              | 61 |
| 5.7.2.1 dmpInt1 . . . . .                              | 61 |
| 5.7.2.2 fifoOvf . . . . .                              | 61 |
| 5.7.2.3 fifoWm . . . . .                               | 61 |
| 5.7.2.4 i2cMstInt . . . . .                            | 61 |
| 5.7.2.5 pllRdyInt . . . . .                            | 61 |
| 5.7.2.6 rawDataRdy . . . . .                           | 61 |
| 5.7.2.7 womInt . . . . .                               | 62 |
| 5.8 icm_plat_t Struct Reference . . . . .              | 62 |
| 5.8.1 Detailed Description . . . . .                   | 62 |
| 5.8.2 Member Data Documentation . . . . .              | 62 |
| 5.8.2.1 delay_ms . . . . .                             | 62 |
| 5.8.2.2 i2c_addr . . . . .                             | 62 |
| 5.8.2.3 read . . . . .                                 | 62 |
| 5.8.2.4 spi_cs . . . . .                               | 63 |
| 5.8.2.5 spi_reg_rw_bits . . . . .                      | 63 |
| 5.8.2.6 user . . . . .                                 | 63 |
| 5.8.2.7 write . . . . .                                | 63 |
| 5.9 Interface_7Semi Class Reference . . . . .          | 63 |
| 5.9.1 Detailed Description . . . . .                   | 64 |
| 5.9.2 Constructor & Destructor Documentation . . . . . | 64 |
| 5.9.2.1 ~Interface_7Semi() . . . . .                   | 64 |

|                                                   |    |
|---------------------------------------------------|----|
| 5.9.3 Member Function Documentation               | 64 |
| 5.9.3.1 beginI2C()                                | 64 |
| 5.9.3.2 beginSPI()                                | 64 |
| 5.9.3.3 delay_us()                                | 65 |
| 5.9.3.4 read()                                    | 65 |
| 5.9.3.5 write()                                   | 65 |
| 5.10 SPI_Interface Class Reference                | 65 |
| 5.10.1 Detailed Description                       | 67 |
| 5.10.2 Member Function Documentation              | 67 |
| 5.10.2.1 beginI2C()                               | 67 |
| 5.10.2.2 beginSPI()                               | 67 |
| 5.10.2.3 read()                                   | 68 |
| 5.10.2.4 write()                                  | 68 |
| 5.10.3 Member Data Documentation                  | 68 |
| 5.10.3.1 cs                                       | 68 |
| 5.10.3.2 speed                                    | 68 |
| 5.10.3.3 spi                                      | 68 |
| 6 File Documentation                              | 69 |
| 6.1 examples/gyr_test/gyr_test.ino File Reference | 69 |
| 6.1.1 Detailed Description                        | 70 |
| 6.1.2 Macro Definition Documentation              | 70 |
| 6.1.2.1 I2C_FREQ                                  | 70 |
| 6.1.2.2 ICM_ADDR                                  | 70 |
| 6.1.2.3 SCL_PIN                                   | 71 |
| 6.1.2.4 SDA_PIN                                   | 71 |
| 6.1.3 Function Documentation                      | 71 |
| 6.1.3.1 applySettings()                           | 71 |
| 6.1.3.2 firstStage()                              | 71 |
| 6.1.3.3 loop()                                    | 72 |
| 6.1.3.4 printDobleLinea()                         | 72 |
| 6.1.3.5 printLinea()                              | 72 |
| 6.1.3.6 runSweep()                                | 72 |
| 6.1.3.7 setup()                                   | 73 |
| 6.1.4 Variable Documentation                      | 73 |
| 6.1.4.1 configsDLPF                               | 73 |
| 6.1.4.2 etiquetasGyroAVGCfg                       | 74 |
| 6.1.4.3 etiquetasGyroFSCfg                        | 74 |
| 6.1.4.4 gyroAVGCfg                                | 74 |
| 6.1.4.5 gyroDLPF                                  | 74 |
| 6.1.4.6 gyroFSCfg                                 | 74 |
| 6.1.4.7 imu                                       | 74 |
| 6.1.4.8 N_AVG                                     | 75 |

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 6.1.4.9 N_DLPF . . . . .                                          | 75 |
| 6.1.4.10 N_FS . . . . .                                           | 75 |
| 6.2 gyr_test.ino . . . . .                                        | 75 |
| 6.3 examples/i2c_accel/i2c_accel.ino File Reference . . . . .     | 78 |
| 6.3.1 Macro Definition Documentation . . . . .                    | 78 |
| 6.3.1.1 I2C_FREQ . . . . .                                        | 78 |
| 6.3.1.2 ICM_ADDR . . . . .                                        | 79 |
| 6.3.1.3 SCL_PIN . . . . .                                         | 79 |
| 6.3.1.4 SDA_PIN . . . . .                                         | 79 |
| 6.3.2 Function Documentation . . . . .                            | 79 |
| 6.3.2.1 loop() . . . . .                                          | 79 |
| 6.3.2.2 setup() . . . . .                                         | 79 |
| 6.3.3 Variable Documentation . . . . .                            | 80 |
| 6.3.3.1 imu . . . . .                                             | 80 |
| 6.4 i2c_accel.ino . . . . .                                       | 80 |
| 6.5 examples/i2c_basic/i2c_basic.ino File Reference . . . . .     | 82 |
| 6.5.1 Macro Definition Documentation . . . . .                    | 82 |
| 6.5.1.1 I2C_FREQ . . . . .                                        | 82 |
| 6.5.1.2 ICM_ADDR . . . . .                                        | 82 |
| 6.5.1.3 SCL_PIN . . . . .                                         | 83 |
| 6.5.1.4 SDA_PIN . . . . .                                         | 83 |
| 6.5.2 Function Documentation . . . . .                            | 83 |
| 6.5.2.1 loop() . . . . .                                          | 83 |
| 6.5.2.2 setup() . . . . .                                         | 83 |
| 6.5.3 Variable Documentation . . . . .                            | 84 |
| 6.5.3.1 imu . . . . .                                             | 84 |
| 6.6 i2c_basic.ino . . . . .                                       | 84 |
| 6.7 examples/i2c_gyro/i2c_gyro.ino File Reference . . . . .       | 86 |
| 6.7.1 Macro Definition Documentation . . . . .                    | 86 |
| 6.7.1.1 I2C_FREQ . . . . .                                        | 86 |
| 6.7.1.2 ICM_ADDR . . . . .                                        | 86 |
| 6.7.1.3 SCL_PIN . . . . .                                         | 87 |
| 6.7.1.4 SDA_PIN . . . . .                                         | 87 |
| 6.7.2 Function Documentation . . . . .                            | 87 |
| 6.7.2.1 loop() . . . . .                                          | 87 |
| 6.7.2.2 setup() . . . . .                                         | 87 |
| 6.7.3 Variable Documentation . . . . .                            | 87 |
| 6.7.3.1 imu . . . . .                                             | 87 |
| 6.8 i2c_gyro.ino . . . . .                                        | 88 |
| 6.9 examples/i2c_magneto/i2c_magneto.ino File Reference . . . . . | 89 |
| 6.9.1 Macro Definition Documentation . . . . .                    | 90 |
| 6.9.1.1 I2C_FREQ . . . . .                                        | 90 |
| 6.9.1.2 ICM_ADDR . . . . .                                        | 90 |



|                                                                |     |
|----------------------------------------------------------------|-----|
| 6.9.1.3 SCL_PIN . . . . .                                      | 90  |
| 6.9.1.4 SDA_PIN . . . . .                                      | 90  |
| 6.9.2 Function Documentation . . . . .                         | 90  |
| 6.9.2.1 loop() . . . . .                                       | 90  |
| 6.9.2.2 setup() . . . . .                                      | 91  |
| 6.9.3 Variable Documentation . . . . .                         | 91  |
| 6.9.3.1 imu . . . . .                                          | 91  |
| 6.10 i2c_magneto.ino . . . . .                                 | 91  |
| 6.11 examples/interrupt/interrupt.ino File Reference . . . . . | 93  |
| 6.11.1 Detailed Description . . . . .                          | 94  |
| 6.11.2 Macro Definition Documentation . . . . .                | 94  |
| 6.11.2.1 I2C_FREQ . . . . .                                    | 94  |
| 6.11.2.2 ICM_ADDR . . . . .                                    | 94  |
| 6.11.2.3 PIN_INT . . . . .                                     | 94  |
| 6.11.2.4 SCL_PIN . . . . .                                     | 94  |
| 6.11.2.5 SDA_PIN . . . . .                                     | 94  |
| 6.11.3 Function Documentation . . . . .                        | 94  |
| 6.11.3.1 isrHandler() . . . . .                                | 94  |
| 6.11.3.2 loop() . . . . .                                      | 95  |
| 6.11.3.3 printDobleLinea() . . . . .                           | 95  |
| 6.11.3.4 printLinea() . . . . .                                | 95  |
| 6.11.3.5 setup() . . . . .                                     | 95  |
| 6.11.4 Variable Documentation . . . . .                        | 95  |
| 6.11.4.1 imu . . . . .                                         | 95  |
| 6.11.4.2 intEn . . . . .                                       | 95  |
| 6.11.4.3 isrCount . . . . .                                    | 95  |
| 6.11.4.4 pinCfg . . . . .                                      | 96  |
| 6.12 interrupt.ino . . . . .                                   | 96  |
| 6.13 examples/mag_test/mag_test.ino File Reference . . . . .   | 97  |
| 6.13.1 Detailed Description . . . . .                          | 98  |
| 6.13.2 Macro Definition Documentation . . . . .                | 98  |
| 6.13.2.1 I2C_FREQ . . . . .                                    | 98  |
| 6.13.2.2 ICM_ADDR . . . . .                                    | 98  |
| 6.13.2.3 SCL_PIN . . . . .                                     | 98  |
| 6.13.2.4 SDA_PIN . . . . .                                     | 99  |
| 6.13.3 Function Documentation . . . . .                        | 99  |
| 6.13.3.1 applySettings() . . . . .                             | 99  |
| 6.13.3.2 loop() . . . . .                                      | 99  |
| 6.13.3.3 printDobleLinea() . . . . .                           | 99  |
| 6.13.3.4 printLinea() . . . . .                                | 99  |
| 6.13.3.5 setup() . . . . .                                     | 100 |
| 6.13.4 Variable Documentation . . . . .                        | 100 |
| 6.13.4.1 etiquetasOpModeCfg . . . . .                          | 100 |

|                                                                |     |
|----------------------------------------------------------------|-----|
| 6.13.4.2 imu . . . . .                                         | 100 |
| 6.13.4.3 N_OPM . . . . .                                       | 100 |
| 6.13.4.4 opMode . . . . .                                      | 100 |
| 6.14 mag_test.ino . . . . .                                    | 101 |
| 6.15 examples/spi_accel/spi_accel.ino File Reference . . . . . | 102 |
| 6.15.1 Macro Definition Documentation . . . . .                | 103 |
| 6.15.1.1 SPI_FAST_SPEED . . . . .                              | 103 |
| 6.15.2 Function Documentation . . . . .                        | 103 |
| 6.15.2.1 loop() . . . . .                                      | 103 |
| 6.15.2.2 setup() . . . . .                                     | 103 |
| 6.15.2.3 spi_bus() . . . . .                                   | 104 |
| 6.15.3 Variable Documentation . . . . .                        | 104 |
| 6.15.3.1 imu . . . . .                                         | 104 |
| 6.16 spi_accel.ino . . . . .                                   | 104 |
| 6.17 examples/spi_basic/spi_basic.ino File Reference . . . . . | 106 |
| 6.17.1 Macro Definition Documentation . . . . .                | 106 |
| 6.17.1.1 SPI_FAST_SPEED . . . . .                              | 106 |
| 6.17.2 Function Documentation . . . . .                        | 106 |
| 6.17.2.1 loop() . . . . .                                      | 106 |
| 6.17.2.2 setup() . . . . .                                     | 107 |
| 6.17.2.3 spi_bus() . . . . .                                   | 107 |
| 6.17.3 Variable Documentation . . . . .                        | 107 |
| 6.17.3.1 imu . . . . .                                         | 107 |
| 6.18 spi_basic.ino . . . . .                                   | 107 |
| 6.19 examples/spi_gyro/spi_gyro.ino File Reference . . . . .   | 109 |
| 6.19.1 Macro Definition Documentation . . . . .                | 109 |
| 6.19.1.1 SPI_FAST_SPEED . . . . .                              | 109 |
| 6.19.2 Function Documentation . . . . .                        | 109 |
| 6.19.2.1 loop() . . . . .                                      | 109 |
| 6.19.2.2 setup() . . . . .                                     | 110 |
| 6.19.2.3 spi_bus() . . . . .                                   | 110 |
| 6.19.3 Variable Documentation . . . . .                        | 110 |
| 6.19.3.1 imu . . . . .                                         | 110 |
| 6.20 spi_gyro.ino . . . . .                                    | 110 |
| 6.21 examples/test/test.ino File Reference . . . . .           | 112 |
| 6.21.1 Detailed Description . . . . .                          | 114 |
| 6.21.2 Macro Definition Documentation . . . . .                | 114 |
| 6.21.2.1 I2C_FREQ . . . . .                                    | 114 |
| 6.21.2.2 ICM_ADDR . . . . .                                    | 114 |
| 6.21.2.3 SCL_PIN . . . . .                                     | 114 |
| 6.21.2.4 SDA_PIN . . . . .                                     | 114 |
| 6.21.3 Function Documentation . . . . .                        | 114 |
| 6.21.3.1 applySettings() . . . . .                             | 114 |

|           |                                         |     |
|-----------|-----------------------------------------|-----|
| 6.21.3.2  | firstStage()                            | 115 |
| 6.21.3.3  | loop()                                  | 115 |
| 6.21.3.4  | printDobleLinea()                       | 116 |
| 6.21.3.5  | printLinea()                            | 116 |
| 6.21.3.6  | runSweep()                              | 116 |
| 6.21.3.7  | setup()                                 | 117 |
| 6.21.4    | Variable Documentation                  | 117 |
| 6.21.4.1  | accelAVG                                | 117 |
| 6.21.4.2  | accelCfg                                | 117 |
| 6.21.4.3  | accelDLPF                               | 117 |
| 6.21.4.4  | accelSR                                 | 118 |
| 6.21.4.5  | configsDLPF                             | 118 |
| 6.21.4.6  | etiquetasAccelAVG                       | 118 |
| 6.21.4.7  | etiquetasAccelCfg                       | 118 |
| 6.21.4.8  | etiquetasAccelDLPFCFG                   | 118 |
| 6.21.4.9  | etiquetasSR                             | 118 |
| 6.21.4.10 | imu                                     | 118 |
| 6.21.4.11 | N_DLPF                                  | 118 |
| 6.21.4.12 | N_FS                                    | 119 |
| 6.21.4.13 | total_fail                              | 119 |
| 6.21.4.14 | total_pass                              | 119 |
| 6.21.4.15 | total_warn                              | 119 |
| 6.22      | test.ino                                | 119 |
| 6.23      | examples/test2/test2.ino File Reference | 122 |
| 6.23.1    | Detailed Description                    | 123 |
| 6.23.2    | Macro Definition Documentation          | 124 |
| 6.23.2.1  | I2C_FREQ                                | 124 |
| 6.23.2.2  | ICM_ADDR                                | 124 |
| 6.23.2.3  | SCL_PIN                                 | 124 |
| 6.23.2.4  | SDA_PIN                                 | 124 |
| 6.23.3    | Function Documentation                  | 124 |
| 6.23.3.1  | applySettings()                         | 124 |
| 6.23.3.2  | firstStage()                            | 125 |
| 6.23.3.3  | loop()                                  | 125 |
| 6.23.3.4  | printDobleLinea()                       | 125 |
| 6.23.3.5  | printLinea()                            | 125 |
| 6.23.3.6  | runSweep()                              | 126 |
| 6.23.3.7  | setup()                                 | 126 |
| 6.23.4    | Variable Documentation                  | 127 |
| 6.23.4.1  | accelAVG                                | 127 |
| 6.23.4.2  | accelCfg                                | 127 |
| 6.23.4.3  | accelDLPF                               | 127 |
| 6.23.4.4  | accelSR                                 | 127 |

|           |                                          |     |
|-----------|------------------------------------------|-----|
| 6.23.4.5  | configsDLPF                              | 127 |
| 6.23.4.6  | etiquetasAccelAVG                        | 128 |
| 6.23.4.7  | etiquetasAccelCfg                        | 128 |
| 6.23.4.8  | etiquetasSR                              | 128 |
| 6.23.4.9  | imu                                      | 128 |
| 6.23.4.10 | N_AVG                                    | 128 |
| 6.23.4.11 | N_DLPF                                   | 128 |
| 6.23.4.12 | N_FS                                     | 128 |
| 6.23.4.13 | total_fail                               | 128 |
| 6.23.4.14 | total_pass                               | 128 |
| 6.23.4.15 | total_warn                               | 129 |
| 6.24      | test2.ino                                | 129 |
| 6.25      | README.md File Reference                 | 132 |
| 6.26      | src/7Semi_I2C_Interface.h File Reference | 132 |
| 6.27      | 7Semi_I2C_Interface.h                    | 133 |
| 6.28      | src/7Semi_Interface.h File Reference     | 134 |
| 6.29      | 7Semi_Interface.h                        | 135 |
| 6.30      | src/7Semi_SPI_Interface.h File Reference | 136 |
| 6.31      | 7Semi_SPI_Interface.h                    | 137 |
| 6.32      | src/BusIO_7Semi.h File Reference         | 139 |
| 6.33      | BusIO_7Semi.h                            | 139 |
| 6.34      | src/DevLab_ICM20948.cpp File Reference   | 141 |
| 6.35      | DevLab_ICM20948.cpp                      | 141 |
| 6.36      | src/DevLab_ICM20948.h File Reference     | 158 |
| 6.36.1    | Enumeration Type Documentation           | 159 |
| 6.36.1.1  | ICM20948_Accel_Average                   | 159 |
| 6.36.1.2  | ICM20948_Accel_DLPFCFG                   | 159 |
| 6.36.1.3  | ICM20948_Accel_FullScale                 | 160 |
| 6.36.1.4  | ICM20948_Clock_Source                    | 160 |
| 6.36.1.5  | ICM20948_Gyro_Average                    | 160 |
| 6.36.1.6  | ICM20948_Gyro_DLPF                       | 161 |
| 6.36.1.7  | ICM20948_Gyro_FullScale                  | 161 |
| 6.36.1.8  | ICM20948_Op_Mode                         | 161 |
| 6.37      | DevLab_ICM20948.h                        | 162 |
| 6.38      | src/ICM20948_platform.h File Reference   | 172 |
| 6.39      | ICM20948_platform.h                      | 172 |
| 6.40      | src/ICM20948_regs.h File Reference       | 173 |
| 6.40.1    | Macro Definition Documentation           | 179 |
| 6.40.1.1  | ACCEL_BYPASS_3DB_BW_HZ                   | 179 |
| 6.40.1.2  | ACCEL_BYPASS_NBW_HZ                      | 179 |
| 6.40.1.3  | ACCEL_BYPASS_RATE_HZ                     | 179 |
| 6.40.1.4  | ACCEL_CONFIG                             | 179 |
| 6.40.1.5  | ACCEL_CONFIG_2                           | 179 |

---

|                                           |     |
|-------------------------------------------|-----|
| 6.40.1.6 ACCEL_DEC3_AVG_16 . . . . .      | 179 |
| 6.40.1.7 ACCEL_DEC3_AVG_32 . . . . .      | 179 |
| 6.40.1.8 ACCEL_DEC3_AVG_4 . . . . .       | 179 |
| 6.40.1.9 ACCEL_DEC3_AVG_8 . . . . .       | 179 |
| 6.40.1.10 ACCEL_DLPF0_3DB_BW_HZ . . . . . | 179 |
| 6.40.1.11 ACCEL_DLPF0_NBW_HZ . . . . .    | 179 |
| 6.40.1.12 ACCEL_DLPF1_3DB_BW_HZ . . . . . | 180 |
| 6.40.1.13 ACCEL_DLPF1_NBW_HZ . . . . .    | 180 |
| 6.40.1.14 ACCEL_DLPF2_3DB_BW_HZ . . . . . | 180 |
| 6.40.1.15 ACCEL_DLPF2_NBW_HZ . . . . .    | 180 |
| 6.40.1.16 ACCEL_DLPF3_3DB_BW_HZ . . . . . | 180 |
| 6.40.1.17 ACCEL_DLPF3_NBW_HZ . . . . .    | 180 |
| 6.40.1.18 ACCEL_DLPF4_3DB_BW_HZ . . . . . | 180 |
| 6.40.1.19 ACCEL_DLPF4_NBW_HZ . . . . .    | 180 |
| 6.40.1.20 ACCEL_DLPF5_3DB_BW_HZ . . . . . | 180 |
| 6.40.1.21 ACCEL_DLPF5_NBW_HZ . . . . .    | 180 |
| 6.40.1.22 ACCEL_DLPF6_3DB_BW_HZ . . . . . | 180 |
| 6.40.1.23 ACCEL_DLPF6_NBW_HZ . . . . .    | 180 |
| 6.40.1.24 ACCEL_DLPF7_3DB_BW_HZ . . . . . | 181 |
| 6.40.1.25 ACCEL_DLPF7_NBW_HZ . . . . .    | 181 |
| 6.40.1.26 ACCEL_DLPF_BASE_HZ . . . . .    | 181 |
| 6.40.1.27 ACCEL_DLPF_ODR_HZ . . . . .     | 181 |
| 6.40.1.28 ACCEL_DLPFCFG_0 . . . . .       | 181 |
| 6.40.1.29 ACCEL_DLPFCFG_1 . . . . .       | 181 |
| 6.40.1.30 ACCEL_DLPFCFG_2 . . . . .       | 181 |
| 6.40.1.31 ACCEL_DLPFCFG_3 . . . . .       | 181 |
| 6.40.1.32 ACCEL_DLPFCFG_4 . . . . .       | 181 |
| 6.40.1.33 ACCEL_DLPFCFG_5 . . . . .       | 181 |
| 6.40.1.34 ACCEL_DLPFCFG_6 . . . . .       | 181 |
| 6.40.1.35 ACCEL_DLPFCFG_7 . . . . .       | 181 |
| 6.40.1.36 ACCEL_DLPFCFG_MASK . . . . .    | 182 |
| 6.40.1.37 ACCEL_DLPFCFG_SHIFT . . . . .   | 182 |
| 6.40.1.38 ACCEL_FCHOICE . . . . .         | 182 |
| 6.40.1.39 ACCEL_FCHOICE_BYPASS . . . . .  | 182 |
| 6.40.1.40 ACCEL_FCHOICE_DLPF . . . . .    | 182 |
| 6.40.1.41 ACCEL_FS_16G . . . . .          | 182 |
| 6.40.1.42 ACCEL_FS_2G . . . . .           | 182 |
| 6.40.1.43 ACCEL_FS_4G . . . . .           | 182 |
| 6.40.1.44 ACCEL_FS_8G . . . . .           | 182 |
| 6.40.1.45 ACCEL_FS_SEL_MASK . . . . .     | 182 |
| 6.40.1.46 ACCEL_FS_SEL_SHIFT . . . . .    | 182 |
| 6.40.1.47 ACCEL_INTEL_CTRL . . . . .      | 183 |
| 6.40.1.48 ACCEL_INTEL_EN . . . . .        | 183 |

|                                |     |
|--------------------------------|-----|
| 6.40.1.49 ACCEL_INTEL_MODE_INT | 183 |
| 6.40.1.50 ACCEL_SMPLRT_DIV_1   | 183 |
| 6.40.1.51 ACCEL_SMPLRT_DIV_2   | 183 |
| 6.40.1.52 ACCEL_SMPLRT_DIV_MAX | 183 |
| 6.40.1.53 ACCEL_SMPLRT_DIV_MIN | 183 |
| 6.40.1.54 ACCEL_WOM_THR        | 183 |
| 6.40.1.55 ACCEL_XOUT_H         | 183 |
| 6.40.1.56 ACCEL_XOUT_L         | 183 |
| 6.40.1.57 ACCEL_YOUT_H         | 183 |
| 6.40.1.58 ACCEL_YOUT_L         | 183 |
| 6.40.1.59 ACCEL_ZOUT_H         | 184 |
| 6.40.1.60 ACCEL_ZOUT_L         | 184 |
| 6.40.1.61 AK09916_I2C_ADDR     | 184 |
| 6.40.1.62 AK_CNTL2             | 184 |
| 6.40.1.63 AK_CNTL3             | 184 |
| 6.40.1.64 AK_HXL               | 184 |
| 6.40.1.65 AK_ST1               | 184 |
| 6.40.1.66 AK_ST2               | 184 |
| 6.40.1.67 AK_WIA2              | 184 |
| 6.40.1.68 AK_WIA2_VAL          | 184 |
| 6.40.1.69 AUTO_SEL             | 184 |
| 6.40.1.70 AX_ST_EN_REG         | 184 |
| 6.40.1.71 AY_ST_EN_REG         | 185 |
| 6.40.1.72 AZ_ST_EN_REG         | 185 |
| 6.40.1.73 BANK1_REG_BANK_SEL   | 185 |
| 6.40.1.74 BANK2_REG_BANK_SEL   | 185 |
| 6.40.1.75 BANK3_REG_BANK_SEL   | 185 |
| 6.40.1.76 BYPASS_EN            | 185 |
| 6.40.1.77 CLK_STOP             | 185 |
| 6.40.1.78 DEC3_CFG_MASK        | 185 |
| 6.40.1.79 DEC3_CFG_SHIFT       | 185 |
| 6.40.1.80 DELAY_ES_SHADOW      | 185 |
| 6.40.1.81 DELAY_TIME_EN        | 185 |
| 6.40.1.82 DELAY_TIMEH          | 185 |
| 6.40.1.83 DELAY_TIMEL          | 186 |
| 6.40.1.84 dps1000              | 186 |
| 6.40.1.85 dps2000              | 186 |
| 6.40.1.86 dps250               | 186 |
| 6.40.1.87 dps500               | 186 |
| 6.40.1.88 EnhancedRegular      | 186 |
| 6.40.1.89 EXT_SLV_SENS_DATA_00 | 186 |
| 6.40.1.90 EXT_SLV_SENS_DATA_23 | 186 |
| 6.40.1.91 EXT_SYNC_SET_MASK    | 186 |

|                                |     |
|--------------------------------|-----|
| 6.40.1.92 FIFO_CFG             | 186 |
| 6.40.1.93 FIFO_COUNTH          | 186 |
| 6.40.1.94 FIFO_COUNTL          | 186 |
| 6.40.1.95 FIFO_EN_1            | 187 |
| 6.40.1.96 FIFO_EN_2            | 187 |
| 6.40.1.97 FIFO_MODE            | 187 |
| 6.40.1.98 FIFO_R_W             | 187 |
| 6.40.1.99 FIFO_RST             | 187 |
| 6.40.1.100 FSYNC_CONFIG        | 187 |
| 6.40.1.101 FSYNC_INT_MODE_EN   | 187 |
| 6.40.1.102 g16                 | 187 |
| 6.40.1.103 g2                  | 187 |
| 6.40.1.104 g4                  | 187 |
| 6.40.1.105 g8                  | 187 |
| 6.40.1.106 GYRO_AVGCFG_MASK    | 187 |
| 6.40.1.107 GYRO_AVGCFG_SHIFT   | 188 |
| 6.40.1.108 GYRO_BW_MEDIUM      | 188 |
| 6.40.1.109 GYRO_BW_NARROW      | 188 |
| 6.40.1.110 GYRO_BW_ULTRA_WIDE  | 188 |
| 6.40.1.111 GYRO_BW_WIDE        | 188 |
| 6.40.1.112 GYRO_CONFIG_1       | 188 |
| 6.40.1.113 GYRO_CONFIG_2       | 188 |
| 6.40.1.114 GYRO_DLPF_BASE_HZ   | 188 |
| 6.40.1.115 GYRO_DLPF_ODR_HZ    | 188 |
| 6.40.1.116 GYRO_DLPFCFG_0      | 188 |
| 6.40.1.117 GYRO_DLPFCFG_1      | 188 |
| 6.40.1.118 GYRO_DLPFCFG_2      | 188 |
| 6.40.1.119 GYRO_DLPFCFG_3      | 189 |
| 6.40.1.120 GYRO_DLPFCFG_4      | 189 |
| 6.40.1.121 GYRO_DLPFCFG_5      | 189 |
| 6.40.1.122 GYRO_DLPFCFG_6      | 189 |
| 6.40.1.123 GYRO_DLPFCFG_7      | 189 |
| 6.40.1.124 GYRO_DLPFCFG_MASK   | 189 |
| 6.40.1.125 GYRO_DLPFCFG_SHIFT  | 189 |
| 6.40.1.126 GYRO_FCHOICE        | 189 |
| 6.40.1.127 GYRO_FCHOICE_BYPASS | 189 |
| 6.40.1.128 GYRO_FCHOICE_DLPF   | 189 |
| 6.40.1.129 GYRO_FS_1000DPS     | 189 |
| 6.40.1.130 GYRO_FS_2000DPS     | 190 |
| 6.40.1.131 GYRO_FS_250DPS      | 190 |
| 6.40.1.132 GYRO_FS_500DPS      | 190 |
| 6.40.1.133 GYRO_FS_SEL_MASK    | 190 |
| 6.40.1.134 GYRO_FS_SEL_SHIFT   | 190 |

|                                         |     |
|-----------------------------------------|-----|
| 6.40.1.135 GYRO_SMPLRT_DIV . . . . .    | 190 |
| 6.40.1.136 GYRO_SMPLRT_MIN_HZ . . . . . | 190 |
| 6.40.1.137 GYRO_XOUT_H . . . . .        | 190 |
| 6.40.1.138 GYRO_XOUT_L . . . . .        | 190 |
| 6.40.1.139 GYRO_YOUT_H . . . . .        | 190 |
| 6.40.1.140 GYRO_YOUT_L . . . . .        | 190 |
| 6.40.1.141 GYRO_ZOUT_H . . . . .        | 190 |
| 6.40.1.142 GYRO_ZOUT_L . . . . .        | 191 |
| 6.40.1.143 HighAccuracy . . . . .       | 191 |
| 6.40.1.144 Hz10 . . . . .               | 191 |
| 6.40.1.145 Hz15 . . . . .               | 191 |
| 6.40.1.146 Hz2 . . . . .                | 191 |
| 6.40.1.147 Hz20 . . . . .               | 191 |
| 6.40.1.148 Hz25 . . . . .               | 191 |
| 6.40.1.149 Hz30 . . . . .               | 191 |
| 6.40.1.150 Hz6 . . . . .                | 191 |
| 6.40.1.151 Hz8 . . . . .                | 191 |
| 6.40.1.152 I2C_MST_CLK_MASK . . . . .   | 191 |
| 6.40.1.153 I2C_MST_CTRL . . . . .       | 191 |
| 6.40.1.154 I2C_MST_DELAY_CTRL . . . . . | 192 |
| 6.40.1.155 I2C_MST_ODR_CONFIG . . . . . | 192 |
| 6.40.1.156 I2C_MST_P_NSR . . . . .      | 192 |
| 6.40.1.157 I2C_MST_STATUS . . . . .     | 192 |
| 6.40.1.158 I2C_SLV0_ADDR . . . . .      | 192 |
| 6.40.1.159 I2C_SLV0_CTRL . . . . .      | 192 |
| 6.40.1.160 I2C_SLV0_DELAY_EN . . . . .  | 192 |
| 6.40.1.161 I2C_SLV0_DO . . . . .        | 192 |
| 6.40.1.162 I2C_SLV0_REG . . . . .       | 192 |
| 6.40.1.163 I2C_SLV1_ADDR . . . . .      | 192 |
| 6.40.1.164 I2C_SLV1_CTRL . . . . .      | 192 |
| 6.40.1.165 I2C_SLV1_DELAY_EN . . . . .  | 192 |
| 6.40.1.166 I2C_SLV1_DO . . . . .        | 193 |
| 6.40.1.167 I2C_SLV1_REG . . . . .       | 193 |
| 6.40.1.168 I2C_SLV2_ADDR . . . . .      | 193 |
| 6.40.1.169 I2C_SLV2_CTRL . . . . .      | 193 |
| 6.40.1.170 I2C_SLV2_DELAY_EN . . . . .  | 193 |
| 6.40.1.171 I2C_SLV2_DO . . . . .        | 193 |
| 6.40.1.172 I2C_SLV2_REG . . . . .       | 193 |
| 6.40.1.173 I2C_SLV3_ADDR . . . . .      | 193 |
| 6.40.1.174 I2C_SLV3_CTRL . . . . .      | 193 |
| 6.40.1.175 I2C_SLV3_DELAY_EN . . . . .  | 193 |
| 6.40.1.176 I2C_SLV3_DO . . . . .        | 193 |
| 6.40.1.177 I2C_SLV3_REG . . . . .       | 193 |



|                                            |     |
|--------------------------------------------|-----|
| 6.40.1.178 I2C_SLV4_ADDR . . . . .         | 194 |
| 6.40.1.179 I2C_SLV4_CTRL . . . . .         | 194 |
| 6.40.1.180 I2C_SLV4_DELAY_EN . . . . .     | 194 |
| 6.40.1.181 I2C_SLV4_DI . . . . .           | 194 |
| 6.40.1.182 I2C_SLV4_DLY_MASK . . . . .     | 194 |
| 6.40.1.183 I2C_SLV4_DO . . . . .           | 194 |
| 6.40.1.184 I2C_SLV4_REG . . . . .          | 194 |
| 6.40.1.185 I2C_SLVx_BYTE_SW . . . . .      | 194 |
| 6.40.1.186 I2C_SLVx_EN . . . . .           | 194 |
| 6.40.1.187 I2C_SLVx_GRP . . . . .          | 194 |
| 6.40.1.188 I2C_SLVx LENG_MASK . . . . .    | 194 |
| 6.40.1.189 I2C_SLVx_REG_DIS . . . . .      | 194 |
| 6.40.1.190 I2C_SLVx_RNW . . . . .          | 195 |
| 6.40.1.191 INT1_ACTL . . . . .             | 195 |
| 6.40.1.192 INT1_LATCH_INT_EN . . . . .     | 195 |
| 6.40.1.193 INT1_OPEN . . . . .             | 195 |
| 6.40.1.194 INT_ACTL_FSYNC . . . . .        | 195 |
| 6.40.1.195 INT_ANYRD_2CLEAR . . . . .      | 195 |
| 6.40.1.196 INT_DMP_INT1_EN . . . . .       | 195 |
| 6.40.1.197 INT_ENABLE . . . . .            | 195 |
| 6.40.1.198 INT_ENABLE_1 . . . . .          | 195 |
| 6.40.1.199 INT_ENABLE_2 . . . . .          | 195 |
| 6.40.1.200 INT_ENABLE_3 . . . . .          | 195 |
| 6.40.1.201 INT_I2C_MST_INT_EN . . . . .    | 195 |
| 6.40.1.202 INT_PIN_CFG . . . . .           | 196 |
| 6.40.1.203 INT_PLL_RDY_EN . . . . .        | 196 |
| 6.40.1.204 INT_RAW_DATA_0_RDY_EN . . . . . | 196 |
| 6.40.1.205 INT_REG_WOF_EN . . . . .        | 196 |
| 6.40.1.206 INT_STATUS . . . . .            | 196 |
| 6.40.1.207 INT_STATUS_1 . . . . .          | 196 |
| 6.40.1.208 INT_STATUS_2 . . . . .          | 196 |
| 6.40.1.209 INT_STATUS_3 . . . . .          | 196 |
| 6.40.1.210 INT_WOM_INT_EN . . . . .        | 196 |
| 6.40.1.211 INTERNAL_20MHZ . . . . .        | 196 |
| 6.40.1.212 LowPower . . . . .              | 196 |
| 6.40.1.213 LP_ACCEL_CYCLE . . . . .        | 196 |
| 6.40.1.214 LP_CONFIG . . . . .             | 197 |
| 6.40.1.215 LP_GYRO_CYCLE . . . . .         | 197 |
| 6.40.1.216 LP_I2C_MST_CYCLE . . . . .      | 197 |
| 6.40.1.217 MOD_CTRL_USR . . . . .          | 197 |
| 6.40.1.218 MST_LOST_ARB . . . . .          | 197 |
| 6.40.1.219 MST_ODR_CFG_MASK . . . . .      | 197 |
| 6.40.1.220 MST_PASS_THROUGH . . . . .      | 197 |

|                                                   |     |
|---------------------------------------------------|-----|
| 6.40.1.221 MST_SLV0_NACK . . . . .                | 197 |
| 6.40.1.222 MST_SLV1_NACK . . . . .                | 197 |
| 6.40.1.223 MST_SLV2_NACK . . . . .                | 197 |
| 6.40.1.224 MST_SLV3_NACK . . . . .                | 197 |
| 6.40.1.225 MST_SLV4_DONE . . . . .                | 197 |
| 6.40.1.226 MST_SLV4_NACK . . . . .                | 198 |
| 6.40.1.227 MULT_MST_EN . . . . .                  | 198 |
| 6.40.1.228 ODR_ALIGN_EN . . . . .                 | 198 |
| 6.40.1.229 ODR_ALIGN_EN_BIT . . . . .             | 198 |
| 6.40.1.230 PWR_CLKSEL_AUTO . . . . .              | 198 |
| 6.40.1.231 PWR_CLKSEL_INT_20MHZ . . . . .         | 198 |
| 6.40.1.232 PWR_CLKSEL_MASK . . . . .              | 198 |
| 6.40.1.233 PWR_DEVICE_RESET . . . . .             | 198 |
| 6.40.1.234 PWR_DISABLE_ACCEL . . . . .            | 198 |
| 6.40.1.235 PWR_DISABLE_GYRO . . . . .             | 198 |
| 6.40.1.236 PWR_LP_EN . . . . .                    | 198 |
| 6.40.1.237 PWR_MGMT_1 . . . . .                   | 198 |
| 6.40.1.238 PWR_MGMT_2 . . . . .                   | 199 |
| 6.40.1.239 PWR_SLEEP . . . . .                    | 199 |
| 6.40.1.240 PWR_TEMP_DIS . . . . .                 | 199 |
| 6.40.1.241 REG_BANK_SEL . . . . .                 | 199 |
| 6.40.1.242 REG_BANK_SEL_USER_BANK_MASK . . . . .  | 199 |
| 6.40.1.243 REG_BANK_SEL_USER_BANK_SHIFT . . . . . | 199 |
| 6.40.1.244 REG_LP_DMP_EN . . . . .                | 199 |
| 6.40.1.245 Regular . . . . .                      | 199 |
| 6.40.1.246 SELF_TEST_X_ACCEL . . . . .            | 199 |
| 6.40.1.247 SELF_TEST_X_GYRO . . . . .             | 199 |
| 6.40.1.248 SELF_TEST_Y_ACCEL . . . . .            | 199 |
| 6.40.1.249 SELF_TEST_Y_GYRO . . . . .             | 199 |
| 6.40.1.250 SELF_TEST_Z_ACCEL . . . . .            | 200 |
| 6.40.1.251 SELF_TEST_Z_GYRO . . . . .             | 200 |
| 6.40.1.252 STS_DMP_INT1 . . . . .                 | 200 |
| 6.40.1.253 STS_I2C_MST_INT . . . . .              | 200 |
| 6.40.1.254 STS_PLL_RDY_INT . . . . .              | 200 |
| 6.40.1.255 STS_RAW_DATA_0_RDY_INT . . . . .       | 200 |
| 6.40.1.256 STS_WOM_INT . . . . .                  | 200 |
| 6.40.1.257 TEMP_CONFIG . . . . .                  | 200 |
| 6.40.1.258 TEMP_DLPFCFG_MASK . . . . .            | 200 |
| 6.40.1.259 TEMP_DLPFCFG_SHIFT . . . . .           | 200 |
| 6.40.1.260 TEMP_OUT_H . . . . .                   | 200 |
| 6.40.1.261 TEMP_OUT_L . . . . .                   | 200 |
| 6.40.1.262 TIMEBASE_CORRECTION_PLL . . . . .      | 201 |
| 6.40.1.263 USER_BANK_0 . . . . .                  | 201 |

|                                            |     |
|--------------------------------------------|-----|
| 6.40.1.264 USER_BANK_1 . . . . .           | 201 |
| 6.40.1.265 USER_BANK_2 . . . . .           | 201 |
| 6.40.1.266 USER_BANK_3 . . . . .           | 201 |
| 6.40.1.267 USER_CTRL . . . . .             | 201 |
| 6.40.1.268 USER_CTRL_DMP_EN . . . . .      | 201 |
| 6.40.1.269 USER_CTRL_DMP_RST . . . . .     | 201 |
| 6.40.1.270 USER_CTRL_FIFO_EN . . . . .     | 201 |
| 6.40.1.271 USER_CTRL_I2C_IF_DIS . . . . .  | 201 |
| 6.40.1.272 USER_CTRL_I2C_MST_EN . . . . .  | 201 |
| 6.40.1.273 USER_CTRL_I2C_MST_RST . . . . . | 201 |
| 6.40.1.274 USER_CTRL_SRAM_RST . . . . .    | 202 |
| 6.40.1.275 WHO_AM_I . . . . .              | 202 |
| 6.40.1.276 WHO_AM_I_VAL . . . . .          | 202 |
| 6.40.1.277 WOF_DEGLITCH_EN . . . . .       | 202 |
| 6.40.1.278 WOF_EDGE_INT . . . . .          | 202 |
| 6.40.1.279 XA_OFFS_H . . . . .             | 202 |
| 6.40.1.280 XA_OFFS_L . . . . .             | 202 |
| 6.40.1.281 XG_OFFS_USRH . . . . .          | 202 |
| 6.40.1.282 XG_OFFS_USRL . . . . .          | 202 |
| 6.40.1.283 XGYRO_CTEN . . . . .            | 203 |
| 6.40.1.284 YA_OFFS_H . . . . .             | 203 |
| 6.40.1.285 YA_OFFS_L . . . . .             | 203 |
| 6.40.1.286 YG_OFFS_USRH . . . . .          | 203 |
| 6.40.1.287 YG_OFFS_USRL . . . . .          | 203 |
| 6.40.1.288 YGYRO_CTEN . . . . .            | 203 |
| 6.40.1.289 ZA_OFFS_H . . . . .             | 203 |
| 6.40.1.290 ZA_OFFS_L . . . . .             | 203 |
| 6.40.1.291 ZG_OFFS_USRH . . . . .          | 203 |
| 6.40.1.292 ZG_OFFS_USRL . . . . .          | 203 |
| 6.40.1.293 ZGYRO_CTEN . . . . .            | 203 |
| 6.41 ICM20948_regs.h . . . . .             | 203 |
| Index . . . . .                            | 211 |



# Chapter 1

## DevLab ICM20948 Arduino Library

Arduino driver for the module DevLab ICM-20948 9-axis IMU sensor.

This library is a DevLab adaptation derived from the 7Semi ICM20948 Arduino Library. The original MIT license notice from 7semi-solutions is preserved in **LICENSE**. See **NOTICE.md** for attribution and origin details.

The ICM-20948 integrates a 3-axis accelerometer, 3-axis gyroscope, and 3-axis magnetometer (AK09916), enabling motion tracking, orientation sensing, and navigation applications.

---

### 1.1 Features

- 9-axis motion sensing
  - 3-axis Accelerometer
  - 3-axis Gyroscope
  - 3-axis Magnetometer
- Configurable sensor ranges
  - Accel:  $\pm 2g$  /  $\pm 4g$  /  $\pm 8g$  /  $\pm 16g$
  - Gyro:  $\pm 250$  /  $\pm 500$  /  $\pm 1000$  /  $\pm 2000$  dps
- Digital Low Pass Filter configurable(DLPF)
  - Noise reduction
  - Configurable bandwidth
- Adjustable sample rates
  - Accel: up to 1125 Hz
  - Gyro: up to 1100 Hz
- Internal I2C Master (for magnetometer)
  - AK09916 integration via SLV0/SLV4
- ISupports both communication interfaces
  - I<sup>2</sup>C
  - SPI
- Temperature measurement

## 1.2 Connections / Wiring

The ICM-20948 supports both I<sup>2</sup>C and SPI communication.

---

### 1.3 I<sup>2</sup>C Connection

| ICM-20948 Pin | MCU Pin | Notes                     |
|---------------|---------|---------------------------|
| VCC           | 3.3V    | 5V only if board supports |
| GND           | GND     | Common ground             |
| SDA           | SDA     | I <sup>2</sup> C data     |
| SCL           | SCL     | I <sup>2</sup> C clock    |
| AD0           | GND/3V3 | Address select            |

#### 1.3.1 I<sup>2</sup>C Notes

---

- Default I<sup>2</sup>C address: 0x29
  - Supported bus speeds:
    - 100 kHz
    - 400 kHz (recommended)
- 

### 1.4 SPI Connection

| ICM-20948 Pin | MCU Pin |
|---------------|---------|
| VCC           | 3.3V    |
| GND           | GND     |
| SCK           | SCK     |
| MOSI          | MOSI    |
| MISO          | MISO    |
| CS            | GPIO    |

## 1.5 Installation

### 1.5.1 Arduino Library Manager

1. Open Arduino IDE
  2. Go to Library Manager
  3. Search for DevLab ICM20948
  4. Click Install
- 

### 1.5.2 Manual Installation

1. Download repository as ZIP
  2. Arduino IDE Sketch Include Library Add .ZIP Library
-

## 1.6 Library Overview

---

### 1.6.1 Initialize Sensor

- I2C

```
if (!imu.beginI2C(0x69, Wire, 400000))  
{  
  Serial.println("IMU init failed");  
  while (1);  
}
```

- SPI

```
if (!imu.beginSPI(CS_PIN, SPI, 1000000))  
{  
  Serial.println("SPI init failed");  
  while (1);  
}
```

### 1.6.2 Enable Sensors

```
imu.setSensors(true, true, true);
```

- accel enable accelerometer
- gyro enable gyroscope
- temp enable temperatur

## 1.7 Reading Accelerometer

```
float ax, ay, az;  
imu.readAccel(ax, ay, az);
```

- Output in g

## 1.8 Reading Gyroscope

```
float gx, gy, gz;  
imu.readGyro(gx, gy, gz);
```

- Output in degrees/sec

## 1.9 Reading Temperature

```
float temp;  
imu.readTemperature(temp);
```

- Output in °C

## 1.10 Reading Magnetometer

```
imu.initMag();  
  
float mx, my, mz;  
imu.readMag(mx, my, mz);
```

- Output in microtesla (µT)

## 1.11 Setting Accelerometer Scale

```
imu.setAccelScale(G_4);
```

- G\_2, G\_4, G\_8, G\_16

### 1.11.1 Setting Gyroscope Scale

```
imu.setGyroScale(DPS_2000);
```

### 1.11.2 Setting Sample Rate

```
imu.setAccelSampleRate(225);  
imu.setGyroSampleRate(225);
```

### 1.11.3 DLPF Configuration

```
imu.setAccelDLPF(ACCEL_DLPFCFG_3, false);  
imu.setDLPF(GYRO_DLPF_3, false);
```

### 1.11.4 Averaging Configuration

```
imu.setAccelAveraging(AVG_8);
```

## 1.12 Applications

- Robotics (obstacle detection)
- Drones (altitude sensing)
- Wearables (activity tracking)
- Navigation systems (IMU fusion)
- Industrial motion sensing
- Gaming controllers

## 1.13 License

- MIT License
- Original work copyright: 2025 7semi-solutions
- DevLab adaptation maintains the original license notice and attribution.



# Chapter 2

## Hierarchical Index

### 2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                                          |    |
|------------------------------------------|----|
| BusIO_7Semi< Bus > . . . . .             | 11 |
| BusIO_7Semi< Interface_7Semi > . . . . . | 11 |
| configDLPF . . . . .                     | 20 |
| DevLab_ICM20948 . . . . .                | 21 |
| ICM20948_IntEnableConfig . . . . .       | 58 |
| ICM20948_IntPinConfig . . . . .          | 59 |
| ICM20948_IntStatus . . . . .             | 61 |
| icm_plat_t . . . . .                     | 62 |
| Interface_7Semi . . . . .                | 63 |
| I2C_Interface . . . . .                  | 55 |
| SPI_Interface . . . . .                  | 65 |



# Chapter 3

## Class Index

### 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                                 |    |
|-------------------------------------------------|----|
| <a href="#">BusIO_7Semi&lt; Bus &gt;</a>        | 11 |
| <a href="#">configDLPF</a>                      |    |
| Gyroscope DLPF divider and timing configuration | 20 |
| <a href="#">DevLab_ICM20948</a>                 | 21 |
| <a href="#">I2C_Interface</a>                   | 55 |
| <a href="#">ICM20948_IntEnableConfig</a>        | 58 |
| <a href="#">ICM20948_IntPinConfig</a>           | 59 |
| <a href="#">ICM20948_IntStatus</a>              | 61 |
| <a href="#">icm_plat_t</a>                      | 62 |
| <a href="#">Interface_7Semi</a>                 | 63 |
| <a href="#">SPI_Interface</a>                   | 65 |



# Chapter 4

## File Index

### 4.1 File List

Here is a list of all files with brief descriptions:

|                                                              |     |
|--------------------------------------------------------------|-----|
| examples/gyr_test/ <a href="#">gyr_test.ino</a>              |     |
| Gyroscope DLPF and full-scale sweep for the 7Semi ICM-20948  | 69  |
| examples/i2c_accel/ <a href="#">i2c_accel.ino</a>            | 78  |
| examples/i2c_basic/ <a href="#">i2c_basic.ino</a>            | 82  |
| examples/i2c_gyro/ <a href="#">i2c_gyro.ino</a>              | 86  |
| examples/i2c_magneto/ <a href="#">i2c_magneto.ino</a>        | 89  |
| examples/interrupt/ <a href="#">interrupt.ino</a>            |     |
| I2C interrupt example for the 7Semi ICM-20948                | 93  |
| examples/mag_test/ <a href="#">mag_test.ino</a>              |     |
| Magnetometer operation-mode sweep for the 7Semi ICM-20948    | 97  |
| examples/spi_accel/ <a href="#">spi_accel.ino</a>            | 102 |
| examples/spi_basic/ <a href="#">spi_basic.ino</a>            | 106 |
| examples/spi_gyro/ <a href="#">spi_gyro.ino</a>              | 109 |
| examples/test/ <a href="#">test.ino</a>                      |     |
| Accelerometer DLPF validation sketch for the 7Semi ICM-20948 | 112 |
| examples/test2/ <a href="#">test2.ino</a>                    |     |
| Extended accelerometer DLPF, full-scale, and averaging sweep | 122 |
| src/ <a href="#">7Semi_I2C_Interface.h</a>                   | 132 |
| src/ <a href="#">7Semi_Interface.h</a>                       | 134 |
| src/ <a href="#">7Semi_SPI_Interface.h</a>                   | 136 |
| src/ <a href="#">BusIO_7Semi.h</a>                           | 139 |
| src/ <a href="#">DevLab_ICM20948.cpp</a>                     | 141 |
| src/ <a href="#">DevLab_ICM20948.h</a>                       | 158 |
| src/ <a href="#">ICM20948_platform.h</a>                     | 172 |
| src/ <a href="#">ICM20948_regs.h</a>                         | 173 |



# Chapter 5

## Class Documentation

### 5.1 BusIO\_7Semi< Bus > Class Template Reference

#include <BusIO\_7Semi.h>

#### Public Member Functions

- [BusIO\\_7Semi](#) (Bus &busRef)
- bool [read](#) (uint8\_t reg, uint8\_t &value)
- bool [read](#) (uint8\_t reg, uint16\_t &value)
- bool [read](#) (uint8\_t reg, uint8\_t \*data, uint32\_t len)
- bool [write](#) (uint8\_t reg, uint8\_t value)
- bool [write](#) (uint8\_t reg, uint16\_t value)
- bool [write](#) (uint8\_t reg, const uint8\_t \*data, uint32\_t len)
- bool [readBits](#) (uint8\_t reg, uint8\_t pos, uint8\_t len, uint8\_t &value)
- bool [readBits](#) (uint8\_t reg, uint8\_t pos, uint8\_t len, uint16\_t &value)
- bool [writeBits](#) (uint8\_t reg, uint8\_t pos, uint8\_t len, uint8\_t value)
- bool [writeBits](#) (uint8\_t reg, uint8\_t pos, uint8\_t len, uint16\_t value)
- bool [readBit](#) (uint8\_t reg, uint8\_t pos, uint8\_t &value)
- bool [readBit](#) (uint8\_t reg, uint8\_t pos, uint16\_t &value)
- bool [writeBit](#) (uint8\_t reg, uint8\_t pos, uint8\_t value)
- bool [writeBit](#) (uint8\_t reg, uint8\_t pos, uint16\_t value)

#### 5.1.1 Detailed Description

```
template<typename Bus>
class BusIO_7Semi< Bus >
```

Definition at line 5 of file [BusIO\\_7Semi.h](#).

#### 5.1.2 Constructor & Destructor Documentation

##### 5.1.2.1 BusIO\_7Semi()

```
template<typename Bus >
BusIO\_7Semi< Bus >::BusIO\_7Semi (
    Bus & busRef ) [inline]
```

Constructor

- Stores interface reference

Definition at line 14 of file [BusIO\\_7Semi.h](#).

### 5.1.3 Member Function Documentation

#### 5.1.3.1 read() [1/3]

```
template<typename Bus >
bool BusIO_7Semi< Bus >::read (
    uint8_t reg,
    uint16_t & value ) [inline]
```

read 16-bit

Definition at line 23 of file [BusIO\\_7Semi.h](#).

#### 5.1.3.2 read() [2/3]

```
template<typename Bus >
bool BusIO_7Semi< Bus >::read (
    uint8_t reg,
    uint8_t & value ) [inline]
```

read 8-bit

Definition at line 17 of file [BusIO\\_7Semi.h](#).



The diagram illustrates the state transitions for the ICM20948 sensor driver. The states are organized into columns, with transitions occurring between them based on specific events. The 'BusIO\_7Semi::read' state is highlighted in grey.

**States (Nodes):**

- DevLab\_ICM20948::auxReadByte
- DevLab\_ICM20948::auxRead Response
- DevLab\_ICM20948::auxRead SensorData
- DevLab\_ICM20948::auxWriteByte
- DevLab\_ICM20948::auxWrite Command
- DevLab\_ICM20948::checkInt Status
- DevLab\_ICM20948::getAccelDLPF
- DevLab\_ICM20948::getAccel Offset
- DevLab\_ICM20948::getAccel SampleRate
- DevLab\_ICM20948::getClock
- DevLab\_ICM20948::getDLPF
- DevLab\_ICM20948::getGyro Offset
- DevLab\_ICM20948::getGyro SampleRate
- DevLab\_ICM20948::getAccel Scale
- DevLab\_ICM20948::getGyro Scale
- BusIO\_7Semi::readBit
- DevLab\_ICM20948::getAccel Averaging
- loop
- DevLab\_ICM20948::readAccel
- BusIO\_7Semi::readBits
- DevLab\_ICM20948::readGyro
- loop
- DevLab\_ICM20948::readTemperature
- DevLab\_ICM20948::readMag
- DevLab\_ICM20948::beginI2C
- DevLab\_ICM20948::beginSPI
- DevLab\_ICM20948::auxMaster Enable
- DevLab\_ICM20948::setDLPF
- DevLab\_ICM20948::setAccelDLPF
- DevLab\_ICM20948::setAccel Averaging
- DevLab\_ICM20948::setAccel Scale
- DevLab\_ICM20948::setGyro Averaging
- DevLab\_ICM20948::setGyro Scale
- BusIO\_7Semi::writeBit
- BusIO\_7Semi::writeBits
- BusIO\_7Semi::writeBits
- BusIO\_7Semi::writeBit

**Transitions (Edges):**

- DevLab\_ICM20948::auxRead12bit → DevLab\_ICM20948::auxRead SensorData
- DevLab\_ICM20948::getSensors → DevLab\_ICM20948::getAccel Scale
- DevLab\_ICM20948::getSensors → DevLab\_ICM20948::getGyro Scale
- DevLab\_ICM20948::getSensors → BusIO\_7Semi::readBit
- DevLab\_ICM20948::getLowPower → BusIO\_7Semi::readBit
- DevLab\_ICM20948::getAccel Scale → BusIO\_7Semi::readBits
- DevLab\_ICM20948::getGyro Scale → BusIO\_7Semi::readBits
- BusIO\_7Semi::readBit → BusIO\_7Semi::readBits
- DevLab\_ICM20948::getAccel Averaging → BusIO\_7Semi::readBits
- loop → DevLab\_ICM20948::readAccel
- loop → BusIO\_7Semi::readBits
- loop → DevLab\_ICM20948::readGyro
- loop → DevLab\_ICM20948::readTemperature
- loop → DevLab\_ICM20948::readMag
- DevLab\_ICM20948::beginI2C → DevLab\_ICM20948::readWhoAmI
- DevLab\_ICM20948::beginSPI → BusIO\_7Semi::writeBit
- DevLab\_ICM20948::auxMaster Enable → BusIO\_7Semi::writeBit
- DevLab\_ICM20948::setDLPF → BusIO\_7Semi::writeBits
- DevLab\_ICM20948::setAccelDLPF → BusIO\_7Semi::writeBits
- DevLab\_ICM20948::setAccel Averaging → BusIO\_7Semi::writeBits
- DevLab\_ICM20948::setAccel Scale → BusIO\_7Semi::writeBits
- DevLab\_ICM20948::setGyro Averaging → BusIO\_7Semi::writeBits
- DevLab\_ICM20948::setGyro Scale → BusIO\_7Semi::writeBits
- BusIO\_7Semi::writeBit → BusIO\_7Semi::writeBits

**Key Features:**

- The diagram uses a color-coded system: blue for standard states, red for specific states (e.g., 'runSweep', 'firstStage', 'applySettings'), and grey for the 'BusIO\_7Semi::read' state.
- Transitions are labeled with the event that triggers the state change, such as 'DevLab\_ICM20948::getSensors' or 'BusIO\_7Semi::read'.
- The diagram is organized into several columns, with states on the left and transitions on the right.

```
template<typename Bus >
bool BusIO_7Semi< Bus >::read (
    uint8_t reg,
    uint8_t * data,
    uint32_t len ) [inline]
```

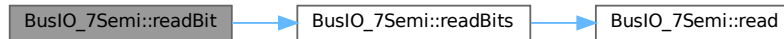
Definition at line 33 of file BusIO\_7Semi.h.

```
template<typename Bus >
```

```
bool BusIO_7Semi< Bus >::readBit (
    uint8_t reg,
    uint8_t pos,
    uint16_t & value ) [inline]
```

Definition at line 142 of file [BusIO\\_7Semi.h](#).

Here is the call graph for this function:

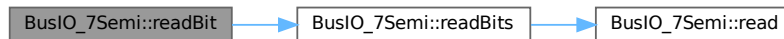


#### 5.1.3.5 readBit() [2/2]

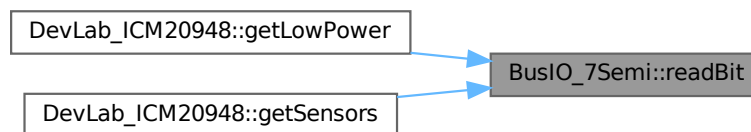
```
template<typename Bus >
bool BusIO_7Semi< Bus >::readBit (
    uint8_t reg,
    uint8_t pos,
    uint8_t & value ) [inline]
```

Definition at line 137 of file [BusIO\\_7Semi.h](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.1.3.6 readBits() [1/2]

```
template<typename Bus >
bool BusIO_7Semi< Bus >::readBits (
    uint8_t reg,
    uint8_t pos,
    uint8_t len,
    uint16_t & value ) [inline]
```

readBits (16-bit register)

Definition at line 78 of file [BusIO\\_7Semi.h](#).

Here is the call graph for this function:



#### 5.1.3.7 readBits() [2/2]

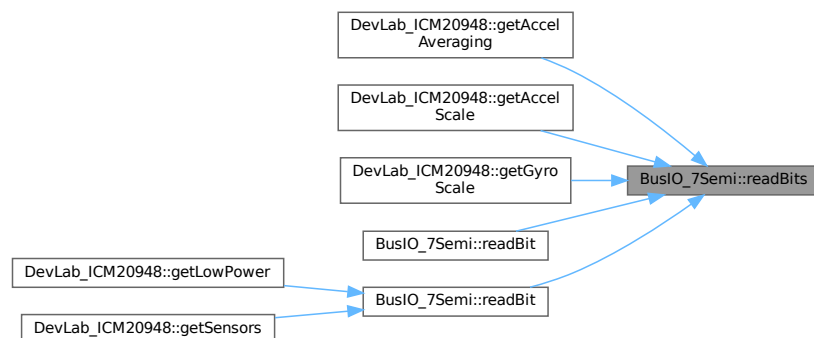
```

template<typename Bus >
bool BusIO_7Semi< Bus >::readBits (
    uint8_t reg,
    uint8_t pos,
    uint8_t len,
    uint8_t & value ) [inline]
  
```

readBits (8-bit register)  
 Definition at line 60 of file [BusIO\\_7Semi.h](#).  
 Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.1.3.8 write() [1/3]

```

template<typename Bus >
bool BusIO_7Semi< Bus >::write (
  
```

```
uint8_t reg,  
const uint8_t * data,  
uint32_t len ) [inline]
```

write burst

Definition at line 52 of file [BusIO\\_7Semi.h](#).

#### 5.1.3.9 write() [2/3]

```
template<typename Bus >  
bool BusIO_7Semi< Bus >::write (  
    uint8_t reg,  
    uint16_t value ) [inline]
```

write 16-bit

Definition at line 45 of file [BusIO\\_7Semi.h](#).

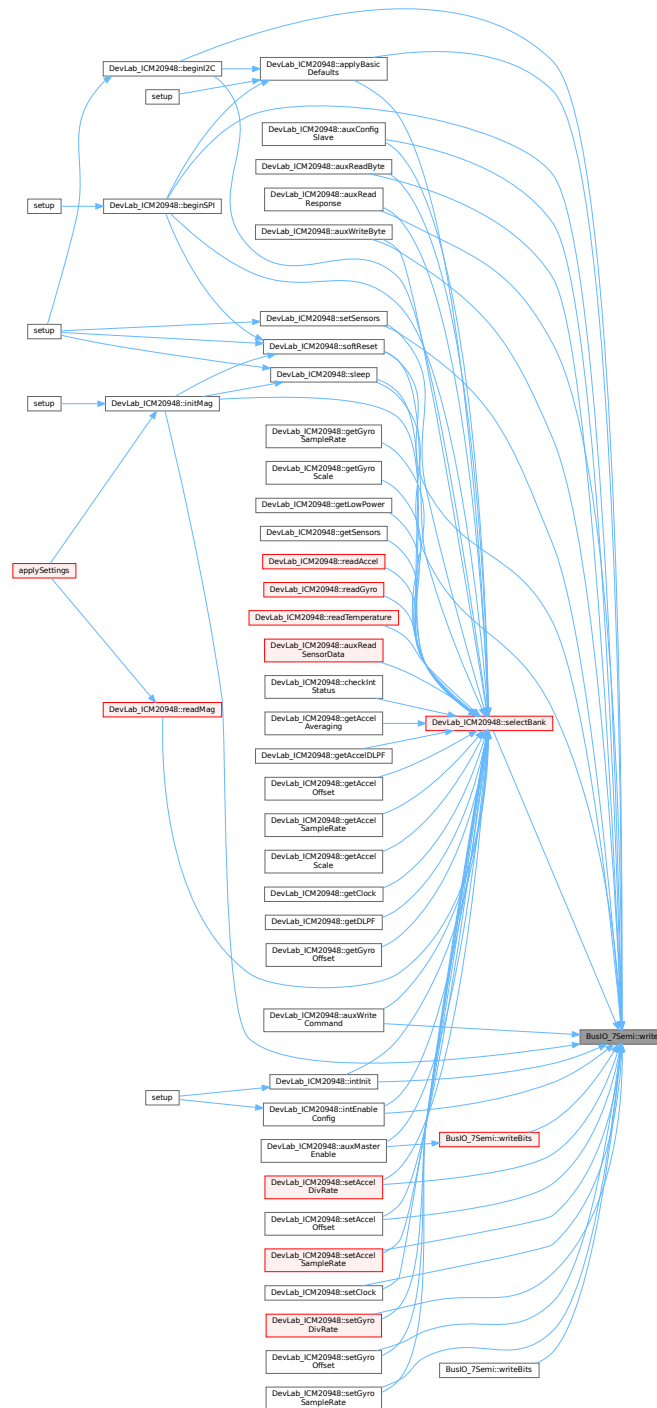
#### 5.1.3.10 write() [3/3]

```
template<typename Bus >  
bool BusIO_7Semi< Bus >::write (  
    uint8_t reg,  
    uint8_t value ) [inline]
```

write 8-bit

Definition at line 39 of file [BusIO\\_7Semi.h](#).

Here is the caller graph for this function:

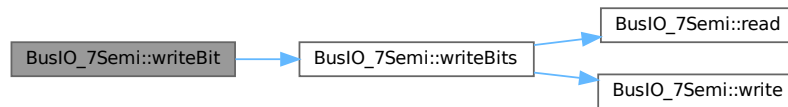


#### 5.1.3.11 writeBit() [1/2]

```
template<typename Bus >
bool BusIO_7Semi< Bus >::writeBit (
    uint8_t reg,
    uint8_t pos,
    uint16_t value ) [inline]
```

Definition at line 151 of file [BusIO\\_7Semi.h](#).

Here is the call graph for this function:



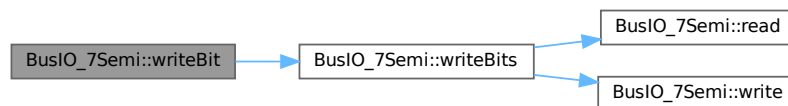
#### 5.1.3.12 writeBit() [2/2]

```

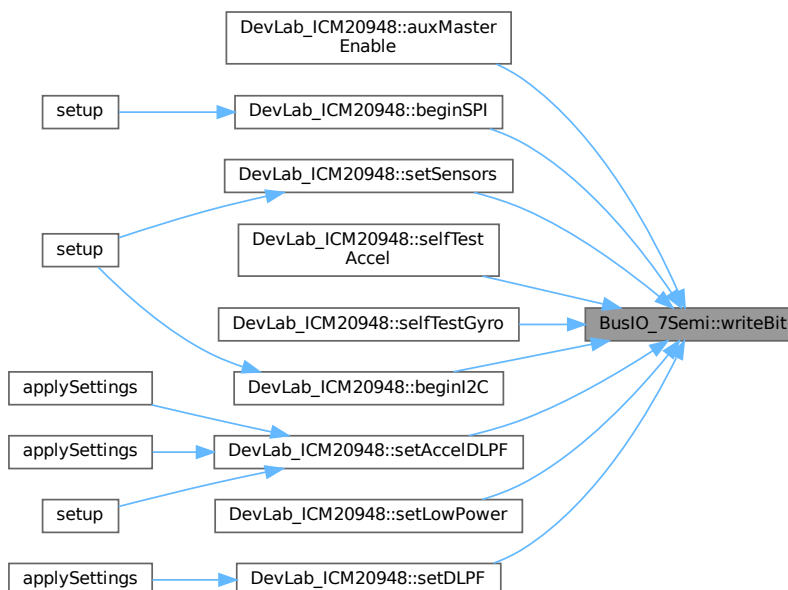
template<typename Bus >
bool BusIO_7Semi< Bus >::writeBit (
    uint8_t reg,
    uint8_t pos,
    uint8_t value ) [inline]
  
```

Definition at line 147 of file [BusIO\\_7Semi.h](#).

Here is the call graph for this function:



Here is the caller graph for this function:



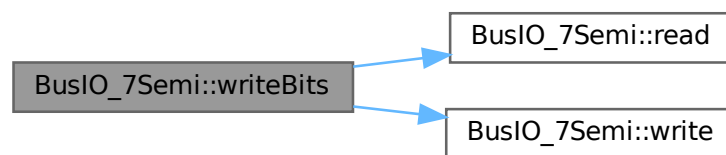
## 5.1.3.13 writeBits() [1/2]

```
template<typename Bus >
bool BusIO_7Semi< Bus >::writeBits (
    uint8_t reg,
    uint8_t pos,
    uint8_t len,
    uint16_t value ) [inline]
writeBits (16-bit register)
```

- Modifies specific bits in 16-bit register

Definition at line 120 of file [BusIO\\_7Semi.h](#).

Here is the call graph for this function:



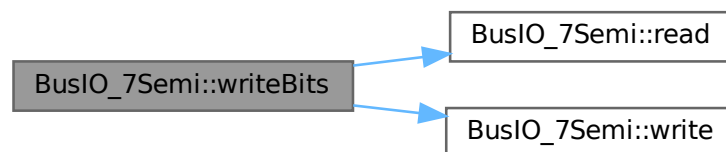
## 5.1.3.14 writeBits() [2/2]

```
template<typename Bus >
bool BusIO_7Semi< Bus >::writeBits (
    uint8_t reg,
    uint8_t pos,
    uint8_t len,
    uint8_t value ) [inline]
writeBits (8-bit register)
```

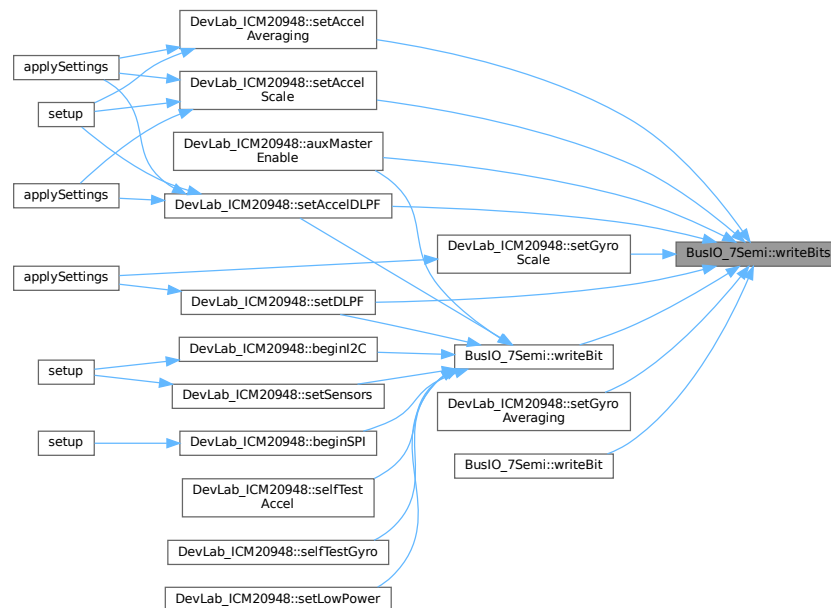
- Modifies specific bits in 8-bit register

Definition at line 98 of file [BusIO\\_7Semi.h](#).

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- [src/BusIO\\_7Semi.h](#)

## 5.2 configDLPF Struct Reference

Gyroscope DLPF divider and timing configuration.

Public Attributes

- `uint8_t` [dlpf\\_idx](#)  
Index into the gyroDLPF lookup table.
- `uint16_t` [div](#)  
Sample-rate divider written to the IMU register.
- `float` [nbw\\_hz](#)  
Noise bandwidth in hertz for reporting.
- `float` [odr\\_hz](#)  
Output data rate in hertz for timing calculations.
- `const char *` [nombre](#)  
Text label printed with each captured sample.

### 5.2.1 Detailed Description

Gyroscope DLPF divider and timing configuration.

Accelerometer DLPF divider and timing configuration.

Definition at line 26 of file [gyr\\_test.ino](#).

### 5.2.2 Member Data Documentation

#### 5.2.2.1 div

`uint16_t` [configDLPF::div](#)



Sample-rate divider written to the IMU register.  
Definition at line 30 of file [gyr\\_test.ino](#).

#### 5.2.2.2 dlpf\_idx

`uint8_t configDLPF::dlpf_idx`  
Index into the gyroDLPF lookup table.  
Index into the accelDLPF lookup table.  
Definition at line 28 of file [gyr\\_test.ino](#).

#### 5.2.2.3 nbw\_hz

`float configDLPF::nbw_hz`  
Noise bandwidth in hertz for reporting.  
Definition at line 32 of file [gyr\\_test.ino](#).

#### 5.2.2.4 nombre

`const char * configDLPF::nombre`  
Text label printed with each captured sample.  
Definition at line 36 of file [gyr\\_test.ino](#).

#### 5.2.2.5 odr\_hz

`float configDLPF::odr_hz`  
Output data rate in hertz for timing calculations.  
Definition at line 34 of file [gyr\\_test.ino](#).  
The documentation for this struct was generated from the following files:

- [examples/gyr\\_test/gyr\\_test.ino](#)
- [examples/test/test.ino](#)
- [examples/test2/test2.ino](#)

## 5.3 DevLab\_ICM20948 Class Reference

`#include <DevLab_ICM20948.h>`

### Public Member Functions

- [DevLab\\_ICM20948](#) ()
- [beginI2C](#) (uint8\_t address=0x69, TwoWire &i2cPort=Wire, uint32\_t i2cSpeed=400000)
- [beginSPI](#) (uint8\_t csPin, SPIClass &spiPort=SPI, uint32\_t spiSpeed=1000000)
- [readWhoAmI](#) (uint8\_t &whoAmI)
- [selectBank](#) (uint8\_t bank)
- [softReset](#) ()
- [sleep](#) (bool en)
- [applyBasicDefaults](#) ()
- [readAccel](#) (float &x, float &y, float &z)
- [readGyro](#) (float &x, float &y, float &z)
- [readTemperature](#) (float &temperature)
- [readMag](#) (float &x, float &y, float &z)
- [initMag](#) ()
- [setMagOpMode](#) (ICM20948\_Op\_Mode opMode)
- [setLowPower](#) (bool enable)
- [getLowPower](#) (bool &enable)
- [setClock](#) (ICM20948\_Clock\_Source clock)
- [getClock](#) (uint8\_t &clock)
- [setGyroSampleRate](#) (float sampleRate)

- bool [getGyroSampleRate](#) (float &sampleRate)
- bool [setGyroDivRate](#) (uint8\_t divisor)
- bool [setDLPF](#) (ICM20948\_Gyro\_DLPF dlpf, bool bypass)
- bool [getDLPF](#) (uint8\_t &dlpf, bool &bypass)
- bool [setGyroScale](#) (ICM20948\_Gyro\_FullScale fullScale)
- bool [setGyroAveraging](#) (ICM20948\_Gyro\_Average avg)
- bool [getGyroScale](#) (uint8\_t &fullScale)
- bool [selfTestGyro](#) (bool x, bool y, bool z)
- bool [setAccelDivRate](#) (uint16\_t divisor)
- bool [setAccelSampleRate](#) (uint16\_t sampleRate)
- bool [getAccelSampleRate](#) (float &sampleRate)
- bool [selfTestAccel](#) (bool x, bool y, bool z)
- bool [setAccelScale](#) (ICM20948\_Accel\_FullScale fullScale)
- bool [getAccelScale](#) (uint8\_t &fullScale)
- bool [setAccelDLPF](#) (uint8\_t dlpf, bool bypass)
- bool [getAccelDLPF](#) (uint8\_t &dlpf, bool &bypass)
- bool [setAccelAveraging](#) (ICM20948\_Accel\_Average avg)
- bool [getAccelAveraging](#) (uint8\_t &avg)
- bool [setSensors](#) (bool accel\_on, bool gyro\_on, bool temp\_on)
- bool [getSensors](#) (bool &accel\_on, bool &gyro\_on, bool &temp\_on)
- bool [setGyroOffset](#) (uint16\_t offsetX, uint16\_t offsetY, uint16\_t offsetZ)
- bool [getGyroOffset](#) (int16\_t &offsetX, int16\_t &offsetY, int16\_t &offsetZ)
- bool [setAccelOffset](#) (int16\_t offsetX, int16\_t offsetY, int16\_t offsetZ)
- bool [getAccelOffset](#) (int16\_t &offsetX, int16\_t &offsetY, int16\_t &offsetZ)
- bool [intInit](#) (const ICM20948\_IntPinConfig &cfg)
- bool [intEnableConfig](#) (const ICM20948\_IntEnableConfig &cfg)
- bool [checkIntStatus](#) (ICM20948\_IntStatus &status)
- bool [auxMasterEnable](#) (uint8\_t clkFreq)
- bool [auxWriteByte](#) (uint8\_t slaveAddr, uint8\_t reg, uint8\_t data)
- bool [auxReadByte](#) (uint8\_t slaveAddr, uint8\_t reg, uint8\_t &data)
- bool [auxWriteCommand](#) (uint8\_t slaveAddr, uint8\_t cmd)
- bool [auxConfigSlave](#) (uint8\_t slaveAddr, uint8\_t reg, uint8\_t numBytes)
- bool [auxReadSensorData](#) (uint8\_t \*buf, uint8\_t len)
- bool [auxReadResponse](#) (uint8\_t slaveAddr, uint8\_t &data)
- bool [auxRead12bit](#) (uint16\_t &raw)

### 5.3.1 Detailed Description

Class

- Manages ICM-20948 over I2C (SPI path disabled in this cut)
- Caches scale factors for LSB->physical conversions

Definition at line 172 of file [DevLab\\_ICM20948.h](#).

### 5.3.2 Constructor & Destructor Documentation

#### 5.3.2.1 DevLab\_ICM20948()

`DevLab_ICM20948::DevLab_ICM20948 ( )`

- Construct with default I2C address; call `begin()` to attach bus

Definition at line 4 of file [DevLab\\_ICM20948.cpp](#).

### 5.3.3 Member Function Documentation

#### 5.3.3.1 applyBasicDefaults()

```
bool DevLab_ICM20948::applyBasicDefaults ( )
applyBasicDefaults
```

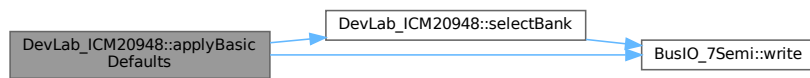
- Apply basic sensor configuration
- Configure power, filters, ranges, and sampling rates

Returns:

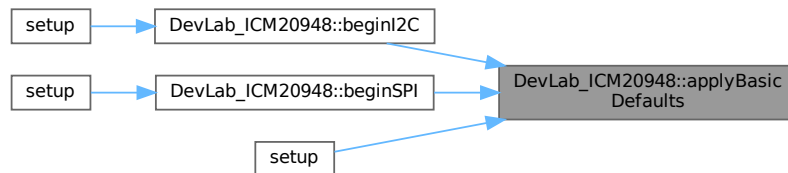
- true Configuration successful
- false Any register write failed

Definition at line 181 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.2 auxConfigSlave()

```
bool DevLab_ICM20948::auxConfigSlave (
    uint8_t slaveAddr,
    uint8_t reg,
    uint8_t numBytes )
auxConfigSlave
```

- Configure SLV0 for automatic periodic reads from an auxiliary I2C slave
- The ICM20948 master will read numBytes starting at reg on every ODR cycle
- Data is deposited into EXT\_SLV\_SENS\_DATA\_00 and subsequent registers
- Call once during initialization; read results with `auxReadSensorData`

Parameters:

- slaveAddr: 7-bit I2C address of the auxiliary slave
- reg: starting register address to read from on the slave

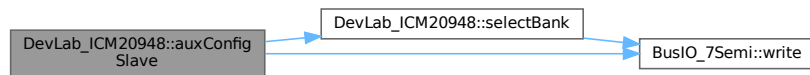
- numBytes: number of bytes to read per cycle (115)

Returns:

- true SLV0 configured successfully
- false Operation failed

Definition at line 1374 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.3 auxMasterEnable()

```
bool DevLab_ICM20948::auxMasterEnable (
    uint8_t clkFreq )
auxMasterEnable
```

- Enable the ICM20948 internal I2C master for auxiliary bus
- Clears `BYPASS_EN` so the aux bus is controlled by the ICM
- Sets `I2C_MST_EN` in `USER_CTRL`
- Configures auxiliary master clock frequency

Parameters:

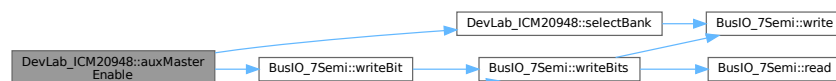
- clkFreq: clock divider value (e.g. 0x07 345.6 kHz, 0x0D 400 kHz)

Returns:

- true Master enabled successfully
- false Operation failed

Definition at line 1260 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.4 auxRead12bit()

```
bool DevLab_ICM20948::auxRead12bit (
    uint16_t & raw )
auxRead12bit
```

- Read a 12-bit value previously configured via `auxConfigSlave` (SLV0, numBytes = 2)
- Assumes little-endian packing: first byte = LSB, second byte = MSB

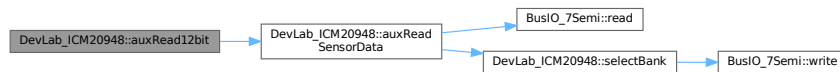
- Result is masked to 12 bits (0-4095)

Returns:

- true Read successful
- false Read failed

Definition at line 1424 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.5 auxReadByte()

```

bool DevLab_ICM20948::auxReadByte (
    uint8_t slaveAddr,
    uint8_t reg,
    uint8_t & data )
  
```

auxReadByte

- Read a single byte from a register of an auxiliary I2C slave via SLV4
- Generates a combined write-then-read transaction (reg byte + repeated START)
- Suitable for slaves that follow standard I2C register-read protocol

Note: slaves that need separate write/read transactions (e.g. command-response firmware) should use `auxWriteCommand + auxReadResponse` instead.

Parameters:

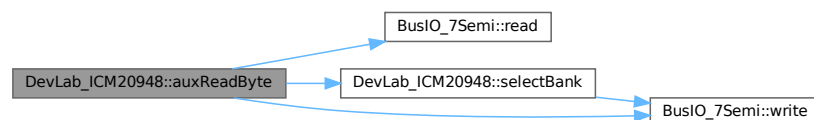
- slaveAddr: 7-bit I2C address of the auxiliary slave
- reg: register address to read from
- data: output byte received from slave

Returns:

- true Read successful
- false Timeout or NACK

Definition at line 1312 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



### 5.3.3.6 auxReadResponse()

```
bool DevLab_ICM20948::auxReadResponse (
    uint8_t slaveAddr,
    uint8_t & data )
auxReadResponse
```

- Read a single response byte from an auxiliary I2C slave via SLV4
- Generates a pure read transaction: [START, addr+R, 1 byte, STOP]
- No register byte is sent (REG\_DIS); slave must already be ready to transmit
- Use after auxWriteCommand to complete a command-response exchange with slaves that require separate write and read transactions (e.g. PY32F0 firmware slaves)

Parameters:

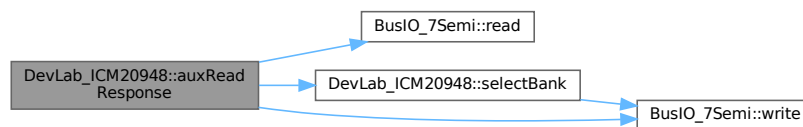
- slaveAddr: 7-bit I2C address of the auxiliary slave
- data: output byte received from slave

Returns:

- true Response received successfully
- false Timeout or NACK

Definition at line 1397 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



### 5.3.3.7 auxReadSensorData()

```
bool DevLab_ICM20948::auxReadSensorData (
    uint8_t * buf,
    uint8_t len )
auxReadSensorData
```

- Read bytes from EXT\_SLV\_SENS\_DATA registers (BANK 0)
- Returns data collected by the ICM20948 master from SLV0SLV3 on the last cycle
- Must call auxConfigSlave first to set up the automatic read

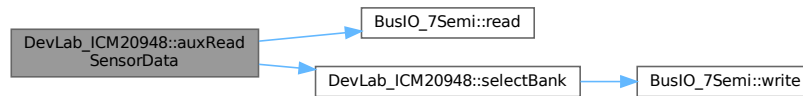
Parameters:

- buf: output buffer to store the received bytes
- len: number of bytes to read (must match numBytes configured in auxConfigSlave)

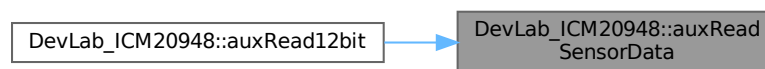
Returns:

- true Read successful
- false Bus error

Definition at line 1391 of file [DevLab\\_ICM20948.cpp](#).  
Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.8 auxWriteByte()

```
bool DevLab_ICM20948::auxWriteByte (
    uint8_t slaveAddr,
    uint8_t reg,
    uint8_t data )
auxWriteByte
```

- Write a single byte to a register of an auxiliary I2C slave via SLV4
- Uses one-shot SLV4 transaction: polls SLV4\_DONE, checks for NACK
- Suitable for configuration writes (not continuous streaming)

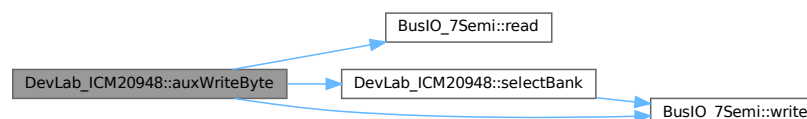
Parameters:

- slaveAddr: 7-bit I2C address of the auxiliary slave
- reg: target register address on the slave
- data: byte to write

Returns:

- true Write acknowledged by slave
- false Timeout or NACK

Definition at line 1281 of file [DevLab\\_ICM20948.cpp](#).  
Here is the call graph for this function:



### 5.3.3.9 auxWriteCommand()

```
bool DevLab_ICM20948::auxWriteCommand (
    uint8_t slaveAddr,
    uint8_t cmd )
auxWriteCommand
```

- Send a single command byte to an auxiliary I2C slave via SLV4
- Uses REG\_DIS: generates [START, addr+W, cmd\_byte, STOP] with no register prefix
- Designed for slaves that use a command-response protocol (no register addressing)

Parameters:

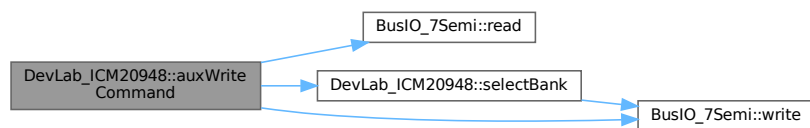
- slaveAddr: 7-bit I2C address of the auxiliary slave
- cmd: command byte to send

Returns:

- true Command acknowledged by slave
- false Timeout or NACK

Definition at line 1346 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



### 5.3.3.10 beginI2C()

```
bool DevLab_ICM20948::beginI2C (
    uint8_t address = 0x69,
    TwoWire & i2cPort = Wire,
    uint32_t i2cSpeed = 400000 )
beginI2C
```

- Initialize ICM20948 using I2C interface
- Sets up communication layer (Interface + BusIO)
- Verifies device identity (WHO\_AM\_I)
- Configures basic power and clock settings

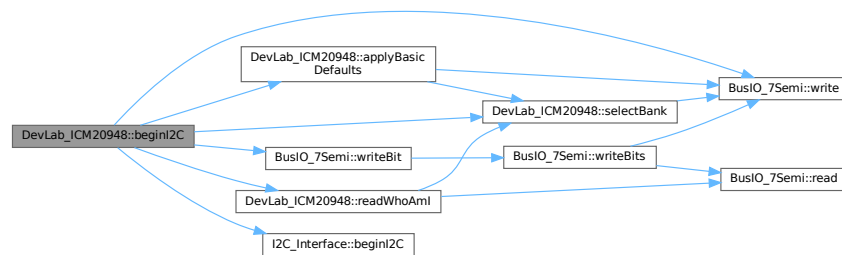
Returns:

- true Device initialized successfully
- false Communication or configuration failed

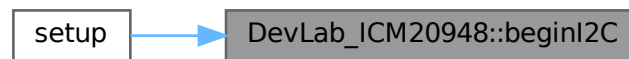
Definition at line 6 of file [DevLab\\_ICM20948.cpp](#).



Here is the call graph for this function:



Here is the caller graph for this function:

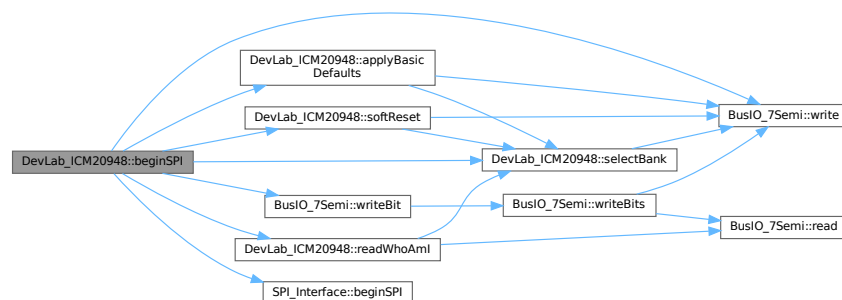


#### 5.3.3.11 beginSPI()

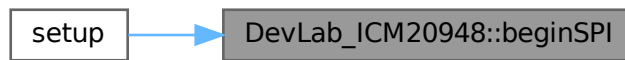
```
bool DevLab_ICM20948::beginSPI (
    uint8_t csPin,
    SPIClass & spiPort = SPI,
    uint32_t spiSpeed = 1000000 )
```

Definition at line 60 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.12 checkIntStatus()

```

bool DevLab_ICM20948::checkIntStatus (
    ICM20948_IntStatus & status )
checkIntStatus
  
```

- Read and parse all four interrupt status registers in a single burst read
- Covers INT\_STATUS (0x19), INT\_STATUS\_1 (0x1A), INT\_STATUS\_2 (0x1B), INT\_STATUS\_3 (0x1C) from BANK 0
- Reading clears the interrupt flags when clearMode = 0 in [ICM20948\\_IntPinConfig](#)
- Typically called inside the INT pin ISR or polled in the main loop

Parameters:

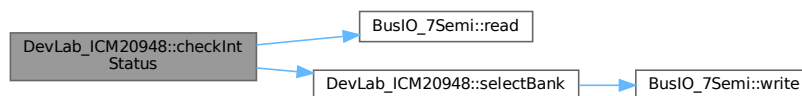
- status: [ICM20948\\_IntStatus](#) struct populated with the current interrupt flags

Returns:

- true Status read successfully
- false Operation failed

Definition at line 1231 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.13 getAccelAveraging()

```

bool DevLab_ICM20948::getAccelAveraging (
    uint8_t & avg )
getAccelAveraging
  
```

- Read accelerometer averaging / decimation setting (DEC3)

Flow:

- Validate bus pointer
- Select USER BANK 2

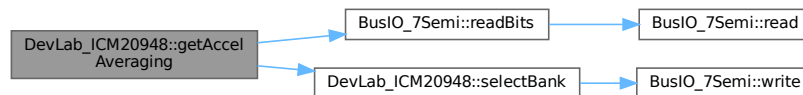
- Read DEC3 bits [1:0]

Returns:

- true Read successful
- false Operation failed

Definition at line 988 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.14 getAccelDLPF()

```
bool DevLab_ICM20948::getAccelDLPF (
    uint8_t & dlpf,
    bool & bypass )
getAccelDLPF
```

- Read accelerometer DLPF configuration
- Returns filter setting and bypass state

Flow:

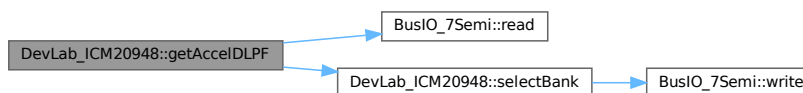
- Validate bus pointer
- Select USER BANK 2
- Read ACCEL\_CONFIG register
- Extract bypass and DLPF bits

Returns:

- true Read successful
- false Operation failed

Definition at line 926 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



## 5.3.3.15 getAccelOffset()

```
bool DevLab_ICM20948::getAccelOffset (
    int16_t & offsetX,
    int16_t & offsetY,
    int16_t & offsetZ )
getAccelOffset
```

- Read accelerometer offset values for X, Y, Z axes
- Reads offset registers from USER BANK 1

Flow:

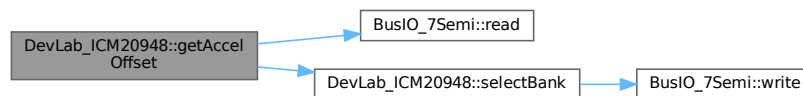
- Validate bus pointer
- Select USER BANK 1
- Read offset registers
- Convert to signed values

Returns:

- true Read successful
- false Operation failed

Definition at line 1151 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



## 5.3.3.16 getAccelSampleRate()

```
bool DevLab_ICM20948::getAccelSampleRate (
    float & sampleRate )
getAccelSampleRate
```

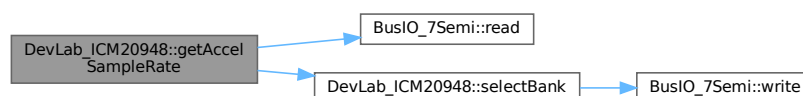
- Get accelerometer output data rate (ODR)
- Computes value from divider registers

Returns:

- true Read successful
- false Operation failed

Definition at line 796 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



## 5.3.3.17 getAccelScale()

```
bool DevLab_ICM20948::getAccelScale (
    uint8_t & fullScale )
getAccelScale
```

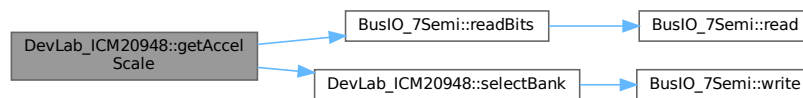
- Get current accelerometer full-scale range (FS\_SEL)

Returns:

- true Read successful
- false Operation failed

Definition at line 878 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



## 5.3.3.18 getClock()

```
bool DevLab_ICM20948::getClock (
    uint8_t & clock )
getClock
```

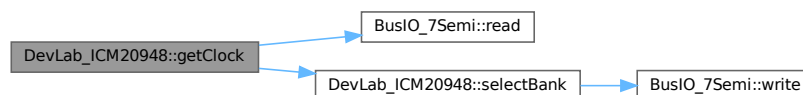
- Read current clock source (CLKSEL [2:0])

Returns:

- true Read successful
- false Operation failed

Definition at line 534 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



## 5.3.3.19 getDLPF()

```
bool DevLab_ICM20948::getDLPF (
    uint8_t & dlpf,
    bool & bypass )
getDLPF
```

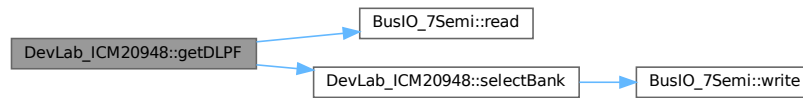
- Read gyroscope DLPF configuration
- Returns filter setting and bypass state

Returns:

- true Read successful
- false Operation failed

Definition at line 619 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.20 getGyroOffset()

```
bool DevLab_ICM20948::getGyroOffset (
    int16_t & offsetX,
    int16_t & offsetY,
    int16_t & offsetZ )
getGyroOffset
```

- Read gyroscope offset values for X, Y, Z axes
- Reads offset registers from USER BANK 2

Flow:

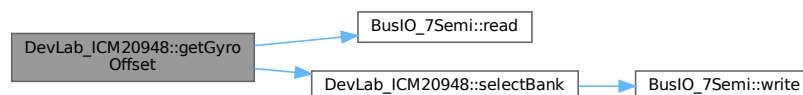
- Validate bus pointer
- Select USER BANK 2
- Read offset registers
- Convert to signed values

Returns:

- true Read successful
- false Operation failed

Definition at line 1099 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



## 5.3.3.21 getGyroSampleRate()

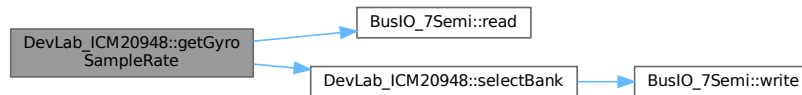
```
bool DevLab_ICM20948::getGyroSampleRate (
    float & sampleRate )
getGyroSampleRate
```

- Get current gyroscope sample rate
- Computes value from divider register

Returns:

- true Read successful
- false Operation failed

Definition at line 571 of file [DevLab\\_ICM20948.cpp](#).  
Here is the call graph for this function:



## 5.3.3.22 getGyroScale()

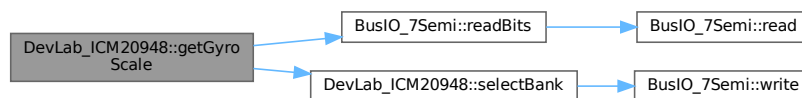
```
bool DevLab_ICM20948::getGyroScale (
    uint8_t & fullScale )
getGyroScale
```

- Get current gyroscope full-scale range (FS\_SEL

Returns:

- true Read successful
- false Operation failed

Definition at line 665 of file [DevLab\\_ICM20948.cpp](#).  
Here is the call graph for this function:



## 5.3.3.23 getLowPower()

```
bool DevLab_ICM20948::getLowPower (
    bool & enable )
getLowPower
```

- Read low power mode status

- Returns state of LP\_EN bit

Returns:

- true Read successful
- false Operation failed

Definition at line 502 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.24 getSensors()

```

bool DevLab_ICM20948::getSensors (
    bool & accel_on,
    bool & gyro_on,
    bool & temp_on )
getSensors
  
```

- Read accel, gyro, and temperature enable state

Flow:

- Validate bus pointer
- Select USER BANK 0
- Read PWR\_MGMT\_2 register
- Read TEMP\_DIS bit
- Decode enable states

Returns:

- true Read successful
- false Operation failed

Definition at line 1039 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:





## 5.3.3.25 initMag()

```
bool DevLab_ICM20948::initMag ( )
initMag
```

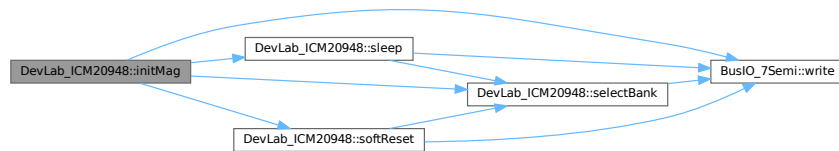
- Initialize AK09916 magnetometer using internal I2C master
- Verifies WHO\_AM\_I and enables continuous mode
- Configures SLV0 for automatic data read

Returns:

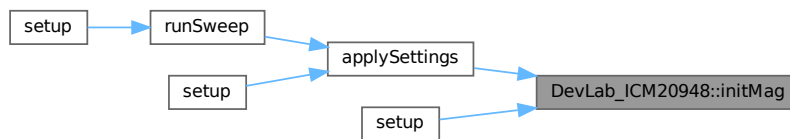
- true Initialization successful
- false Operation failed

Definition at line 334 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 5.3.3.26 intEnableConfig()

```
bool DevLab_ICM20948::intEnableConfig (
    const ICM20948_IntEnableConfig & cfg )
intEnableConfig
```

- Enable or disable individual interrupt sources routed to the INT pin
- Writes INT\_ENABLE (0x10), INT\_ENABLE\_1 (0x11), INT\_ENABLE\_2 (0x12), and INT\_ENABLE\_3 (0x13) in BANK 0
- Must call `intInit()` first to configure the pin electrical behavior

Parameters:

- `cfg`: [ICM20948\\_IntEnableConfig](#) struct with the desired interrupt sources enabled

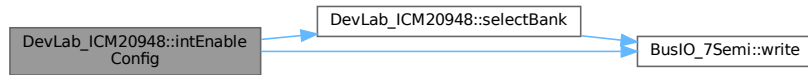
Returns:

- true All four registers written successfully

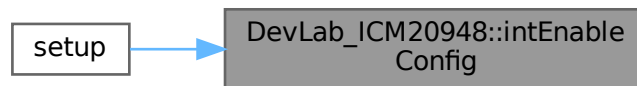
- false Operation failed

Definition at line 1200 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.27 intInit()

```

bool DevLab_ICM20948::intInit (
    const ICM20948\_IntPinConfig & cfg )
intInit
  
```

- Configure the physical behavior of the INT pin (INT\_PIN\_CFG register, BANK 0)
- Sets active level, drive mode (push-pull / open-drain), latch vs pulse mode, clear condition, FSYNC level, FSYNC interrupt enable, and I2C bypass

Parameters:

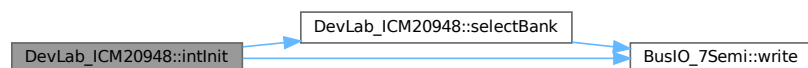
- cfg: [ICM20948\\_IntPinConfig](#) struct with the desired pin settings

Returns:

- true Configuration written successfully
- false Operation failed

Definition at line 1176 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.28 readAccel()

```

bool DevLab_ICM20948::readAccel (
    float & x,
    float & y,
    float & z )
  
```

readAccel

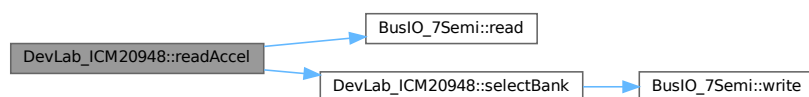
- Read accelerometer data (X, Y, Z)
- Convert raw values to g using LSB scale

Returns:

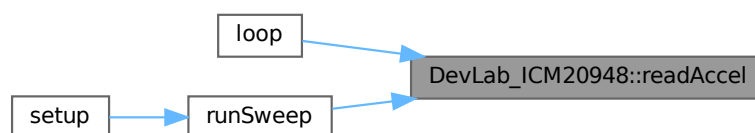
- true Read successful
- false Read failed

Definition at line 238 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 5.3.3.29 readGyro()

```
bool DevLab_ICM20948::readGyro (
    float & x,
    float & y,
    float & z )
readGyro
```

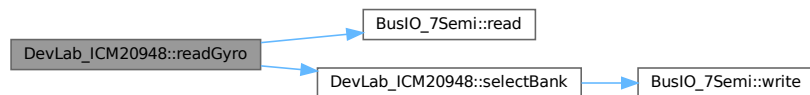
- Read gyroscope data (X, Y, Z)
- Convert raw values to dps using LSB scale

Returns:

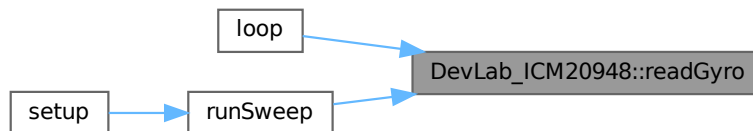
- true Read successful
- false Read failed

Definition at line 258 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



### 5.3.3.30 readMag()

```
bool DevLab_ICM20948::readMag (
    float & x,
    float & y,
    float & z )
readMag
```

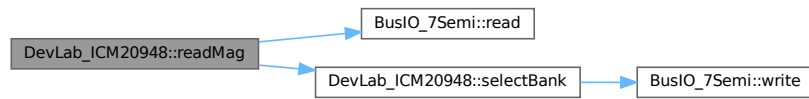
- Read magnetometer data via internal I2C master
- Data comes from EXT\_SLV\_SENS\_DATA registers
- Convert raw values to  $\mu T$

Returns:

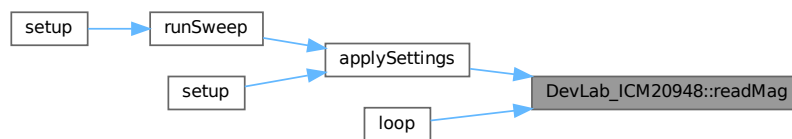
- true Read successful
- false Read failed

Definition at line 310 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.31 readTemperature()

```
bool DevLab_ICM20948::readTemperature (
    float & temperature )
readTemperature
```

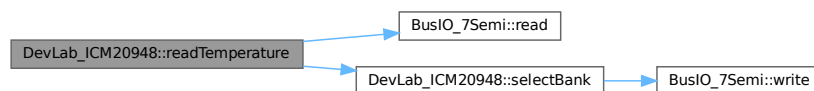
- Read temperature sensor
- Convert raw values to °C

Returns:

- true Read successful
- false Read failed

Definition at line 278 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.32 readWhoAmI()

```

bool DevLab_ICM20948::readWhoAmI (
    uint8_t & whoAmI )
readWhoAmI
  
```

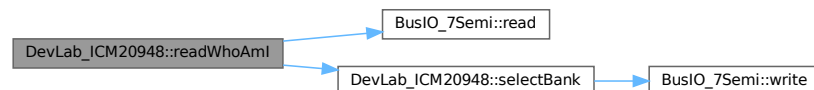
- Read WHO\_AM\_I register (device ID)
- Used during initialization to verify ICM20948
- Expected value: 0xEA

Returns:

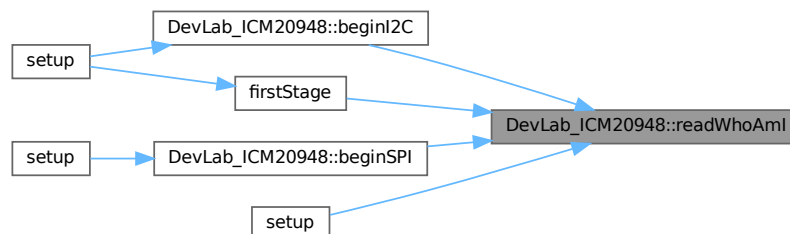
- true Read successful
- false Operation failed

Definition at line 118 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 5.3.3.33 selectBank()

```
bool DevLab_ICM20948::selectBank (
    uint8_t bank )
selectBank
```

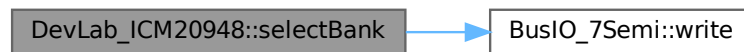
- Select USER BANK (03)
- Used to access different register groups inside ICM20948

Returns:

- true Bank switch successful
- false Write failed

Definition at line 132 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



## 5.3.3.34 selfTestAccel()

```
bool DevLab_ICM20948::selfTestAccel (
    bool x,
    bool y,
    bool z )
selfTestAccel
```

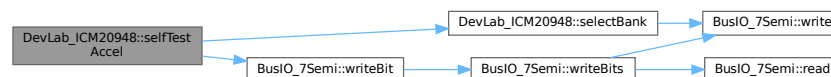
- Enable or disable accelerometer self-test per axis

Returns:

- true Operation successful
- false Operation failed

Definition at line 826 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



## 5.3.3.35 selfTestGyro()

```
bool DevLab_ICM20948::selfTestGyro (
    bool x,
    bool y,
    bool z )
selfTestGyro
```

- Enable or disable gyroscope self-test per axis

Returns:

- true Operation successful
- false Operation failed

Definition at line 683 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.36 setAccelAveraging()

```
bool DevLab_ICM20948::setAccelAveraging (
    ICM20948_Accel_Average avg )
setAccelAveraging
```

- Configure accelerometer averaging / decimation (DEC3)
- Controls internal averaging of accel samples

Flow:

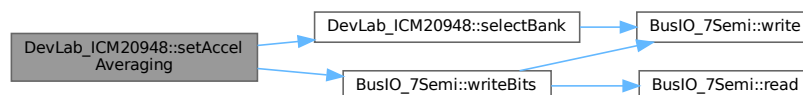
- Validate bus pointer
- Select USER BANK 2
- Write DEC3 bits [1:0]

Returns:

- true Operation successful
- false Operation failed

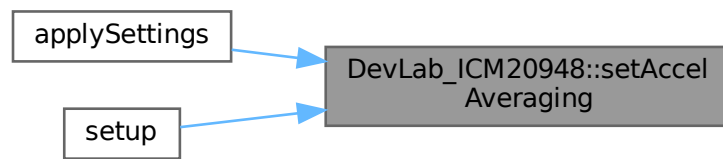
Definition at line 952 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:





Here is the caller graph for this function:

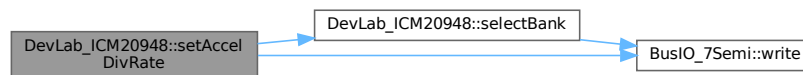


#### 5.3.3.37 setAccelDivRate()

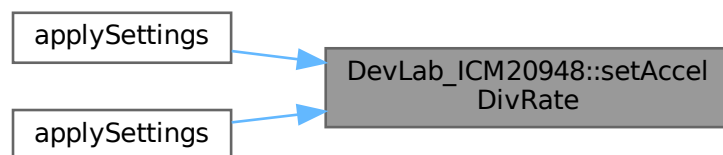
```
bool DevLab_ICM20948::setAccelDivRate (
    uint16_t divisor )
```

Definition at line 748 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.38 setAccelDLPF()

```
bool DevLab_ICM20948::setAccelDLPF (
    uint8_t dlpf,
    bool bypass )
```

`setAccelDLPF`

- Configure accelerometer digital low-pass filter (DLPF)
- Option to bypass filter (full bandwidth)

Flow:

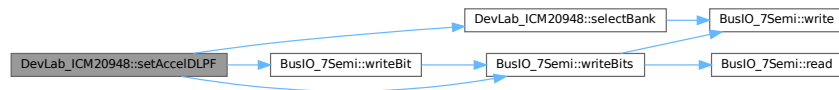
- Validate bus pointer
- Select USER BANK 2
- Set or clear bypass bit
- Configure DLPF bits if not bypassed

Returns:

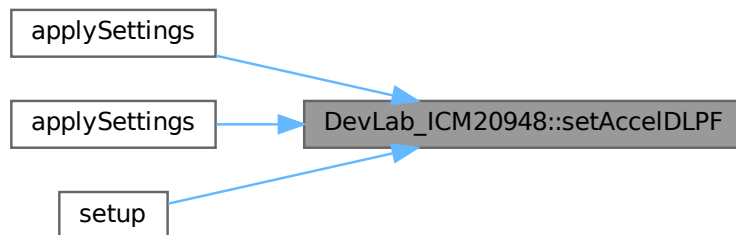
- true Operation successful
- false Operation failed

Definition at line 897 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.39 setAccelOffset()

```

bool DevLab_ICM20948::setAccelOffset (
    int16_t offsetX,
    int16_t offsetY,
    int16_t offsetZ )
setAccelOffset

```

- Set accelerometer offset values for X, Y, Z axes
- Writes offset registers in USER BANK 1

Flow:

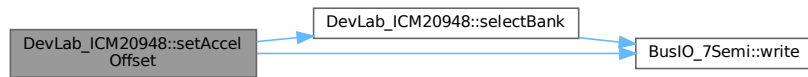
- Validate bus pointer
- Select USER BANK 1
- Pack offsets into byte array
- Write to offset registers

Returns:

- true Operation successful
- false Operation failed

Definition at line 1124 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.40 setAccelSampleRate()

```
bool DevLab_ICM20948::setAccelSampleRate (
    uint16_t sampleRate )
setAccelSampleRate
```

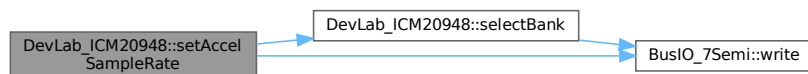
- Set accelerometer output data rate (ODR)
- Uses 1125 Hz base rate with 12-bit divider

Returns:

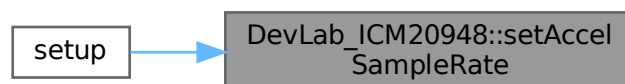
- true Operation successful
- false Operation failed

Definition at line 709 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 5.3.3.41 setAccelScale()

```
bool DevLab_ICM20948::setAccelScale (
    ICM20948_Accel_FullScale fullScale )
setAccelScale
```

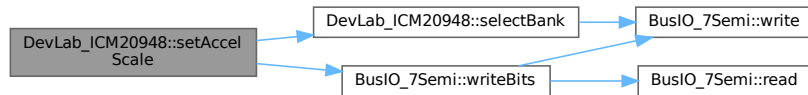
- Set accelerometer full-scale range (FS\_SEL)
- Controls measurement range (±2g, ±4g, ±8g, ±16g)

Returns:

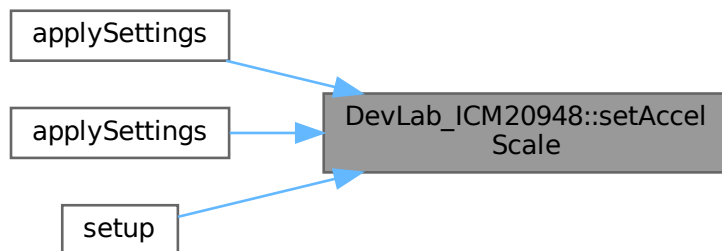
- true Operation successful
- false Operation failed

Definition at line 857 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 5.3.3.42 setClock()

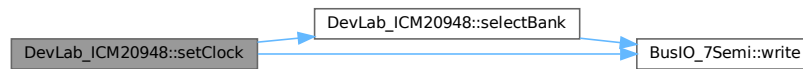
```
bool DevLab_ICM20948::setClock (
    ICM20948_Clock_Source clock )
setClock
```

- Set clock source (CLKSEL [2:0])
- Selects internal clock for sensor operation

Returns:

- true Operation successful
- false Operation failed

Definition at line 520 of file [DevLab\\_ICM20948.cpp](#).  
Here is the call graph for this function:



#### 5.3.3.43 setDLPF()

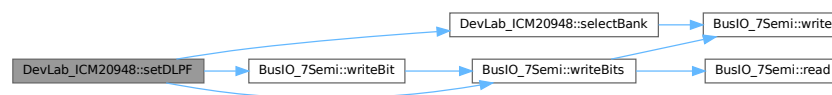
```
bool DevLab_ICM20948::setDLPF (
    ICM20948_Gyro_DLPF dlpf,
    bool bypass )
setDLPF
```

- Configure gyroscope digital low-pass filter (DLPF)
- Option to bypass filter (full bandwidth)

Returns:

- true Operation successful
- false Operation failed

Definition at line 593 of file [DevLab\\_ICM20948.cpp](#).  
Here is the call graph for this function:



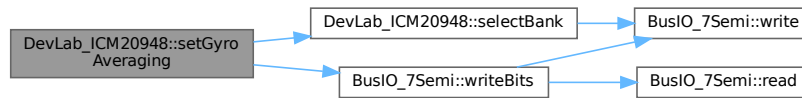
Here is the caller graph for this function:



#### 5.3.3.44 setGyroAveraging()

```
bool DevLab_ICM20948::setGyroAveraging (
    ICM20948_Gyro_Average avg )
Definition at line 970 of file DevLab\_ICM20948.cpp.
```

Here is the call graph for this function:

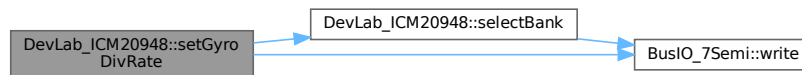


#### 5.3.3.45 setGyroDivRate()

```
bool DevLab_ICM20948::setGyroDivRate (
    uint8_t divisor )
```

Definition at line 780 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.46 setGyroOffset()

```
bool DevLab_ICM20948::setGyroOffset (
    uint16_t offsetX,
    uint16_t offsetY,
    uint16_t offsetZ )
setGyroOffset
```

- Set gyroscope offset values for X, Y, Z axes
- Writes offset registers in USER BANK 2

Flow:

- Validate bus pointer
- Select USER BANK 2
- Pack offsets into byte array

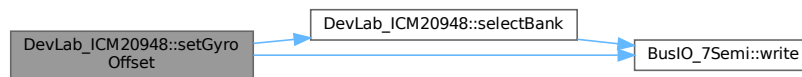
- Write to offset registers

Returns:

- true Operation successful
- false Operation failed

Definition at line 1072 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.47 setGyroSampleRate()

```
bool DevLab_ICM20948::setGyroSampleRate (
    float sampleRate )
setGyroSampleRate
```

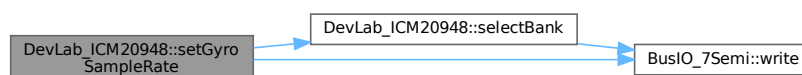
- Set gyroscope sample rate
- Uses internal divider (SRD) based on 1100 Hz base rate

Returns:

- true Operation successful
- false Invalid input or write failed

Definition at line 550 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



#### 5.3.3.48 setGyroScale()

```
bool DevLab_ICM20948::setGyroScale (
    ICM20948\_Gyro\_FullScale fullScale )
setGyroScale
```

- Set gyroscope full-scale range (FS\_SEL)
- Controls sensitivity (dps range)

Flow:

- Validate bus pointer
- Select USER BANK 2

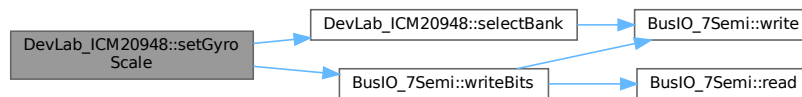
- Write FS\_SEL bits [2:1]

Returns:

- true Operation successful
- false Operation failed

Definition at line 644 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.49 setLowPower()

```
bool DevLab_ICM20948::setLowPower (
    bool enable )
setLowPower
```

- Enable or disable low power mode
- Controls LP\_EN bit in PWR\_MGMT\_1

Returns:

- true Operation successful
- false Operation failed

Definition at line 488 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:





## 5.3.3.50 setMagOpMode()

```
bool DevLab_ICM20948::setMagOpMode (
    ICM20948_Op_Mode opMode )
setMagOpMode
```

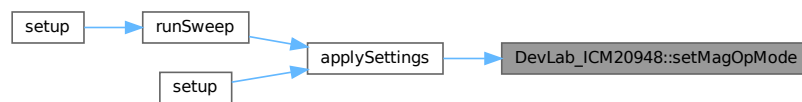
- Set magnetometer operation mode

Returns:

- true Mode set successfully
- false Write failed

Definition at line 410 of file [DevLab\\_ICM20948.cpp](#).

Here is the caller graph for this function:



## 5.3.3.51 setSensors()

```
bool DevLab_ICM20948::setSensors (
    bool accel_on,
    bool gyro_on,
    bool temp_on )
setSensors
```

- Enable or disable accel, gyro, and temperature sensor
- Controls power gating via PWR\_MGMT\_2 and TEMP\_DIS

Flow:

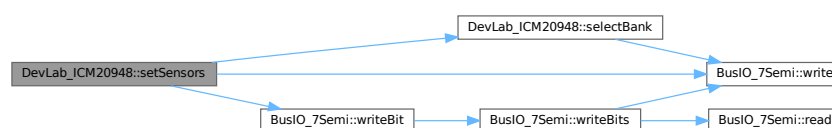
- Validate bus pointer
- Select USER BANK 0
- Build PWR\_MGMT\_2 mask
- Configure temperature enable/disable

Returns:

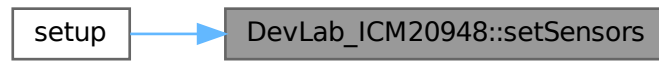
- true Operation successful
- false Operation failed

Definition at line 1006 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 5.3.3.52 sleep()

```
bool DevLab_ICM20948::sleep (
    bool en )
sleep
```

- Enable or disable sleep mode
- Maintains clock source (CLKSEL = 1)

Flow:

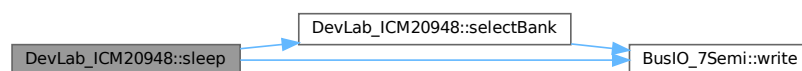
- Select USER BANK 0
- Set or clear SLEEP bit

Returns:

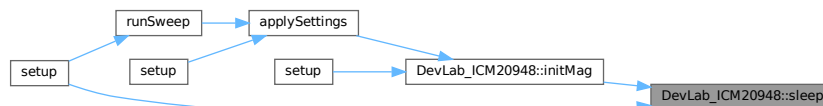
- true Operation successful
- false Operation failed

Definition at line 164 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 5.3.3.53 softReset()

```
bool DevLab_ICM20948::softReset ( )
softReset
```

- Perform software reset of ICM20948
- Resets all internal registers to default state

Flow:

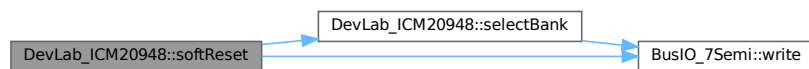
- Select USER BANK 0
- Set DEVICE\_RESET bit
- Wait for device to restart

Returns:

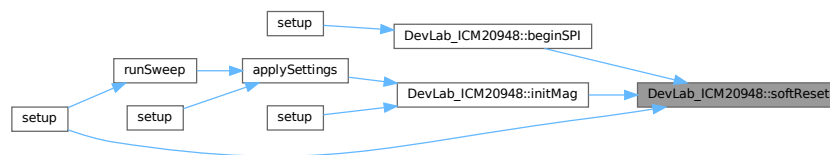
- true Reset successful
- false Operation failed

Definition at line 144 of file [DevLab\\_ICM20948.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



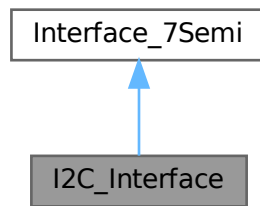
The documentation for this class was generated from the following files:

- [src/DevLab\\_ICM20948.h](#)
- [src/DevLab\\_ICM20948.cpp](#)

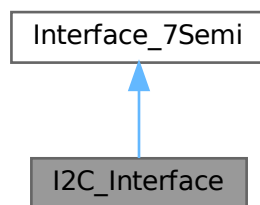
## 5.4 I2C\_Interface Class Reference

```
#include <7Semi_I2C_Interface.h>
```

Inheritance diagram for `I2C_Interface`:



Collaboration diagram for `I2C_Interface`:



#### Public Member Functions

- `bool beginI2C` (`uint8_t` addr, `TwoWire` &wire, `uint32_t` speed, `uint8_t` sda=255, `uint8_t` scl=255)
- `int8_t read` (`uint8_t` reg, `uint8_t` \*data, `uint32_t` len) override
- `bool beginSPI` (`uint8_t`, `SPIClass` &, `uint32_t`, `uint8_t`, `uint8_t`, `uint8_t`) override
- `int8_t write` (`uint8_t` reg, `const uint8_t` \*data, `uint32_t` len) override

#### Public Member Functions inherited from `Interface_7Semi`

- `virtual ~Interface_7Semi` ()

#### Public Attributes

- `TwoWire` \* `i2c` = nullptr
- `uint8_t` `address` = 0

#### Additional Inherited Members

#### Static Protected Member Functions inherited from `Interface_7Semi`

- `static void delay_us` (`uint32_t` us)

### 5.4.1 Detailed Description

Definition at line 5 of file `7Semi_I2C_Interface.h`.

## 5.4.2 Member Function Documentation

### 5.4.2.1 beginI2C()

```
bool I2C_Interface::beginI2C (
    uint8_t address,
    TwoWire & wire,
    uint32_t speed,
    uint8_t sda = 255,
    uint8_t scl = 255 ) [inline], [virtual]
```

[beginI2C\(\)](#)

- Initializes I2C peripheral
- Configures SDA / SCL pins if supported
- Sets communication clock

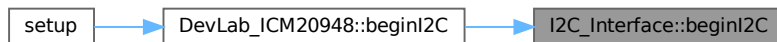
Returns:

- true Initialization successful
- false Initialization failed

Implements [Interface\\_7Semi](#).

Definition at line 11 of file [7Semi\\_I2C\\_Interface.h](#).

Here is the caller graph for this function:



### 5.4.2.2 beginSPI()

```
bool I2C_Interface::beginSPI (
    uint8_t ,
    SPIClass & ,
    uint32_t ,
    uint8_t ,
    uint8_t ,
    uint8_t ) [inline], [override], [virtual]
```

Implements [Interface\\_7Semi](#).

Definition at line 69 of file [7Semi\\_I2C\\_Interface.h](#).

### 5.4.2.3 read()

```
int8_t I2C_Interface::read (
    uint8_t reg,
    uint8_t * data,
    uint32_t len ) [inline], [override], [virtual]
```

[read\(\)](#)

- Reads data from device register

Parameters:

- reg : Register address
- data : Output buffer

- len : Number of bytes

Returns:

- 0 Success
- <0 Error

Implements [Interface\\_7Semi](#).

Definition at line 32 of file [7Semi\\_I2C\\_Interface.h](#).

#### 5.4.2.4 write()

```
int8_t I2C_Interface::write (
    uint8_t reg,
    const uint8_t * data,
    uint32_t len ) [inline], [override], [virtual]
```

[write\(\)](#)

- Writes data to device register

Returns:

- 0 Success
- <0 Error

Implements [Interface\\_7Semi](#).

Definition at line 75 of file [7Semi\\_I2C\\_Interface.h](#).

### 5.4.3 Member Data Documentation

#### 5.4.3.1 address

```
uint8_t I2C_Interface::address = 0
```

Definition at line 9 of file [7Semi\\_I2C\\_Interface.h](#).

#### 5.4.3.2 i2c

```
TwoWire* I2C_Interface::i2c = nullptr
```

Definition at line 8 of file [7Semi\\_I2C\\_Interface.h](#).

The documentation for this class was generated from the following file:

- [src/7Semi\\_I2C\\_Interface.h](#)

## 5.5 ICM20948\_IntEnableConfig Struct Reference

```
#include <DevLab_ICM20948.h>
```

Public Attributes

- uint8\_t [wofEn](#): 1
- uint8\_t [womIntEn](#): 1
- uint8\_t [pllRdyEn](#): 1
- uint8\_t [dmpInt1En](#): 1
- uint8\_t [i2cMstIntEn](#): 1
- uint8\_t [rawDataRdyEn](#): 1
- uint8\_t [fifoOvfEn](#) [5]
- uint8\_t [fifoWmEn](#) [5]

### 5.5.1 Detailed Description

#### ICM20948\_IntEnableConfig

- Selects which interrupt sources are routed to the INT pin
- Covers INT\_ENABLE (0x10), INT\_ENABLE\_1 (0x11), INT\_ENABLE\_2 (0x12), INT\_ENABLE\_3 (0x13)
- FIFO overflow and watermark fields are per-channel arrays [0][4]
- Pass to intEnableConfig() to write all four registers in one call

Definition at line 122 of file [DevLab\\_ICM20948.h](#).

### 5.5.2 Member Data Documentation

#### 5.5.2.1 dmpInt1En

uint8\_t ICM20948\_IntEnableConfig::dmpInt1En

Definition at line 128 of file [DevLab\\_ICM20948.h](#).

#### 5.5.2.2 fifoOvfEn

uint8\_t ICM20948\_IntEnableConfig::fifoOvfEn[5]

Definition at line 135 of file [DevLab\\_ICM20948.h](#).

#### 5.5.2.3 fifoWmEn

uint8\_t ICM20948\_IntEnableConfig::fifoWmEn[5]

Definition at line 138 of file [DevLab\\_ICM20948.h](#).

#### 5.5.2.4 i2cMstIntEn

uint8\_t ICM20948\_IntEnableConfig::i2cMstIntEn

Definition at line 129 of file [DevLab\\_ICM20948.h](#).

#### 5.5.2.5 pllRdyEn

uint8\_t ICM20948\_IntEnableConfig::pllRdyEn

Definition at line 127 of file [DevLab\\_ICM20948.h](#).

#### 5.5.2.6 rawDataRdyEn

uint8\_t ICM20948\_IntEnableConfig::rawDataRdyEn

Definition at line 132 of file [DevLab\\_ICM20948.h](#).

#### 5.5.2.7 wofEn

uint8\_t ICM20948\_IntEnableConfig::wofEn

Definition at line 125 of file [DevLab\\_ICM20948.h](#).

#### 5.5.2.8 womIntEn

uint8\_t ICM20948\_IntEnableConfig::womIntEn

Definition at line 126 of file [DevLab\\_ICM20948.h](#).

The documentation for this struct was generated from the following file:

- [src/DevLab\\_ICM20948.h](#)

## 5.6 ICM20948\_IntPinConfig Struct Reference

```
#include <DevLab_ICM20948.h>
```

## Public Attributes

- uint8\_t [activeLevel](#): 1
- uint8\_t [driveMode](#): 1
- uint8\_t [latchMode](#): 1
- uint8\_t [clearMode](#): 1
- uint8\_t [fsyncActLevel](#): 1
- uint8\_t [fsyncIntEn](#): 1
- uint8\_t [bypassEn](#): 1

### 5.6.1 Detailed Description

#### [ICM20948\\_IntPinConfig](#)

- Physical configuration of the INT pin (INT\_PIN\_CFG register, BANK 0)
- Controls electrical behavior and clear condition of the interrupt line
- Pass to `intInit()` to apply settings

Definition at line [103](#) of file [DevLab\\_ICM20948.h](#).

### 5.6.2 Member Data Documentation

#### 5.6.2.1 [activeLevel](#)

uint8\_t [ICM20948\\_IntPinConfig::activeLevel](#)

Definition at line [105](#) of file [DevLab\\_ICM20948.h](#).

#### 5.6.2.2 [bypassEn](#)

uint8\_t [ICM20948\\_IntPinConfig::bypassEn](#)

Definition at line [111](#) of file [DevLab\\_ICM20948.h](#).

#### 5.6.2.3 [clearMode](#)

uint8\_t [ICM20948\\_IntPinConfig::clearMode](#)

Definition at line [108](#) of file [DevLab\\_ICM20948.h](#).

#### 5.6.2.4 [driveMode](#)

uint8\_t [ICM20948\\_IntPinConfig::driveMode](#)

Definition at line [106](#) of file [DevLab\\_ICM20948.h](#).

#### 5.6.2.5 [fsyncActLevel](#)

uint8\_t [ICM20948\\_IntPinConfig::fsyncActLevel](#)

Definition at line [109](#) of file [DevLab\\_ICM20948.h](#).

#### 5.6.2.6 [fsyncIntEn](#)

uint8\_t [ICM20948\\_IntPinConfig::fsyncIntEn](#)

Definition at line [110](#) of file [DevLab\\_ICM20948.h](#).

#### 5.6.2.7 [latchMode](#)

uint8\_t [ICM20948\\_IntPinConfig::latchMode](#)

Definition at line [107](#) of file [DevLab\\_ICM20948.h](#).

The documentation for this struct was generated from the following file:

- [src/DevLab\\_ICM20948.h](#)



## 5.7 ICM20948\_IntStatus Struct Reference

```
#include <DevLab_ICM20948.h>
```

Public Attributes

- uint8\_t [womInt](#): 1
- uint8\_t [pllRdyInt](#): 1
- uint8\_t [dmpInt1](#): 1
- uint8\_t [i2cMstInt](#): 1
- uint8\_t [rawDataRdy](#): 1
- uint8\_t [fifoOvf](#) [5]
- uint8\_t [fifoWm](#) [5]

### 5.7.1 Detailed Description

[ICM20948\\_IntStatus](#)

- Holds the parsed state of the four interrupt status registers
- Covers INT\_STATUS (0x19), INT\_STATUS\_1 (0x1A), INT\_STATUS\_2 (0x1B), INT\_STATUS\_3 (0x1C)
- Populated by `checkIntStatus()` via a single 4-byte burst read
- Reading these registers clears the interrupt flags (when `clearMode = 0` in `IntPinConfig`)

Definition at line 149 of file [DevLab\\_ICM20948.h](#).

### 5.7.2 Member Data Documentation

#### 5.7.2.1 dmpInt1

uint8\_t ICM20948\_IntStatus::dmpInt1

Definition at line 154 of file [DevLab\\_ICM20948.h](#).

#### 5.7.2.2 fifoOvf

uint8\_t ICM20948\_IntStatus::fifoOvf[5]

Definition at line 161 of file [DevLab\\_ICM20948.h](#).

#### 5.7.2.3 fifoWm

uint8\_t ICM20948\_IntStatus::fifoWm[5]

Definition at line 164 of file [DevLab\\_ICM20948.h](#).

#### 5.7.2.4 i2cMstInt

uint8\_t ICM20948\_IntStatus::i2cMstInt

Definition at line 155 of file [DevLab\\_ICM20948.h](#).

#### 5.7.2.5 pllRdyInt

uint8\_t ICM20948\_IntStatus::pllRdyInt

Definition at line 153 of file [DevLab\\_ICM20948.h](#).

#### 5.7.2.6 rawDataRdy

uint8\_t ICM20948\_IntStatus::rawDataRdy

Definition at line 158 of file [DevLab\\_ICM20948.h](#).

### 5.7.2.7 womInt

uint8\_t ICM20948\_IntStatus::womInt

Definition at line 152 of file [DevLab\\_ICM20948.h](#).

The documentation for this struct was generated from the following file:

- [src/DevLab\\_ICM20948.h](#)

## 5.8 icm\_plat\_t Struct Reference

```
#include <ICM20948_platform.h>
```

Public Attributes

- int(\* [read](#))(uint8\_t reg, uint8\_t \*buf, size\_t len, void \*user)
- int(\* [write](#))(uint8\_t reg, const uint8\_t \*buf, size\_t len, void \*user)
- uint8\_t(\* [spi\\_reg\\_rw\\_bits](#))(uint8\_t reg, int is\_write)
- void(\* [delay\\_ms](#))(uint32\_t ms)
- void \* [user](#)
- void(\* [spi\\_cs](#))(int level, void \*user)
- uint8\_t [i2c\\_addr](#)

### 5.8.1 Detailed Description

7Semi comment style

- Platform glue for bus IO, delays, and timing
- Implement these for your system (I2C or SPI)

Definition at line 13 of file [ICM20948\\_platform.h](#).

### 5.8.2 Member Data Documentation

#### 5.8.2.1 delay\_ms

```
void(* icm_plat_t::delay_ms)(uint32_t ms)
```

- Millisecond delay

Definition at line 32 of file [ICM20948\\_platform.h](#).

#### 5.8.2.2 i2c\_addr

```
uint8_t icm_plat_t::i2c_addr
```

- Device address (I2C) or ignored for SPI if you wrap inside user

Definition at line 45 of file [ICM20948\\_platform.h](#).

#### 5.8.2.3 read

```
int(* icm_plat_t::read)(uint8_t reg, uint8_t *buf, size_t len, void *user)
```

- Bus read: read **len** bytes from **reg** into **buf**
- Return 0 on success, nonzero on error

Definition at line 18 of file [ICM20948\\_platform.h](#).

## 5.8.2.4 spi\_cs

```
void(* icm_plat_t::spi_cs) (int level, void *user)
```

- If using SPI: set chip select active (1) or inactive (0)
- For I2C: leave as NULL

Definition at line 41 of file [ICM20948\\_platform.h](#).

## 5.8.2.5 spi\_reg\_rw\_bits

```
uint8_t(* icm_plat_t::spi_reg_rw_bits) (uint8_t reg, int is_write)
```

- Optional: SPI transfers may need register R/W bit mangling
- For I2C, leave as NULL

Definition at line 28 of file [ICM20948\\_platform.h](#).

## 5.8.2.6 user

```
void* icm_plat_t::user
```

- User context pointer passed to read/write

Definition at line 36 of file [ICM20948\\_platform.h](#).

## 5.8.2.7 write

```
int(* icm_plat_t::write) (uint8_t reg, const uint8_t *buf, size_t len, void *user)
```

- Bus write: write **len** bytes from **buf** to **reg**
- Return 0 on success, nonzero on error

Definition at line 23 of file [ICM20948\\_platform.h](#).

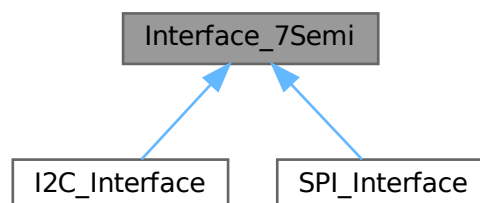
The documentation for this struct was generated from the following file:

- [src/ICM20948\\_platform.h](#)

## 5.9 Interface\_7Semi Class Reference

```
#include <7Semi_Interface.h>
```

Inheritance diagram for Interface\_7Semi:



## Public Member Functions

- virtual [~Interface\\_7Semi](#) ()
- virtual bool [beginI2C](#) (uint8\_t address, TwoWire &wire, uint32\_t speed, uint8\_t sda=255, uint8\_t scl=255)=0
- virtual bool [beginSPI](#) (uint8\_t cs\_pin, SPIClass &wire, uint32\_t speed, uint8\_t csk=255, uint8\_t miso=255, uint8\_t mosi=255)=0
- virtual int8\_t [read](#) (uint8\_t reg, uint8\_t \*data, uint32\_t len)=0
- virtual int8\_t [write](#) (uint8\_t reg, const uint8\_t \*data, uint32\_t len)=0

## Static Protected Member Functions

- static void [delay\\_us](#) (uint32\_t us)

## 5.9.1 Detailed Description

## 7Semi Universal Interface Layer

- Abstract communication layer for I2C / SPI
- Ensures sensor drivers remain hardware independent

Definition at line 15 of file [7Semi\\_Interface.h](#).

## 5.9.2 Constructor &amp; Destructor Documentation

## 5.9.2.1 ~Interface\_7Semi()

```
virtual Interface_7Semi::~~Interface_7Semi ( ) [inline], [virtual]
```

Definition at line 18 of file [7Semi\\_Interface.h](#).

## 5.9.3 Member Function Documentation

## 5.9.3.1 beginI2C()

```
virtual bool Interface_7Semi::beginI2C (
    uint8_t address,
    TwoWire & wire,
    uint32_t speed,
    uint8_t sda = 255,
    uint8_t scl = 255 ) [pure virtual]
```

[beginI2C\(\)](#)

- Initializes I2C peripheral
- Configures SDA / SCL pins if supported
- Sets communication clock

Returns:

- true Initialization successful
- false Initialization failed

Implemented in [I2C\\_Interface](#), and [SPI\\_Interface](#).

## 5.9.3.2 beginSPI()

```
virtual bool Interface_7Semi::beginSPI (
    uint8_t cs_pin,
    SPIClass & wire,
    uint32_t speed,
    uint8_t csk = 255,
    uint8_t miso = 255,
    uint8_t mosi = 255 ) [pure virtual]
```

Implemented in [SPI\\_Interface](#), and [I2C\\_Interface](#).

#### 5.9.3.3 delay\_us()

```
static void Interface_7Semi::delay_us (
    uint32_t us ) [inline], [static], [protected]
```

Delay wrapper

Definition at line 82 of file [7Semi\\_Interface.h](#).

#### 5.9.3.4 read()

```
virtual int8_t Interface_7Semi::read (
    uint8_t reg,
    uint8_t * data,
    uint32_t len ) [pure virtual]
```

[read\(\)](#)

- Reads data from device register

Parameters:

- reg : Register address
- data : Output buffer
- len : Number of bytes

Returns:

- 0 Success
- <0 Error

Implemented in [I2C\\_Interface](#), and [SPI\\_Interface](#).

#### 5.9.3.5 write()

```
virtual int8_t Interface_7Semi::write (
    uint8_t reg,
    const uint8_t * data,
    uint32_t len ) [pure virtual]
```

[write\(\)](#)

- Writes data to device register

Returns:

- 0 Success
- <0 Error

Implemented in [I2C\\_Interface](#), and [SPI\\_Interface](#).

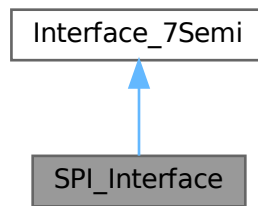
The documentation for this class was generated from the following file:

- [src/7Semi\\_Interface.h](#)

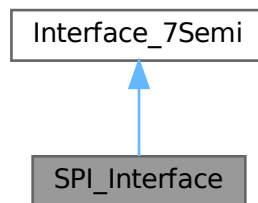
## 5.10 SPI\_Interface Class Reference

```
#include <7Semi_SPI_Interface.h>
```

Inheritance diagram for SPI\_Interface:



Collaboration diagram for SPI\_Interface:



#### Public Member Functions

- bool [beginSPI](#) (uint8\_t csPin, SPIClass &spiPort, uint32\_t spiSpeed, uint8\_t sck=255, uint8\_t miso=255, uint8\_t mosi=255) override
- bool [beginI2C](#) (uint8\_t, TwoWire &, uint32\_t, uint8\_t, uint8\_t) override
- int8\_t [read](#) (uint8\_t reg, uint8\_t \*data, uint32\_t len) override
- int8\_t [write](#) (uint8\_t reg, const uint8\_t \*data, uint32\_t len) override

#### Public Member Functions inherited from [Interface\\_7Semi](#)

- virtual [~Interface\\_7Semi](#) ()

#### Public Attributes

- SPIClass \* [spi](#) = nullptr
- uint8\_t [cs](#) = 255
- uint32\_t [speed](#) = 1000000

#### Additional Inherited Members

#### Static Protected Member Functions inherited from [Interface\\_7Semi](#)

- static void [delay\\_us](#) (uint32\_t us)

### 5.10.1 Detailed Description

7Semi SPI Interface Implementation

Definition at line 7 of file [7Semi\\_SPI\\_Interface.h](#).

### 5.10.2 Member Function Documentation

#### 5.10.2.1 beginI2C()

```
bool SPI_Interface::beginI2C (
    uint8_t address,
    TwoWire & wire,
    uint32_t speed,
    uint8_t sda,
    uint8_t scl ) [inline], [override], [virtual]
```

[beginI2C\(\)](#)

- Initializes I2C peripheral
- Configures SDA / SCL pins if supported
- Sets communication clock

Returns:

- true Initialization successful
- false Initialization failed

Implements [Interface\\_7Semi](#).

Definition at line 51 of file [7Semi\\_SPI\\_Interface.h](#).

#### 5.10.2.2 beginSPI()

```
bool SPI_Interface::beginSPI (
    uint8_t csPin,
    SPIClass & spiPort,
    uint32_t spiSpeed,
    uint8_t sck = 255,
    uint8_t miso = 255,
    uint8_t mosi = 255 ) [inline], [override], [virtual]
```

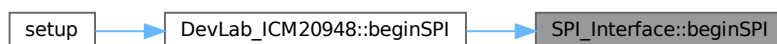
[beginSPI\(\)](#)

- Initialize SPI interface
- Configure CS and SPI bus

Implements [Interface\\_7Semi](#).

Definition at line 21 of file [7Semi\\_SPI\\_Interface.h](#).

Here is the caller graph for this function:



## 5.10.2.3 read()

```
int8_t SPI_Interface::read (
    uint8_t reg,
    uint8_t * data,
    uint32_t len ) [inline], [override], [virtual]
```

read()

- Read multiple bytes from register

Send register address (read)

Read data

Implements [Interface\\_7Semi](#).

Definition at line 62 of file [7Semi\\_SPI\\_Interface.h](#).

## 5.10.2.4 write()

```
int8_t SPI_Interface::write (
    uint8_t reg,
    const uint8_t * data,
    uint32_t len ) [inline], [override], [virtual]
```

write()

- Write multiple bytes to register

Send register address (write)

Write data

Implements [Interface\\_7Semi](#).

Definition at line 108 of file [7Semi\\_SPI\\_Interface.h](#).

## 5.10.3 Member Data Documentation

## 5.10.3.1 cs

```
uint8_t SPI_Interface::cs = 255
```

Definition at line 12 of file [7Semi\\_SPI\\_Interface.h](#).

## 5.10.3.2 speed

```
uint32_t SPI_Interface::speed = 1000000
```

Definition at line 13 of file [7Semi\\_SPI\\_Interface.h](#).

## 5.10.3.3 spi

```
SPIClass* SPI_Interface::spi = nullptr
```

Definition at line 11 of file [7Semi\\_SPI\\_Interface.h](#).

The documentation for this class was generated from the following file:

- [src/7Semi\\_SPI\\_Interface.h](#)



# Chapter 6

## File Documentation

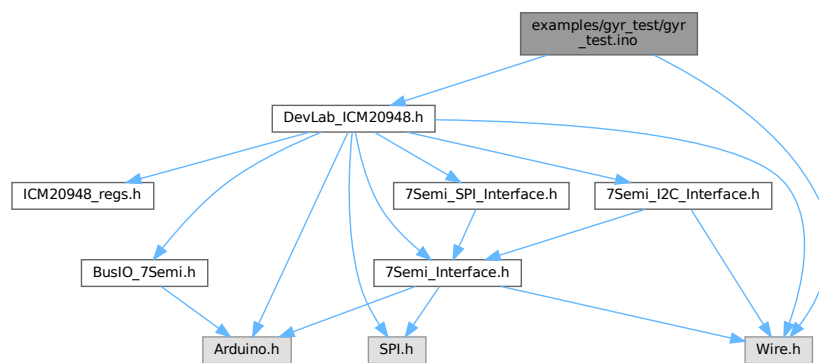
### 6.1 examples/gyr\_test/gyr\_test.ino File Reference

Gyroscope DLPF and full-scale sweep for the 7Semi ICM-20948.

```
#include <Wire.h>
```

```
#include <DevLab_ICM20948.h>
```

Include dependency graph for gyr\_test.ino:



#### Classes

- struct `configDLPF`  
Gyroscope DLPF divider and timing configuration.

#### Macros

- `#define SDA_PIN 6`  
I2C SDA pin used by the example board.
- `#define SCL_PIN 7`  
I2C SCL pin used by the example board.
- `#define I2C_FREQ 400000UL`  
I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.

#### Functions

- void `printLinea()`

- Print a separator line to Serial.
- void `printDobleLinea ()`  
Print a double separator line to Serial.
- void `firstStage ()`  
Verify the IMU identity register.
- bool `applySettings (uint8_t fs_idx, const configDLPF &cfg)`  
Apply one gyroscope full-scale and DLPF configuration.
- void `runSweep ()`  
Run the gyroscope validation sweep and print each captured sample.
- void `setup ()`  
Initialize I2C, reset/wake the IMU, enable gyro, and start the sweep.
- void `loop ()`  
Empty loop; this sketch runs the gyroscope sweep once in `setup()`.

## Variables

- const `configDLPF configsDLPF []`
- const `uint8_t N_DLPF = sizeof(configsDLPF) / sizeof(configsDLPF[0])`  
Number of gyroscope DLPF configurations.
- const `ICM20948_Gyro_FullScale gyroFSCfg []`  
Gyroscope full-scale ranges exercised by the sweep.
- const `char * etiquetasGyroFSCfg [] = {"250", "500", "1000", "2000"}`  
Text labels matching gyroFSCfg.
- const `uint8_t N_FS = 4`  
Number of gyroscope full-scale ranges.
- const `ICM20948_Gyro_DLPF gyroDLPF []`  
Gyroscope DLPF enum values matching configsDLPF.
- const `ICM20948_Gyro_Average gyroAVGCfg []`  
Gyroscope averaging settings available for experiments.
- const `char * etiquetasGyroAVGCfg [] = {"1", "2", "4", "8", "16", "32", "64", "128"}`  
Text labels matching gyroAVGCfg.
- const `uint8_t N_AVG = 8`  
Number of gyroscope averaging settings.
- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.

## 6.1.1 Detailed Description

Gyroscope DLPF and full-scale sweep for the 7Semi ICM-20948.

Applies each gyroscope filter/full-scale configuration over I2C and prints repeated samples for validation and noise comparison.

Definition in file `gyr_test.ino`.

## 6.1.2 Macro Definition Documentation

### 6.1.2.1 I2C\_FREQ

```
#define I2C_FREQ 400000UL
```

I2C bus speed in hertz.

Definition at line 18 of file `gyr_test.ino`.

### 6.1.2.2 ICM\_ADDR

```
#define ICM_ADDR 0x69
```

ICM-20948 I2C address selected by the AD0 pin.

Definition at line 20 of file `gyr_test.ino`.

### 6.1.2.3 SCL\_PIN

```
#define SCL_PIN 7
```

I2C SCL pin used by the example board.

Definition at line 16 of file [gyr\\_test.ino](#).

### 6.1.2.4 SDA\_PIN

```
#define SDA_PIN 6
```

I2C SDA pin used by the example board.

Definition at line 14 of file [gyr\\_test.ino](#).

## 6.1.3 Function Documentation

### 6.1.3.1 applySettings()

```
bool applySettings (
    uint8_t fs_idx,
    const configDLPF & cfg )
```

Apply one gyroscope full-scale and DLPF configuration.

Parameters

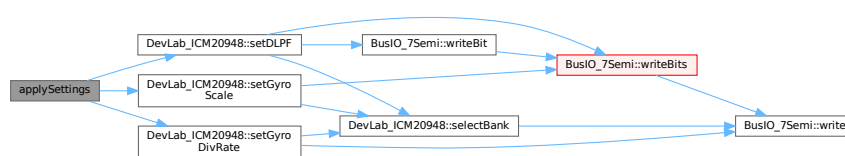
|        |                                        |
|--------|----------------------------------------|
| fs_idx | Index into gyroFSCfg.                  |
| cfg    | DLPF timing and divider configuration. |

Returns

true if all configuration writes succeeded, otherwise false.

Definition at line 133 of file [gyr\\_test.ino](#).

Here is the call graph for this function:



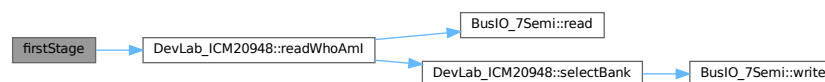
### 6.1.3.2 firstStage()

```
void firstStage ( )
```

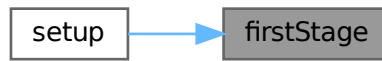
Verify the IMU identity register.

Definition at line 117 of file [gyr\\_test.ino](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 6.1.3.3 `loop()`

```
void loop ( )
```

Empty loop; this sketch runs the gyroscope sweep once in `setup()`.

Definition at line 248 of file `gyr_test.ino`.

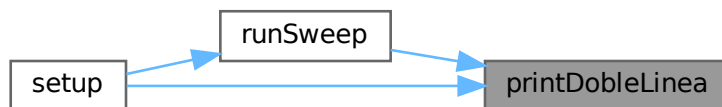
#### 6.1.3.4 `printDobleLinea()`

```
void printDobleLinea ( )
```

Print a double separator line to Serial.

Definition at line 112 of file `gyr_test.ino`.

Here is the caller graph for this function:



#### 6.1.3.5 `printLinea()`

```
void printLinea ( )
```

Print a separator line to Serial.

Definition at line 110 of file `gyr_test.ino`.

#### 6.1.3.6 `runSweep()`

```
void runSweep ( )
```

Run the gyroscope validation sweep and print each captured sample.

Definition at line 158 of file `gyr_test.ino`.

```
graph LR; A[setup] --> B[runSweep]
```

```
void setup ( )
```

Here is the call graph for this function:

[illegible]

#### 6.1.4.1 configsDLPF

Initial value:

Generated on Tue Jun 16 2026 17:30:42 for DevLab ICM20948 by Doxygen

```
{ 2, 0, 154.3f, 1125.0f, "119,5 ; 154,3 ; 1125.0 " },
{ 3, 2, 73.3f, 375.0f, "51,2 ; 73,3 ; 375 " },
{ 4, 5, 35.9f, 187.5f, "23,9 ; 35,9 ; 187.5 " },
{ 5, 14, 17.8f, 75.0f, "11,6 ; 17,8 ; 75.0 " },
{ 6, 29, 8.9f, 37.5f, "5,7 ; 8,9 ; 37.5 " },
{ 7, 0, 376.5f, 1125.0f, "361,4 ; 376,5 ; 1125 " },
}
```

Definition at line 41 of file [gyr\\_test.ino](#).

#### 6.1.4.2 etiquetasGyroAVGCfg

```
const char* etiquetasGyroAVGCfg[ ] = {"1","2","4","8","16","32","64","128"}
```

Text labels matching gyroAVGCfg.

Definition at line 102 of file [gyr\\_test.ino](#).

#### 6.1.4.3 etiquetasGyroFSCfg

```
const char* etiquetasGyroFSCfg[ ] = {"250", "500", "1000", "2000"}
```

Text labels matching gyroFSCfg.

Definition at line 73 of file [gyr\\_test.ino](#).

#### 6.1.4.4 gyroAVGCfg

```
const ICM20948_Gyro_Average gyroAVGCfg[ ]
```

Initial value:

```
= {
  ICM20948_Gyro_Average::AVG_1,
  ICM20948_Gyro_Average::AVG_2,
  ICM20948_Gyro_Average::AVG_4,
  ICM20948_Gyro_Average::AVG_8,
  ICM20948_Gyro_Average::AVG_16,
  ICM20948_Gyro_Average::AVG_32,
  ICM20948_Gyro_Average::AVG_64,
  ICM20948_Gyro_Average::AVG_128
}
```

Gyroscope averaging settings available for experiments.

Definition at line 90 of file [gyr\\_test.ino](#).

#### 6.1.4.5 gyroDLPF

```
const ICM20948_Gyro_DLPF gyroDLPF[ ]
```

Initial value:

```
= {
  ICM20948_Gyro_DLPF::DLPF_196HZ,
  ICM20948_Gyro_DLPF::DLPF_151HZ,
  ICM20948_Gyro_DLPF::DLPF_119HZ,
  ICM20948_Gyro_DLPF::DLPF_51HZ,
  ICM20948_Gyro_DLPF::DLPF_23HZ,
  ICM20948_Gyro_DLPF::DLPF_11HZ,
  ICM20948_Gyro_DLPF::DLPF_5HZ,
  ICM20948_Gyro_DLPF::DLPF_361HZ
}
```

Gyroscope DLPF enum values matching configsDLPF.

Definition at line 78 of file [gyr\\_test.ino](#).

#### 6.1.4.6 gyroFSCfg

```
const ICM20948_Gyro_FullScale gyroFSCfg[ ]
```

Initial value:

```
= {
  ICM20948_Gyro_FullScale::DPS_250,
  ICM20948_Gyro_FullScale::DPS_500,
  ICM20948_Gyro_FullScale::DPS_1000,
  ICM20948_Gyro_FullScale::DPS_2000
}
```

Gyroscope full-scale ranges exercised by the sweep.

Definition at line 65 of file [gyr\\_test.ino](#).

#### 6.1.4.7 imu

[DevLab\\_ICM20948](#) imu

Global ICM-20948 driver instance.

Definition at line 107 of file [gyr\\_test.ino](#).

#### 6.1.4.8 N\_AVG

```
const uint8_t N_AVG = 8
```

Number of gyroscope averaging settings.

Definition at line 104 of file [gyr\\_test.ino](#).

#### 6.1.4.9 N\_DLPF

```
const uint8_t N_DLPF = sizeof(configsDLPF) / sizeof(configsDLPF[0])
```

Number of gyroscope DLPF configurations.

Definition at line 62 of file [gyr\\_test.ino](#).

#### 6.1.4.10 N\_FS

```
const uint8_t N_FS = 4
```

Number of gyroscope full-scale ranges.

Definition at line 75 of file [gyr\\_test.ino](#).

## 6.2 gyr\_test.ino

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file gyr_test.ino
00003  * @brief Gyroscope DLPF and full-scale sweep for the 7Semi ICM-20948.
00004  * @details Applies each gyroscope filter/full-scale configuration over I2C and
00005  * prints repeated samples for validation and noise comparison.
00006  */
00007
00008 #include <Wire.h>
00009 #include <DevLab_ICM20948.h>
00010 // -----
00011 // PINES I2C – ESP32C6 NANO Unit Electronics
00012 // -----
00013 /** @brief I2C SDA pin used by the example board. */
00014 #define SDA_PIN 6
00015 /** @brief I2C SCL pin used by the example board. */
00016 #define SCL_PIN 7
00017 /** @brief I2C bus speed in hertz. */
00018 #define I2C_FREQ 400000UL
00019 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00020 #define ICM_ADDR 0x69
00021
00022 // -----
00023 // STRUCT
00024 // -----
00025 /** @brief Gyroscope DLPF divider and timing configuration. */
00026 struct configDLPF {
00027     /** @brief Index into the gyroDLPF lookup table. */
00028     uint8_t dlpf_idx;
00029     /** @brief Sample-rate divider written to the IMU register. */
00030     uint16_t div;
00031     /** @brief Noise bandwidth in hertz for reporting. */
00032     float nbw_hz;
00033     /** @brief Output data rate in hertz for timing calculations. */
00034     float odr_hz;
00035     /** @brief Text label printed with each captured sample. */
00036     const char* nombre;
00037 };
00038 //id,div,nbw_hz,odr_hz,nombre : {DLPF, NBW , ODR}
00039 /*
00040 /** @brief Gyroscope DLPF configurations exercised by the sweep. */
00041 const configDLPF configsDLPF[] = {
00042     { 0, 0, 229.8f, 1125.0f, "196,6 ; 229,8 ; 1125.0 " },
00043     { 1, 0, 187.6f, 1125.0f, "151,8 ; 187.6 ; 1125.0 " },
00044     { 2, 0, 154.3f, 1125.0f, "119,5 ; 154,3 ; 1125.0 " },
00045     { 3, 2, 73.3f, 375.0f, "51,2 ; 73,3 ; 375 " },
00046     { 4, 5, 35.9f, 187.5f, "23,9 ; 35,9 ; 187.5 " },
00047     { 5, 14, 17.8f, 75.0f, "11,6 ; 17,8 ; 75.0 " },
00048     { 6, 29, 8.9f, 37.5f, "5,7 ; 8,9 ; 37.5 " },
00049     { 7, 0, 376.5f, 1125.0f, "361,4 ; 376,5 ; 1125 " },
00050 };*/
00051 const configDLPF configsDLPF[] = {
00052     { 0, 0, 229.8f, 1125.0f, "196,6 ; 229,8 ; 1125.0 " },
```

```

00053 { 1, 0, 187.6f, 1125.0f, "151,8 ; 187.6 ; 1125.0 " },
00054 { 2, 0, 154.3f, 1125.0f, "119,5 ; 154,3 ; 1125.0 " },
00055 { 3, 2, 73.3f, 375.0f, "51,2 ; 73,3 ; 375 " },
00056 { 4, 5, 35.9f, 187.5f, "23,9 ; 35,9 ; 187.5 " },
00057 { 5, 14, 17.8f, 75.0f, "11,6 ; 17,8 ; 75.0 " },
00058 { 6, 29, 8.9f, 37.5f, "5,7 ; 8,9 ; 37.5 " },
00059 { 7, 0, 376.5f, 1125.0f, "361,4 ; 376,5 ; 1125 " },
00060 };
00061 /** @brief Number of gyroscope DLPF configurations. */
00062 const uint8_t N_DLPF = sizeof(configsDLPF) / sizeof(configsDLPF[0]);
00063
00064 /** @brief Gyroscope full-scale ranges exercised by the sweep. */
00065 const ICM20948_Gyro_FullScale gyroFSCfg[] = {
00066     ICM20948_Gyro_FullScale::DPS_250,
00067     ICM20948_Gyro_FullScale::DPS_500,
00068     ICM20948_Gyro_FullScale::DPS_1000,
00069     ICM20948_Gyro_FullScale::DPS_2000
00070 };
00071
00072 /** @brief Text labels matching gyroFSCfg. */
00073 const char* etiquetasGyroFSCfg[] = {"250", "500", "1000", "2000"};
00074 /** @brief Number of gyroscope full-scale ranges. */
00075 const uint8_t N_FS = 4;
00076
00077 /** @brief Gyroscope DLPF enum values matching configsDLPF. */
00078 const ICM20948_Gyro_DLPF gyroDLPF[] = {
00079     ICM20948_Gyro_DLPF::DLPF_196HZ,
00080     ICM20948_Gyro_DLPF::DLPF_151HZ,
00081     ICM20948_Gyro_DLPF::DLPF_119HZ,
00082     ICM20948_Gyro_DLPF::DLPF_51HZ,
00083     ICM20948_Gyro_DLPF::DLPF_23HZ,
00084     ICM20948_Gyro_DLPF::DLPF_11HZ,
00085     ICM20948_Gyro_DLPF::DLPF_5HZ,
00086     ICM20948_Gyro_DLPF::DLPF_361HZ
00087 };
00088
00089 /** @brief Gyroscope averaging settings available for experiments. */
00090 const ICM20948_Gyro_Average gyroAVGCfg[] = {
00091     ICM20948_Gyro_Average::AVG_1,
00092     ICM20948_Gyro_Average::AVG_2,
00093     ICM20948_Gyro_Average::AVG_4,
00094     ICM20948_Gyro_Average::AVG_8,
00095     ICM20948_Gyro_Average::AVG_16,
00096     ICM20948_Gyro_Average::AVG_32,
00097     ICM20948_Gyro_Average::AVG_64,
00098     ICM20948_Gyro_Average::AVG_128
00099 };
00100
00101 /** @brief Text labels matching gyroAVGCfg. */
00102 const char* etiquetasGyroAVGCfg[] = {"1", "2", "4", "8", "16", "32", "64", "128"};
00103 /** @brief Number of gyroscope averaging settings. */
00104 const uint8_t N_AVG = 8;
00105
00106 /** @brief Global ICM-20948 driver instance. */
00107 DevLab_ICM20948 imu;
00108
00109 /** @brief Print a separator line to Serial. */
00110 void printLinea() {
00111     Serial.println(F("-----"));
00112 }
00113 /** @brief Print a double separator line to Serial. */
00114 void printDobleLinea() {
00115     Serial.println(F("====="));
00116 }
00117
00118 /**
00119  * @brief Verify the IMU identity register.
00120  */
00121 void firstStage() {
00122     uint8_t who;
00123     if (!imu.readWhoAmI(who) || who != 0xEA) {
00124         Serial.println(F("ERROR: WHO_AM_I mismatch"));
00125         while (1) delay(200);
00126     }
00127     Serial.printf("WHO_AM_I = 0x%02X OK\n", who);
00128 }
00129
00130 /**
00131  * @brief Apply one gyroscope full-scale and DLPF configuration.
00132  *
00133  * @param fs_idx Index into gyroFSCfg.
00134  * @param cfg DLPF timing and divider configuration.
00135  * @return true if all configuration writes succeeded, otherwise false.
00136  */
00137 bool applySettings(uint8_t fs_idx, const configDLPF &cfg) {
00138     ICM20948_Gyro_DLPF dlpf_val = gyroDLPF[cfg.dlpf_idx];
00139     bool ok = true;
00140
00141     ok &= imu.setGyroScale(gyroFSCfg[fs_idx]);

```



```

00138
00139
00140 // Paso 1: escribe DLPFCFG en bits [5:3]
00141 // SetDLPF false -> Activa DLPF
00142 // SetDLPF true -> Desactiva DLPF
00143 ok &= imu.setDLPF(dlpf_val, false);
00144
00145 //ok &= imu.setGyroAveraging(gyroAVGCfg[avg]);
00146 // Paso 2: activa FCHOICE=1 (DLPF activo) sin tocar DLPFCFG
00147 //ok &= imu.setAccelDLPF(0, true);
00148
00149 // DIV directo al registro, sin conversion interna
00150 ok &= imu.setGyroDivRate(cfg.div);
00151
00152 return ok;
00153 }
00154
00155 /**
00156  * @brief Run the gyroscope validation sweep and print each captured sample.
00157  */
00158 void runSweep(){
00159     uint16_t n = 0;
00160     printDobleLinea();
00161     Serial.println(F(" ICM-20948 | UNIT Electronics"));
00162     Serial.println(F(" MODO VERIFICACION - FS=+-2g fijo, 3 lecturas por config"));
00163     Serial.println(F(" Sensor PLANO y QUIETO | Az esperado: ~+1.000g"));
00164     printDobleLinea();
00165     for (uint8_t i = 0; i < N_FS ; i++){
00166         for(uint8_t d = 0; d < N_DLPF; d++){
00167
00168             bool cfg_ok = applySettings(i,configsDLPF[d]);
00169             if(!cfg_ok){
00170                 Serial.println(F("ERROR: applySettings fallo"));
00171             }
00172
00173             // Esperar que el filtro estabilice (10 periodos del ODR, min 50ms)
00174             uint32_t settle_ms = max((uint32_t)(10000.0f / configsDLPF[d].odr_hz),
00175                                     (uint32_t)50);
00176             delay(settle_ms);
00177             // Tomar 3 lecturas espaciadas por el periodo del ODR
00178             uint32_t periodo_ms = max((uint32_t)(1000.0f / configsDLPF[d].odr_hz),
00179                                     (uint32_t)1);
00180             float gx, gy, gz;
00181             for (uint8_t r = 0; r < 100; r++) {
00182                 n++;
00183                 delay(periodo_ms);
00184                 if (imu.readGyro(gx, gy, gz)) {
00185                     Serial.printf("%d ; %s ; %s ; %.4f ; %.4f ; %.4f\n",
00186                                   n,
00187                                   configsDLPF[d].nombre, // "246 ; 265 ; 1125"
00188                                   etiquetasGyroFSCfg[i], // "+-2G"
00189                                   //etiquetasGyroAVGCfg[j], // "8"
00190                                   gx, gy, gz);
00191                 } else {
00192                     Serial.println(F(" [?] readGyr() fallo"));
00193                 }
00194             }
00195         }
00196     }
00197 }
00198 }
00199
00200 /**
00201  * @brief Initialize I2C, reset/wake the IMU, enable gyro, and start the sweep.
00202  */
00203 void setup() {
00204     // put your setup code here, to run once:
00205     Serial.begin(115200);
00206     delay(200);
00207
00208     printDobleLinea();
00209     Serial.println(F(" ICM-20948 VALIDACION ACELEROMETRO"));
00210     printDobleLinea();
00211
00212     Wire.begin(SDA_PIN, SCL_PIN);
00213
00214     if (!imu.beginI2C(ICM_ADDR, Wire, I2C_FREQ)) {
00215         Serial.println(F("ERROR: beginI2C() fallo"));
00216         while (1) delay(200);
00217     }
00218
00219     firstStage();
00220
00221     // softReset pone el chip a dormir
00222     // Despues del reset hay que despertar el dispositivo
00223     imu.softReset();
00224     delay(100);

```

```

00225  imu.sleep(false);    // limpia SLEEP bit, CLKSEL=1
00226  delay(50);
00227
00228  if (!imu.setSensors(false, true, false)) {
00229      Serial.println(F("ERROR: setSensors() fallo"));
00230  }
00231
00232  Serial.println(F("  Sensor listo.\n"));
00233  Serial.println(F("  Sensor PLANO y QUIETO sobre la mesa.));
00234  Serial.println(F("  Iniciando en 5 segundos..."));
00235  for (int i = 5; i > 0; i--) {
00236      Serial.printf("  %d...\n", i);
00237      delay(1000);
00238  }
00239
00240  //for (int runs = 0; runs < 20 ; runs++){
00241      runSweep();
00242  //}
00243 }
00244
00245 /**
00246  * @brief Empty loop; this sketch runs the gyroscope sweep once in setup().
00247  */
00248 void loop() {
00249     // put your main code here, to run repeatedly:
00250
00251 }

```

## 6.3 examples/i2c\_accel/i2c\_accel.ino File Reference

```

#include <Wire.h>
#include <DevLab_ICM20948.h>

```

### Macros

- `#define SDA_PIN 6`  
I2C SDA pin used by the example board.
- `#define SCL_PIN 7`  
I2C SCL pin used by the example board.
- `#define I2C_FREQ 400000UL`  
I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.

### Functions

- void `setup ()`  
Configure the IMU for accelerometer-only operation over I2C.
- void `loop ()`  
Read and print accelerometer data in g.

### Variables

- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.

## 6.3.1 Macro Definition Documentation

### 6.3.1.1 I2C\_FREQ

```
#define I2C_FREQ 400000UL
```

I2C bus speed in hertz.

Definition at line 40 of file `i2c_accel.ino`.

### 6.3.1.2 ICM\_ADDR

```
#define ICM_ADDR 0x69
```

ICM-20948 I2C address selected by the AD0 pin.

Definition at line 42 of file [i2c\\_accel.ino](#).

### 6.3.1.3 SCL\_PIN

```
#define SCL_PIN 7
```

I2C SCL pin used by the example board.

Definition at line 38 of file [i2c\\_accel.ino](#).

### 6.3.1.4 SDA\_PIN

```
#define SDA_PIN 6
```

I2C SDA pin used by the example board.

Definition at line 36 of file [i2c\\_accel.ino](#).

## 6.3.2 Function Documentation

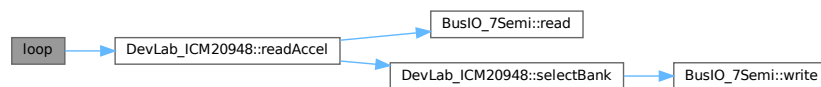
### 6.3.2.1 loop()

```
void loop ( )
```

Read and print accelerometer data in g.

Definition at line 121 of file [i2c\\_accel.ino](#).

Here is the call graph for this function:



### 6.3.2.2 setup()

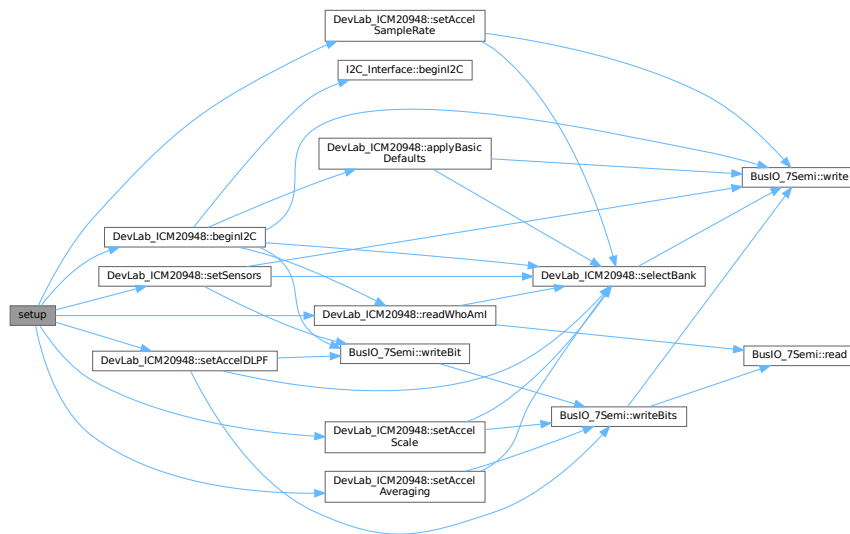
```
void setup ( )
```

Configure the IMU for accelerometer-only operation over I2C.

The function initializes the bus, verifies the device identity, enables the accelerometer, and applies range, DLPF, averaging, and sample-rate settings.

Definition at line 53 of file [i2c\\_accel.ino](#).

Here is the call graph for this function:



### 6.3.3 Variable Documentation

#### 6.3.3.1 imu

[DevLab\\_ICM20948](#) imu

Global ICM-20948 driver instance.

Definition at line 45 of file [i2c\\_accel.ino](#).

## 6.4 i2c\_accel.ino

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * @file      I2C_Accel.ino
00003  * @brief     Minimal I2C bring-up for 7Semi ICM-20948 on UNO/ESP32 +
00004  *           continuous accelerometer readout
00005  *
00006  * Features
00007  * - I2C init on default/custom pins
00008  * - IMU beginI2C() on I2C (addr 0x68/0x69)
00009  * - Manual sensor enable (setSensors)
00010  * - Accel config: scale, DLPF, averaging, sample rate
00011  * - Read accel (g) at ~2 Hz print
00012  *
00013  * Notes
00014  * - WHO_AM_I must be 0xEA
00015  * - Sensors must be explicitly enabled
00016  * - Bank switching handled internally by library
00017  *
00018  * Wiring (Arduino UNO I2C)
00019  * - SDA -> A4
00020  * - SCL -> A5
00021  * - VCC -> 3V3 (or 5V if your board supports it)
00022  * - GND -> GND
00023  *
00024  * Wiring (ESP32 default I2C)
00025  * - SDA -> GPIO21
00026  * - SCL -> GPIO22
00027  * - VCC -> 3V3
00028  * - GND -> GND
00029  *****/
00030
00031 #include <Wire.h>
00032 #include <DevLab_ICM20948.h>
00033
00034 /* ===== User Config ===== */
00035 /** @brief I2C SDA pin used by the example board. */
00036 #define SDA_PIN 6
00037 /** @brief I2C SCL pin used by the example board. */

```

```

00038 #define SCL_PIN 7
00039 /** @brief I2C bus speed in hertz. */
00040 #define I2C_FREQ 400000UL
00041 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00042 #define ICM_ADDR 0x69
00043
00044 /** @brief Global ICM-20948 driver instance. */
00045 DevLab_ICM20948 imu;
00046
00047 /**
00048  * @brief Configure the IMU for accelerometer-only operation over I2C.
00049  *
00050  * The function initializes the bus, verifies the device identity, enables the
00051  * accelerometer, and applies range, DLPF, averaging, and sample-rate settings.
00052  */
00053 void setup() {
00054     Serial.begin(115200);
00055     delay(200);
00056     Serial.println(F("ICM-20948 (I2C) - Accel Example"));
00057     Wire.begin(SDA_PIN, SCL_PIN);
00058     /* Initialize IMU using I2C. */
00059     if (!imu.beginI2C(ICM_ADDR, Wire, 400000)) {
00060         Serial.println(F("ERROR: ICM-20948 beginI2C() failed.));
00061         while (1) delay(200);
00062     }
00063
00064     Serial.println(F("ICM-20948 ready.));
00065
00066     /* Verify device identity. */
00067     uint8_t who;
00068     if (imu.readWhoAmI(who)) {
00069         Serial.print("WHO_AM_I: 0x");
00070         Serial.println(who, HEX);
00071     }
00072
00073     /* Enable only accelerometer
00074      * - accel = true
00075      * - gyro = false
00076      * - temp = false
00077      */
00078     if (!imu.setSensors(true, false, false)) {
00079         Serial.println(F("setSensors failed.));
00080     }
00081
00082     /* ----- ACCEL CONFIG -----
00083      * - Scale: ±2/±4/±8/±16 g
00084      * - DLPF: noise filtering
00085      * - Averaging: noise reduction
00086      * - Sample rate: output frequency
00087      */
00088
00089     /* Set accelerometer full-scale range. */
00090     if (!imu.setAccelScale(ICM20948_Accel_FullScale::G_4)) {
00091         Serial.println(F("setAccelScale failed.));
00092     }
00093
00094     /* Enable DLPF (recommended)
00095      * - bypass = false → DLPF enabled
00096      */
00097     if (!imu.setAccelDLPF(ACCEL_DLPFCFG_3, false)) {
00098         Serial.println(F("setAccelDLPF failed.));
00099     }
00100
00101     /* Set averaging (noise reduction). */
00102     if (!imu.setAccelAveraging(ICM20948_Accel_Average::AVG_8)) {
00103         Serial.println(F("setAccelAveraging failed.));
00104     }
00105
00106     /* Set output data rate (ODR)
00107      * - Base = 1125 Hz
00108      * - ODR = 1125 / (1 + divider)
00109      * - Example: 225 Hz
00110      */
00111     if (!imu.setAccelSampleRate(225)) {
00112         Serial.println(F("setAccelSampleRate failed.));
00113     }
00114
00115     delay(10);
00116 }
00117
00118 /**
00119  * @brief Read and print accelerometer data in g.
00120  */
00121 void loop() {
00122     float ax, ay, az;
00123
00124     /* Read accelerometer data

```

```

00125  * - Output in g units
00126  * - Returns true on success
00127  */
00128  if (imu.readAccel(ax, ay, az)) {
00129      Serial.print("ACCEL [g]: ");
00130      Serial.print(ax, 3);
00131      Serial.print(", ");
00132      Serial.print(ay, 3);
00133      Serial.print(", ");
00134      Serial.println(az, 3);
00135  } else {
00136      Serial.println(F("ACCEL read failed"));
00137  }
00138
00139  Serial.println(F("-----"));
00140  delay(500);
00141 }

```

## 6.5 examples/i2c\_basic/i2c\_basic.ino File Reference

```

#include <Wire.h>
#include <DevLab_ICM20948.h>

```

### Macros

- `#define SDA_PIN 6`  
I2C SDA pin used by the example board.
- `#define SCL_PIN 7`  
I2C SCL pin used by the example board.
- `#define I2C_FREQ 400000UL`  
I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.

### Functions

- `void setup ()`  
Configure Serial, I2C, sensor identity check, and enabled sensors.
- `void loop ()`  
Read accelerometer, gyroscope, magnetometer, and temperature data.

### Variables

- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.

## 6.5.1 Macro Definition Documentation

### 6.5.1.1 I2C\_FREQ

```
#define I2C_FREQ 400000UL
```

I2C bus speed in hertz.

Definition at line 36 of file [i2c\\_basic.ino](#).

### 6.5.1.2 ICM\_ADDR

```
#define ICM_ADDR 0x69
```

ICM-20948 I2C address selected by the AD0 pin.

Definition at line 38 of file [i2c\\_basic.ino](#).

### 6.5.1.3 SCL\_PIN

```
#define SCL_PIN 7
```

I2C SCL pin used by the example board.

Definition at line 34 of file [i2c\\_basic.ino](#).

### 6.5.1.4 SDA\_PIN

```
#define SDA_PIN 6
```

I2C SDA pin used by the example board.

Definition at line 32 of file [i2c\\_basic.ino](#).

## 6.5.2 Function Documentation

### 6.5.2.1 loop()

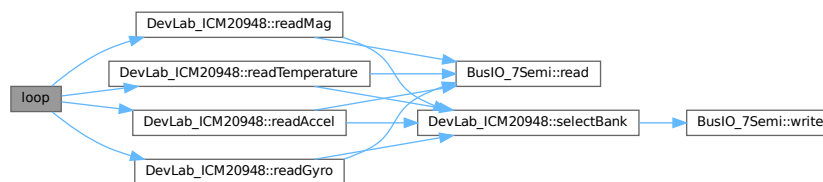
```
void loop ( )
```

Read accelerometer, gyroscope, magnetometer, and temperature data.

Values are printed periodically to the Serial monitor in engineering units.

Definition at line 90 of file [i2c\\_basic.ino](#).

Here is the call graph for this function:



### 6.5.2.2 setup()

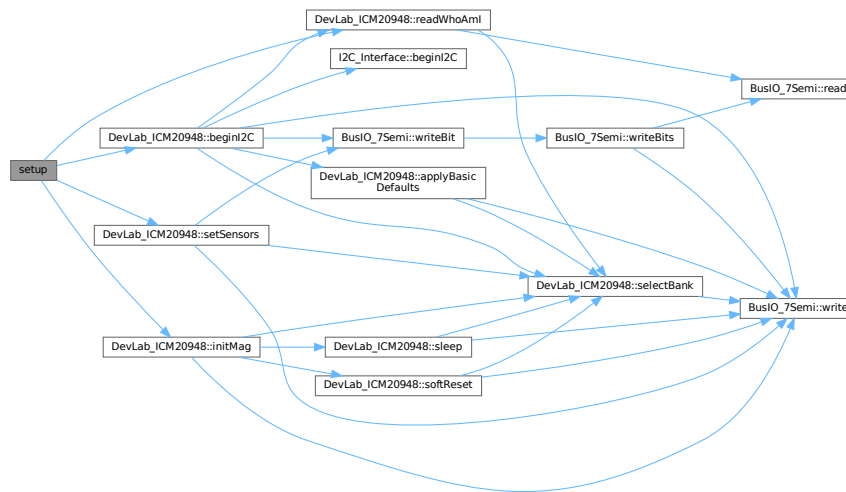
```
void setup ( )
```

Configure Serial, I2C, sensor identity check, and enabled sensors.

The sketch stops in an infinite loop if the IMU cannot be initialized or the WHO\_AM\_I register does not match the expected ICM-20948 value.

Definition at line 49 of file [i2c\\_basic.ino](#).

Here is the call graph for this function:



## 6.5.3 Variable Documentation

### 6.5.3.1 imu

[DevLab\\_ICM20948](#) imu

Global ICM-20948 driver instance.

Definition at line 41 of file [i2c\\_basic.ino](#).

## 6.6 i2c\_basic.ino

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * @file    I2C_Basic.ino
00003  * @brief    Minimal I2C bring-up for 7Semi ICM-20948 +
00004  *           basic Accel/Gyro/Temp/Mag readout (updated API).
00005  *
00006  * Features
00007  * - I2C init (UNO / ESP32)
00008  * - beginI2C() initialization
00009  * - Manual sensor enable (setSensors)
00010  * - Read Accel / Gyro / Temp / Mag
00011  *
00012  * Notes
00013  * - WHO_AM_I must be 0xEA
00014  * - Sensors must be explicitly enabled
00015  * - Magnetometer requires initMag()
00016  *
00017  * Wiring (Arduino UNO I2C)
00018  * - SDA -> A4
00019  * - SCL -> A5
00020  * - VCC -> 3V3 (or 5V if supported)
00021  * - GND -> GND
00022  *
00023  * Wiring (ESP32 default I2C)
00024  * - SDA -> GPIO21
00025  * - SCL -> GPIO22
00026  *****/
00027 #include <Wire.h>
00028 #include <DevLab_ICM20948.h>
00029
00030 /* ===== User Config ===== */
00031 /** @brief I2C SDA pin used by the example board. */
00032 #define SDA_PIN 6
00033 /** @brief I2C SCL pin used by the example board. */
00034 #define SCL_PIN 7
00035 /** @brief I2C bus speed in hertz. */
00036 #define I2C_FREQ 400000UL
00037 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00038 #define ICM_ADDR 0x69
00039

```



```

00040 /** @brief Global ICM-20948 driver instance. */
00041 DevLab_ICM20948 imu;
00042
00043 /**
00044  * @brief Configure Serial, I2C, sensor identity check, and enabled sensors.
00045  *
00046  * The sketch stops in an infinite loop if the IMU cannot be initialized or
00047  * the WHO_AM_I register does not match the expected ICM-20948 value.
00048  */
00049 void setup() {
00050     Serial.begin(115200);
00051     delay(200);
00052     Serial.println(F("ICM-20948 - I2C Basic"));
00053     Wire.begin(SDA_PIN, SCL_PIN);
00054     /* Initialize IMU. */
00055     if (!imu.beginI2C(ICM_ADDR, Wire, 400000)) {
00056         Serial.println(F("ERROR: beginI2C() failed"));
00057         while (1) delay(200);
00058     }
00059
00060     /* WHO_AM_I check. */
00061     uint8_t who;
00062     if (!imu.readWhoAmI(who) || who != 0xEA) {
00063         Serial.println(F("ERROR: WHO_AM_I mismatch"));
00064         while (1) delay(200);
00065     }
00066
00067     Serial.print(F("WHO_AM_I = 0x"));
00068     Serial.println(who, HEX);
00069
00070     /* Enable all sensors. */
00071     if (!imu.setSensors(true, true, true)) {
00072         Serial.println(F("ERROR: setSensors failed"));
00073     }
00074
00075     /* Initialize magnetometer. */
00076     if (!imu.initMag()) {
00077         Serial.println(F("Mag init failed"));
00078     } else {
00079         Serial.println(F("Mag initialized"));
00080     }
00081
00082     Serial.println(F("Ready.\n"));
00083 }
00084
00085 /**
00086  * @brief Read accelerometer, gyroscope, magnetometer, and temperature data.
00087  *
00088  * Values are printed periodically to the Serial monitor in engineering units.
00089  */
00090 void loop() {
00091     float ax, ay, az;
00092     float gx, gy, gz;
00093     float mx, my, mz;
00094     float tC;
00095
00096     /* Read accelerometer. */
00097     if (imu.readAccel(ax, ay, az)) {
00098         Serial.print(F("ACC [g]: "));
00099         Serial.print(ax, 3); Serial.print(", ");
00100         Serial.print(ay, 3); Serial.print(", ");
00101         Serial.println(az, 3);
00102     } else {
00103         Serial.println(F("ACC read failed"));
00104     }
00105
00106     /* Read gyroscope. */
00107     if (imu.readGyro(gx, gy, gz)) {
00108         Serial.print(F("GYR [dps]: "));
00109         Serial.print(gx, 2); Serial.print(", ");
00110         Serial.print(gy, 2); Serial.print(", ");
00111         Serial.println(gz, 2);
00112     } else {
00113         Serial.println(F("GYR read failed"));
00114     }
00115
00116     /* Read magnetometer. */
00117     if (imu.readMag(mx, my, mz)) {
00118         Serial.print(F("MAG [uT]: "));
00119         Serial.print(mx, 2); Serial.print(", ");
00120         Serial.print(my, 2); Serial.print(", ");
00121         Serial.println(mz, 2);
00122     } else {
00123         Serial.println(F("MAG read failed"));
00124     }
00125
00126     /* Read temperature. */

```

```

00127  if (imu.readTemperature(tc)) {
00128      Serial.print(F("TMP [C]: "));
00129      Serial.println(tc, 2);
00130  } else {
00131      Serial.println(F("TMP read failed"));
00132  }
00133
00134  Serial.println(F("-----"));
00135  delay(500);
00136 }
00137
00138 /* Note on magnetometer:
00139 * - Uses internal I2C master (AK09916 via ICM20948)
00140 * - initMag() must be called before readMag()
00141 * - If values are zero:
00142 *   -> check initMag()
00143 *   -> verify power + pull-ups
00144 */

```

## 6.7 examples/i2c\_gyro/i2c\_gyro.ino File Reference

```

#include <Wire.h>
#include <DevLab_ICM20948.h>

```

### Macros

- `#define SDA_PIN 6`  
I2C SDA pin used by the example board.
- `#define SCL_PIN 7`  
I2C SCL pin used by the example board.
- `#define I2C_FREQ 400000UL`  
I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.

### Functions

- `void setup ()`  
Configure the IMU for gyroscope-only operation over I2C.
- `void loop ()`  
Read and print gyroscope data in degrees per second.

### Variables

- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.

## 6.7.1 Macro Definition Documentation

### 6.7.1.1 I2C\_FREQ

```
#define I2C_FREQ 400000UL
```

I2C bus speed in hertz.

Definition at line 45 of file `i2c_gyro.ino`.

### 6.7.1.2 ICM\_ADDR

```
#define ICM_ADDR 0x69
```

ICM-20948 I2C address selected by the AD0 pin.

Definition at line 47 of file `i2c_gyro.ino`.

### 6.7.1.3 SCL\_PIN

```
#define SCL_PIN 7
```

I2C SCL pin used by the example board.

Definition at line 43 of file [i2c\\_gyro.ino](#).

### 6.7.1.4 SDA\_PIN

```
#define SDA_PIN 6
```

I2C SDA pin used by the example board.

Definition at line 41 of file [i2c\\_gyro.ino](#).

## 6.7.2 Function Documentation

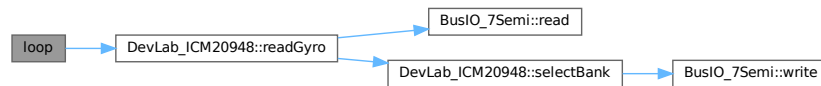
### 6.7.2.1 loop()

```
void loop ( )
```

Read and print gyroscope data in degrees per second.

Definition at line 125 of file [i2c\\_gyro.ino](#).

Here is the call graph for this function:



### 6.7.2.2 setup()

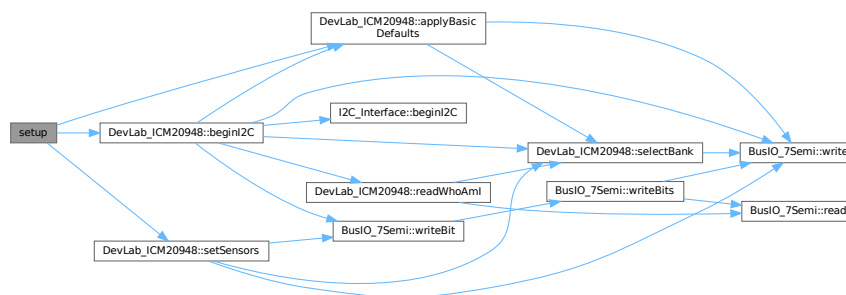
```
void setup ( )
```

Configure the IMU for gyroscope-only operation over I2C.

The function initializes the I2C interface, applies safe defaults, enables the gyroscope, and configures DLPF, full-scale range, and output data rate.

Definition at line 58 of file [i2c\\_gyro.ino](#).

Here is the call graph for this function:



## 6.7.3 Variable Documentation

### 6.7.3.1 imu

```
DevLab_ICM20948 imu
```

Global ICM-20948 driver instance.

Definition at line 50 of file [i2c\\_gyro.ino](#).

## 6.8 i2c\_gyro.ino

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * @file      I2C_Gyro.ino
00003  * @brief     Minimal I2C bring-up for 7Semi ICM-20948 on ESP32/UNO +
00004  *           continuous gyroscope readout.
00005  *
00006  * Features
00007  * - I2C init on default/custom pins
00008  * - IMU begin() on I2C (addr 0x68/0x69)
00009  * - Safe defaults (applyBasicDefaults)
00010  * - Optional sensor gating: gyro only
00011  * - Gyro config: DLPF, full-scale, ODR, AVG
00012  * - Read gyro (dps) at ~2 Hz print
00013  *
00014  * Wiring (Arduino UNO, I2C)
00015  * - SDA -> A4
00016  * - SCL -> A5
00017  * - VCC -> 3V3 (or 5V if your board supports it)
00018  * - GND -> GND
00019  *
00020  * Wiring (ESP32, I2C)
00021  * - SDA -> GPIO21 (default)
00022  * - SCL -> GPIO22 (default)
00023  * - VCC -> 3V3
00024  * - GND -> GND
00025  *
00026  * Address
00027  * - Default ICM-20948 I2C address is 0x68 (AD0=LOW). If AD0=HIGH, use 0x69.
00028  *****/
00029
00030 #include <Wire.h>
00031 #include <DevLab_ICM20948.h>
00032
00033 /* ===== User Config =====
00034  * - Edit pins/addr if needed
00035  * - UNO uses fixed SDA=A4, SCL=A5 (no need to call Wire.begin with pins)
00036  * - ESP32 can use default Wire (SDA=21, SCL=22) or custom pins via Wire.begin(sda,scl)
00037  */
00038
00039 /* ===== User Config ===== */
00040 /** @brief I2C SDA pin used by the example board. */
00041 #define SDA_PIN 6
00042 /** @brief I2C SCL pin used by the example board. */
00043 #define SCL_PIN 7
00044 /** @brief I2C bus speed in hertz. */
00045 #define I2C_FREQ 400000UL
00046 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00047 #define ICM_ADDR 0x69
00048
00049 /** @brief Global ICM-20948 driver instance. */
00050 DevLab_ICM20948 imu;
00051
00052 /**
00053  * @brief Configure the IMU for gyroscope-only operation over I2C.
00054  *
00055  * The function initializes the I2C interface, applies safe defaults, enables
00056  * the gyroscope, and configures DLPF, full-scale range, and output data rate.
00057  */
00058 void setup() {
00059     Serial.begin(115200);
00060     delay(200);
00061     Serial.println(F("ICM-20948 - I2C Basic"));
00062     Wire.begin(SDA_PIN, SCL_PIN);
00063     /* Initialize IMU. */
00064     if (!imu.beginI2C(ICM_ADDR, Wire, 400000)) {
00065         Serial.println(F("ERROR: beginI2C() failed"));
00066         while (1) delay(200);
00067     }
00068
00069     /* Apply safe defaults. */
00070     if (!imu.applyBasicDefaults()) {
00071         Serial.println(F("ERROR: applyBasicDefaults() failed.));
00072         while (1) delay(200);
00073     }
00074
00075     Serial.println(F("ICM-20948 ready.));
00076
00077     /* Optional: gate sensors
00078      * - setSensors(accel_on, gyro_on, temp_on)
00079      * - Here: enable only gyro (accel/temp off)
00080      */
00081     if (!imu.setSensors(/*accel*/ false, /*gyro*/ true, /*temp*/ false)) {
00082         Serial.println(F("setSensors failed.));
00083     }

```

```

00084
00085 /* ----- GYRO DLPF Config (reference) -----
00086 * GyroConfig(DLPFCFG, FS_SEL, FCHOICE, XGYRO, YGYRO, ZGYRO, AVGCFG)
00087 * - DLPF      : GYRO_DLPFCFG_0..7 (use 3 or 4 for balanced settings)
00088 * - FS_SEL    : dps250 / dps500 / dps1000 / dps2000
00089 * - FCHOICE    : false → DLPF path (recommended), true → bypass (very wide BW)
00090 * - XGYRO/YGYRO/ZGYRO : per-axis enable (true = enable axis)
00091 * - AVGCFG     : 0..7 internal averaging (higher = more smoothing, more delay)
00092 * FCHOICE=0 (DLPF on):
00093 * - GYRO_DLPFCFG_1 : 3dB ≈ 196.6 Hz, NBW ≈ 229.8 Hz
00094 * - GYRO_DLPFCFG_2 : 3dB ≈ 151.8 Hz, NBW ≈ 187.6 Hz
00095 * - GYRO_DLPFCFG_3 : 3dB ≈ 119.5 Hz, NBW ≈ 154.3 Hz ← good default
00096 * - GYRO_DLPFCFG_4 : 3dB ≈ 51.2 Hz, NBW ≈ 73.3 Hz
00097 * - GYRO_DLPFCFG_5 : 3dB ≈ 23.9 Hz, NBW ≈ 35.9 Hz
00098 * - GYRO_DLPFCFG_6 : 3dB ≈ 5.7 Hz, NBW ≈ 8.3 Hz
00099 * - GYRO_DLPFCFG_7 : 3dB ≈ 473 Hz, NBW ≈ 499 Hz
00100 *
00101 * FCHOICE=1 (Bypass) : very wide (use with care; highest noise)
00102 */
00103 if (!imu.GyroConfig(/*DLPFCFG*/ GYRO_DLPFCFG_4,
00104 /*FS_SEL*/ dps2000,
00105 /*FCHOICE*/ true, // DLPF ON (recommended)
00106 /*X*/ true, /*Y*/ true, /*Z*/ true,
00107 /*AVGCFG*/ 0)) {
00108   Serial.println(F("GyroConfig failed.));
00109 }
00110
00111 /* Set gyroscope output data rate (ODR)
00112 * - Gyro_SMPLRT(rate_hz)
00113 * - Base (DLPF path) = 1100 Hz
00114 * - Valid range: ~4.3-1100 Hz
00115 * - Example: 1000 Hz
00116 */
00117 if (!imu.Gyro_SMPLRT(1000)) {
00118   Serial.println(F("Gyro_SMPLRT failed.));
00119 }
00120 }
00121
00122 /**
00123 * @brief Read and print gyroscope data in degrees per second.
00124 */
00125 void loop() {
00126   float gx, gy, gz; // gyro X/Y/Z in dps
00127
00128   /* - readGyro(x,y,z)
00129    * - Returns:
00130    * - true on success;
00131    * - Output:
00132    * - gx/gy/gz in dps
00133    */
00134   if (imu.readGyro(gx, gy, gz)) {
00135     Serial.print("GYRO [dps]: ");
00136     Serial.print(gx, 3);
00137     Serial.print(", ");
00138     Serial.print(gy, 3);
00139     Serial.print(", ");
00140     Serial.println(gz, 3);
00141   } else {
00142     Serial.println(F("GYRO read failed"));
00143   }
00144
00145   Serial.println(F("-----"));
00146   delay(500);
00147 }

```

## 6.9 examples/i2c\_magneto/i2c\_magneto.ino File Reference

```

#include <Wire.h>
#include <DevLab_ICM20948.h>

```

### Macros

- #define [SDA\\_PIN](#) 6  
I2C SDA pin used by the example board.
- #define [SCL\\_PIN](#) 7  
I2C SCL pin used by the example board.
- #define [I2C\\_FREQ](#) 400000UL

- I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.

## Functions

- `void setup ()`  
Configure I2C, verify the IMU, and initialize the magnetometer.
- `void loop ()`  
Read and print magnetometer data in microtesla.

## Variables

- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.

## 6.9.1 Macro Definition Documentation

### 6.9.1.1 I2C\_FREQ

`#define I2C_FREQ 400000UL`  
I2C bus speed in hertz.  
Definition at line 42 of file `i2c_magneto.ino`.

### 6.9.1.2 ICM\_ADDR

`#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.  
Definition at line 44 of file `i2c_magneto.ino`.

### 6.9.1.3 SCL\_PIN

`#define SCL_PIN 7`  
I2C SCL pin used by the example board.  
Definition at line 40 of file `i2c_magneto.ino`.

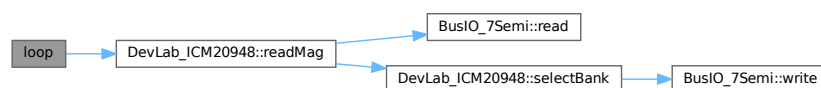
### 6.9.1.4 SDA\_PIN

`#define SDA_PIN 6`  
I2C SDA pin used by the example board.  
Definition at line 38 of file `i2c_magneto.ino`.

## 6.9.2 Function Documentation

### 6.9.2.1 loop()

`void loop ( )`  
Read and print magnetometer data in microtesla.  
Definition at line 97 of file `i2c_magneto.ino`.  
Here is the call graph for this function:



## 6.9.2.2 setup()

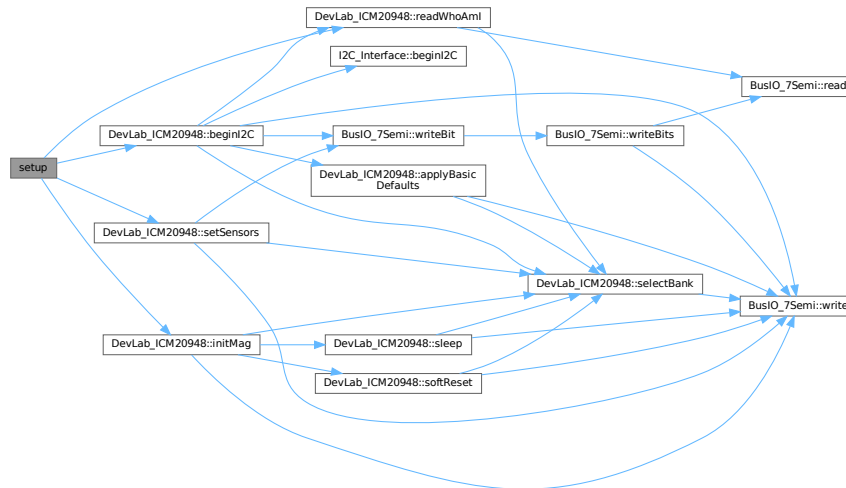
```
void setup ( )
```

Configure I2C, verify the IMU, and initialize the magnetometer.

The magnetometer is accessed through the ICM-20948 internal I2C master, so `initMag()` must succeed before `loop()` can read magnetic field data.

Definition at line 55 of file [i2c\\_magneto.ino](#).

Here is the call graph for this function:



## 6.9.3 Variable Documentation

## 6.9.3.1 imu

[DevLab\\_ICM20948](#) `imu`

Global ICM-20948 driver instance.

Definition at line 47 of file [i2c\\_magneto.ino](#).

## 6.10 i2c\_magneto.ino

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * @file      I2C_Magneto.ino
00003  * @brief     Minimal I2C bring-up for 7Semi ICM-20948 +
00004  *            AK09916 magnetometer readout (updated API).
00005  *
00006  * Features
00007  * - I2C init on default/custom pins
00008  * - IMU beginI2C() initialization
00009  * - Manual sensor enable
00010  * - Magnetometer initialization (initMag)
00011  * - Read magnetometer (uT) at ~2 Hz
00012  *
00013  * Wiring (Arduino UNO, I2C)
00014  * - SDA -> A4
00015  * - SCL -> A5
00016  * - VCC -> 3V3 (or 5V if board supports it)
00017  * - GND -> GND
00018  *
00019  * Wiring (ESP32, I2C default)
00020  * - SDA -> GPIO21
00021  * - SCL -> GPIO22
00022  * - VCC -> 3V3
00023  * - GND -> GND
00024  *
00025  * Notes
00026  * - WHO_AM_I must be 0xEA
00027  * - initMag() must be called before readMag()
00028  * - Magnetometer runs via internal I2C master
00029  */

```

```

00030  * Author   : 7Semi
00031  * License : MIT
00032  *****/
00033 #include <Wire.h>
00034 #include <DevLab_ICM20948.h>
00035
00036 /* ===== User Config ===== */
00037 /** @brief I2C SDA pin used by the example board. */
00038 #define SDA_PIN 6
00039 /** @brief I2C SCL pin used by the example board. */
00040 #define SCL_PIN 7
00041 /** @brief I2C bus speed in hertz. */
00042 #define I2C_FREQ 400000UL
00043 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00044 #define ICM_ADDR 0x69
00045
00046 /** @brief Global ICM-20948 driver instance. */
00047 DevLab_ICM20948 imu;
00048
00049 /**
00050  * @brief Configure I2C, verify the IMU, and initialize the magnetometer.
00051  *
00052  * The magnetometer is accessed through the ICM-20948 internal I2C master, so
00053  * initMag() must succeed before loop() can read magnetic field data.
00054  */
00055 void setup() {
00056     Serial.begin(115200);
00057     delay(200);
00058     Serial.println(F("ICM-20948 (I2C) - Magnetometer Example"));
00059     Wire.begin(SDA_PIN, SCL_PIN);
00060     /* Initialize IMU. */
00061     if (!imu.beginI2C(ICM_ADDR, Wire, 400000)) {
00062         Serial.println(F("ERROR: beginI2C() failed."));
00063         while (1) delay(200);
00064     }
00065
00066     Serial.println(F("ICM-20948 ready (I2C)."));
00067
00068     /* Verify device. */
00069     uint8_t who;
00070     if (!imu.readWhoAmI(who) || who != 0xEA) {
00071         Serial.println(F("ERROR: WHO_AM_I mismatch"));
00072         while (1) delay(200);
00073     }
00074
00075     Serial.print(F("WHO_AM_I = 0x"));
00076     Serial.println(who, HEX);
00077
00078     /* Enable required sensors (mag uses internal I2C master)
00079     * - accel/gyro not strictly required but safe to keep ON
00080     */
00081     if (!imu.setSensors(true, true, false)) {
00082         Serial.println(F("setSensors failed."));
00083     }
00084
00085     /* Initialize magnetometer. */
00086     if (!imu.initMag()) {
00087         Serial.println(F("ERROR: initMag() failed"));
00088         while (1) delay(200);
00089     }
00090
00091     Serial.println(F("Magnetometer ready"));
00092 }
00093
00094 /**
00095  * @brief Read and print magnetometer data in microtesla.
00096  */
00097 void loop() {
00098     float mx, my, mz;
00099
00100     /* Read magnetometer data
00101     * - Output in microtesla (uT)
00102     * - Returns true on success
00103     */
00104     if (imu.readMag(mx, my, mz)) {
00105         Serial.print(F("MAG [uT]: "));
00106         Serial.print(mx, 2); Serial.print(F(", "));
00107         Serial.print(my, 2); Serial.print(F(", "));
00108         Serial.println(mz, 2);
00109     } else {
00110         Serial.println(F("Mag read failed"));
00111     }
00112
00113     Serial.println(F("-----"));
00114     delay(500);
00115 }

```



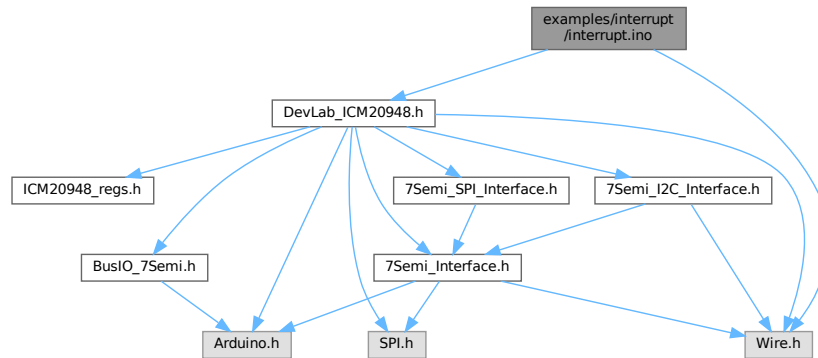
## 6.11 examples/interrupt/interrupt.ino File Reference

I2C interrupt example for the 7Semi ICM-20948.

```
#include <Wire.h>
```

```
#include <DevLab_ICM20948.h>
```

Include dependency graph for interrupt.ino:



### Macros

- `#define SDA_PIN 6`  
I2C SDA pin used by the example board.
- `#define SCL_PIN 7`  
I2C SCL pin used by the example board.
- `#define I2C_FREQ 400000UL`  
I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.
- `#define PIN_INT D7`  
MCU pin connected to the IMU interrupt output.

### Functions

- `void printLinea ()`  
Print a separator line to Serial.
- `void printDobleLinea ()`  
Print a double separator line to Serial.
- `void IRAM_ATTR isrHandler ()`  
Interrupt service routine for the IMU data-ready signal.
- `void setup ()`  
Initialize I2C, verify the IMU, configure interrupts, and attach ISR.
- `void loop ()`  
Report interrupt events when the ISR count changes.

### Variables

- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.
- `ICM20948_IntPinConfig pinCfg`  
Interrupt pin electrical and latch configuration.

- [ICM20948\\_IntEnableConfig intEn](#)  
Interrupt enable flags used by this example.
- `volatile uint32_t isrCount = 0`  
Number of interrupt events observed by the ISR.

### 6.11.1 Detailed Description

I2C interrupt example for the 7Semi ICM-20948.

Configures the IMU raw-data-ready interrupt pin and counts ISR events on an Arduino-compatible board.

Definition in file [interrupt.ino](#).

### 6.11.2 Macro Definition Documentation

#### 6.11.2.1 I2C\_FREQ

```
#define I2C_FREQ 400000UL
```

I2C bus speed in hertz.

Definition at line 16 of file [interrupt.ino](#).

#### 6.11.2.2 ICM\_ADDR

```
#define ICM_ADDR 0x69
```

ICM-20948 I2C address selected by the AD0 pin.

Definition at line 18 of file [interrupt.ino](#).

#### 6.11.2.3 PIN\_INT

```
#define PIN_INT D7
```

MCU pin connected to the IMU interrupt output.

Definition at line 20 of file [interrupt.ino](#).

#### 6.11.2.4 SCL\_PIN

```
#define SCL_PIN 7
```

I2C SCL pin used by the example board.

Definition at line 14 of file [interrupt.ino](#).

#### 6.11.2.5 SDA\_PIN

```
#define SDA_PIN 6
```

I2C SDA pin used by the example board.

Definition at line 12 of file [interrupt.ino](#).

### 6.11.3 Function Documentation

#### 6.11.3.1 isrHandler()

```
void IRAM_ATTR isrHandler ( )
```

Interrupt service routine for the IMU data-ready signal.

Definition at line 52 of file [interrupt.ino](#).

Here is the caller graph for this function:



## 6.11.3.2 loop()

```
void loop ( )
```

Report interrupt events when the ISR count changes.

Definition at line 94 of file [interrupt.ino](#).

## 6.11.3.3 printDobleLinea()

```
void printDobleLinea ( )
```

Print a double separator line to Serial.

Definition at line 28 of file [interrupt.ino](#).

## 6.11.3.4 printLinea()

```
void printLinea ( )
```

Print a separator line to Serial.

Definition at line 26 of file [interrupt.ino](#).

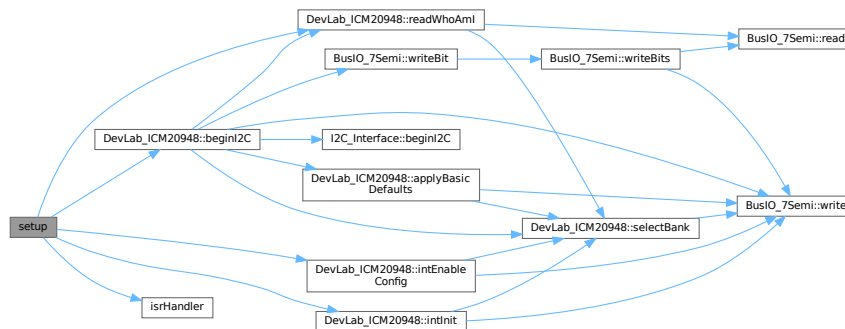
## 6.11.3.5 setup()

```
void setup ( )
```

Initialize I2C, verify the IMU, configure interrupts, and attach ISR.

Definition at line 59 of file [interrupt.ino](#).

Here is the call graph for this function:



## 6.11.4 Variable Documentation

## 6.11.4.1 imu

[DevLab\\_ICM20948](#) imu

Global ICM-20948 driver instance.

Definition at line 23 of file [interrupt.ino](#).

## 6.11.4.2 intEn

[ICM20948\\_IntEnableConfig](#) intEn

Initial value:

```
= {
  .rawDataRdyEn = 1
}
```

Interrupt enable flags used by this example.

Definition at line 42 of file [interrupt.ino](#).

## 6.11.4.3 isrCount

```
volatile uint32_t isrCount = 0
```

Number of interrupt events observed by the ISR.

Definition at line 47 of file [interrupt.ino](#).

#### 6.11.4.4 pinCfg

`ICM20948_IntPinConfig pinCfg`

Initial value:

```
= {
    .activeLevel = 1,
    .driveMode   = 0,
    .latchMode   = 0,
    .clearMode   = 1,
    .fsyncActLevel = 0,
    .fsyncIntEn  = 0,
    .bypassEn    = 0
}
```

Interrupt pin electrical and latch configuration.

Definition at line 31 of file `interrupt.ino`.

## 6.12 interrupt.ino

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file interrupt.ino
00003  * @brief I2C interrupt example for the 7Semi ICM-20948.
00004  * @details Configures the IMU raw-data-ready interrupt pin and counts ISR
00005  * events on an Arduino-compatible board.
00006  */
00007
00008 #include <Wire.h>
00009 #include <DevLab_ICM20948.h>
00010
00011 /** @brief I2C SDA pin used by the example board. */
00012 #define SDA_PIN 6
00013 /** @brief I2C SCL pin used by the example board. */
00014 #define SCL_PIN 7
00015 /** @brief I2C bus speed in hertz. */
00016 #define I2C_FREQ 400000UL
00017 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00018 #define ICM_ADDR 0x69
00019 /** @brief MCU pin connected to the IMU interrupt output. */
00020 #define PIN_INT D7 // ajusta a tu pin real
00021
00022 /** @brief Global ICM-20948 driver instance. */
00023 DevLab_ICM20948 imu;
00024
00025 /** @brief Print a separator line to Serial. */
00026 void printLinea() {
00027     Serial.println(F("-----")); }
00028 /** @brief Print a double separator line to Serial. */
00029 void printDobleLinea() {
00030     Serial.println(F("=====")); }
00031
00032 /** @brief Interrupt pin electrical and latch configuration. */
00033 ICM20948_IntPinConfig pinCfg = {
00034     .activeLevel = 1,
00035     .driveMode   = 0,
00036     .latchMode   = 0,
00037     .clearMode   = 1,
00038     .fsyncActLevel = 0,
00039     .fsyncIntEn  = 0,
00040     .bypassEn    = 0
00041 };
00042
00043 /** @brief Interrupt enable flags used by this example. */
00044 ICM20948_IntEnableConfig intEn = {
00045     .rawDataRdyEn = 1 // inicializacion directa
00046 };
00047
00048 /** @brief Number of interrupt events observed by the ISR. */
00049 volatile uint32_t isrCount = 0;
00050
00051 /**
00052  * @brief Interrupt service routine for the IMU data-ready signal.
00053  */
00054 void IRAM_ATTR isrHandler() {
00055     isrCount++;
00056 }
00057
00058 /**
00059  * @brief Initialize I2C, verify the IMU, configure interrupts, and attach ISR.
00060  */
00061 void setup() {
00062     Serial.begin(115200);
00063     delay(200);
00064 }
```

```

00063 Wire.begin(SDA_PIN, SCL_PIN);
00064
00065 if (!imu.beginI2C(ICM_ADDR, Wire, I2C_FREQ)) {
00066     Serial.println(F("ERROR: beginI2C() failed."));
00067     while (1) delay(200);
00068 }
00069 Serial.println(F("ICM-20948 ready."));
00070
00071 uint8_t who;
00072 if (!imu.readWhoAmI(who) || who != 0xEA) {
00073     Serial.println(F("ERROR: WHO_AM_I mismatch"));
00074     while (1) delay(200);
00075 }
00076 Serial.printf("WHO_AM_I = 0x%02X\n", who);
00077
00078 if(!imu.intInit(pinCfg)){
00079     Serial.println(F("ERROR: Initialization interruption failed"));
00080     while(1) delay(200);
00081 }; // nombre correcto
00082 if(!imu.intEnableConfig(intEn)){
00083     Serial.println(F("ERROR: Initialization interruption failed"));
00084     while(1) delay(200);
00085 }
00086
00087 attachInterrupt(digitalPinToInterrupt(PIN_INT), isrHandler, FALLING);
00088 Serial.println(F("Interrupt configured. Waiting..."));
00089 }
00090
00091 /**
00092  * @brief Report interrupt events when the ISR count changes.
00093  */
00094 void loop() {
00095     static uint32_t lastCount = 0;
00096
00097     if (isrCount != lastCount) {
00098         lastCount = isrCount;
00099         Serial.printf(">>> ISR disparada! Total: %lu\n", isrCount);
00100     }
00101 }

```

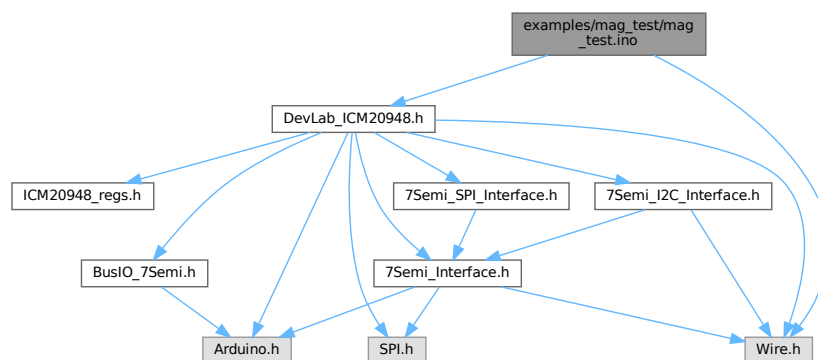
## 6.13 examples/mag\_test/mag\_test.ino File Reference

Magnetometer operation-mode sweep for the 7Semi ICM-20948.

```
#include <Wire.h>
```

```
#include <DevLab_ICM20948.h>
```

Include dependency graph for mag\_test.ino:



### Macros

- `#define SDA_PIN 6`  
I2C SDA pin used by the example board.
- `#define SCL_PIN 7`  
I2C SCL pin used by the example board.
- `#define I2C_FREQ 400000UL`

- I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.

## Functions

- `void printLinea ()`  
Print a separator line to Serial.
- `void printDobleLinea ()`  
Print a double separator line to Serial.
- `void applySettings ()`  
Initialize the magnetometer and collect samples for each mode.
- `void setup ()`  
Initialize I2C, verify the IMU, enable sensors, and start the sweep.
- `void loop ()`  
Empty loop; this sketch runs the magnetometer sweep once in `setup()`.

## Variables

- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.
- `const ICM20948_Op_Mode opMode []`  
Magnetometer operating modes tested by the sweep.
- `const char * etiquetasOpModeCfg []`  
Text labels matching the operating modes printed in the sweep.
- `const uint8_t N_OPM = 6`  
Number of printable magnetometer operation modes.

### 6.13.1 Detailed Description

Magnetometer operation-mode sweep for the 7Semi ICM-20948.

Initializes the AK09916 magnetometer through the ICM-20948 internal I2C master, iterates through continuous measurement modes, and prints CSV-like magnetic field samples to the Serial monitor.

Definition in file [mag\\_test.ino](#).

### 6.13.2 Macro Definition Documentation

#### 6.13.2.1 I2C\_FREQ

```
#define I2C_FREQ 400000UL
```

I2C bus speed in hertz.

Definition at line 19 of file [mag\\_test.ino](#).

#### 6.13.2.2 ICM\_ADDR

```
#define ICM_ADDR 0x69
```

ICM-20948 I2C address selected by the AD0 pin.

Definition at line 21 of file [mag\\_test.ino](#).

#### 6.13.2.3 SCL\_PIN

```
#define SCL_PIN 7
```

I2C SCL pin used by the example board.

Definition at line 17 of file [mag\\_test.ino](#).

#### 6.13.2.4 SDA\_PIN

```
#define SDA_PIN 6
```

I2C SDA pin used by the example board.

Definition at line 15 of file [mag\\_test.ino](#).

### 6.13.3 Function Documentation

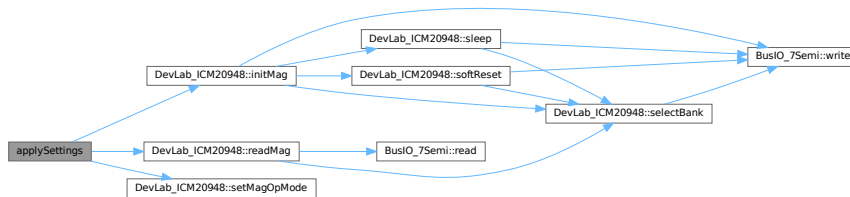
#### 6.13.3.1 applySettings()

```
void applySettings ( )
```

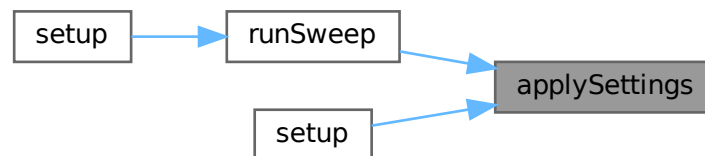
Initialize the magnetometer and collect samples for each mode.

Definition at line 52 of file [mag\\_test.ino](#).

Here is the call graph for this function:



Here is the caller graph for this function:



#### 6.13.3.2 loop()

```
void loop ( )
```

Empty loop; this sketch runs the magnetometer sweep once in [setup\(\)](#).

Definition at line 137 of file [mag\\_test.ino](#).

#### 6.13.3.3 printDobleLinea()

```
void printDobleLinea ( )
```

Print a double separator line to Serial.

Definition at line 29 of file [mag\\_test.ino](#).

#### 6.13.3.4 printLinea()

```
void printLinea ( )
```

Print a separator line to Serial.

Definition at line 27 of file [mag\\_test.ino](#).

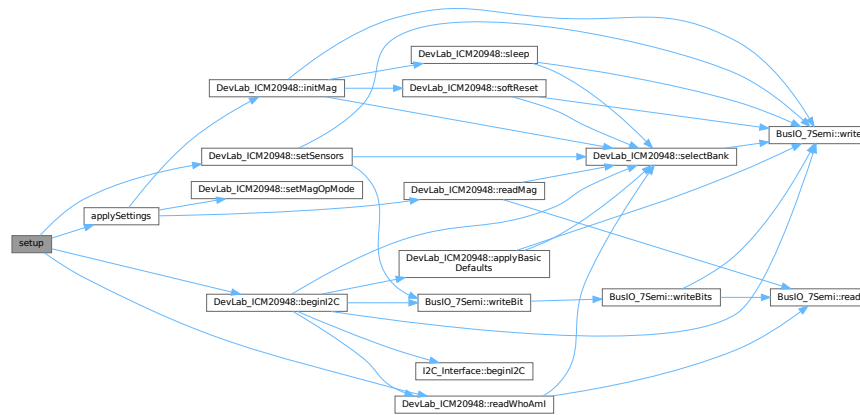
### 6.13.3.5 setup()

```
void setup ( )
```

Initialize I2C, verify the IMU, enable sensors, and start the sweep.

Definition at line 90 of file [mag\\_test.ino](#).

Here is the call graph for this function:



## 6.13.4 Variable Documentation

### 6.13.4.1 etiquetasOpModeCfg

```
const char* etiquetasOpModeCfg[ ]
```

Initial value:

```
= {
  "PowerDown", "SingleMeasure", "Continuo1", "Continuo2", "Continuo3", "Continuo4"
}
```

Text labels matching the operating modes printed in the sweep.

Definition at line 43 of file [mag\\_test.ino](#).

### 6.13.4.2 imu

```
DevLab_ICM20948 imu
```

Global ICM-20948 driver instance.

Definition at line 24 of file [mag\\_test.ino](#).

### 6.13.4.3 N\_OPM

```
const uint8_t N_OPM = 6
```

Number of printable magnetometer operation modes.

Definition at line 47 of file [mag\\_test.ino](#).

### 6.13.4.4 opMode

```
const ICM20948_Op_Mode opMode[ ]
```

Initial value:

```
= {
  ICM20948_Op_Mode::MODE_POWER_DOWN,
  ICM20948_Op_Mode::MODE_SINGLE_MEASUREMENT,
  ICM20948_Op_Mode::MODE_CONTINUOUS_1,
  ICM20948_Op_Mode::MODE_CONTINUOUS_2,
  ICM20948_Op_Mode::MODE_CONTINUOUS_3,
  ICM20948_Op_Mode::MODE_CONTINUOUS_4,
  ICM20948_Op_Mode::MODE_SELF_TEST
}
```

Magnetometer operating modes tested by the sweep.

Definition at line 32 of file [mag\\_test.ino](#).



## 6.14 mag\_test.ino

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file mag_test.ino
00003  * @brief Magnetometer operation-mode sweep for the 7Semi ICM-20948.
00004  * @details Initializes the AK09916 magnetometer through the ICM-20948 internal
00005  * I2C master, iterates through continuous measurement modes, and prints CSV-like
00006  * magnetic field samples to the Serial monitor.
00007  */
00008
00009 #include <Wire.h>
00010 #include <DevLab_ICM20948.h>
00011 // -----
00012 // PINES I2C - ESP32C6 NANO Unit Electronics
00013 // -----
00014 /** @brief I2C SDA pin used by the example board. */
00015 #define SDA_PIN 6
00016 /** @brief I2C SCL pin used by the example board. */
00017 #define SCL_PIN 7
00018 /** @brief I2C bus speed in hertz. */
00019 #define I2C_FREQ 400000UL
00020 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00021 #define ICM_ADDR 0x69
00022
00023 /** @brief Global ICM-20948 driver instance. */
00024 DevLab_ICM20948 imu;
00025
00026 /** @brief Print a separator line to Serial. */
00027 void printLinea() {
00028     Serial.println(F("-----"));
00029 }
00030 /** @brief Print a double separator line to Serial. */
00031 void printDobleLinea() {
00032     Serial.println(F("====="));
00033 }
00034
00035 /** @brief Magnetometer operating modes tested by the sweep. */
00036 const ICM20948_Op_Mode opMode[] = {
00037     ICM20948_Op_Mode::MODE_POWER_DOWN,
00038     ICM20948_Op_Mode::MODE_SINGLE_MEASUREMENT,
00039     ICM20948_Op_Mode::MODE_CONTINUOUS_1,
00040     ICM20948_Op_Mode::MODE_CONTINUOUS_2,
00041     ICM20948_Op_Mode::MODE_CONTINUOUS_3,
00042     ICM20948_Op_Mode::MODE_CONTINUOUS_4,
00043     ICM20948_Op_Mode::MODE_SELF_TEST
00044 };
00045
00046 /** @brief Text labels matching the operating modes printed in the sweep. */
00047 const char* etiquetasOpModeCfg[] = {
00048     "PowerDown", "SingleMeasure", "Continuo1", "Continuo2", "Continuo3", "Continuo4"
00049 };
00050
00051 /** @brief Number of printable magnetometer operation modes. */
00052 const uint8_t N_OPM = 6;
00053
00054 /**
00055  * @brief Initialize the magnetometer and collect samples for each mode.
00056  */
00057 void applySettings(){
00058     /** Initialize magnetometer. */
00059     if (!imu.initMag()) {
00060         Serial.println(F("ERROR: initMag() failed"));
00061         while (1) delay(200);
00062     }
00063     for(int i = 2; i < N_OPM; i++){
00064         imu.setMagOpMode(opMode[i]);
00065         delay(100);
00066         Serial.println(F("Magnetometer ready"));
00067         float mx, my, mz;
00068
00069         /** Read magnetometer data
00070          * - Output in microtesla (uT)
00071          * - Returns true on success
00072          */
00073         for(int r = 0; r < 500 ; r++){
00074             if (imu.readMag(mx, my, mz)) {
00075                 Serial.printf("[%d]", r);
00076                 Serial.print(",");
00077                 Serial.print(etiquetasOpModeCfg[i]);
00078                 Serial.print(",");
00079                 Serial.print(mx, 2); Serial.print(F(" "));
00080                 Serial.print(my, 2); Serial.print(F(" "));
00081                 Serial.println(mz, 2);
00082             } else {
00083                 Serial.println(F("Mag read failed"));
00084             }
00085         }
00086     }
00087     Serial.println(F("-----"));
00088 }

```

```

00082     delay(500);
00083 }
00084
00085 }
00086
00087 /**
00088  * @brief Initialize I2C, verify the IMU, enable sensors, and start the sweep.
00089  */
00090 void setup() {
00091     // put your setup code here, to run once:
00092     Serial.begin(115200);
00093     delay(200);
00094     Serial.println(F("ICM-20948 (I2C) - Magnetometer Example"));
00095
00096     Wire.begin(SDA_PIN,SCL_PIN);
00097     /* Initialize IMU. */
00098     if (!imu.beginI2C(ICM_ADDR, Wire, 400000)) {
00099         Serial.println(F("ERROR: beginI2C() failed."));
00100         while (1) delay(200);
00101     }
00102
00103     Serial.println(F("ICM-20948 ready (I2C)."));
00104
00105     /* Verify device. */
00106     uint8_t who;
00107     if (!imu.readWhoAmI(who) || who != 0xEA) {
00108         Serial.println(F("ERROR: WHO_AM_I mismatch"));
00109         while (1) delay(200);
00110     }
00111
00112     Serial.print(F("WHO_AM_I = 0x"));
00113     Serial.println(who, HEX);
00114
00115     /* Enable required sensors (mag uses internal I2C master)
00116      * - accel/gyro not strictly required but safe to keep ON
00117      */
00118     if (!imu.setSensors(true, true, false)) {
00119         Serial.println(F("setSensors failed."));
00120     }
00121
00122     Serial.println(F("  Sensor listo.\n"));
00123     Serial.println(F("  Sensor PLANO y QUIETO sobre la mesa."));
00124     Serial.println(F("  Iniciando en 5 segundos..."));
00125     for (int i = 5; i > 0; i--) {
00126         Serial.printf("  %d...\n", i);
00127         delay(1000);
00128     }
00129
00130     applySettings();
00131
00132 }
00133
00134 /**
00135  * @brief Empty loop; this sketch runs the magnetometer sweep once in setup().
00136  */
00137 void loop() {
00138     // put your main code here, to run repeatedly:
00139
00140 }

```

## 6.15 examples/spi\_accel/spi\_accel.ino File Reference

#include <DevLab\_ICM20948.h>

### Macros

- #define [SPI\\_FAST\\_SPEED](#) 1000000  
SPI clock used during sensor initialization and reads.

### Functions

- SPIClass [spi\\_bus](#) (SPI)  
SPI bus object used by the IMU driver.
- void [setup](#) ()  
Configure the IMU for accelerometer-only operation over SPI.
- void [loop](#) ()

Read and print accelerometer data in g.

#### Variables

- [DevLab\\_ICM20948 imu](#)

Global ICM-20948 driver instance.

### 6.15.1 Macro Definition Documentation

#### 6.15.1.1 SPI\_FAST\_SPEED

```
#define SPI_FAST_SPEED 1000000
```

SPI clock used during sensor initialization and reads.

Definition at line 33 of file [spi\\_accel.ino](#).

### 6.15.2 Function Documentation

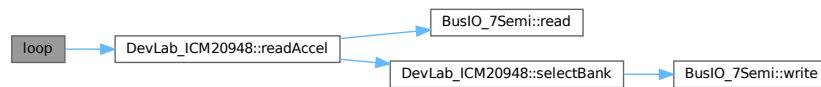
#### 6.15.2.1 loop()

```
void loop ( )
```

Read and print accelerometer data in g.

Definition at line 116 of file [spi\\_accel.ino](#).

Here is the call graph for this function:



#### 6.15.2.2 setup()

```
void setup ( )
```

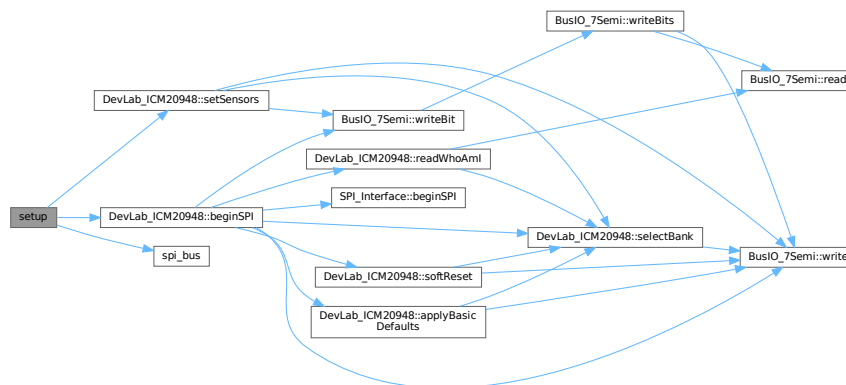
Configure the IMU for accelerometer-only operation over SPI.

Note

The sketch expects a board-level CS\_PIN definition to select the chip-select pin used by beginSPI().

Definition at line 47 of file [spi\\_accel.ino](#).

Here is the call graph for this function:

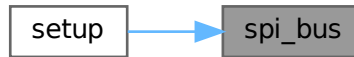


### 6.15.2.3 spi\_bus()

SPIClass spi\_bus (  
    SPI )

SPI bus object used by the IMU driver.

Here is the caller graph for this function:



## 6.15.3 Variable Documentation

### 6.15.3.1 imu

[DevLab\\_ICM20948](#) imu

Global ICM-20948 driver instance.

Definition at line 36 of file [spi\\_accel.ino](#).

## 6.16 spi\_accel.ino

[Go to the documentation of this file.](#)

```

00001 /*****
00002  * @file      SPI_Accel.ino
00003  * @brief     Minimal SPI bring-up for 7Semi ICM-20948 on ESP32 +
00004  *           continuous accelerometer readout.
00005  *
00006  * Features
00007  * - SPI init on custom pins (ESP32 VSPI)
00008  * - IMU begin() on SPI (CS pin user-defined)
00009  * - Safe defaults (applyBasicDefaults)
00010  * - Optional sensor gating: accel only
00011  * - Accel config: DLPF, full-scale, ODR, DEC3
00012  * - Read accel (g) at ~2 Hz print
00013  *
00014  * Wiring (Arduino UNO)
00015  * - SCK -> GPIO13
00016  * - MOSI(SDA) -> GPIO12
00017  * - MISO(SDO) -> GPIO11
00018  * - CS -> GPIO10 (changeable)
00019  * - VCC -> 3V3
00020  * - GND -> GND
00021  * Wiring (ESP32 VSPI default)
00022  * - SCK -> GPIO18
00023  * - MOSI -> GPIO23
00024  * - MISO -> GPIO19
00025  * - CS -> GPIO5 (changeable)
00026  * - VCC -> 3V3
00027  * - GND -> GND
00028  *****/
00029
00030 #include <DevLab_ICM20948.h>
00031
00032 /** @brief SPI clock used during sensor initialization and reads. */
00033 #define SPI_FAST_SPEED 1000000
00034
00035 /** @brief Global ICM-20948 driver instance. */
00036 DevLab_ICM20948 imu;
00037
00038 /** @brief SPI bus object used by the IMU driver. */
00039 SPIClass spi_bus(SPI); // usa el bus SPI por defecto del Arduino
00040
00041 /**
00042  * @brief Configure the IMU for accelerometer-only operation over SPI.
00043  *
00044  * @note The sketch expects a board-level CS_PIN definition to select the
00045  * chip-select pin used by beginSPI().
00046  */
  
```

```

00047 void setup() {
00048   Serial.begin(115200);
00049   delay(200);
00050
00051
00052   if (!imu.beginSPI(CS_PIN, spi_bus, 1000000)) {
00053     Serial.println("ERROR: beginSPI() failed");
00054     while (1) delay(200);
00055   }
00056
00057
00058   Serial.println(F("ICM-20948 ready."));
00059
00060   /* Optional: gate sensors
00061    * - setSensors(accel_on, gyro_on, temp_on)
00062    * - Here: enable only accel (gyro/temp off)
00063    */
00064   if (!imu.setSensors(/*accel*/ true, /*gyro*/ false, /*temp*/ false)) {
00065     Serial.println(F("setSensors failed."));
00066   }
00067   /* ----- ACCEL DLPF Config (reference) -----
00068    * ACCEL_DLPFCFG_0 (0..7) - 3dB & Noise Bandwidth (datasheet)
00069    * DLPF path ODR = 1125 / (1 + ACCEL_SMPLRT_DIV), DIV: 0..4095
00070    *
00071    * FCHOICE=0 (Bypass) : 3dB ≈ 1209 Hz, NBW ≈ 1248 Hz, Rate ≈ 4500 Hz
00072    * FCHOICE=1 (DLPF on):
00073    * - ACCEL_DLPFCFG_0 : 3dB ≈ 246.0 Hz, NBW ≈ 265.0 Hz
00074    * - ACCEL_DLPFCFG_1 : 3dB ≈ 246.0 Hz, NBW ≈ 265.0 Hz
00075    * - ACCEL_DLPFCFG_2 : 3dB ≈ 111.4 Hz, NBW ≈ 136.0 Hz
00076    * - ACCEL_DLPFCFG_3 : 3dB ≈ 50.4 Hz, NBW ≈ 68.8 Hz ← good default
00077    * - ACCEL_DLPFCFG_4 : 3dB ≈ 23.9 Hz, NBW ≈ 34.4 Hz
00078    * - ACCEL_DLPFCFG_5 : 3dB ≈ 11.5 Hz, NBW ≈ 17.0 Hz
00079    * - ACCEL_DLPFCFG_6 : 3dB ≈ 5.7 Hz, NBW ≈ 8.3 Hz
00080    * - ACCEL_DLPFCFG_7 : 3dB ≈ 473 Hz, NBW ≈ 499 Hz
00081    */
00082
00083   /* Configure accelerometer
00084    * AccelConfigure(DLPF, FS_SEL, dlpf_enable, dec3, stX, stY, stZ)
00085    * - DLPF : ACCEL_DLPFCFG_0..7 (use 3 for balanced setting)
00086    * - FS_SEL : 0..3 => ±2/±4/±8/±16 g (g2=0, g4=1, g8=2, g16=3)
00087    * - dlpf_enable: true to use DLPF path (recommended)
00088    * - dec3 : 0..3 (DEC3_CFG averaging)
00089    *   0 = ACCEL_DEC3_AVG_1_OR_4 (depends on FCHOICE)
00090    *   1 = ACCEL_DEC3_AVG_8
00091    *   2 = ACCEL_DEC3_AVG_16
00092    *   3 = ACCEL_DEC3_AVG_32
00093    * - stX/Y/Z : enable self-test (false here)
00094    */
00095   if (!imu.AccelConfigure(ACCEL_DLPFCFG_3, /*FS_SEL*/ g4,
00096     /*dlpf_enable*/ true, // DLPF really ON
00097     /*dec3*/ ACCEL_DEC3_AVG_8,
00098     /*stX*/ false, /*stY*/ false, /*stZ*/ false)) {
00099     Serial.println(F("AccelConfigure failed."));
00100   }
00101
00102   /* Set accelerometer output data rate (ODR)
00103    * - Accel_SMPLRT(rate_hz)
00104    * - Base (DLPF path) = 1125 Hz
00105    * - Valid range: 1-1125 Hz
00106    * - Example: 225 Hz
00107    */
00108   if (!imu.Accel_SMPLRT(225)) {
00109     Serial.println(F("Accel_SMPLRT failed."));
00110   }
00111 }
00112
00113 /**
00114  * @brief Read and print accelerometer data in g.
00115  */
00116 void loop() {
00117   float ax, ay, az; // accel X/Y/Z in g
00118
00119   /* - readAccel(x,y,z)
00120    * - Returns:
00121    *   - true on success;
00122    *   - Output:
00123    *     - ax/ay/az in g
00124    */
00125   if (imu.readAccel(ax, ay, az)) {
00126     Serial.print("ACCEL [g]: ");
00127     Serial.print(ax, 3);
00128     Serial.print(", ");
00129     Serial.print(ay, 3);
00130     Serial.print(", ");
00131     Serial.println(az, 3);
00132   } else {
00133     Serial.println(F("ACCEL read failed"));
00134   }
00135 }

```

```

00134 }
00135
00136 Serial.println(F("-----"));
00137 delay(500);
00138 }

```

## 6.17 examples/spi\_basic/spi\_basic.ino File Reference

```
#include <DevLab_ICM20948.h>
```

### Macros

- `#define SPI_FAST_SPEED 1000000`  
SPI clock used during sensor initialization and reads.

### Functions

- SPIClass `spi_bus` (SPI)  
SPI bus object used by the IMU driver.
- void `setup` ()  
Initialize Serial and start the ICM-20948 over SPI.
- void `loop` ()  
Read and print accelerometer, gyroscope, and temperature data.

### Variables

- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.

## 6.17.1 Macro Definition Documentation

### 6.17.1.1 SPI\_FAST\_SPEED

```
#define SPI_FAST_SPEED 1000000
```

SPI clock used during sensor initialization and reads.

Definition at line 34 of file `spi_basic.ino`.

## 6.17.2 Function Documentation

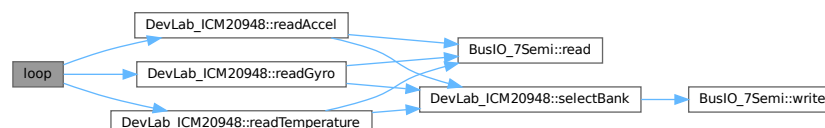
### 6.17.2.1 loop()

```
void loop ( )
```

Read and print accelerometer, gyroscope, and temperature data.

Definition at line 65 of file `spi_basic.ino`.

Here is the call graph for this function:



## 6.17.2.2 setup()

```
void setup ( )
```

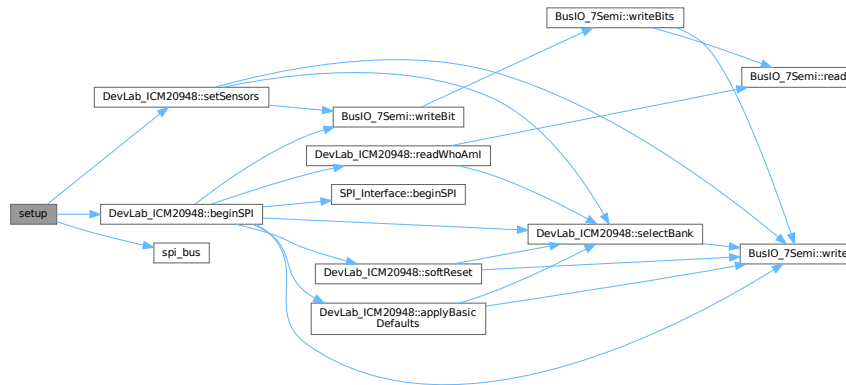
Initialize Serial and start the ICM-20948 over SPI.

Note

The sketch expects a board-level CS\_PIN definition to select the chip-select pin used by beginSPI().

Definition at line 48 of file [spi\\_basic.ino](#).

Here is the call graph for this function:



## 6.17.2.3 spi\_bus()

```
SPIClass spi_bus (
    SPI )
```

SPI bus object used by the IMU driver.

Here is the caller graph for this function:



## 6.17.3 Variable Documentation

## 6.17.3.1 imu

```
DevLab_ICM20948 imu
```

Global ICM-20948 driver instance.

Definition at line 37 of file [spi\\_basic.ino](#).

## 6.18 spi\_basic.ino

[Go to the documentation of this file.](#)

```
00001 /*****
00002  * @file    SPI_Basic.ino
00003  * @brief   Minimal SPI bring-up for 7Semi ICM-20948 +
00004  *          basic Accel/Gyro/Temp readout (updated API).
00005  */
```

```

00005  *
00006  * Features
00007  * - SPI init (UNO / ESP32)
00008  * - beginSPI() initialization
00009  * - Manual sensor enable
00010  * - Read Accel / Gyro / Temp
00011  *
00012  * Notes
00013  * - WHO_AM_I must be 0xEA
00014  * - Sensors must be explicitly enabled
00015  * - SPI speed ~1MHz recommended during init
00016  *
00017  * Wiring (Arduino UNO SPI)
00018  * - SCK -> D13
00019  * - MOSI -> D11
00020  * - MISO -> D12
00021  * - CS -> D10 (changeable; update CS_PIN)
00022  * - VCC -> 3V3
00023  * - GND -> GND
00024  *
00025  * Wiring (ESP32 VSPI default)
00026  * - SCK -> D13
00027  * - MOSI -> D11
00028  * - MISO -> D12
00029  * - CS -> D10
00030  *****/
00031 #include <DevLab_ICM20948.h>
00032
00033 /** @brief SPI clock used during sensor initialization and reads. */
00034 #define SPI_FAST_SPEED 1000000
00035
00036 /** @brief Global ICM-20948 driver instance. */
00037 DevLab_ICM20948 imu;
00038
00039 /** @brief SPI bus object used by the IMU driver. */
00040 SPIClass spi_bus(SPI);
00041
00042 /**
00043  * @brief Initialize Serial and start the ICM-20948 over SPI.
00044  *
00045  * @note The sketch expects a board-level CS_PIN definition to select the
00046  * chip-select pin used by beginSPI().
00047  */
00048 void setup() {
00049     Serial.begin(115200);
00050     delay(200);
00051
00052
00053     if (!imu.beginSPI(CS_PIN, spi_bus, 1000000)) {
00054         Serial.println("ERROR: beginSPI() failed");
00055         while (1) delay(200);
00056     }
00057
00058     imu.setSensors(true, true, true);
00059     Serial.println("Ready.");
00060 }
00061
00062 /**
00063  * @brief Read and print accelerometer, gyroscope, and temperature data.
00064  */
00065 void loop() {
00066     float ax, ay, az;
00067     float gx, gy, gz;
00068     float tC;
00069
00070     /* Read accelerometer. */
00071     if (imu.readAccel(ax, ay, az)) {
00072         Serial.print(F("ACC [g]: "));
00073         Serial.print(ax, 3); Serial.print(", ");
00074         Serial.print(ay, 3); Serial.print(", ");
00075         Serial.println(az, 3);
00076     } else {
00077         Serial.println(F("ACC read failed"));
00078     }
00079
00080     /* Read gyroscope. */
00081     if (imu.readGyro(gx, gy, gz)) {
00082         Serial.print(F("GYR [dps]: "));
00083         Serial.print(gx, 2); Serial.print(", ");
00084         Serial.print(gy, 2); Serial.print(", ");
00085         Serial.println(gz, 2);
00086     } else {
00087         Serial.println(F("GYR read failed"));
00088     }
00089
00090     /* Read temperature. */
00091     if (imu.readTemperature(tC)) {

```



```

00092     Serial.print(F("TMP [C]: "));
00093     Serial.println(tc, 2);
00094   } else {
00095     Serial.println(F("TMP read failed"));
00096   }
00097
00098   Serial.println(F("-----"));
00099   delay(500);
00100 }

```

## 6.19 examples/spi\_gyro/spi\_gyro.ino File Reference

`#include <DevLab_ICM20948.h>`

### Macros

- `#define SPI_FAST_SPEED 1000000`  
SPI clock used during sensor initialization and reads.

### Functions

- SPIClass `spi_bus` (SPI)  
SPI bus object used by the IMU driver.
- void `setup` ()  
Configure the IMU for gyroscope-only operation over SPI.
- void `loop` ()  
Read and print gyroscope data in degrees per second.

### Variables

- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.

## 6.19.1 Macro Definition Documentation

### 6.19.1.1 SPI\_FAST\_SPEED

```
#define SPI_FAST_SPEED 1000000
```

SPI clock used during sensor initialization and reads.

Definition at line 33 of file `spi_gyro.ino`.

## 6.19.2 Function Documentation

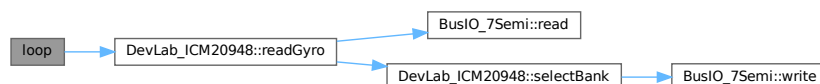
### 6.19.2.1 loop()

```
void loop ( )
```

Read and print gyroscope data in degrees per second.

Definition at line 117 of file `spi_gyro.ino`.

Here is the call graph for this function:





```

00008 * - IMU begin() on SPI (CS pin user-defined)
00009 * - Safe defaults (applyBasicDefaults)
00010 * - Optional sensor gating: gyro only
00011 * - Gyro config: DLPF, full-scale, ODR, AVG
00012 * - Read gyro (dps) at ~2 Hz print
00013 *
00014 * Wiring (Arduino UNO)
00015 * - SCK -> GPIO13
00016 * - MOSI(SDA) -> GPIO11
00017 * - MISO(SDO) -> GPIO12
00018 * - CS -> GPIO10 (changeable)
00019 * - VCC -> 3V3
00020 * - GND -> GND
00021 * Wiring (ESP32 VSPI default)
00022 * - SCK -> GPIO18
00023 * - MOSI -> GPIO23
00024 * - MISO -> GPIO19
00025 * - CS -> GPIO5 (changeable)
00026 * - VCC -> 3V3
00027 * - GND -> GND
00028 *****/
00029
00030 #include <DevLab_ICM20948.h>
00031
00032 /** @brief SPI clock used during sensor initialization and reads. */
00033 #define SPI_FAST_SPEED 1000000
00034
00035 /** @brief Global ICM-20948 driver instance. */
00036 DevLab_ICM20948 imu;
00037
00038 /** @brief SPI bus object used by the IMU driver. */
00039 SPIClass spi_bus(SPI);
00040
00041 /**
00042  * @brief Configure the IMU for gyroscope-only operation over SPI.
00043  *
00044  * @note The sketch expects a board-level CS_PIN definition to select the
00045  * chip-select pin used by beginSPI().
00046  */
00047 void setup() {
00048     Serial.begin(115200);
00049     delay(200);
00050     Serial.println(F("ICM-20948 (SPI) - Gyro Example"));
00051
00052     /* Init SPI bus on your pins. */
00053     // SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, CS_PIN);
00054
00055     if (!imu.beginSPI(CS_PIN, spi_bus, 1000000)) {
00056         Serial.println("ERROR: beginSPI() failed");
00057         while (1) delay(200);
00058     }
00059
00060     /* Apply safe defaults. */
00061     if (!imu.applyBasicDefaults()) {
00062         Serial.println(F("ERROR: applyBasicDefaults() failed."));
00063         while (1) delay(200);
00064     }
00065
00066     Serial.println(F("ICM-20948 ready."));
00067
00068     /* Optional: gate sensors
00069     * - setSensors(accel_on, gyro_on, temp_on)
00070     * - Here: enable only gyro (accel/temp off)
00071     */
00072     if (!imu.setSensors(/*accel*/ false, /*gyro*/ true, /*temp*/ false)) {
00073         Serial.println(F("setSensors failed."));
00074     }
00075
00076     /* ----- GYRO DLPF Config (reference) -----
00077     * GyroConfig(DLPFCFG, FS_SEL, FCHOICE, XGYRO, YGYRO, ZGYRO, AVGCFG)
00078     * - DLPF : GYRO_DLPFCFG_0..7 (use 3 for balanced setting)
00079     * - FS_SEL : dps250 / dps500 / dps1000 / dps2000
00080     * - FCHOICE : false -> DLPF path (recommended), true -> bypass (very wide BW)
00081     * - XGYRO/YGYRO/ZGYRO : per-axis enable (true = enable axis)
00082     * - AVGCFG : 0..7 internal averaging (higher = more smoothing, more delay)
00083     * FCHOICE=0 (DLPF on):
00084     * - GYRO_DLPFCFG_0 : 3dB ≈ 12106 Hz, NBW ≈ 12316 Hz (very wide)
00085     * - GYRO_DLPFCFG_1 : 3dB ≈ 196.6 Hz, NBW ≈ 229.8 Hz
00086     * - GYRO_DLPFCFG_2 : 3dB ≈ 151.8 Hz, NBW ≈ 187.6 Hz
00087     * - GYRO_DLPFCFG_3 : 3dB ≈ 119.5 Hz, NBW ≈ 154.3 Hz ← good default
00088     * - GYRO_DLPFCFG_4 : 3dB ≈ 51.2 Hz, NBW ≈ 73.3 Hz
00089     * - GYRO_DLPFCFG_5 : 3dB ≈ 23.9 Hz, NBW ≈ 35.9 Hz
00090     * - GYRO_DLPFCFG_6 : 3dB ≈ 5.7 Hz, NBW ≈ 8.3 Hz
00091     * - GYRO_DLPFCFG_7 : 3dB ≈ 473 Hz, NBW ≈ 499 Hz
00092     *
00093     * FCHOICE=1 (Bypass) : very wide (use with care; highest noise)
00094     */

```

```

00095  if (!imu.GyroConfig(/*DLPFCFG*/ GYRO_DLPFCFG_4,
00096                      /*FS_SEL*/ dps2000,
00097                      /*FCHOICE*/ true, // DLPF ON
00098                      /*X*/ true, /*Y*/ true, /*Z*/ true,
00099                      /*AVGCFG*/ 0)) {
00100      Serial.println(F("GyroConfig failed."));
00101  }
00102
00103  /* Set gyroscope output data rate (ODR)
00104   * - Gyro_SMPLRT(rate_hz)
00105   * - Base (DLPF path) = 1100 Hz
00106   * - Valid range: ~4.3-1100 Hz
00107   * - Example: 1000 Hz
00108   */
00109  if (!imu.Gyro_SMPLRT(1000)) {
00110      Serial.println(F("Gyro_SMPLRT failed."));
00111  }
00112 }
00113
00114 /**
00115  * @brief Read and print gyroscope data in degrees per second.
00116  */
00117 void loop() {
00118     float gx, gy, gz; // gyro X/Y/Z in dps
00119
00120     /* - readGyro(x,y,z)
00121      * - Returns:
00122      * - true on success;
00123      * - Output:
00124      * - gx/gy/gz in dps
00125      */
00126     if (imu.readGyro(gx, gy, gz)) {
00127         Serial.print("GYRO [dps]: ");
00128         Serial.print(gx, 3);
00129         Serial.print(", ");
00130         Serial.print(gy, 3);
00131         Serial.print(", ");
00132         Serial.println(gz, 3);
00133     } else {
00134         Serial.println(F("GYRO read failed"));
00135     }
00136 }
00137
00138 Serial.println(F("-----"));
00139 delay(500);
00140 }

```

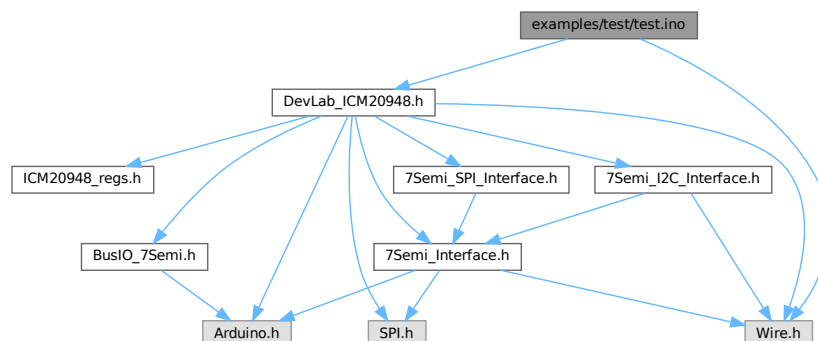
## 6.21 examples/test/test.ino File Reference

Accelerometer DLPF validation sketch for the 7Semi ICM-20948.

```
#include <Wire.h>
```

```
#include <DevLab_ICM20948.h>
```

Include dependency graph for test.ino:



### Classes

- struct [configDLPF](#)

Gyroscope DLPF divider and timing configuration.

## Macros

- `#define SDA_PIN 6`  
I2C SDA pin used by the example board.
- `#define SCL_PIN 7`  
I2C SCL pin used by the example board.
- `#define I2C_FREQ 400000UL`  
I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.

## Functions

- void `printLinea ()`  
Print a separator line to Serial.
- void `printDobleLinea ()`  
Print a double separator line to Serial.
- bool `applySettings (uint8_t fs_idx, const configDLPF &cfg)`  
Apply one accelerometer full-scale and DLPF configuration.
- void `firstStage ()`  
Verify the IMU identity register.
- void `runSweep ()`  
Run the accelerometer DLPF validation sweep and print samples.
- void `setup ()`  
Initialize I2C, reset/wake the IMU, enable sensors, and start sweep.
- void `loop ()`  
Optional post-sweep loop for manual accelerometer reads.

## Variables

- const `configDLPF configsDLPF []`  
Accelerometer DLPF configurations exercised by this validation.
- const `uint8_t N_DLPF = sizeof(configsDLPF) / sizeof(configsDLPF[0])`  
Number of accelerometer DLPF configurations.
- const `ICM20948_Accel_FullScale accelCfg [] = { ICM20948_Accel_FullScale::G_2, ICM20948_Accel_FullScale::G_4, ICM20948_Accel_FullScale::G_8, ICM20948_Accel_FullScale::G_16 }`  
Accelerometer full-scale ranges available for validation.
- const `char * etiquetasAccelCfg [] = { "+2G", "+4G", "+8G", "+16G" }`  
Text labels matching accelCfg.
- const `uint8_t N_FS = 4`  
Number of accelerometer full-scale ranges.
- const `ICM20948_Accel_DLPFCFG accelDLPF [] = { ICM20948_Accel_DLPFCFG::DLPF0_246HZ, ICM20948_Accel_DLPFCFG::DLPF_246HZ, ICM20948_Accel_DLPFCFG::DLPF_111HZ, ICM20948_Accel_DLPFCFG::DLPF_50HZ, ICM20948_Accel_DLPFCFG::DLPF_24HZ, ICM20948_Accel_DLPFCFG::DLPF_12HZ, ICM20948_Accel_DLPFCFG::DLPF_6HZ, ICM20948_Accel_DLPFCFG::DLPF_3.5HZ }`  
Accelerometer DLPF enum values matching configsDLPF.
- const `char * etiquetasAccelDLPFCFG [] = { "246/265", "246/265", "111/136", "50.4/68.8", "23.4/34.4", "11.5/17.0", "5.7/8.3", "473/499" }`  
Text labels describing accelDLPF bandwidth pairs.
- const `ICM20948_Accel_Average accelAVG [] = { ICM20948_Accel_Average::AVG_1_OR_4, ICM20948_Accel_Average::AVG_8, ICM20948_Accel_Average::AVG_16, ICM20948_Accel_Average::AVG_32 }`

- Accelerometer averaging settings available for validation.
- `const char * etiquetasAccelAVG [] = {"1 OR 4", "8", "16", "32"}`  
Text labels matching accelAVG.
- `const uint16_t accelSR [] = {0, 1, 3, 5, 7, 10, 15, 22, 31, 63, 127, 255, 513, 1022, 2044, 4095}`  
Raw accelerometer sample-rate divider values for experiments.
- `const char * etiquetasSR [] = { "1125.0", "562.5", "281.3", "187.5", "140.6", "102.3", "70.3", "48.9", "35.2", "17.6", "8.8", "4.4", "2.2", "1.1", "0.55", "0.27" }`  
Text labels matching accelSR output data rates.
- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.
- `uint8_t total_pass = 0`  
Count of validation passes reserved for future checks.
- `uint8_t total_fail = 0`  
Count of validation failures reserved for future checks.
- `uint8_t total_warn = 0`  
Count of validation warnings reserved for future checks.

### 6.21.1 Detailed Description

Accelerometer DLFP validation sketch for the 7Semi ICM-20948.

Runs an I2C accelerometer sweep across DLFP settings, prints sample data to Serial, and is intended for bench validation with the sensor flat and stationary.

Definition in file [test.ino](#).

### 6.21.2 Macro Definition Documentation

#### 6.21.2.1 I2C\_FREQ

```
#define I2C_FREQ 400000UL
```

I2C bus speed in hertz.

Definition at line 19 of file [test.ino](#).

#### 6.21.2.2 ICM\_ADDR

```
#define ICM_ADDR 0x69
```

ICM-20948 I2C address selected by the AD0 pin.

Definition at line 25 of file [test.ino](#).

#### 6.21.2.3 SCL\_PIN

```
#define SCL_PIN 7
```

I2C SCL pin used by the example board.

Definition at line 17 of file [test.ino](#).

#### 6.21.2.4 SDA\_PIN

```
#define SDA_PIN 6
```

I2C SDA pin used by the example board.

Definition at line 15 of file [test.ino](#).

### 6.21.3 Function Documentation

#### 6.21.3.1 applySettings()

```
bool applySettings (
    uint8_t fs_idx,
    const configDLFP & cfg )
```

Apply one accelerometer full-scale and DLFP configuration.

## Parameters

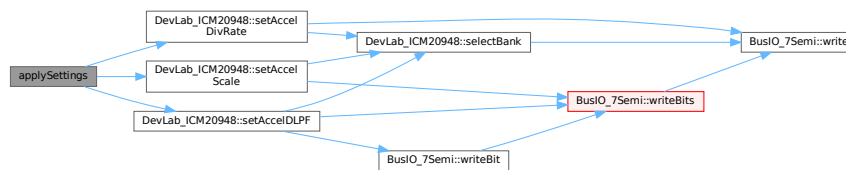
|        |                                        |
|--------|----------------------------------------|
| fs_idx | Index into accelCfg.                   |
| cfg    | DLPF timing and divider configuration. |

## Returns

true if all configuration writes succeeded, otherwise false.

Definition at line 114 of file [test.ino](#).

Here is the call graph for this function:



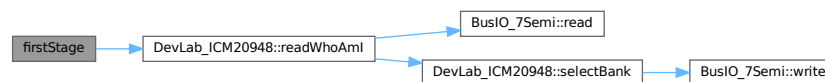
## 6.21.3.2 firstStage()

```
void firstStage ( )
```

Verify the IMU identity register.

Definition at line 131 of file [test.ino](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 6.21.3.3 loop()

```
void loop ( )
```

Optional post-sweep loop for manual accelerometer reads.

Definition at line 225 of file [test.ino](#).

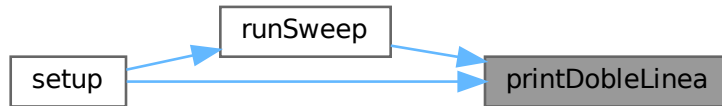
#### 6.21.3.4 printDobleLinea()

```
void printDobleLinea ( )
```

Print a double separator line to Serial.

Definition at line 103 of file [test.ino](#).

Here is the caller graph for this function:



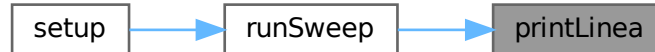
#### 6.21.3.5 printLinea()

```
void printLinea ( )
```

Print a separator line to Serial.

Definition at line 98 of file [test.ino](#).

Here is the caller graph for this function:



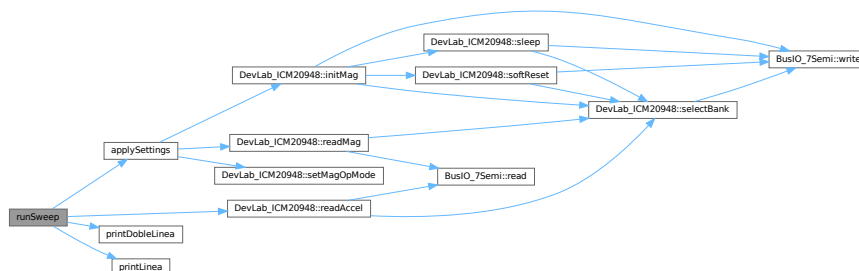
#### 6.21.3.6 runSweep()

```
void runSweep ( )
```

Run the accelerometer DLPF validation sweep and print samples.

Definition at line 144 of file [test.ino](#).

Here is the call graph for this function:





Here is the caller graph for this function:



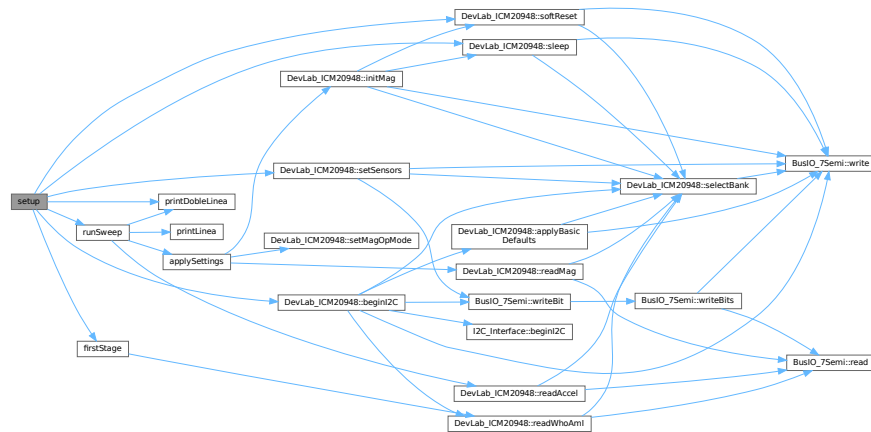
### 6.21.3.7 setup()

`void setup ( )`

Initialize I2C, reset/wake the IMU, enable sensors, and start sweep.

Definition at line 186 of file [test.ino](#).

Here is the call graph for this function:



## 6.21.4 Variable Documentation

### 6.21.4.1 accelAVG

```
const ICM20948_Accel_Average accelAVG[] = {ICM20948_Accel_Average::AVG_1_OR_4 , ICM20948_Accel_Average::AVG_8,
ICM20948_Accel_Average::AVG_16, ICM20948_Accel_Average::AVG_32}
```

Accelerometer averaging settings available for validation.

Definition at line 72 of file [test.ino](#).

### 6.21.4.2 accelCfg

```
const ICM20948_Accel_FullScale accelCfg[] = { ICM20948_Accel_FullScale::G_2, ICM20948_Accel_FullScale::G_4,
ICM20948_Accel_FullScale::G_8, ICM20948_Accel_FullScale::G_16}
```

Accelerometer full-scale ranges available for validation.

Definition at line 60 of file [test.ino](#).

### 6.21.4.3 accelDLPF

```
const ICM20948_Accel_DLPFCFG accelDLPF[] = { ICM20948_Accel_DLPFCFG::DLPF0_246HZ, ICM20948_Accel_DLPFCFG::DLPF1_111HZ,
ICM20948_Accel_DLPFCFG::DLPF2_111HZ, ICM20948_Accel_DLPFCFG::DLPF3_50HZ, ICM20948_Accel_DLPFCFG::DLPF4_24HZ,
ICM20948_Accel_DLPFCFG::DLPF5_12HZ, ICM20948_Accel_DLPFCFG::DLPF6_6HZ, ICM20948_Accel_DLPFCFG::DLPF7_473HZ
}
```

Accelerometer DLPF enum values matching configsDLPF.

Definition at line 67 of file [test.ino](#).

#### 6.21.4.4 accelSR

```
const uint16_t accelSR[] = {0, 1, 3, 5, 7, 10, 15, 22, 31, 63, 127, 255, 513, 1022, 2044, 4095}
```

Raw accelerometer sample-rate divider values for experiments.

Definition at line 77 of file [test.ino](#).

#### 6.21.4.5 configsDLPF

```
const configDLPF configsDLPF[]
```

Initial value:

```
= {
    { 0, 0, 265.0f, 1125.0f, "DLPF0_246 | NBW=265Hz | ODR=1125Hz (4.2x)" },
    { 1, 0, 265.0f, 1125.0f, "DLPF_246 | NBW=265Hz | ODR=1125Hz (4.2x)" },
    { 2, 1, 136.0f, 562.5f, "DLPF_111 | NBW=136Hz | ODR= 562Hz (4.1x)" },
    { 3, 3, 68.8f, 281.3f, "DLPF_50 | NBW= 69Hz | ODR= 281Hz (4.1x)" },
    { 4, 7, 34.4f, 140.6f, "DLPF_24 | NBW= 34Hz | ODR= 141Hz (4.1x)" },
    { 5, 15, 17.0f, 70.3f, "DLPF_12 | NBW= 17Hz | ODR= 70Hz (4.1x)" },
    { 6, 31, 8.3f, 35.2f, "DLPF_6 | NBW= 8Hz | ODR= 35Hz (4.2x)" },
    { 7, 0, 499.0f, 1125.0f, "DLPF_473 | NBW=499Hz | ODR=1125Hz (2.3x)" },
}
```

Accelerometer DLPF configurations exercised by this validation.

Definition at line 42 of file [test.ino](#).

#### 6.21.4.6 etiquetasAccelAVG

```
const char* etiquetasAccelAVG[] = {"1 OR 4", "8", "16", "32"}
```

Text labels matching accelAVG.

Definition at line 74 of file [test.ino](#).

#### 6.21.4.7 etiquetasAccelCfg

```
const char* etiquetasAccelCfg[] = {"+-2G", "+-4G", "+-8G", "+-16G"}
```

Text labels matching accelCfg.

Definition at line 62 of file [test.ino](#).

#### 6.21.4.8 etiquetasAccelDLPFCFG

```
const char* etiquetasAccelDLPFCFG[] = { "246/265", "246/265", "111/136", "50.4/68.8", "23.↵
9/34.4", "11.5/17.0", "5.7/8.3", "473/499"}
```

Text labels describing accelDLPF bandwidth pairs.

Definition at line 69 of file [test.ino](#).

#### 6.21.4.9 etiquetasSR

```
const char* etiquetasSR[] = { "1125.0", "562.5", "281.3", "187.5", "140.6", "102.3", "70.3",
"48.9", "35.2", "17.6", "8.8", "4.4", "2.2", "1.1", "0.55", "0.27"}
```

Text labels matching accelSR output data rates.

Definition at line 79 of file [test.ino](#).

#### 6.21.4.10 imu

[DevLab\\_ICM20948](#) imu

Global ICM-20948 driver instance.

Definition at line 84 of file [test.ino](#).

#### 6.21.4.11 N\_DLPF

```
const uint8_t N_DLPF = sizeof(configsDLPF) / sizeof(configsDLPF[0])
```

Number of accelerometer DLPF configurations.

Definition at line 57 of file [test.ino](#).

## 6.21.4.12 N\_FS

```
const uint8_t N_FS = 4
```

Number of accelerometer full-scale ranges.

Definition at line 64 of file [test.ino](#).

## 6.21.4.13 total\_fail

```
uint8_t total_fail = 0
```

Count of validation failures reserved for future checks.

Definition at line 89 of file [test.ino](#).

## 6.21.4.14 total\_pass

```
uint8_t total_pass = 0
```

Count of validation passes reserved for future checks.

Definition at line 87 of file [test.ino](#).

## 6.21.4.15 total\_warn

```
uint8_t total_warn = 0
```

Count of validation warnings reserved for future checks.

Definition at line 91 of file [test.ino](#).

## 6.22 test.ino

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file test.ino
00003  * @brief Accelerometer DLPF validation sketch for the 7Semi ICM-20948.
00004  * @details Runs an I2C accelerometer sweep across DLPF settings, prints sample
00005  * data to Serial, and is intended for bench validation with the sensor flat and
00006  * stationary.
00007  */
00008
00009 #include <Wire.h>
00010 #include <DevLab_ICM20948.h>
00011 // -----
00012 // PINES I2C - ESP32C6 NANO Unit Electronics
00013 // -----
00014 /** @brief I2C SDA pin used by the example board. */
00015 #define SDA_PIN 6
00016 /** @brief I2C SCL pin used by the example board. */
00017 #define SCL_PIN 7
00018 /** @brief I2C bus speed in hertz. */
00019 #define I2C_FREQ 400000UL // Fast Mode 400 kHz
00020
00021 // -----
00022 // DIRECCIONES I2C
00023 // -----
00024 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00025 #define ICM_ADDR 0x69 // AD0 = LOW (GND)
00026
00027 /** @brief Accelerometer DLPF divider and timing configuration. */
00028 struct configDLPF {
00029     /** @brief Index into the accelDLPF lookup table. */
00030     uint8_t dlpf_idx;
00031     /** @brief Sample-rate divider written to the IMU register. */
00032     uint16_t div; // el valor REAL del registro, 0-4095, sin ambigüedad
00033     /** @brief Noise bandwidth in hertz for reporting. */
00034     float nbw_hz;
00035     /** @brief Output data rate in hertz for timing calculations. */
00036     float odr_hz; // = 1125.0f / (1 + div), para calcular tiempos
00037     /** @brief Text label printed with each captured sample. */
00038     const char* nombre;
00039 };
00040
00041 /** @brief Accelerometer DLPF configurations exercised by this validation. */
00042 const configDLPF configsDLPF[] = {
00043     // idx div NBW(Hz) ODR(Hz) nombre
00044     { 0, 0, 265.0f, 1125.0f, "DLPF0_246 | NBW=265Hz | ODR=1125Hz (4.2x)" },
00045     { 1, 0, 265.0f, 1125.0f, "DLPF_246 | NBW=265Hz | ODR=1125Hz (4.2x)" },
00046     { 2, 1, 136.0f, 562.5f, "DLPF_111 | NBW=136Hz | ODR= 562Hz (4.1x)" },
00047     { 3, 3, 68.8f, 281.3f, "DLPF_50 | NBW= 69Hz | ODR= 281Hz (4.1x)" },
00048     { 4, 7, 34.4f, 140.6f, "DLPF_24 | NBW= 34Hz | ODR= 141Hz (4.1x)" },
00049 }
```

```

00049 { 5, 15, 17.0f, 70.3f, "DLPF_12 | NBW= 17Hz | ODR= 70Hz (4.1x)" },
00050 { 6, 31, 8.3f, 35.2f, "DLPF_6 | NBW= 8Hz | ODR= 35Hz (4.2x)" },
00051 { 7, 0, 499.0f, 1125.0f, "DLPF_473 | NBW=499Hz | ODR=1125Hz (2.3x)" },
00052 };
00053 //-----
00054 // ARRAYS DE CONFIGURACIÓN
00055 // -----
00056 /** @brief Number of accelerometer DLPF configurations. */
00057 const uint8_t N_DLPF = sizeof(configsDLPF) / sizeof(configsDLPF[0]);
00058
00059 /** @brief Accelerometer full-scale ranges available for validation. */
00060 const ICM20948_Accel_FullScale accelCf[] = { ICM20948_Accel_FullScale::G_2,
ICM20948_Accel_FullScale::G_4, ICM20948_Accel_FullScale::G_8, ICM20948_Accel_FullScale::G_16};
00061 /** @brief Text labels matching accelCf. */
00062 const char* etiquetasAccelCf[] = {"+-2G", "+-4G", "+-8G", "+-16G"};
00063 /** @brief Number of accelerometer full-scale ranges. */
00064 const uint8_t N_FS = 4;
00065
00066 /** @brief Accelerometer DLPF enum values matching configsDLPF. */
00067 const ICM20948_Accel_DLPFCFG accelDLPF[] = { ICM20948_Accel_DLPFCFG::DLPF0_246HZ,
ICM20948_Accel_DLPFCFG::DLPF_246HZ, ICM20948_Accel_DLPFCFG::DLPF_111HZ,
ICM20948_Accel_DLPFCFG::DLPF_50HZ, ICM20948_Accel_DLPFCFG::DLPF_24HZ,
ICM20948_Accel_DLPFCFG::DLPF_12HZ, ICM20948_Accel_DLPFCFG::DLPF_6HZ, ICM20948_Accel_DLPFCFG::DLPF_473HZ
};
00068 /** @brief Text labels describing accelDLPF bandwidth pairs. */
00069 const char* etiquetasAccelDLPFCFG[] = { "246/265", "246/265", "111/136", "50.4/68.8", "23.9/34.4",
"11.5/17.0", "5.7/8.3", "473/499"};
00070
00071 /** @brief Accelerometer averaging settings available for validation. */
00072 const ICM20948_Accel_Average accelAVG[] = {ICM20948_Accel_Average::AVG_1_OR_4
,ICM20948_Accel_Average::AVG_8, ICM20948_Accel_Average::AVG_16, ICM20948_Accel_Average::AVG_32};
00073 /** @brief Text labels matching accelAVG. */
00074 const char* etiquetasAccelAVG[] = {"1 OR 4", "8", "16", "32"};
00075
00076 /** @brief Raw accelerometer sample-rate divider values for experiments. */
00077 const uint16_t accelSR[] = {0, 1, 3, 5, 7, 10, 15, 22, 31, 63, 127, 255, 513, 1022, 2044, 4095};
00078 /** @brief Text labels matching accelSR output data rates. */
00079 const char* etiquetasSR[] = { "1125.0", "562.5", "281.3", "187.5", "140.6", "102.3", "70.3", "48.9",
"35.2", "17.6", "8.8", "4.4", "2.2", "1.1", "0.55", "0.27"};
00080 // -----
00081 // INSTANCIA DEL SENSOR
00082 // -----
00083 /** @brief Global ICM-20948 driver instance. */
00084 DevLab_ICM20948 imu;
00085
00086 /** @brief Count of validation passes reserved for future checks. */
00087 uint8_t total_pass = 0;
00088 /** @brief Count of validation failures reserved for future checks. */
00089 uint8_t total_fail = 0;
00090 /** @brief Count of validation warnings reserved for future checks. */
00091 uint8_t total_warn = 0;
00092
00093 // =====
00094 // UTILIDADES DE IMPRESIÓN
00095 // =====
00096
00097 /** @brief Print a separator line to Serial. */
00098 void printLinea() {
00099     Serial.println(F("-----"));
00100 }
00101
00102 /** @brief Print a double separator line to Serial. */
00103 void printDobleLinea() {
00104     Serial.println(F("====="));
00105 }
00106
00107 /**
00108  * @brief Apply one accelerometer full-scale and DLPF configuration.
00109  *
00110  * @param fs_idx Index into accelCf.
00111  * @param cfg DLPF timing and divider configuration.
00112  * @return true if all configuration writes succeeded, otherwise false.
00113  */
00114 bool applySettings(uint8_t fs_idx, const configDLPF &cfg){
00115     uint8_t dlpf_val = (uint8_t)accelDLPF[cfg.dlpf_idx];
00116     bool ok = true;
00117
00118     ok &= imu.setAccelScale(accelCf[fs_idx]);
00119
00120     ok &= imu.setAccelDLPF(dlpf_val, false);
00121
00122     ok &= imu.setAccelDivRate(cfg.div);
00123
00124     return ok;
00125 }
00126
00127

```

```

00128 /**
00129  * @brief Verify the IMU identity register.
00130  */
00131 void firstStage(){
00132     uint8_t who;
00133     if (!imu.readWhoAmI(who) || who != 0xEA) {
00134         Serial.println(F("ERROR: WHO_AM_I mismatch"));
00135         while (1) delay(200);
00136     }
00137     Serial.print(F("WHO_AM_I = 0x"));
00138     Serial.println(who, HEX);
00139 }
00140
00141 /**
00142  * @brief Run the accelerometer DLPF validation sweep and print samples.
00143  */
00144 void runSweep(){
00145     printDobleLinea();
00146     Serial.println("Acelerometro ICM-200948 | UNIT Electronics");
00147     Serial.printf("%d DLPFS x %d FS = %d combinaciones\n", N_DLPF, N_FS, N_DLPF * N_FS);
00148     Serial.println(F("Coloque el sensor plano y quieto mirando hacia arriba"));
00149     printDobleLinea();
00150     for(uint8_t d = 0; d < N_DLPF; d++){
00151         Serial.printf(" BLOQUE %d/%d: %s\n", d+1, N_DLPF, configsDLPF[d].nombre);
00152         Serial.printf(" ODR = %.1fHz | Delay = %lums | Periodo=
00153         %.1fms\n", configsDLPF[d].odr_hz, max((uint32_t)(10000.0f/configsDLPF[d].odr_hz), (uint32_t)50),
00154         1000.0f/configsDLPF[d].odr_hz);
00155         printDobleLinea();
00156         bool cfg_ok = applySettings(configsDLPF[d].dlpf_idx, configsDLPF[d]);
00157         if (!cfg_ok) {
00158             Serial.println(F(" ERROR: applySettings() fallo"));
00159             continue;
00160         }
00161         // Esperar que el filtro estabilice (10 periodos del ODR, min 50ms)
00162         uint32_t settle_ms = max((uint32_t)(10000.0f / configsDLPF[d].odr_hz),
00163         (uint32_t)50);
00164         delay(settle_ms);
00165
00166         // Tomar 3 lecturas espaciadas por el periodo del ODR
00167         uint32_t periodo_ms = max((uint32_t)(1000.0f / configsDLPF[d].odr_hz),
00168         (uint32_t)1);
00169         float ax, ay, az;
00170         for (uint8_t r = 0; r < 3; r++) {
00171             delay(periodo_ms);
00172             if (imu.readAccel(ax, ay, az)) {
00173                 Serial.printf(" [%d] Ax=%.3f Ay=%.3f Az=%.3f g\n",
00174                 r+1, ax, ay, az);
00175             } else {
00176                 Serial.println(F(" [%?] readAccel() fallo"));
00177             }
00178         }
00179         printLinea();
00180     }
00181 }
00182
00183 /**
00184  * @brief Initialize I2C, reset/wake the IMU, enable sensors, and start sweep.
00185  */
00186 void setup() {
00187     Serial.begin(115200);
00188     delay(200);
00189     Serial.println(F("ICM-20948 - I2C Basic"));
00190     printDobleLinea();
00191     Serial.println(F(" ICM-20948 VALIDACION ACELEROMETRO"));
00192     printDobleLinea();
00193     // I2C init (UNO/ESP32 defaults). For ESP32 custom pins: Wire.begin(SDA,SCL);
00194     Wire.begin(SDA_PIN, SCL_PIN);
00195     // Attach IMU
00196     if (!imu.beginI2C(ICM_ADDR, Wire, I2C_FREQ)) {
00197         Serial.println(F("ERROR: begin(I2C) failed"));
00198         while (1) delay(200);
00199     }
00200 }
00201
00202 firstStage();
00203
00204 imu.softReset();
00205
00206 imu.sleep(false);
00207 delay(50);
00208 /* Enable all sensors. */
00209 if (!imu.setSensors(true, true, true)) {
00210     Serial.println(F("ERROR: setSensors failed"));
00211 }
00212 Serial.println(F(" Sensor listo.\n"));
00213 Serial.println(F(" Sensor PLANO y QUIETO sobre la mesa."));

```

```

00214 Serial.println(F(" Iniciando en 5 segundos..."));
00215 for (int i = 5; i > 0; i--) {
00216     Serial.printf(" %d...\n", i);
00217     delay(1000);
00218 }
00219 runSweep();

00220 }
00221
00222 /**
00223  * @brief Optional post-sweep loop for manual accelerometer reads.
00224  */
00225 void loop() {
00226     /*float ax, ay, az;
00227     if (imu.readAccel(ax, ay, az)) {
00228         Serial.printf("X:%.3f Y:%.3f Z:%.3f\n", ax, ay, az);
00229     }
00230     delay(200);*/
00231 }

```

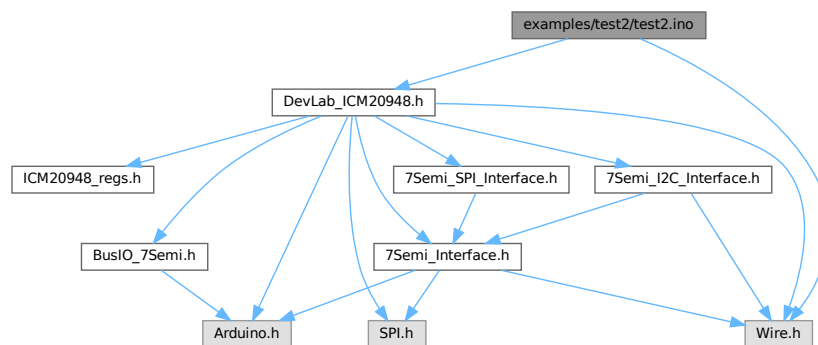
## 6.23 examples/test2/test2.ino File Reference

Extended accelerometer DLPF, full-scale, and averaging sweep.

`#include <Wire.h>`

`#include <DevLab_ICM20948.h>`

Include dependency graph for test2.ino:



### Classes

- struct `configDLPF`  
Gyroscope DLPF divider and timing configuration.

### Macros

- `#define SDA_PIN 6`  
I2C SDA pin used by the example board.
- `#define SCL_PIN 7`  
I2C SCL pin used by the example board.
- `#define I2C_FREQ 400000UL`  
I2C bus speed in hertz.
- `#define ICM_ADDR 0x69`  
ICM-20948 I2C address selected by the AD0 pin.

### Functions

- void `printLinea ()`  
Print a separator line to Serial.

- void `printDobleLinea` ()  
Print a double separator line to Serial.
- void `firstStage` ()  
Verify the IMU identity register.
- bool `applySettings` (uint8\_t fs\_idx, const `configDLPF` &cfg, uint8\_t avg)  
Apply one accelerometer full-scale, DLPF, and averaging configuration.
- void `runSweep` ()  
Run the full accelerometer validation matrix and print samples.
- void `setup` ()  
Initialize I2C, reset/wake the IMU, enable sensors, and start sweep.
- void `loop` ()  
Optional post-sweep loop for manual accelerometer reads.

#### Variables

- const `configDLPF configsDLPF` []  
Accelerometer DLPF configurations exercised by this sweep.
- const uint8\_t `N_DLPF` = sizeof(`configsDLPF`) / sizeof(`configsDLPF`[0])  
Number of accelerometer DLPF configurations.
- const `ICM20948_Accel_FullScale accelCfg` []  
Accelerometer full-scale ranges exercised by the sweep.
- const char \* `etiquetasAccelCfg` [] = { "+-2G", "+-4G", "+-8G", "+-16G" }  
Text labels matching accelCfg.
- const uint8\_t `N_FS` = 4  
Number of accelerometer full-scale ranges.
- const `ICM20948_Accel_DLPFCFG accelDLPF` []  
Accelerometer DLPF enum values matching configsDLPF.
- const `ICM20948_Accel_Average accelAVG` []  
Accelerometer averaging settings exercised by the sweep.
- const char \* `etiquetasAccelAVG` [] = { "14", "8", "16", "32" }  
Text labels matching accelAVG.
- const uint8\_t `N_AVG` = 4  
Number of accelerometer averaging settings.
- const uint16\_t `accelSR` []  
Raw accelerometer sample-rate divider values for experiments.
- const char \* `etiquetasSR` []  
Text labels matching accelSR output data rates.
- `DevLab_ICM20948 imu`  
Global ICM-20948 driver instance.
- uint8\_t `total_pass` = 0  
Count of validation passes reserved for future checks.
- uint8\_t `total_fail` = 0  
Count of validation failures reserved for future checks.
- uint8\_t `total_warn` = 0  
Count of validation warnings reserved for future checks.

#### 6.23.1 Detailed Description

Extended accelerometer DLPF, full-scale, and averaging sweep.

Runs a larger I2C validation matrix for the 7Semi ICM-20948 accelerometer and prints CSV-like samples for later analysis.

Definition in file `test2.ino`.

## 6.23.2 Macro Definition Documentation

### 6.23.2.1 I2C\_FREQ

```
#define I2C_FREQ 400000UL
```

I2C bus speed in hertz.

Definition at line 19 of file [test2.ino](#).

### 6.23.2.2 ICM\_ADDR

```
#define ICM_ADDR 0x69
```

ICM-20948 I2C address selected by the AD0 pin.

Definition at line 21 of file [test2.ino](#).

### 6.23.2.3 SCL\_PIN

```
#define SCL_PIN 7
```

I2C SCL pin used by the example board.

Definition at line 17 of file [test2.ino](#).

### 6.23.2.4 SDA\_PIN

```
#define SDA_PIN 6
```

I2C SDA pin used by the example board.

Definition at line 15 of file [test2.ino](#).

## 6.23.3 Function Documentation

### 6.23.3.1 applySettings()

```
bool applySettings (
    uint8_t fs_idx,
    const configDLPF & cfg,
    uint8_t avg )
```

Apply one accelerometer full-scale, DLPF, and averaging configuration.

Parameters

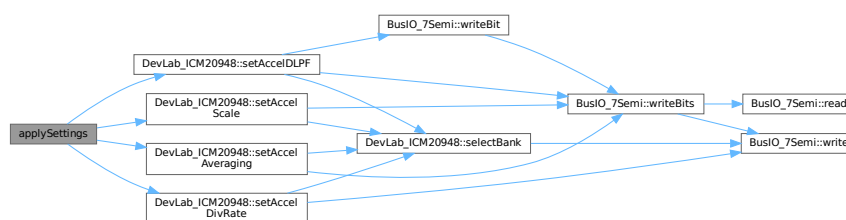
|        |                                        |
|--------|----------------------------------------|
| fs_idx | Index into accelCfg.                   |
| cfg    | DLPF timing and divider configuration. |
| avg    | Index into accelAVG.                   |

Returns

true if all configuration writes succeeded, otherwise false.

Definition at line 149 of file [test2.ino](#).

Here is the call graph for this function:





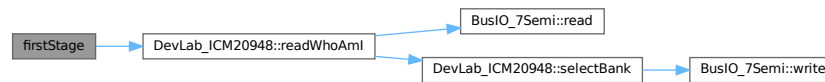
## 6.23.3.2 firstStage()

```
void firstStage ( )
```

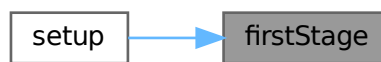
Verify the IMU identity register.

Definition at line 123 of file [test2.ino](#).

Here is the call graph for this function:



Here is the caller graph for this function:



## 6.23.3.3 loop()

```
void loop ( )
```

Optional post-sweep loop for manual accelerometer reads.

Definition at line 298 of file [test2.ino](#).

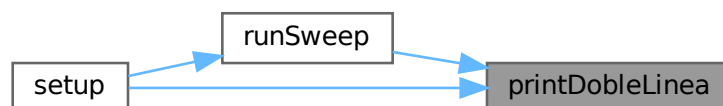
## 6.23.3.4 printDobleLinea()

```
void printDobleLinea ( )
```

Print a double separator line to Serial.

Definition at line 116 of file [test2.ino](#).

Here is the caller graph for this function:



## 6.23.3.5 printLinea()

```
void printLinea ( )
```

Print a separator line to Serial.

Definition at line 114 of file [test2.ino](#).

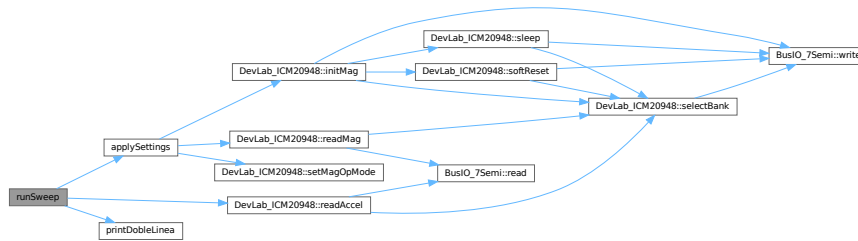
### 6.23.3.6 runSweep()

```
void runSweep ( )
```

Run the full accelerometer validation matrix and print samples.

Definition at line 183 of file [test2.ino](#).

Here is the call graph for this function:



Here is the caller graph for this function:



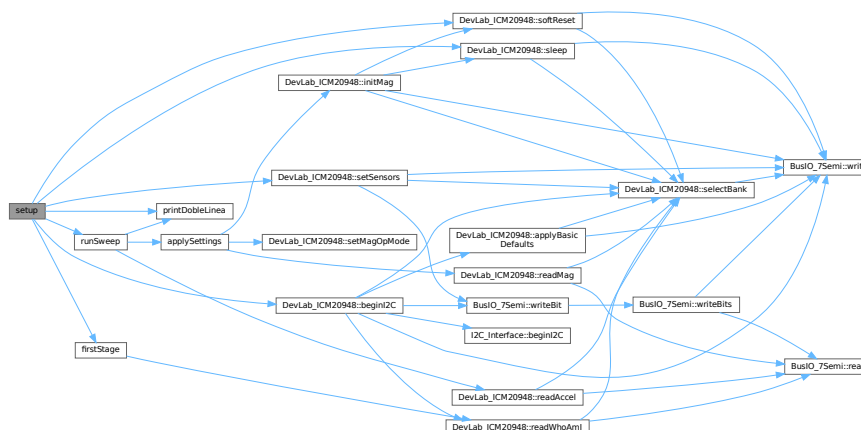
### 6.23.3.7 setup()

```
void setup ( )
```

Initialize I2C, reset/wake the IMU, enable sensors, and start sweep.

Definition at line 250 of file [test2.ino](#).

Here is the call graph for this function:



## 6.23.4 Variable Documentation

### 6.23.4.1 accelAVG

const [ICM20948\\_Accel\\_Average](#) accelAVG[ ]

Initial value:

```
= {
  ICM20948_Accel_Average::AVG_1_OR_4,
  ICM20948_Accel_Average::AVG_8,
  ICM20948_Accel_Average::AVG_16,
  ICM20948_Accel_Average::AVG_32
}
```

Accelerometer averaging settings exercised by the sweep.

Definition at line 82 of file [test2.ino](#).

### 6.23.4.2 accelCfg

const [ICM20948\\_Accel\\_FullScale](#) accelCfg[ ]

Initial value:

```
= {
  ICM20948_Accel_FullScale::G_2,
  ICM20948_Accel_FullScale::G_4,
  ICM20948_Accel_FullScale::G_8,
  ICM20948_Accel_FullScale::G_16
}
```

Accelerometer full-scale ranges exercised by the sweep.

Definition at line 58 of file [test2.ino](#).

### 6.23.4.3 accelDLPF

const [ICM20948\\_Accel\\_DLPFCFG](#) accelDLPF[ ]

Initial value:

```
= {
  ICM20948_Accel_DLPFCFG::DLPF0_246HZ,
  ICM20948_Accel_DLPFCFG::DLPF_246HZ,
  ICM20948_Accel_DLPFCFG::DLPF_111HZ,
  ICM20948_Accel_DLPFCFG::DLPF_50HZ,
  ICM20948_Accel_DLPFCFG::DLPF_24HZ,
  ICM20948_Accel_DLPFCFG::DLPF_12HZ,
  ICM20948_Accel_DLPFCFG::DLPF_6HZ,
  ICM20948_Accel_DLPFCFG::DLPF_473HZ
}
```

Accelerometer DLPF enum values matching configsDLPF.

Definition at line 70 of file [test2.ino](#).

### 6.23.4.4 accelSR

const uint16\_t accelSR[ ]

Initial value:

```
= {
  0, 1, 3, 5, 7, 10, 15, 22, 31, 63, 127, 255, 513, 1022, 2044, 4095
}
```

Raw accelerometer sample-rate divider values for experiments.

Definition at line 93 of file [test2.ino](#).

### 6.23.4.5 configsDLPF

const [configDLPF](#) configsDLPF[ ]

Initial value:

```
= {
  { 0, 0, 265.0f, 1125.0f, "246 ; 265 ; 1125 " },
  { 1, 0, 265.0f, 1125.0f, "246 ; 265 ; 1125 " },
  { 2, 1, 136.0f, 562.5f, "111 ; 136 ; 562 " },
  { 3, 2, 68.8f, 375.0f, "50 ; 69 ; 375 " },
  { 4, 7, 34.4f, 140.6f, "24 ; 34 ; 141 " },
  { 5, 15, 17.0f, 70.3f, "12 ; 17 ; 70 " },
  { 6, 29, 8.3f, 37.5f, "6 ; 8 ; 35 " },
  { 7, 0, 499.0f, 1125.0f, "473 ; 499 ; 1125 " },
}
```

Accelerometer DLPF configurations exercised by this sweep.

Definition at line 41 of file [test2.ino](#).

#### 6.23.4.6 etiquetasAccelAVG

```
const char* etiquetasAccelAVG[] = { "14", "8", "16", "32" }
```

Text labels matching accelAVG.

Definition at line 89 of file [test2.ino](#).

#### 6.23.4.7 etiquetasAccelCfg

```
const char* etiquetasAccelCfg[] = { "+-2G", "+-4G", "+-8G", "+-16G" }
```

Text labels matching accelCfg.

Definition at line 65 of file [test2.ino](#).

#### 6.23.4.8 etiquetasSR

```
const char* etiquetasSR[]
```

Initial value:

```
= {  
  "1125.0", "562.5", "281.3", "187.5", "140.6", "102.3", "70.3",  
  "48.9", "35.2", "17.6", "8.8", "4.4", "2.2", "1.1", "0.55", "0.27"  
}
```

Text labels matching accelSR output data rates.

Definition at line 97 of file [test2.ino](#).

#### 6.23.4.9 imu

```
DevLab_ICM20948 imu
```

Global ICM-20948 driver instance.

Definition at line 104 of file [test2.ino](#).

#### 6.23.4.10 N\_AVG

```
const uint8_t N_AVG = 4
```

Number of accelerometer averaging settings.

Definition at line 91 of file [test2.ino](#).

#### 6.23.4.11 N\_DLPF

```
const uint8_t N_DLPF = sizeof(configsDLPF) / sizeof(configsDLPF[0])
```

Number of accelerometer DLPF configurations.

Definition at line 52 of file [test2.ino](#).

#### 6.23.4.12 N\_FS

```
const uint8_t N_FS = 4
```

Number of accelerometer full-scale ranges.

Definition at line 67 of file [test2.ino](#).

#### 6.23.4.13 total\_fail

```
uint8_t total_fail = 0
```

Count of validation failures reserved for future checks.

Definition at line 108 of file [test2.ino](#).

#### 6.23.4.14 total\_pass

```
uint8_t total_pass = 0
```

Count of validation passes reserved for future checks.

Definition at line 106 of file [test2.ino](#).

## 6.23.4.15 total\_warn

```
uint8_t total_warn = 0
```

Count of validation warnings reserved for future checks.

Definition at line 110 of file [test2.ino](#).

## 6.24 test2.ino

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file test2.ino
00003  * @brief Extended accelerometer DLPF, full-scale, and averaging sweep.
00004  * @details Runs a larger I2C validation matrix for the 7Semi ICM-20948
00005  * accelerometer and prints CSV-like samples for later analysis.
00006  */
00007
00008 #include <Wire.h>
00009 #include <DevLab_ICM20948.h>
00010
00011 // -----
00012 // PINES I2C - ESP32C6 NANO Unit Electronics
00013 // -----
00014 /** @brief I2C SDA pin used by the example board. */
00015 #define SDA_PIN 6
00016 /** @brief I2C SCL pin used by the example board. */
00017 #define SCL_PIN 7
00018 /** @brief I2C bus speed in hertz. */
00019 #define I2C_FREQ 400000UL
00020 /** @brief ICM-20948 I2C address selected by the AD0 pin. */
00021 #define ICM_ADDR 0x69
00022
00023 // -----
00024 // STRUCT
00025 // -----
00026 /** @brief Accelerometer DLPF divider and timing configuration. */
00027 struct configDLPF {
00028     /** @brief Index into the accelDLPF lookup table. */
00029     uint8_t dlpf_idx;
00030     /** @brief Sample-rate divider written to the IMU register. */
00031     uint16_t div;
00032     /** @brief Noise bandwidth in hertz for reporting. */
00033     float nbw_hz;
00034     /** @brief Output data rate in hertz for timing calculations. */
00035     float odr_hz;
00036     /** @brief Text label printed with each captured sample. */
00037     const char* nombre;
00038 };
00039 //id,div,nbw_hz,odr_hz,nombre : {DLPF, NBW , ODR}
00040 /** @brief Accelerometer DLPF configurations exercised by this sweep. */
00041 const configDLPF configsDLPF[] = {
00042     { 0, 0, 265.0f, 1125.0f, "246 ; 265 ; 1125 " },
00043     { 1, 0, 265.0f, 1125.0f, "246 ; 265 ; 1125 " },
00044     { 2, 1, 136.0f, 562.5f, "111 ; 136 ; 562 " },
00045     { 3, 2, 68.8f, 375.0f, "50 ; 69 ; 375 " },
00046     { 4, 7, 34.4f, 140.6f, "24 ; 34 ; 141 " },
00047     { 5, 15, 17.0f, 70.3f, "12 ; 17 ; 70 " },
00048     { 6, 29, 8.3f, 37.5f, "6 ; 8 ; 35 " },
00049     { 7, 0, 499.0f, 1125.0f, "473 ; 499 ; 1125 " },
00050 };
00051 /** @brief Number of accelerometer DLPF configurations. */
00052 const uint8_t N_DLPF = sizeof(configsDLPF) / sizeof(configsDLPF[0]);
00053
00054 // -----
00055 // ARRAYS ORIGINALES
00056 // -----
00057 /** @brief Accelerometer full-scale ranges exercised by the sweep. */
00058 const ICM20948_Accel_FullScale accelCfg[] = {
00059     ICM20948_Accel_FullScale::G_2,
00060     ICM20948_Accel_FullScale::G_4,
00061     ICM20948_Accel_FullScale::G_8,
00062     ICM20948_Accel_FullScale::G_16
00063 };
00064 /** @brief Text labels matching accelCfg. */
00065 const char* etiquetasAccelCfg[] = { "+-2G", "+-4G", "+-8G", "+-16G " };
00066 /** @brief Number of accelerometer full-scale ranges. */
00067 const uint8_t N_FS = 4;
00068
00069 /** @brief Accelerometer DLPF enum values matching configsDLPF. */
00070 const ICM20948_Accel_DLPFCFG accelDLPF[] = {
00071     ICM20948_Accel_DLPFCFG::DLPF0_246HZ,
00072     ICM20948_Accel_DLPFCFG::DLPF_246HZ,
00073     ICM20948_Accel_DLPFCFG::DLPF_111HZ,
00074     ICM20948_Accel_DLPFCFG::DLPF_50HZ,
```

```

00075 ICM20948_Accel_DLPFCFG::DLPF_24HZ,
00076 ICM20948_Accel_DLPFCFG::DLPF_12HZ,
00077 ICM20948_Accel_DLPFCFG::DLPF_6HZ,
00078 ICM20948_Accel_DLPFCFG::DLPF_473HZ
00079 };
00080
00081 /** @brief Accelerometer averaging settings exercised by the sweep. */
00082 const ICM20948_Accel_Average accelAVG[] = {
00083     ICM20948_Accel_Average::AVG_1_OR_4,
00084     ICM20948_Accel_Average::AVG_8,
00085     ICM20948_Accel_Average::AVG_16,
00086     ICM20948_Accel_Average::AVG_32
00087 };
00088 /** @brief Text labels matching accelAVG. */
00089 const char* etiquetasAccelAVG[] = { "14", "8", "16", "32" };
00090 /** @brief Number of accelerometer averaging settings. */
00091 const uint8_t N_AVG = 4;
00092 /** @brief Raw accelerometer sample-rate divider values for experiments. */
00093 const uint16_t accelSR[] = {
00094     0, 1, 3, 5, 7, 10, 15, 22, 31, 63, 127, 255, 513, 1022, 2044, 4095
00095 };
00096 /** @brief Text labels matching accelSR output data rates. */
00097 const char* etiquetasSR[] = {
00098     "1125.0", "562.5", "281.3", "187.5", "140.6", "102.3", "70.3",
00099     "48.9", "35.2", "17.6", "8.8", "4.4", "2.2", "1.1", "0.55", "0.27"
00100 };
00101
00102 // -----
00103 /** @brief Global ICM-20948 driver instance. */
00104 DevLab_ICM20948 imu;
00105 /** @brief Count of validation passes reserved for future checks. */
00106 uint8_t total_pass = 0;
00107 /** @brief Count of validation failures reserved for future checks. */
00108 uint8_t total_fail = 0;
00109 /** @brief Count of validation warnings reserved for future checks. */
00110 uint8_t total_warn = 0;
00111
00112 /** @brief Print a separator line to Serial. */
00113 void printLinea() {
00114     Serial.println(F("-----"));
00115 }
00116 /** @brief Print a double separator line to Serial. */
00117 void printDobleLinea() {
00118     Serial.println(F("====="));
00119 }
00120 // -----
00121 // WHO_AM_I
00122 // -----
00123 /**
00124  * @brief Verify the IMU identity register.
00125  */
00126 void firstStage() {
00127     uint8_t who;
00128     if (!imu.readWhoAmI(who) || who != 0xEA) {
00129         Serial.println(F("ERROR: WHO_AM_I mismatch"));
00130         while (1) delay(200);
00131     }
00132     Serial.printf("WHO_AM_I = 0x%02X OK\n", who);
00133 }
00134 // -----
00135 // APLICAR CONFIGURACION
00136 // -----
00137 /**
00138  * @brief Apply one accelerometer full-scale, DLPF, and averaging configuration.
00139  *
00140  * @param fs_idx Index into accelCfg.
00141  * @param cfg DLPF timing and divider configuration.
00142  * @param avg Index into accelAVG.
00143  * @return true if all configuration writes succeeded, otherwise false.
00144  */
00145 bool applySettings(uint8_t fs_idx, const configDLPF &cfg, uint8_t avg) {
00146     uint8_t dlpf_val = (uint8_t)accelDLPF[cfg.dlpf_idx];
00147     bool ok = true;
00148
00149     ok &= imu.setAccelScale(accelCfg[fs_idx]);
00150
00151     // Paso 1: escribe DLPFCFG en bits [5:3]
00152     ok &= imu.setAccelDLPF(dlpf_val, false);
00153
00154     ok &= imu.setAccelAveraging(accelAVG[avg]);

```

```

00160 // Paso 2: activa FCHOICE=1 (DLPF activo) sin tocar DLPFCFG
00161 //ok &= imu.setAccelDLPF(0, true);
00162
00163 // DIV directo al registro, sin conversion interna
00164 ok &= imu.setAccelDivRate(cfg.div);
00165
00166 return ok;
00167 }
00168
00169 // -----
00170 // SWEEP - MODULO VERIFICACION
00171 //
00172 // Por cada configuracion DLPF:
00173 // 1. Aplica la config (FS fijo en +-2g para verificacion)
00174 // 2. Espera settle del filtro
00175 // 3. Imprime 3 lecturas reales
00176 //
00177 // Cuando las lecturas muestren Az ~1g en reposo,
00178 // el sensor esta funcionando y se puede activar el sweep completo.
00179 // -----
00180 /**
00181  * @brief Run the full accelerometer validation matrix and print samples.
00182  */
00183 void runSweep() {
00184
00185     uint16_t n = 0;
00186     printDobleLinea();
00187     Serial.println(F(" ICM-20948 | UNIT Electronics"));
00188     Serial.println(F(" MODULO VERIFICACION - FS=+-2g fijo, 3 lecturas por config"));
00189     Serial.println(F(" Sensor PLANO y QUIETO | Az esperado: ~+1.000g"));
00190     printDobleLinea();
00191     for(uint8_t i = 0; i < N_AVG; i++){
00192         for(uint8_t j = 0; j < N_FS;j++){
00193             for (uint8_t d = 0; d < N_DLPF; d++) {
00194
00195                 //Serial.println();
00196                 //printLinea();
00197                 /*Serial.printf(" Config %d/%d: %s\n", d+1, N_DLPF, configsDLPF[d].nombre);
00198                 Serial.printf(" ODR=%.1fHz | Periodo=%.1fms\n",
00199                     configsDLPF[d].odr_hz,
00200                     1000.0f / configsDLPF[d].odr_hz);*/
00201
00202                 // --- FIX: aplicar la configuracion antes de leer ---
00203                 bool cfg_ok = applySettings(j, configsDLPF[d],i); // FS=0 -> +-2g
00204                 if (!cfg_ok) {
00205                     Serial.println(F(" ERROR: applySettings() fallo"));
00206                     continue;
00207                 }
00208
00209                 // Esperar que el filtro estabilice (10 periodos del ODR, min 50ms)
00210                 uint32_t settle_ms = max((uint32_t)(1000.0f / configsDLPF[d].odr_hz),
00211                     (uint32_t)50);
00212                 delay(settle_ms);
00213
00214                 // Tomar 3 lecturas espaciadas por el periodo del ODR
00215                 uint32_t periodo_ms = max((uint32_t)(1000.0f / configsDLPF[d].odr_hz),
00216                     (uint32_t)1);
00217                 float ax, ay, az;
00218                 for (uint8_t r = 0; r < 100; r++) {
00219                     n++;
00220                     delay(periodo_ms);
00221                     if (imu.readAccel(ax, ay, az)) {
00222                         Serial.printf("%d ; %s ; %s ; %s ; %.4f ; %.4f ; %.4f\n",
00223                             n,
00224                             configsDLPF[d].nombre, // "246 ; 265 ; 1125"
00225                             etiquetasAccelCfg[i], // "+-2g"
00226                             etiquetasAccelAVG[j], // "8"
00227                             ax, ay, az);
00228                     } else {
00229                         Serial.println(F(" [?] readAccel() fallo"));
00230                     }
00231                 }
00232             }
00233         }
00234     }
00235     Serial.println();
00236     printDobleLinea();
00237     Serial.println(F(" VERIFICACION COMPLETA"));
00238     Serial.println(F(" Si Az ~ +1.0g en todos los bloques:"));
00239     Serial.println(F(" el sensor funciona correctamente.));
00240     Serial.println(F(" Activar sweep completo cuando este listo.));
00241     printDobleLinea();
00242 }
00243
00244 // -----
00245 // SETUP
00246 // -----

```

```

00247 /**
00248  * @brief Initialize I2C, reset/wake the IMU, enable sensors, and start sweep.
00249  */
00250 void setup() {
00251     Serial.begin(115200);
00252     delay(200);
00253     Serial.println(F("ICM-20948 - Verificacion"));
00254
00255     printDobleLinea();
00256     Serial.println(F("   ICM-20948 VALIDACION ACELEROMETRO"));
00257     printDobleLinea();
00258
00259     Wire.begin(SDA_PIN, SCL_PIN);
00260
00261     if (!imu.beginI2C(ICM_ADDR, Wire, I2C_FREQ)) {
00262         Serial.println(F("ERROR: beginI2C() fallo"));
00263         while (1) delay(200);
00264     }
00265
00266     firstStage();
00267
00268     // softReset pone el chip a dormir
00269     // Despues del reset hay que despertar el dispositivo
00270     imu.softReset();
00271     delay(100);
00272     imu.sleep(false); // limpia SLEEP bit, CLKSEL=1
00273     delay(50);
00274
00275     if (!imu.setSensors(true, true, true)) {
00276         Serial.println(F("ERROR: setSensors() fallo"));
00277     }
00278
00279     Serial.println(F("  Sensor listo.\n"));
00280     Serial.println(F("  Sensor PLANO y QUIETO sobre la mesa.));
00281     Serial.println(F("  Iniciando en 5 segundos..."));
00282     for (int i = 5; i > 0; i--) {
00283         Serial.printf("  %d...\n", i);
00284         delay(1000);
00285     }
00286
00287     //for (int runs = 0; runs < 20 ; runs++){
00288         runSweep();
00289     //}
00290 }
00291
00292 // -----
00293 // LOOP - lectura simple post-sweep
00294 // -----
00295 /**
00296  * @brief Optional post-sweep loop for manual accelerometer reads.
00297  */
00298 void loop() {
00299     /*float ax, ay, az;
00300     if (imu.readAccel(ax, ay, az)) {
00301         Serial.printf("X:%.3f  Y:%.3f  Z:%.3f\n", ax, ay, az);
00302     }
00303     delay(500);*/
00304 }

```

## 6.25 README.md File Reference

## 6.26 src/7Semi\_I2C\_Interface.h File Reference

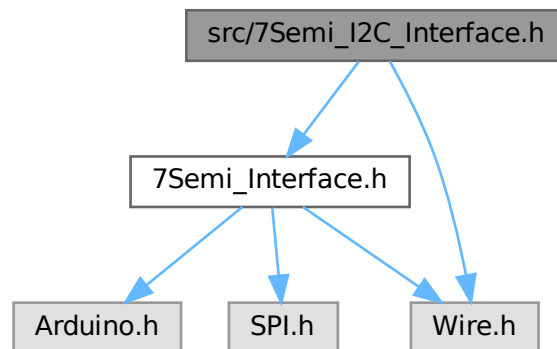
```

#include "7Semi_Interface.h"
#include <Wire.h>

```



Include dependency graph for 7Semi\_I2C\_Interface.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [I2C\\_Interface](#)

## 6.27 7Semi\_I2C\_Interface.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "7Semi_Interface.h"
00003 #include <Wire.h>
00004
00005 class I2C_Interface : public Interface_7Semi
00006 {
00007 public:
00008     TwoWire *i2c = nullptr;
00009     uint8_t address = 0;
00010
00011     bool beginI2C(uint8_t addr, TwoWire &wire, uint32_t speed,
00012                 uint8_t sda = 255, uint8_t scl = 255)
00013     {
00014         address = addr;
00015         i2c = &wire;
00016
00017         #if defined(ESP32)
00018             if (sda != 255 && scl != 255)
00019                 i2c->begin(sda, scl);
00020             else
00021                 i2c->begin();
00022         #else
00023             i2c->begin();
00024         #endif
00025         // i2c->setClock(speed);
00026
00027         // Probe device
00028         i2c->beginTransaction(address);
00029         return (i2c->endTransmission() == 0);
00030     }
00031
00032     int8_t read(uint8_t reg, uint8_t *data, uint32_t len) override
00033     {
00034         if (!i2c)
  
```

```

00035         return -1;
00036         i2c->beginTransaction(address);
00037         i2c->write(reg );
00038
00039         if (i2c->endTransmission(false) != 0)
00040             return -1;
00041
00042         delay(1);
00043
00044         if (len > 255)
00045             return -3;
00046         uint8_t received = i2c->requestFrom(address, (uint8_t)len);
00047         if (received != len)
00048             return -2;
00049
00050         for (uint32_t i = 0; i < len; i++)
00051         {
00052             if (!i2c->available())
00053                 return -4;
00054             data[i] = i2c->read();
00055         }
00056         // Serial.print("R-Reg: ");
00057         // Serial.print(reg, HEX);
00058         // Serial.print(" |Data: ");
00059         // for (int i = 0; i < len; i++)
00060         // {
00061         //     Serial.print(" ");
00062         //     Serial.print(data[i], HEX);
00063         // }
00064         // Serial.println();
00065
00066         return 0;
00067     }
00068
00069     bool beginSPI(uint8_t, SPIClass &, uint32_t,
00070                  uint8_t, uint8_t, uint8_t) override
00071     {
00072         return false;
00073     }
00074
00075     int8_t write(uint8_t reg, const uint8_t *data, uint32_t len) override
00076     {
00077         if (!i2c)
00078             return -1;
00079
00080         i2c->beginTransaction(address);
00081         i2c->write(reg);
00082         for (uint32_t i = 0; i < len; i++)
00083             i2c->write(data[i]);
00084
00085         uint8_t status = i2c->endTransmission();
00086
00087         // Serial.print("W-Reg: ");
00088         // Serial.print(reg, HEX);
00089         // Serial.print(" |Data: ");
00090         // for (int i = 0; i < len; i++)
00091         // {
00092         //     Serial.print(" ");
00093         //     Serial.print(data[i], HEX);
00094         // }
00095         // Serial.println();
00096         return (status == 0) ? 0 : -status;
00097     }
00098 };

```

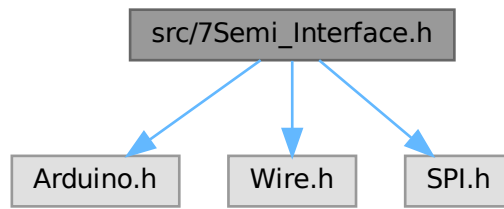
## 6.28 src/7Semi\_Interface.h File Reference

```

#include <Arduino.h>
#include <Wire.h>
#include <SPI.h>

```

Include dependency graph for 7Semi\_Interface.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Interface\\_7Semi](#)

## 6.29 7Semi\_Interface.h

[Go to the documentation of this file.](#)

```

00001 // #pragma once
00002 #ifndef INTERFACE_7SEMI_H
00003 #define INTERFACE_7SEMI_H
00004
00005 #include <Arduino.h>
00006 #include <Wire.h>
00007 #include <SPI.h>
00008
00009 /**
00010  * 7Semi Universal Interface Layer
00011  *
00012  * - Abstract communication layer for I2C / SPI
00013  * - Ensures sensor drivers remain hardware independent
00014  */
00015 class Interface_7Semi {
00016 public:
00017     virtual ~Interface_7Semi() {}
00018
00019     /**
00020      * beginI2C()
00021      *
00022      * - Initializes I2C peripheral
00023      * - Configures SDA / SCL pins if supported
00024      * - Sets communication clock
00025      *
00026      * Returns:
00027      * - true → Initialization successful
00028      * - false → Initialization failed
00029      */
00030     virtual bool beginI2C(
00031         uint8_t address,
00032         TwoWire &wire,
00033         uint32_t speed,
00034         uint8_t sda = 255,
00035         uint8_t scl = 255) = 0;
00036
00037     virtual bool beginSPI(
00038         uint8_t cs_pin,
  
```

```

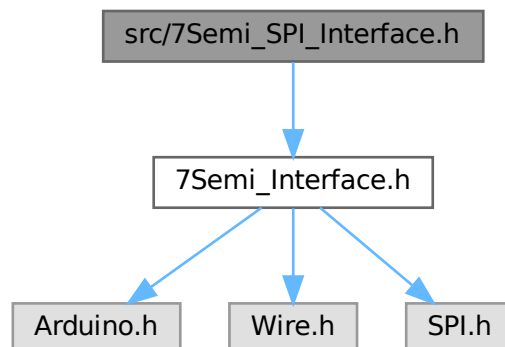
00040     SPIClass &wire,
00041     uint32_t speed,
00042     uint8_t csk = 255,
00043     uint8_t miso = 255,
00044     uint8_t mosi = 255) = 0;
00045
00046
00047     /**
00048     * read()
00049     *
00050     * - Reads data from device register
00051     *
00052     * Parameters:
00053     * - reg : Register address
00054     * - data : Output buffer
00055     * - len : Number of bytes
00056     *
00057     * Returns:
00058     * - 0 → Success
00059     * - <0 → Error
00060     */
00061     virtual int8_t read(
00062         uint8_t reg,
00063         uint8_t *data,
00064         uint32_t len) = 0;
00065
00066     /**
00067     * write()
00068     *
00069     * - Writes data to device register
00070     *
00071     * Returns:
00072     * - 0 → Success
00073     * - <0 → Error
00074     */
00075     virtual int8_t write(
00076         uint8_t reg,
00077         const uint8_t *data,
00078         uint32_t len) = 0;
00079
00080 protected:
00081     /** Delay wrapper */
00082     static void delay_us(uint32_t us)
00083     {
00084         delayMicroseconds(us);
00085     }
00086 };
00087 #endif

```

### 6.30 src/7Semi\_SPI\_Interface.h File Reference

#include "7Semi\_Interface.h"

Include dependency graph for 7Semi\_SPI\_Interface.h:



```

graph TD
    Root["c07Tern_05_10Feb13"] --> Node1["c07CenLab_C020848.1"]
    Node1 --> Leaf1["examplengr1_06Feb13"]
    Node1 --> Leaf2["examplengr2_06Feb13"]
    Node1 --> Leaf3["examplengr3_06Feb13"]
    Node1 --> Leaf4["examplengr4_06Feb13"]
    Node1 --> Leaf5["examplengr5_06Feb13"]
    Node1 --> Leaf6["examplengr6_06Feb13"]
    Node1 --> Leaf7["examplengr7_06Feb13"]
    Node1 --> Leaf8["examplengr8_06Feb13"]
    Node1 --> Leaf9["examplengr9_06Feb13"]
    Node1 --> Leaf10["examplengr10_06Feb13"]
    Node1 --> Leaf11["examplengr11_06Feb13"]
    Node1 --> Leaf12["examplengr12_06Feb13"]
  
```

- class `SPI_Interface`

[Go to the documentation of this file.](#)

Generated on Tue Jun 16 2026 17:30:42 for DevLab ICM20948 by Doxygen

```

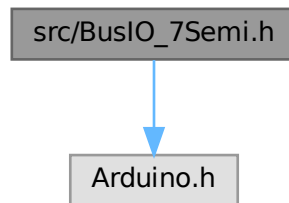
00067     SPISettings settings(speed, MSBFIRST, SPI_MODE0);
00068
00069     spi->beginTransaction(settings);
00070     digitalWrite(cs, LOW);
00071     delayMicroseconds(10);
00072
00073     /**
00074      * Send register address (read)
00075      */
00076     spi->transfer(reg | 0x80);
00077
00078     delayMicroseconds(10);
00079
00080     /**
00081      * Read data
00082      */
00083     for (uint32_t i = 0; i < len; i++)
00084         data[i] = spi->transfer(0x00);
00085
00086     digitalWrite(cs, HIGH);
00087     delayMicroseconds(10);
00088     spi->endTransaction();
00089
00090     // Serial.print("R-Reg: ");
00091     // Serial.print(reg, HEX);
00092     // Serial.print(" |Data: ");
00093     // for (int i = 0; i < len; i++)
00094     // {
00095     //     Serial.print(" ");
00096     //     Serial.print(data[i], HEX);
00097     // }
00098     // Serial.println();
00099
00100     return 0;
00101 }
00102
00103 /**
00104  * write()
00105  *
00106  * - Write multiple bytes to register
00107  */
00108 int8_t write(uint8_t reg, const uint8_t *data, uint32_t len) override
00109 {
00110     if (!spi || !data || len == 0)
00111         return -1;
00112
00113     SPISettings settings(speed, MSBFIRST, SPI_MODE0);
00114
00115     spi->beginTransaction(settings);
00116     digitalWrite(cs, LOW);
00117     delayMicroseconds(10);
00118
00119     /**
00120      * Send register address (write)
00121      */
00122     spi->transfer(reg & 0x7F);
00123
00124     /**
00125      * Write data
00126      */
00127     for (uint32_t i = 0; i < len; i++)
00128         spi->transfer(data[i]);
00129
00130     digitalWrite(cs, HIGH);
00131     delayMicroseconds(10);
00132     spi->endTransaction();
00133
00134     // Serial.print("W-Reg: ");
00135     // Serial.print(reg, HEX);
00136     // Serial.print(" |Data: ");
00137     // for (int i = 0; i < len; i++)
00138     // {
00139     //     Serial.print(" ");
00140     //     Serial.print(data[i], HEX);
00141     // }
00142     // Serial.println();
00143
00144     return 0;
00145 }
00146 };

```

## 6.32 src/BusIO\_7Semi.h File Reference

```
#include <Arduino.h>
```

Include dependency graph for BusIO\_7Semi.h:



This graph shows which files directly or indirectly include this file:



## Classes

- class BusIO\_7Semi< Bus >

## 6.33 BusIO\_7Semi.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <Arduino.h>
00003
00004 template <typename Bus>
00005 class BusIO_7Semi
00006 {
00007
00008 public:
00009     /**
00010      * Constructor
00011      *
00012      * - Stores interface reference
00013      */
00014     BusIO_7Semi(Bus &busRef) : bus(busRef) {}
00015
00016     /** read 8-bit */
00017     inline bool read(uint8_t reg, uint8_t &value)
00018     {
00019         return (bus.read(reg, &value, 1) == 0);
00020     }
00021
00022     /** read 16-bit */
00023     inline bool read(uint8_t reg, uint16_t &value)
00024     {
00025         uint8_t data[2];
00026         if (bus.read(reg, data, 2) != 0)
00027             return false;
00028         value = (data[0] << 8) | data[1];
00029         return true;
00030     }
00031
00032     /** read burst */
00033     inline bool read(uint8_t reg, uint8_t *data, uint32_t len)
00034     {
00035         return (bus.read(reg, data, len) == 0);
00036     }

```

```

00037
00038 /** write 8-bit */
00039 inline bool write(uint8_t reg, uint8_t value)
00040 {
00041     return (bus.write(reg, &value, 1) == 0);
00042 }
00043
00044 /** write 16-bit */
00045 inline bool write(uint8_t reg, uint16_t value)
00046 {
00047     uint8_t data[2] = {(uint8_t)(value >> 8), (uint8_t)(value & 0xFF)};
00048     return (bus.write(reg, data, 2) == 0);
00049 }
00050
00051 /** write burst */
00052 inline bool write(uint8_t reg, const uint8_t *data, uint32_t len)
00053 {
00054     return (bus.write(reg, data, len) == 0);
00055 }
00056
00057 /**
00058  * readBits (8-bit register)
00059  */
00060 bool readBits(uint8_t reg, uint8_t pos, uint8_t len, uint8_t &value)
00061 {
00062     if (len == 0 || len > 8 || (pos + len) > 8)
00063         return false;
00064
00065     uint8_t reg_val;
00066     if (!read(reg, reg_val))
00067         return false;
00068
00069     uint8_t mask = ((uint8_t)1 << len) - 1;
00070
00071     value = (reg_val >> pos) & mask;
00072     return true;
00073 }
00074
00075 /**
00076  * readBits (16-bit register)
00077  */
00078 bool readBits(uint8_t reg, uint8_t pos, uint8_t len, uint16_t &value)
00079 {
00080     if (len == 0 || len > 16 || (pos + len) > 16)
00081         return false;
00082
00083     uint8_t reg_val;
00084     if (!read(reg, reg_val))
00085         return false;
00086
00087     uint16_t mask = ((uint16_t)1 << len) - 1;
00088
00089     value = (reg_val >> pos) & mask;
00090     return true;
00091 }
00092
00093 /**
00094  * writeBits (8-bit register)
00095  *
00096  * - Modifies specific bits in 8-bit register
00097  */
00098 bool writeBits(uint8_t reg, uint8_t pos, uint8_t len, uint8_t value)
00099 {
00100     if (len == 0 || len > 8 || (pos + len) > 8)
00101         return false;
00102
00103     uint8_t reg_val;
00104     if (!read(reg, reg_val))
00105         return false;
00106
00107     uint8_t mask = ((uint8_t)1 << len) - 1;
00108
00109     reg_val &= ~(mask << pos);
00110     reg_val |= (value & mask) << pos;
00111
00112     return write(reg, reg_val);
00113 }
00114
00115 /**
00116  * writeBits (16-bit register)
00117  *
00118  * - Modifies specific bits in 16-bit register
00119  */
00120 bool writeBits(uint8_t reg, uint8_t pos, uint8_t len, uint16_t value)
00121 {
00122     if (len == 0 || len > 16 || (pos + len) > 16)
00123         return false;

```



```

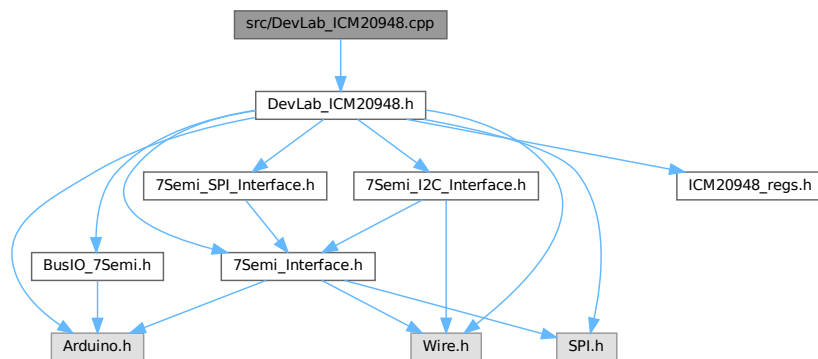
00124
00125     uint8_t reg_val;
00126     if (!read(reg, reg_val))
00127         return false;
00128
00129     uint16_t mask = ((uint16_t)1 << len) - 1;
00130
00131     reg_val &= ~(mask << pos);
00132     reg_val |= (value & mask) << pos;
00133
00134     return write(reg, reg_val);
00135 }
00136
00137 bool readBit(uint8_t reg, uint8_t pos, uint8_t &value)
00138 {
00139     return readBits(reg, pos, 1, value);
00140 }
00141
00142 bool readBit(uint8_t reg, uint8_t pos, uint16_t &value)
00143 {
00144     return readBits(reg, pos, 1, value);
00145 }
00146
00147 bool writeBit(uint8_t reg, uint8_t pos, uint8_t value)
00148 {
00149     return writeBits(reg, pos, 1, (uint8_t)value );
00150 }
00151 bool writeBit(uint8_t reg, uint8_t pos, uint16_t value)
00152 {
00153     return writeBits(reg, pos, 1, value);
00154 }
00155
00156 private:
00157     Bus &bus;
00158 };

```

## 6.34 src/DevLab\_ICM20948.cpp File Reference

#include "DevLab\_ICM20948.h"

Include dependency graph for DevLab\_ICM20948.cpp:



## 6.35 DevLab\_ICM20948.cpp

[Go to the documentation of this file.](#)

```

00001 #include "DevLab_ICM20948.h"
00002
00003 /** - Construct with default I2C address; call begin() to attach bus */
00004 DevLab_ICM20948::DevLab_ICM20948() {}
00005
00006 bool DevLab_ICM20948::beginI2C(uint8_t address, TwoWire &i2cPort, uint32_t i2cSpeed)
00007 {
00008     // Free previous BusIO instance
00009     if (bus)
00010     {
00011         delete bus;
00012         bus = nullptr;

```

```

00013     }
00014
00015     // Assign I2C interface implementation
00016     iface = &i2c;
00017
00018     // Initialize I2C (400kHz) and probe device
00019     if (!i2c.beginI2C(address, i2cPort, i2cSpeed))
00020         return false;
00021
00022     // Create BusIO abstraction layer
00023     bus = new BusIO_7Semi<Interface_7Semi>(*iface);
00024     if (!bus)
00025         return false;
00026
00027     // Allow device to stabilize after power-up
00028     delay(100);
00029
00030     // Read WHO_AM_I register
00031     uint8_t who_am_i;
00032     if (!readWhoAmI(who_am_i))
00033         return false;
00034
00035     // Validate device ID (expected 0xEA)
00036     if (who_am_i != WHO_AM_I_VAL)
00037         return false;
00038
00039     if (!selectBank(2))
00040         return false;
00041
00042     if (!bus->write(ODR_ALIGN_EN, (uint8_t)0x01))
00043         return false;
00044
00045     // Enable I2C interface (I2C_IF_DIS = 0)
00046     if (!bus->writeBit(USER_CTRL, 4, (uint8_t)0x00))
00047         return false;
00048
00049     // Set clock source to PLL (auto select best clock)
00050     if (!bus->write(PWR_MGMT_1, (uint8_t)0x01))
00051         return false;
00052
00053     if(!applyBasicDefaults())
00054         return false;
00055
00056     // Initialization successful
00057     return true;
00058 }
00059
00060 bool DevLab_ICM20948::beginSPI(uint8_t csPin, SPIClass &spiPort, uint32_t spiSpeed)
00061 {
00062     // Free previous BusIO instance
00063     if (bus)
00064     {
00065         delete bus;
00066         bus = nullptr;
00067     }
00068
00069     // Assign SPI interface implementation
00070     iface = &spi;
00071
00072     // Initialize SPI (1MHz) and probe device
00073     if (!spi.beginSPI(csPin, spiPort, spiSpeed))
00074         return false;
00075
00076     // Create BusIO abstraction layer
00077     bus = new BusIO_7Semi<Interface_7Semi>(*iface);
00078     if (!bus)
00079         return false;
00080
00081     softReset();
00082
00083     // Allow device to stabilize after power-up
00084     delay(100);
00085
00086     // Read WHO_AM_I register
00087     uint8_t who_am_i;
00088
00089     if (!readWhoAmI(who_am_i))
00090         return false;
00091
00092     // Validate device ID (expected 0xEA)
00093     if (who_am_i != WHO_AM_I_VAL)
00094         return false;
00095
00096     if (!selectBank(2))
00097         return false;
00098
00099

```

```

00100     if (!bus->write(ODR_ALIGN_EN, (uint8_t)0x01))
00101         return false;
00102
00103     // Enable I2C interface (I2C_IF_DIS = 0)
00104     if (!bus->writeBit(USER_CTRL, 4, (uint8_t)0x01))
00105         return false;
00106
00107     // Wake device from sleep mode (SLEEP = 0)
00108     if (!bus->write(PWR_MGMT_1, (uint8_t)0x01))
00109         return false;
00110
00111     if(!applyBasicDefaults())
00112         return false;
00113
00114     // Initialization successful
00115     return true;
00116 }
00117
00118 bool DevLab_ICM20948::readWhoAmI(uint8_t &whoAmI)
00119 {
00120     // Select USER BANK 0
00121     if (!selectBank(0))
00122         return false;
00123
00124     // Read WHO_AM_I register
00125     if (!bus->read(WHO_AM_I, whoAmI))
00126         return false;
00127
00128     // Success
00129     return true;
00130 }
00131
00132 bool DevLab_ICM20948::selectBank(uint8_t bank)
00133 {
00134     if (bank > 3)
00135         return false;
00136
00137     // Mask bank value to valid range (0-3) and shift to bits [5:4]
00138     uint8_t v = (bank & 0x03) << 4;
00139
00140     // Write bank selection to REG_BANK_SEL register
00141     return bus->write(REG_BANK_SEL, v);
00142 }
00143
00144 bool DevLab_ICM20948::softReset()
00145 {
00146     // Select USER BANK 0
00147     if (!selectBank(0))
00148         return false;
00149
00150     // Set DEVICE_RESET bit (bit 7)
00151     if (!bus->write(PWR_MGMT_1, (uint8_t)0x80))
00152         return false;
00153
00154     // Allow device to reset
00155     delay(100);
00156
00157     selectBank(2);
00158     bus->write(ODR_ALIGN_EN, (uint8_t)0x01);
00159
00160     // Success
00161     return true;
00162 }
00163
00164 bool DevLab_ICM20948::sleep(bool en)
00165 {
00166     // Select USER BANK 0
00167     if (!selectBank(0))
00168         return false;
00169     uint8_t v = 1;
00170     if(en)
00171         v|= 1<<6;
00172
00173     // Set or clear SLEEP bit (bit 6)
00174     if (!bus->write(PWR_MGMT_1, v))
00175         return false;
00176
00177     // Success
00178     return true;
00179 }
00180
00181 bool DevLab_ICM20948::applyBasicDefaults()
00182 {
00183     // Select USER BANK 0
00184     if (!selectBank(0))
00185         return false;
00186

```

```

00187 // Wake device and set clock source (CLKSEL=1, SLEEP=0)
00188 if (!bus->write(PWR_MGMT_1, (uint8_t)0x01))
00189     return false;
00190
00191 if (!bus->write(PWR_MGMT_2, (uint8_t)0x00))
00192     return false;
00193
00194 // Allow device to stabilize
00195 delay(10);
00196
00197 // Disable duty-cycling (LP mode off)
00198 if (!bus->write(LP_CONFIG, (uint8_t)0x00))
00199     return false;
00200
00201 // Select USER BANK 2
00202 if (!selectBank(2))
00203     return false;
00204
00205 // Configure gyro (DLPF enabled, FS = 2000 dps)
00206 if (!bus->write(GYRO_CONFIG_1, (uint8_t)0x1F))
00207     return false;
00208
00209 // Set gyro sample rate divider (SRD = 0 → max rate)
00210 if (!bus->write(GYRO_SMPLRT_DIV, (uint8_t)0x00))
00211     return false;
00212
00213 // Configure accel (DLPF enabled, FS = 16g)
00214 if (!bus->write(ACCEL_CONFIG, (uint8_t)0x1F))
00215     return false;
00216
00217 // Set accel sample rate divider high byte
00218 if (!bus->write(ACCEL_SMPLRT_DIV_1, (uint8_t)0x00))
00219     return false;
00220
00221 // Set accel sample rate divider low byte
00222 if (!bus->write(ACCEL_SMPLRT_DIV_2, (uint8_t)0x00))
00223     return false;
00224
00225 // Select USER BANK 0
00226 if (!selectBank(0))
00227     return false;
00228
00229 // Configure interrupt pin (open-drain, active-low, latch)
00230 if (!bus->write(INT_PIN_CFG, (uint8_t)0x30))
00231     return false;
00232
00233 // Configuration successful
00234 return true;
00235 }
00236 }
00237
00238 bool DevLab_ICM20948::readAccel(float &x, float &y, float &z)
00239 {
00240     // Select USER BANK 0
00241     if (!selectBank(0))
00242         return false;
00243
00244     // Read 6 bytes (X, Y, Z)
00245     uint8_t raw[6];
00246     if (!bus->read(ACCEL_XOUT_H, raw, 6))
00247         return false;
00248
00249     // Convert raw data to g
00250     x = (int16_t)((raw[0] << 8) | raw[1]) / mg_per_lsb;
00251     y = (int16_t)((raw[2] << 8) | raw[3]) / mg_per_lsb;
00252     z = (int16_t)((raw[4] << 8) | raw[5]) / mg_per_lsb;
00253
00254     // Success
00255     return true;
00256 }
00257
00258 bool DevLab_ICM20948::readGyro(float &x, float &y, float &z)
00259 {
00260     // Select USER BANK 0
00261     if (!selectBank(0))
00262         return false;
00263
00264     // Read 6 bytes (X, Y, Z)
00265     uint8_t raw[6];
00266     if (!bus->read(GYRO_XOUT_H, raw, 6))
00267         return false;
00268
00269     // Convert raw data to dps
00270     x = (int16_t)((raw[0] << 8) | raw[1]) / degree_per_second;
00271     y = (int16_t)((raw[2] << 8) | raw[3]) / degree_per_second;
00272     z = (int16_t)((raw[4] << 8) | raw[5]) / degree_per_second;
00273

```

```

00274 // Success
00275 return true;
00276 }
00277
00278 bool DevLab_ICM20948::readTemperature(float &temperature)
00279 {
00280 // Select USER BANK 0
00281 if (!selectBank(0))
00282 return false;
00283
00284 // Initialize accumulator
00285 temperature = 0;
00286
00287 // Read and average 5 samples
00288 for (int i = 0; i < 5; i++)
00289 {
00290 // Read temperature registers
00291 uint8_t raw[2];
00292 if (!bus->read(TEMP_OUT_H, raw, 2))
00293 return false;
00294
00295 // Convert raw to signed value
00296 int16_t temp = (int16_t)((raw[0] << 8) | raw[1]);
00297
00298 // Convert to °C and accumulate
00299 temperature += temp / 333.87f + 21.0f;
00300 }
00301
00302 // Compute average temperature
00303 temperature /= 5.0f;
00304
00305 // Success
00306 return true;
00307 }
00308
00309
00310 bool DevLab_ICM20948::readMag(float &x, float &y, float &z)
00311 {
00312 // Select USER BANK 0
00313 if (!selectBank(0))
00314 return false;
00315
00316 // Read 8 bytes (ST1 + XYZ + ST2)
00317 uint8_t buf[8];
00318 if (!bus->read(EXT_SLV_SENS_DATA_00, buf, 8))
00319 return false;
00320
00321 int16_t mx = (int16_t)(buf[1] | (buf[2] << 8));
00322 int16_t my = (int16_t)(buf[3] | (buf[4] << 8));
00323 int16_t mz = (int16_t)(buf[5] | (buf[6] << 8));
00324
00325 // Convert to µT (AK09916 sensitivity)
00326 x = mx * 0.15f;
00327 y = my * 0.15f;
00328 z = mz * 0.15f;
00329
00330 // Success
00331 return true;
00332 }
00333
00334 bool DevLab_ICM20948::initMag()
00335 {
00336 if (!bus)
00337 return false;
00338
00339 softReset();
00340
00341 sleep(false);
00342
00343 if (!selectBank(0))
00344 return false;
00345 // Enable I2C master mode and disable I2C slave mode
00346 if (!bus->write(USER_CTRL, (uint8_t)USER_CTRL_I2C_MST_EN))
00347 return false;
00348
00349 if (!selectBank(3))
00350 return false;
00351
00352 if (!bus->write(I2C_MST_CTRL, (uint8_t)0x07))
00353 return false;
00354
00355 delay(10);
00356
00357 //Soft Reset
00358 writeSlave4(AK_CNTL3, 0x01);
00359
00360 delay(100);

```

```

00361
00362     uint8_t i, who;
00363     for (i = 0; i < 10; i++)
00364     {
00365         readSlave4(AK_WIA2, who);
00366         if (who == AK_WIA2_VAL)
00367         {
00368             //Mode 3 (continuous measurement 100Hz)
00369             writeSlave4(AK_CNTL2, 0x08);
00370
00371             delay(10);
00372
00373             // ---- Setup auto-read (SLV0) ----
00374             if (!selectBank(3))
00375                 return false;
00376
00377             if (!bus->write(I2C_SLV0_ADDR, (uint8_t)(AK09916_I2C_ADDR | 0x80)))
00378                 return false;
00379
00380             if (!bus->write(I2C_SLV0_REG, (uint8_t)AK_ST1))
00381                 return false;
00382
00383             if (!bus->write(I2C_SLV0_CTRL, (uint8_t)(I2C_SLVx_EN | 8)))
00384                 return false;
00385
00386             return true;
00387         }
00388
00389         writeSlave4(AK_CNTL2, 0x08);
00390
00391         delay(10);
00392
00393         // ---- Setup auto-read (SLV0) ----
00394         if (!selectBank(3))
00395             return false;
00396
00397         if (!bus->write(I2C_SLV0_ADDR, (uint8_t)(AK09916_I2C_ADDR | 0x80)))
00398             return false;
00399
00400         if (!bus->write(I2C_SLV0_REG, (uint8_t)AK_ST1))
00401             return false;
00402
00403         if (!bus->write(I2C_SLV0_CTRL, (uint8_t)(I2C_SLVx_EN | 8)))
00404             return false;
00405
00406         return false;
00407     }
00408 }
00409
00410 bool DevLab_ICM20948::setMagOpMode(ICM20948_Op_Mode opMode)
00411 {
00412     // Write operation mode to magnetometer
00413     writeSlave4(AK_CNTL2, (uint8_t)opMode);
00414
00415     return true;
00416 }
00417
00418 bool DevLab_ICM20948::writeSlave4(uint8_t reg, uint8_t value)
00419 {
00420     // Select USER BANK 3
00421     if (!selectBank(3))
00422         return false;
00423
00424     // 7 - 1:r , 0:w
00425     //[6:0]: I2C slave address (AK09916_I2C_ADDR)
00426     // Set slave address (write)
00427     if (!bus->write(I2C_SLV4_ADDR, (uint8_t)AK09916_I2C_ADDR))
00428         return false;
00429
00430     // Set data to write to slave
00431     if (!bus->write(I2C_SLV4_DO, value))
00432         return false;
00433
00434     // Set target register
00435     if (!bus->write(I2C_SLV4_REG, reg))
00436         return false;
00437
00438     // Start transaction (EN bit)
00439     if (!bus->write(I2C_SLV4_CTRL, (uint8_t)128))
00440         return false;
00441
00442     // Wait for completion (with timeout)
00443     uint32_t start = millis();
00444     uint8_t status;
00445
00446     while (millis() - start < 10)
00447     {
00448         if (bus->read(I2C_SLV4_CTRL, status))

```

```

00448         if (!(status & 0x80))
00449             return true;
00450     }
00451
00452     // Timeout
00453     return false;
00454 }
00455
00456 bool DevLab_ICM20948::readSlave4(uint8_t reg, uint8_t &value)
00457 {
00458     // Select USER BANK 3
00459     if (!selectBank(3))
00460         return false;
00461
00462     // Set slave address (read)
00463     if (!bus->write(I2C_SLV4_ADDR, (uint8_t)(AK09916_I2C_ADDR | 0x80)))
00464         return false;
00465
00466     // Set target register
00467     if (!bus->write(I2C_SLV4_REG, reg))
00468         return false;
00469
00470     // Start transaction
00471     if (!bus->write(I2C_SLV4_CTRL, (uint8_t)128))
00472         return false;
00473     // Wait for completion
00474     uint32_t start = millis();
00475     uint8_t status;
00476     while (millis() - start < 10)
00477     {
00478         if (bus->read(I2C_SLV4_CTRL, status))
00479             if (!(status & 0x80))
00480             { if (!bus->read(I2C_SLV4_DI, value))
00481                 return false;
00482                 return true;
00483             }
00484     }
00485     return false;
00486 }
00487
00488 bool DevLab_ICM20948::setLowPower(bool enable)
00489 {
00490     // Select USER BANK 0
00491     if (!selectBank(0))
00492         return false;
00493
00494     // Set or clear LP_EN bit (bit 5)
00495     if (!bus->writeBit(PWR_MGMT_1, 5, (uint8_t)enable))
00496         return false;
00497
00498     // Success
00499     return true;
00500 }
00501
00502 bool DevLab_ICM20948::getLowPower(bool &enable)
00503 {
00504     // Select USER BANK 0
00505     if (!selectBank(0))
00506         return false;
00507
00508     // Read LP_EN bit (bit 5)
00509     uint8_t v = 0;
00510     if (!bus->readBit(PWR_MGMT_1, 5, v))
00511         return false;
00512
00513     // Convert bit value to boolean
00514     enable = (v == 1);
00515
00516     // Success
00517     return true;
00518 }
00519
00520 bool DevLab_ICM20948::setClock(ICM20948_Clock_Source clock)
00521 {
00522     // Select USER BANK 0
00523     if (!selectBank(0))
00524         return false;
00525
00526     // Set CLKSEL bits [2:0]
00527     if (!bus->write(PWR_MGMT_1, (uint8_t)clock))
00528         return false;
00529
00530     // Success
00531     return true;
00532 }
00533
00534 bool DevLab_ICM20948::getClock(uint8_t &clock)

```

```

00535 {
00536     // Select USER BANK 0
00537     if (!selectBank(0))
00538         return false;
00539
00540     // Read CLKSEL bits [2:0]
00541     if (!bus->read(PWR_MGMT_1, clock))
00542         return false;
00543
00544     clock &= 0x07;
00545
00546     // Success
00547     return true;
00548 }
00549
00550 bool DevLab_ICM20948::setGyroSampleRate(float sampleRate)
00551 {
00552     // Validate sample rate range (approx 4.3 Hz to 1100 Hz)
00553     if ((sampleRate < 4.3f) || (sampleRate > 1100.0f))
00554         return false;
00555
00556     // Compute divider value
00557     uint8_t v = (uint8_t)((1100.0f / sampleRate) - 1.0f);
00558
00559     // Select USER BANK 2
00560     if (!selectBank(2))
00561         return false;
00562
00563     // Write sample rate divider
00564     if (!bus->write(GYRO_SMPLRT_DIV, v))
00565         return false;
00566
00567     // Success
00568     return true;
00569 }
00570
00571 bool DevLab_ICM20948::getGyroSampleRate(float &sampleRate)
00572 {
00573     // Validate bus pointer
00574     if (!bus)
00575         return false;
00576
00577     // Select USER BANK 2
00578     if (!selectBank(2))
00579         return false;
00580
00581     // Read sample rate divider
00582     uint8_t v = 0;
00583     if (!bus->read(GYRO_SMPLRT_DIV, v))
00584         return false;
00585
00586     // Compute sample rate
00587     sampleRate = 1100.0f / (1.0f + v);
00588
00589     // Success
00590     return true;
00591 }
00592
00593 bool DevLab_ICM20948::setDLPF(ICM20948_Gyro_DLPF dlpf, bool bypass)
00594 {
00595     // Validate bus pointer
00596     if (!bus)
00597         return false;
00598
00599     // Select USER BANK 2
00600     if (!selectBank(2))
00601         return false;
00602
00603     // Enable or disable DLPF bypass (bit 0)
00604     if (!bus->writeBit(GYRO_CONFIG_1, 0, (uint8_t)bypass))
00605         return false;
00606
00607     // If bypass enabled, skip DLPF configuration
00608     if (bypass)
00609         return true;
00610
00611     // Set DLPF configuration bits [5:3]
00612     if (!bus->writeBits(GYRO_CONFIG_1, 3, 3, (uint8_t)dlpf))
00613         return false;
00614
00615     // Success
00616     return true;
00617 }
00618
00619 bool DevLab_ICM20948::getDLPF(uint8_t &dlpf, bool &bypass)
00620 {
00621     // Validate bus pointer

```



```

00622     if (!bus)
00623         return false;
00624
00625     // Select USER BANK 2
00626     if (!selectBank(2))
00627         return false;
00628
00629     // Read GYRO_CONFIG_1 register
00630     uint8_t v = 0;
00631     if (!bus->read(GYRO_CONFIG_1, v))
00632         return false;
00633
00634     // Extract bypass bit (bit 0)
00635     bypass = (v & 0x01);
00636
00637     // Extract DLPF bits [5:3]
00638     dlpf = (v >> 3) & 0x07;
00639
00640     // Success
00641     return true;
00642 }
00643
00644 bool DevLab_ICM20948::setGyroScale(ICM20948_Gyro_FullScale fullScale)
00645 {
00646     // Validate bus pointer
00647     if (!bus)
00648         return false;
00649
00650     // Select USER BANK 2
00651     if (!selectBank(2))
00652         return false;
00653
00654     // Set full-scale range bits [2:1]
00655     if (!bus->writeBits(GYRO_CONFIG_1, 1, 2, (uint8_t)fullScale))
00656         return false;
00657
00658     // Update internal scale (LSB per g)
00659     degree_per_second = 32768.0f / (250.0f * (1 << ((uint8_t)fullScale)));
00660
00661     // Success
00662     return true;
00663 }
00664
00665 bool DevLab_ICM20948::getGyroScale(uint8_t &fullScale)
00666 {
00667     // Validate bus pointer
00668     if (!bus)
00669         return false;
00670
00671     // Select USER BANK 2
00672     if (!selectBank(2))
00673         return false;
00674
00675     // Read full-scale range bits [2:1]
00676     if (!bus->readBits(GYRO_CONFIG_1, 1, 2, fullScale))
00677         return false;
00678
00679     // Success
00680     return true;
00681 }
00682
00683 bool DevLab_ICM20948::selfTestGyro(bool x, bool y, bool z)
00684 {
00685     // Validate bus pointer
00686     if (!bus)
00687         return false;
00688
00689     // Select USER BANK 2
00690     if (!selectBank(2))
00691         return false;
00692
00693     // Set Z-axis self-test (bit 3)
00694     if (!bus->writeBit(GYRO_CONFIG_2, 3, (uint8_t)z))
00695         return false;
00696
00697     // Set Y-axis self-test (bit 4)
00698     if (!bus->writeBit(GYRO_CONFIG_2, 4, (uint8_t)y))
00699         return false;
00700
00701     // Set X-axis self-test (bit 5)
00702     if (!bus->writeBit(GYRO_CONFIG_2, 5, (uint8_t)x))
00703         return false;
00704
00705     // Success
00706     return true;
00707 }
00708

```

```

00709 bool DevLab_ICM20948::setAccelSampleRate(uint16_t sampleRate)
00710 {
00711     // Validate bus pointer
00712     if (!bus)
00713         return false;
00714
00715     // Select USER BANK 2
00716     if (!selectBank(2))
00717         return false;
00718
00719     // Validate input
00720     if (sampleRate == 0)
00721         return false;
00722
00723     // Clamp maximum rate
00724     if (sampleRate >= 1125)
00725         return false;
00726
00727     // Compute divider (rounded)
00728     uint16_t div = (1125.0f / sampleRate) - 1;
00729
00730     if (div > 4095u)
00731         div = 4095u;
00732
00733     // Split divider into high and low bytes
00734     uint8_t msb = (uint8_t)((div >> 8) & 0xFF);
00735     uint8_t lsb = (uint8_t)(div & 0xFF);
00736
00737     // Write divider high byte
00738     if (!bus->write(ACCEL_SMPLRT_DIV_1, msb))
00739         return false;
00740
00741     // Write divider low byte
00742     if (!bus->write(ACCEL_SMPLRT_DIV_2, lsb))
00743         return false;
00744
00745     // Success
00746     return true;
00747 }
00748 bool DevLab_ICM20948::setAccelDivRate(uint16_t divisor)
00749 {
00750     if (!bus)
00751         return false;
00752
00753     // Select USER BANK 2
00754     if (!selectBank(2))
00755         return false;
00756
00757     // Validate input
00758     if (divisor == 0)
00759         return false;
00760
00761     if (divisor > 4095u)
00762         divisor = 4095u;
00763
00764     // Split divider into high and low bytes
00765     uint8_t msb = (uint8_t)((divisor >> 8) & 0xFF);
00766     uint8_t lsb = (uint8_t)(divisor & 0xFF);
00767
00768     // Write divider high byte
00769     if (!bus->write(ACCEL_SMPLRT_DIV_1, msb))
00770         return false;
00771
00772     // Write divider low byte
00773     if (!bus->write(ACCEL_SMPLRT_DIV_2, lsb))
00774         return false;
00775
00776     // Success
00777     return true;
00778 }
00779
00780 bool DevLab_ICM20948::setGyroDivRate(uint8_t divisor)
00781 {
00782     if (!bus)
00783         return false;
00784
00785     // Select USER BANK 2
00786     if (!selectBank(2))
00787         return false;
00788
00789     // Write sample rate divider
00790     if (!bus->write(GYRO_SMPLRT_DIV, divisor))
00791         return false;
00792
00793     // Success
00794     return true;
00795 }

```

```

00796 bool DevLab_ICM20948::getAccelSampleRate(float &sampleRate)
00797 {
00798     // Validate bus pointer
00799     if (!bus)
00800         return false;
00801
00802     // Select USER BANK 2
00803     if (!selectBank(2))
00804         return false;
00805
00806     // Read divider high byte
00807     uint8_t msb = 0;
00808     if (!bus->read(ACCEL_SMPLRT_DIV_1, msb))
00809         return false;
00810
00811     // Read divider low byte
00812     uint8_t lsb = 0;
00813     if (!bus->read(ACCEL_SMPLRT_DIV_2, lsb))
00814         return false;
00815
00816     // Combine 12-bit divider
00817     uint16_t div = ((msb & 0x0F) << 8) | lsb;
00818
00819     // Compute sample rate
00820     sampleRate = 1125.0f / (1.0f + div);
00821
00822     // Success
00823     return true;
00824 }
00825
00826 bool DevLab_ICM20948::selfTestAccel(bool x, bool y, bool z)
00827 {
00828     // Validate bus pointer
00829     if (!bus)
00830         return false;
00831
00832     // Select USER BANK 2
00833     if (!selectBank(2))
00834         return false;
00835
00836     if (x)
00837     { // Set X-axis self-test (bit 7)
00838         if (!bus->writeBit(ACCEL_CONFIG_2, 7, (uint8_t)x))
00839             return false;
00840     }
00841
00842     if (y)
00843     { // Set Y-axis self-test (bit 6)
00844         if (!bus->writeBit(ACCEL_CONFIG_2, 6, (uint8_t)y))
00845             return false;
00846     }
00847
00848     if (z)
00849     { // Set Z-axis self-test (bit 5)
00850         if (!bus->writeBit(ACCEL_CONFIG_2, 5, (uint8_t)z))
00851             return false;
00852     }
00853
00854     // Success
00855     return true;
00856 }
00857 bool DevLab_ICM20948::setAccelScale(ICM20948_Accel_FullScale fullScale)
00858 {
00859     // Validate bus pointer
00860     if (!bus)
00861         return false;
00862
00863     // Select USER BANK 2
00864     if (!selectBank(2))
00865         return false;
00866
00867     // Set full-scale bits [2:1]
00868     if (!bus->writeBits(ACCEL_CONFIG, 1, 2, (uint8_t)fullScale))
00869         return false;
00870
00871     // Update internal scale (LSB per g)
00872     mg_per_lsb = 16384.0f / (1 << (uint8_t)fullScale);
00873
00874     // Success
00875     return true;
00876 }
00877
00878 bool DevLab_ICM20948::getAccelScale(uint8_t &fullScale)
00879 {
00880     // Validate bus pointer
00881     if (!bus)
00882         return false;

```

```

00883
00884 // Select USER BANK 2
00885 if (!selectBank(2))
00886     return false;
00887
00888 // Read full-scale bits [2:1]
00889 if (!bus->readBits(ACCEL_CONFIG, 1, 2, fullScale))
00890     return false;
00891
00892 // Success
00893 return true;
00894 }
00895
00896
00897 bool DevLab_ICM20948::setAccelDLPF(uint8_t dlpf, bool bypass)
00898 {
00899     // Validate bus pointer
00900     if (!bus)
00901         return false;
00902
00903     // Select USER BANK 2
00904     if (!selectBank(2))
00905         return false;
00906
00907     // Set or clear DLPF bypass (bit 0)
00908     if (!bus->writeBit(ACCEL_CONFIG, 0, (uint8_t)bypass))
00909         return false;
00910
00911     // If bypass enabled, skip DLPF configuration
00912     if (bypass)
00913         return true;
00914
00915     // Limit DLPF value to valid range
00916     dlpf &= 0x07;
00917
00918     // Set DLPF configuration bits [5:3]
00919     if (!bus->writeBits(ACCEL_CONFIG, 3, 3, dlpf))
00920         return false;
00921
00922     // Success
00923     return true;
00924 }
00925
00926 bool DevLab_ICM20948::getAccelDLPF(uint8_t &dlpf, bool &bypass)
00927 {
00928     // Validate bus pointer
00929     if (!bus)
00930         return false;
00931
00932     // Select USER BANK 2
00933     if (!selectBank(2))
00934         return false;
00935
00936     // Read ACCEL_CONFIG register
00937     uint8_t v = 0;
00938     if (!bus->read(ACCEL_CONFIG, v))
00939         return false;
00940
00941     // Extract bypass bit (bit 0)
00942     bypass = (v & 0x01);
00943
00944     // Extract DLPF bits [5:3]
00945     dlpf = (v >> 3) & 0x07;
00946
00947     // Success
00948     return true;
00949 }
00950
00951
00952 bool DevLab_ICM20948::setAccelAveraging(ICM20948_Accel_Average avg)
00953 {
00954     // Validate bus pointer
00955     if (!bus)
00956         return false;
00957
00958     // Select USER BANK 2
00959     if (!selectBank(2))
00960         return false;
00961
00962     // Set DEC3 averaging bits [1:0]
00963     if (!bus->writeBits(ACCEL_CONFIG_2, 0, 2, (uint8_t)avg))
00964         return false;
00965
00966     // Success
00967     return true;
00968 }
00969

```

```

00970 bool DevLab_ICM20948::setGyroAveraging(ICM20948_Gyro_Average avg)
00971 {
00972     // Validate bus pointer
00973     if (!bus)
00974         return false;
00975
00976     // Select USER BANK 2
00977     if (!selectBank(2))
00978         return false;
00979
00980     // Set DEC3 averaging bits [1:0]
00981     if (!bus->writeBits(GYRO_CONFIG_2, 0, 2, (uint8_t)avg))
00982         return false;
00983
00984     // Success
00985     return true;
00986 }
00987
00988 bool DevLab_ICM20948::getAccelAveraging(uint8_t &avg)
00989 {
00990     // Validate bus pointer
00991     if (!bus)
00992         return false;
00993
00994     // Select USER BANK 2
00995     if (!selectBank(2))
00996         return false;
00997
00998     // Read DEC3 averaging bits [1:0]
00999     if (!bus->readBits(ACCEL_CONFIG_2, 0, 2, avg))
01000         return false;
01001
01002     // Success
01003     return true;
01004 }
01005
01006 bool DevLab_ICM20948::setSensors(bool accel_on, bool gyro_on, bool temp_on)
01007 {
01008     // Validate bus pointer
01009     if (!bus)
01010         return false;
01011
01012     // Select USER BANK 0
01013     if (!selectBank(0))
01014         return false;
01015
01016     // Build PWR_MGMT_2 mask
01017     uint8_t v = 0;
01018
01019     // Disable accel axes if not enabled
01020     if (!accel_on)
01021         v |= 0x38; // bits [5:3]
01022
01023     // Disable gyro axes if not enabled
01024     if (!gyro_on)
01025         v |= 0x07; // bits [2:0]
01026
01027     // Write sensor enable/disable mask
01028     if (!bus->write(PWR_MGMT_2, v))
01029         return false;
01030
01031     // Set or clear TEMP_DIS bit (bit 3)
01032     if (!bus->writeBit(PWR_MGMT_1, 3, (uint8_t)!temp_on))
01033         return false;
01034
01035     // Success
01036     return true;
01037 }
01038
01039 bool DevLab_ICM20948::getSensors(bool &accel_on, bool &gyro_on, bool &temp_on)
01040 {
01041     // Validate bus pointer
01042     if (!bus)
01043         return false;
01044
01045     // Select USER BANK 0
01046     if (!selectBank(0))
01047         return false;
01048
01049     // Read PWR_MGMT_2 register
01050     uint8_t v = 0;
01051     if (!bus->read(PWR_MGMT_2, v))
01052         return false;
01053
01054     // Decode gyro state (bits [2:0])
01055     gyro_on = ((v & 0x07) == 0);
01056 }

```

```

01057 // Decode accel state (bits [5:3])
01058 accel_on = ((v & 0x38) == 0);
01059
01060 // Read TEMP_DIS bit (bit 3)
01061 uint8_t t = 0;
01062 if (!bus->readBit(PWR_MGMT_1, 3, t))
01063     return false;
01064
01065 // Decode temperature state
01066 temp_on = (t == 0);
01067
01068 // Success
01069 return true;
01070 }
01071
01072 bool DevLab_ICM20948::setGyroOffset(uint16_t offsetX, uint16_t offsetY, uint16_t offsetZ)
01073 {
01074     // Validate bus pointer
01075     if (!bus)
01076         return false;
01077
01078     // Select USER BANK 2
01079     if (!selectBank(2))
01080         return false;
01081
01082     // Pack offset values into byte array
01083     uint8_t data[6] = {
01084         (uint8_t)(offsetX >> 8),
01085         (uint8_t)(offsetX & 0xFF),
01086         (uint8_t)(offsetY >> 8),
01087         (uint8_t)(offsetY & 0xFF),
01088         (uint8_t)(offsetZ >> 8),
01089         (uint8_t)(offsetZ & 0xFF)};
01090
01091     // Write offset registers
01092     if (!bus->write(XG_OFFS_USRH, data, 6))
01093         return false;
01094
01095     // Success
01096     return true;
01097 }
01098
01099 bool DevLab_ICM20948::getGyroOffset(int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ)
01100 {
01101     // Validate bus pointer
01102     if (!bus)
01103         return false;
01104
01105     // Select USER BANK 2
01106     if (!selectBank(2))
01107         return false;
01108
01109     // Read offset registers
01110     uint8_t data[6];
01111     if (!bus->read(XG_OFFS_USRH, data, 6))
01112         return false;
01113
01114     // Convert to signed values
01115     offsetX = (int16_t)((data[0] << 8) | data[1]);
01116     offsetY = (int16_t)((data[2] << 8) | data[3]);
01117     offsetZ = (int16_t)((data[4] << 8) | data[5]);
01118
01119     // Success
01120     return true;
01121 }
01122
01123
01124 bool DevLab_ICM20948::setAccelOffset(int16_t offsetX, int16_t offsetY, int16_t offsetZ)
01125 {
01126     // Validate bus pointer
01127     if (!bus)
01128         return false;
01129
01130     // Select USER BANK 1 (accel offsets are here)
01131     if (!selectBank(1))
01132         return false;
01133
01134     // Pack offset values into byte array
01135     uint8_t data[6] = {
01136         (uint8_t)(offsetX >> 8),
01137         (uint8_t)(offsetX & 0xFF),
01138         (uint8_t)(offsetY >> 8),
01139         (uint8_t)(offsetY & 0xFF),
01140         (uint8_t)(offsetZ >> 8),
01141         (uint8_t)(offsetZ & 0xFF)};
01142
01143     // Write offset registers

```

```

01144     if (!bus->write(XA_OFFS_H, data, 6))
01145         return false;
01146
01147     // Success
01148     return true;
01149 }
01150
01151 bool DevLab_ICM20948::getAccelOffset(int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ)
01152 {
01153     // Validate bus pointer
01154     if (!bus)
01155         return false;
01156
01157     // Select USER BANK 1
01158     if (!selectBank(1))
01159         return false;
01160
01161     // Read offset registers
01162     uint8_t data[6];
01163     if (!bus->read(XA_OFFS_H, data, 6))
01164         return false;
01165
01166     // Convert to signed values
01167     offsetX = (int16_t)((data[0] << 8) | data[1]);
01168     offsetY = (int16_t)((data[2] << 8) | data[3]);
01169     offsetZ = (int16_t)((data[4] << 8) | data[5]);
01170
01171     // Success
01172     return true;
01173 }
01174
01175
01176 bool DevLab_ICM20948::intInit(const ICM20948_IntPinConfig &cfg){
01177     // Validate bus pointer
01178     if (!bus)
01179         return false;
01180
01181     if(!selectBank(0))
01182         return false;
01183
01184     uint8_t reg = (cfg.activeLevel << 7) |
01185                  (cfg.driveMode << 6) |
01186                  (cfg.latchMode << 5) |
01187                  (cfg.clearMode << 4) |
01188                  (cfg.fsyncActLevel << 3) |
01189                  (cfg.fsyncIntEn << 2) |
01190                  (cfg.bypassEn << 1);
01191
01192     // Initialize INT1
01193     // Configure interrupt pin (open-drain, active-low, latch)
01194     if (!bus->write(INT_PIN_CFG, reg))
01195         return false;
01196
01197     // Configuration successful
01198     return true;
01199 }
01200
01201 bool DevLab_ICM20948::intEnableConfig(const ICM20948_IntEnableConfig &cfg)
01202 {
01203     if (!bus) return false;
01204     if (!selectBank(0)) return false;
01205
01206     // INT_ENABLE
01207     uint8_t reg0 = (cfg.wofEn << 7) |
01208                  (cfg.womIntEn << 3) |
01209                  (cfg.pllRdyEn << 2) |
01210                  (cfg.dmpInt1En << 1) |
01211                  (cfg.i2cMstIntEn << 0);
01212
01213     if (!bus->write(INT_ENABLE, reg0)) return false;
01214
01215     // INT_ENABLE_1
01216     if (!bus->write(INT_ENABLE_1, (uint8_t)(cfg.rawDataRdyEn & 0x01))) return false;
01217
01218     // INT_ENABLE_2 – construye máscara desde el array
01219     uint8_t ovfMask = 0;
01220     for (uint8_t i = 0; i < 5; i++)
01221         ovfMask |= (cfg.fifoOvfEn[i] ? BIT(i) : 0);
01222     if (!bus->write(INT_ENABLE_2, ovfMask)) return false;
01223
01224     // INT_ENABLE_3 – igual
01225     uint8_t wmMask = 0;
01226     for (uint8_t i = 0; i < 5; i++)
01227         wmMask |= (cfg.fifoWmEn[i] ? BIT(i) : 0);
01228     if (!bus->write(INT_ENABLE_3, wmMask)) return false;
01229
01230     return true;
01231 }
01232
01233

```

```

01231 bool DevLab_ICM20948::checkIntStatus(ICM20948_IntStatus &status)
01232 {
01233     if (!bus)                return false;
01234     if (!selectBank(0)) return false;
01235
01236     // Leer los 4 registros de status en una sola operación de burst
01237     uint8_t raw[4];
01238     if (!bus->read(INT_STATUS, raw, 4)) return false;
01239
01240     // INT_STATUS (0x19)
01241     status.womInt    = (raw[0] & STS_WOM_INT)    ? 1 : 0;
01242     status.pllRdyInt = (raw[0] & STS_PLL_RDY_INT) ? 1 : 0;
01243     status.dmpInt1   = (raw[0] & STS_DMP_INT1)   ? 1 : 0;
01244     status.i2cMstInt = (raw[0] & STS_I2C_MST_INT) ? 1 : 0;
01245
01246     // INT_STATUS_1 (0x1A)
01247     status.rawDataRdy = (raw[1] & STS_RAW_DATA_0_RDY_INT) ? 1 : 0;
01248
01249     // INT_STATUS_2 (0x1B) – FIFO overflow canal por canal
01250     for (uint8_t i = 0; i < 5; i++)
01251         status.fifoOvf[i] = (raw[2] & BIT(i)) ? 1 : 0;
01252
01253     // INT_STATUS_3 (0x1C) – FIFO watermark canal por canal
01254     for (uint8_t i = 0; i < 5; i++)
01255         status.fifoWm[i] = (raw[3] & BIT(i)) ? 1 : 0;
01256
01257     return true;
01258 }
01259
01260 bool DevLab_ICM20948::auxMasterEnable(uint8_t clkFreq)
01261 {
01262     // BANK 0: asegurarse que BYPASS_EN=0 e I2C_MST_EN=1
01263     if (!selectBank(0)) return false;
01264
01265     // Limpiar BYPASS_EN (bit 1 de INT_PIN_CFG)
01266     if (!bus->writeBit(INT_PIN_CFG, 1, (uint8_t)0)) return false;
01267
01268     // Activar I2C_MST_EN (bit 5 de USER_CTRL)
01269     if (!bus->writeBit(USER_CTRL, 5, (uint8_t)1)) return false;
01270
01271     // BANK 3: configurar clock del master auxiliar
01272     if (!selectBank(3)) return false;
01273
01274     // clkFreq típico: 0x07 = ~345.6 kHz | 0x0D = ~400 kHz
01275     if (!bus->writeBits(I2C_MST_CTRL, 0, 4, clkFreq)) return false;
01276
01277     // Volver a BANK 0
01278     return selectBank(0);
01279 }
01280
01281 bool DevLab_ICM20948::auxWriteByte(uint8_t slaveAddr, uint8_t reg, uint8_t data)
01282 {
01283     if (!selectBank(3)) return false;
01284
01285     // Dirección del slave, bit 7=0 para escritura
01286     if (!bus->write(I2C_SLV4_ADDR, (uint8_t)(slaveAddr & 0x7F))) return false;
01287
01288     // Registro destino que queremos escribir en el slave
01289     if (!bus->write(I2C_SLV4_REG, reg)) return false;
01290
01291     // Dato a escribir
01292     if (!bus->write(I2C_SLV4_DO, data)) return false;
01293
01294     // Disparar transacción (I2C_SLV4_EN, se auto-limpia al terminar)
01295     if (!bus->write(I2C_SLV4_CTRL, (uint8_t)I2C_SLVx_EN)) return false;
01296
01297     // Esperar a que complete – verificar SLV4_DONE en BANK 0
01298     if (!selectBank(0)) return false;
01299
01300     uint8_t status = 0;
01301     uint8_t timeout = 50;
01302     do {
01303         delay(1);
01304         if (!bus->read(I2C_MST_STATUS, status)) return false;
01305         if (--timeout == 0) return false; // timeout
01306     } while (!(status & MST_SLV4_DONE));
01307
01308     // Verificar que no hubo NACK
01309     return !(status & MST_SLV4_NACK);
01310 }
01311
01312 bool DevLab_ICM20948::auxReadByte(uint8_t slaveAddr, uint8_t reg, uint8_t &data)
01313 {
01314     if (!selectBank(3)) return false;
01315
01316     // Dirección del slave, bit 7=1 para lectura
01317     if (!bus->write(I2C_SLV4_ADDR, (uint8_t)((slaveAddr & 0x7F) | I2C_SLVx_RNW))) return false;

```



```

01318
01319 // Registro origen que queremos leer del slave
01320 if (!bus->write(I2C_SLV4_REG, reg)) return false;
01321
01322 // Disparar transacción (I2C_SLV4_EN, se auto-limpia al terminar)
01323 if (!bus->write(I2C_SLV4_CTRL, (uint8_t)I2C_SLVx_EN)) return false;
01324
01325 // Esperar a que complete – verificar SLV4_DONE en BANK 0
01326 if (!selectBank(0)) return false;
01327
01328 uint8_t status = 0;
01329 uint8_t timeout = 50;
01330 do {
01331     delay(1);
01332     if (!bus->read(I2C_MST_STATUS, status)) return false;
01333     if (--timeout == 0) return false; // timeout
01334 } while (!(status & MST_SLV4_DONE));
01335
01336 // Verificar que no hubo NACK
01337 if (status & MST_SLV4_NACK) return false;
01338
01339 // Leer el dato recibido del slave (BANK 3)
01340 if (!selectBank(3)) return false;
01341 if (!bus->read(I2C_SLV4_DI, data)) return false;
01342
01343 return selectBank(0);
01344 }
01345
01346 bool DevLab_ICM20948::auxWriteCommand(uint8_t slaveAddr, uint8_t cmd)
01347 {
01348     if (!selectBank(3)) return false;
01349
01350     // Dirección del slave, bit 7=0 para escritura
01351     if (!bus->write(I2C_SLV4_ADDR, (uint8_t)(slaveAddr & 0x7F))) return false;
01352
01353     // Byte de comando a enviar (único byte de la transacción)
01354     if (!bus->write(I2C_SLV4_D0, cmd)) return false;
01355
01356     // EN + REG_DIS: el SLV4 envía solo I2C_SLV4_D0, sin byte de registro
01357     if (!bus->write(I2C_SLV4_CTRL, (uint8_t)(I2C_SLVx_EN | I2C_SLVx_REG_DIS))) return false;
01358
01359     // Esperar a que complete – verificar SLV4_DONE en BANK 0
01360     if (!selectBank(0)) return false;
01361
01362     uint8_t status = 0;
01363     uint8_t timeout = 50;
01364     do {
01365         delay(1);
01366         if (!bus->read(I2C_MST_STATUS, status)) return false;
01367         if (--timeout == 0) return false; // timeout
01368     } while (!(status & MST_SLV4_DONE));
01369
01370     // Verificar que no hubo NACK
01371     return !(status & MST_SLV4_NACK);
01372 }
01373
01374 bool DevLab_ICM20948::auxConfigSlave(uint8_t slaveAddr, uint8_t reg, uint8_t numBytes)
01375 {
01376     if (!selectBank(3)) return false;
01377
01378     // Slave 0: dirección + bit READ
01379     if (!bus->write(I2C_SLV0_ADDR, (uint8_t)(slaveAddr | I2C_SLVx_RNW))) return false;
01380
01381     // Registro de inicio
01382     if (!bus->write(I2C_SLV0_REG, reg)) return false;
01383
01384     // Habilitar + número de bytes
01385     if (!bus->write(I2C_SLV0_CTRL, (uint8_t)(I2C_SLVx_EN | (numBytes & 0x0F)))) return false;
01386
01387     return selectBank(0);
01388 }
01389
01390 // Para leer los datos que el ICM leyó automáticamente:
01391 bool DevLab_ICM20948::auxReadSensorData(uint8_t *buf, uint8_t len)
01392 {
01393     if (!selectBank(0)) return false;
01394     return bus->read(EXT_SLV_SENS_DATA_00, buf, len);
01395 }
01396
01397 bool DevLab_ICM20948::auxReadResponse(uint8_t slaveAddr, uint8_t &data)
01398 {
01399     if (!selectBank(3)) return false;
01400
01401     // Slave address con bit READ, sin byte de registro (REG_DIS)
01402     // Genera: [START, addr+R, lee 1 byte, STOP]
01403     if (!bus->write(I2C_SLV4_ADDR, (uint8_t)((slaveAddr & 0x7F) | I2C_SLVx_RNW))) return false;
01404     if (!bus->write(I2C_SLV4_CTRL, (uint8_t)(I2C_SLVx_EN | I2C_SLVx_REG_DIS))) return false;

```



- class [DevLab\\_ICM20948](#)

#### Enumerations

- enum class [ICM20948\\_Op\\_Mode](#) : uint8\_t {  
[MODE\\_POWER\\_DOWN](#) = 0x00 , [MODE\\_SINGLE\\_MEASUREMENT](#) = 0x01 , [MODE\\_CONTINUOUS\\_1](#) = 0x02 , [MODE\\_CONTINUOUS\\_2](#) = 0x04 ,  
[MODE\\_CONTINUOUS\\_3](#) = 0x06 , [MODE\\_CONTINUOUS\\_4](#) = 0x08 , [MODE\\_SELF\\_TEST](#) = 0x10 }
- enum class [ICM20948\\_Clock\\_Source](#) : uint8\_t {  
[INTERNAL\\_20MHZ\\_0](#) = 0x00 , [AUTO\\_PLL\\_1](#) = 0x01 , [AUTO\\_PLL\\_2](#) = 0x02 ,  
[AUTO\\_PLL\\_3](#) = 0x03 ,  
[AUTO\\_PLL\\_4](#) = 0x04 , [AUTO\\_PLL\\_5](#) = 0x05 , [INTERNAL\\_20MHZ\\_6](#) = 0x06 ,  
[STOP\\_CLOCK](#) = 0x07 }
- enum class [ICM20948\\_Gyro\\_DLPF](#) : uint8\_t {  
[DLPF\\_196HZ](#) = 0x00 , [DLPF\\_151HZ](#) = 0x01 , [DLPF\\_119HZ](#) = 0x02 , [DLPF\\_51HZ](#) = 0x03 ,  
[DLPF\\_23HZ](#) = 0x04 , [DLPF\\_11HZ](#) = 0x05 , [DLPF\\_5HZ](#) = 0x06 , [DLPF\\_361HZ](#) = 0x07 }
- enum class [ICM20948\\_Gyro\\_Average](#) : uint8\_t {  
[AVG\\_1](#) = 0x00 , [AVG\\_2](#) = 0x01 , [AVG\\_4](#) = 0x02 , [AVG\\_8](#) = 0x03 ,  
[AVG\\_16](#) = 0x04 , [AVG\\_32](#) = 0x05 , [AVG\\_64](#) = 0x06 , [AVG\\_128](#) = 0x07 }
- enum class [ICM20948\\_Gyro\\_FullScale](#) : uint8\_t { [DPS\\_250](#) = 0x00 , [DPS\\_500](#) = 0x01 ,  
[DPS\\_1000](#) = 0x02 , [DPS\\_2000](#) = 0x03 }
- enum class [ICM20948\\_Accel\\_FullScale](#) : uint8\_t { [G\\_2](#) = 0x00 , [G\\_4](#) = 0x01 , [G\\_8](#) = 0x02 ,  
[G\\_16](#) = 0x03 }
- enum class [ICM20948\\_Accel\\_Average](#) : uint8\_t { [AVG\\_1\\_OR\\_4](#) = 0x00 , [AVG\\_8](#) = 0x01 ,  
[AVG\\_16](#) = 0x02 , [AVG\\_32](#) = 0x03 }
- enum class [ICM20948\\_Accel\\_DLPFCFG](#) : uint8\_t {  
[DLPF0\\_246HZ](#) = 0x00 , [DLPF\\_246HZ](#) = 0x01 , [DLPF\\_111HZ](#) = 0x02 , [DLPF\\_50HZ](#) = 0x03 ,  
[DLPF\\_24HZ](#) = 0x04 , [DLPF\\_12HZ](#) = 0x05 , [DLPF\\_6HZ](#) = 0x06 , [DLPF\\_473HZ](#) = 0x07 }

### 6.36.1 Enumeration Type Documentation

#### 6.36.1.1 ICM20948\_Accel\_Average

enum class [ICM20948\\_Accel\\_Average](#) : uint8\_t [strong]

##### Enumerator

|                            |  |
|----------------------------|--|
| <a href="#">AVG_1_OR_4</a> |  |
| <a href="#">AVG_8</a>      |  |
| <a href="#">AVG_16</a>     |  |
| <a href="#">AVG_32</a>     |  |

Definition at line 76 of file [DevLab\\_ICM20948.h](#).

#### 6.36.1.2 ICM20948\_Accel\_DLPFCFG

enum class [ICM20948\\_Accel\\_DLPFCFG](#) : uint8\_t [strong]

##### Enumerator

|                             |  |
|-----------------------------|--|
| <a href="#">DLPF0_246HZ</a> |  |
| <a href="#">DLPF_246HZ</a>  |  |
| <a href="#">DLPF_111HZ</a>  |  |
| <a href="#">DLPF_50HZ</a>   |  |
| <a href="#">DLPF_24HZ</a>   |  |

Enumerator

|            |  |
|------------|--|
| DLPF_12HZ  |  |
| DLPF_6HZ   |  |
| DLPF_473HZ |  |

Definition at line 84 of file [DevLab\\_ICM20948.h](#).

### 6.36.1.3 ICM20948\_Accel\_FullScale

```
enum class ICM20948_Accel_FullScale : uint8_t [strong]
```

Enumerator

|      |  |
|------|--|
| G_2  |  |
| G_4  |  |
| G_8  |  |
| G_16 |  |

Definition at line 68 of file [DevLab\\_ICM20948.h](#).

### 6.36.1.4 ICM20948\_Clock\_Source

```
enum class ICM20948_Clock_Source : uint8_t [strong]
```

Enumerator

|                       |  |
|-----------------------|--|
| INTERNAL_20MHZ↔<br>_0 |  |
| AUTO_PLL_1            |  |
| AUTO_PLL_2            |  |
| AUTO_PLL_3            |  |
| AUTO_PLL_4            |  |
| AUTO_PLL_5            |  |
| INTERNAL_20MHZ↔<br>_6 |  |
| STOP_CLOCK            |  |

Definition at line 25 of file [DevLab\\_ICM20948.h](#).

### 6.36.1.5 ICM20948\_Gyro\_Average

```
enum class ICM20948_Gyro_Average : uint8_t [strong]
```

Enumerator

|        |  |
|--------|--|
| AVG_1  |  |
| AVG_2  |  |
| AVG_4  |  |
| AVG_8  |  |
| AVG_16 |  |
| AVG_32 |  |
| AVG_64 |  |

Enumerator

|         |  |
|---------|--|
| AVG_128 |  |
|---------|--|

Definition at line 49 of file [DevLab\\_ICM20948.h](#).

#### 6.36.1.6 ICM20948\_Gyro\_DLPF

enum class [ICM20948\\_Gyro\\_DLPF](#) : uint8\_t [strong]

Enumerator

|            |  |
|------------|--|
| DLPF_196HZ |  |
| DLPF_151HZ |  |
| DLPF_119HZ |  |
| DLPF_51HZ  |  |
| DLPF_23HZ  |  |
| DLPF_11HZ  |  |
| DLPF_5HZ   |  |
| DLPF_361HZ |  |

Definition at line 37 of file [DevLab\\_ICM20948.h](#).

#### 6.36.1.7 ICM20948\_Gyro\_FullScale

enum class [ICM20948\\_Gyro\\_FullScale](#) : uint8\_t [strong]

Enumerator

|          |  |
|----------|--|
| DPS_250  |  |
| DPS_500  |  |
| DPS_1000 |  |
| DPS_2000 |  |

Definition at line 60 of file [DevLab\\_ICM20948.h](#).

#### 6.36.1.8 ICM20948\_Op\_Mode

enum class [ICM20948\\_Op\\_Mode](#) : uint8\_t [strong]

Enumerator

|                         |  |
|-------------------------|--|
| MODE_POWER_DOWN         |  |
| MODE_SINGLE_MEASUREMENT |  |
| MODE_CONTINUOUS_1       |  |
| MODE_CONTINUOUS_2       |  |
| MODE_CONTINUOUS_3       |  |
| MODE_CONTINUOUS_4       |  |
| MODE_SELF_TEST          |  |

Definition at line 14 of file [DevLab\\_ICM20948.h](#).

## 6.37 DevLab\_ICM20948.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ICM20948_DEVLAB_H
00002 #define ICM20948_DEVLAB_H
00003
00004 #include <Arduino.h>
00005 #include <Wire.h>
00006 #include <SPI.h>
00007 #include "ICM20948_regs.h"
00008
00009 #include "7Semi_Interface.h"
00010 #include "7Semi_I2C_Interface.h"
00011 #include "7Semi_SPI_Interface.h"
00012 #include "BusIO_7Semi.h"
00013
00014 enum class ICM20948_Op_Mode : uint8_t
00015 {
00016     MODE_POWER_DOWN = 0x00,
00017     MODE_SINGLE_MEASUREMENT = 0x01,
00018     MODE_CONTINUOUS_1 = 0x02,
00019     MODE_CONTINUOUS_2 = 0x04,
00020     MODE_CONTINUOUS_3 = 0x06,
00021     MODE_CONTINUOUS_4 = 0x08,
00022     MODE_SELF_TEST = 0x10
00023 };
00024
00025 enum class ICM20948_Clock_Source : uint8_t
00026 {
00027     INTERNAL_20MHZ_0 = 0x00, // Internal oscillator
00028     AUTO_PLL_1 = 0x01,      // Auto select (PLL if ready)
00029     AUTO_PLL_2 = 0x02,
00030     AUTO_PLL_3 = 0x03,
00031     AUTO_PLL_4 = 0x04,
00032     AUTO_PLL_5 = 0x05,
00033     INTERNAL_20MHZ_6 = 0x06, // Internal oscillator
00034     STOP_CLOCK = 0x07      // Stops clock, timing generator reset
00035 };
00036
00037 enum class ICM20948_Gyro_DLPF : uint8_t
00038 {
00039     DLPF_196HZ = 0x00, // BW ≈ 196.6 Hz, NBW ≈ 229.8 Hz
00040     DLPF_151HZ = 0x01, // BW ≈ 151.8 Hz, NBW ≈ 187.6 Hz
00041     DLPF_119HZ = 0x02, // BW ≈ 119.5 Hz, NBW ≈ 154.3 Hz
00042     DLPF_51HZ = 0x03,  // BW ≈ 51.2 Hz,  NBW ≈ 73.3 Hz
00043     DLPF_23HZ = 0x04,  // BW ≈ 23.9 Hz,  NBW ≈ 35.9 Hz
00044     DLPF_11HZ = 0x05,  // BW ≈ 11.6 Hz,  NBW ≈ 17.8 Hz
00045     DLPF_5HZ = 0x06,   // BW ≈ 5.7 Hz,   NBW ≈ 8.9 Hz
00046     DLPF_361HZ = 0x07 // BW ≈ 361.4 Hz, NBW ≈ 376.5 Hz
00047 };
00048
00049 enum class ICM20948_Gyro_Average : uint8_t
00050 {
00051     AVG_1 = 0x00,      // 1x Average (no averaging)
00052     AVG_2 = 0x01,      // 2x Average
00053     AVG_4 = 0x02,      // 4x Average
00054     AVG_8 = 0x03,      // 8x Average
00055     AVG_16 = 0x04,     // Average 16 samples
00056     AVG_32 = 0x05,     // Average 32 samples
00057     AVG_64 = 0x06,     // Average 64 samples
00058     AVG_128 = 0x07     // Average 128 samples
00059 };
00060 enum class ICM20948_Gyro_FullScale : uint8_t
00061 {
00062     DPS_250 = 0x00,    // ±250 dps
00063     DPS_500 = 0x01,    // ±500 dps
00064     DPS_1000 = 0x02,   // ±1000 dps
00065     DPS_2000 = 0x03    // ±2000 dps
00066 };
00067
00068 enum class ICM20948_Accel_FullScale : uint8_t
00069 {
00070     G_2 = 0x00, // ±2g
00071     G_4 = 0x01, // ±4g
00072     G_8 = 0x02, // ±8g
00073     G_16 = 0x03 // ±16g
00074 };
00075
00076 enum class ICM20948_Accel_Average : uint8_t
00077 {
00078     AVG_1_OR_4 = 0x00, // Depends on ACCEL_FCHOICE
00079     AVG_8 = 0x01,      // Average 8 samples
00080     AVG_16 = 0x02,     // Average 16 samples
00081     AVG_32 = 0x03      // Average 32 samples
00082 };
00083

```

```

00084 enum class ICM20948_Accel_DLPFCFG : uint8_t
00085 {
00086     DLPF0_246HZ = 0x00, // BW ≈ 246 Hz, NBW ≈ 265 Hz
00087     DLPF_246HZ = 0x01, // BW ≈ 246 Hz, NBW ≈ 265 Hz
00088     DLPF_111HZ = 0x02, // BW ≈ 111 Hz, NBW ≈ 136 Hz
00089     DLPF_50HZ = 0x03, // BW ≈ 50 Hz, NBW ≈ 68.8 Hz
00090     DLPF_24HZ = 0x04, // BW ≈ 24 Hz, NBW ≈ 34.4 Hz
00091     DLPF_12HZ = 0x05, // BW ≈ 12 Hz, NBW ≈ 17 Hz
00092     DLPF_6HZ = 0x06, // BW ≈ 6 Hz, NBW ≈ 8.3 Hz
00093     DLPF_473HZ = 0x07, // BW ≈ 473 Hz, NBW ≈ 499 Hz
00094 };
00095
00096 /**
00097  * ICM20948_IntPinConfig
00098  *
00099  * - Physical configuration of the INT pin (INT_PIN_CFG register, BANK 0)
00100  * - Controls electrical behavior and clear condition of the interrupt line
00101  * - Pass to intInit() to apply settings
00102  */
00103 struct ICM20948_IntPinConfig
00104 {
00105     uint8_t activeLevel : 1; // bit 7 | 0 = active high,      1 = active low
00106     uint8_t driveMode : 1; // bit 6 | 0 = push-pull,        1 = open-drain
00107     uint8_t latchMode : 1; // bit 5 | 0 = pulse 50µs,       1 = latch held
00108     uint8_t clearMode : 1; // bit 4 | 0 = read INT_STATUS,  1 = any read
00109     uint8_t fsyncActLevel : 1; // bit 3 | 0 = FSYNC active high, 1 = FSYNC active low
00110     uint8_t fsyncIntEn : 1; // bit 2 | 0 = FSYNC disabled,  1 = FSYNC as interrupt
00111     uint8_t bypassEn : 1; // bit 1 | 0 = normal,          1 = I2C bypass mode
00112 };
00113
00114 /**
00115  * ICM20948_IntEnableConfig
00116  *
00117  * - Selects which interrupt sources are routed to the INT pin
00118  * - Covers INT_ENABLE (0x10), INT_ENABLE_1 (0x11), INT_ENABLE_2 (0x12), INT_ENABLE_3 (0x13)
00119  * - FIFO overflow and watermark fields are per-channel arrays [0]-[4]
00120  * - Pass to intEnableConfig() to write all four registers in one call
00121  */
00122 struct ICM20948_IntEnableConfig
00123 {
00124     // --- INT_ENABLE (0x10) ---
00125     uint8_t wofEn : 1; // Wake on FSYNC
00126     uint8_t womIntEn : 1; // Wake on motion
00127     uint8_t pllRdyEn : 1; // PLL ready
00128     uint8_t dmpInt1En : 1; // DMP interrupt
00129     uint8_t i2cMstIntEn : 1; // I2C master interrupt
00130
00131     // --- INT_ENABLE_1 (0x11) ---
00132     uint8_t rawDataRdyEn : 1; // Raw data ready
00133
00134     // --- INT_ENABLE_2 (0x12) --- canal por canal
00135     uint8_t fifoOvfEn[5]; // [0]-[4]: 1 = overflow interrupt ON para ese canal
00136
00137     // --- INT_ENABLE_3 (0x13) --- canal por canal
00138     uint8_t fifoWmEn[5]; // [0]-[4]: 1 = watermark interrupt ON para ese canal
00139 };
00140
00141 /**
00142  * ICM20948_IntStatus
00143  *
00144  * - Holds the parsed state of the four interrupt status registers
00145  * - Covers INT_STATUS (0x19), INT_STATUS_1 (0x1A), INT_STATUS_2 (0x1B), INT_STATUS_3 (0x1C)
00146  * - Populated by checkIntStatus() via a single 4-byte burst read
00147  * - Reading these registers clears the interrupt flags (when clearMode = 0 in IntPinConfig)
00148  */
00149 struct ICM20948_IntStatus
00150 {
00151     // --- INT_STATUS (0x19) ---
00152     uint8_t womInt : 1; // Wake on motion ocurrió
00153     uint8_t pllRdyInt : 1; // PLL listo
00154     uint8_t dmpInt1 : 1; // DMP generó interrupción
00155     uint8_t i2cMstInt : 1; // I2C master generó interrupción
00156
00157     // --- INT_STATUS_1 (0x1A) ---
00158     uint8_t rawDataRdy : 1; // Datos de sensores listos para leer
00159
00160     // --- INT_STATUS_2 (0x1B) ---
00161     uint8_t fifoOvf[5]; // [0]-[4]: FIFO overflow por canal
00162
00163     // --- INT_STATUS_3 (0x1C) ---
00164     uint8_t fifoWm[5]; // [0]-[4]: FIFO watermark por canal
00165 };
00166
00167 /**
00168  * Class
00169  * - Manages ICM-20948 over I2C (SPI path disabled in this cut)
00170  * - Caches scale factors for LSB->physical conversions

```

```

00171 */
00172 class DevLab_ICM20948
00173 {
00174 public:
00175     DevLab_ICM20948();
00176
00177     /**
00178      * beginI2C
00179      *
00180      * - Initialize ICM20948 using I2C interface
00181      * - Sets up communication layer (Interface + BusIO)
00182      * - Verifies device identity (WHO_AM_I)
00183      * - Configures basic power and clock settings
00184      *
00185      * Returns:
00186      * - true → Device initialized successfully
00187      * - false → Communication or configuration failed
00188      */
00189     bool beginI2C(uint8_t address = 0x69, TwoWire &i2cPort = Wire, uint32_t i2cSpeed = 400000);
00190
00191     bool beginSPI(uint8_t csPin, SPIClass &spiPort = SPI, uint32_t spiSpeed = 1000000);
00192
00193     /**
00194      * readWhoAmI
00195      *
00196      * - Read WHO_AM_I register (device ID)
00197      * - Used during initialization to verify ICM20948
00198      * - Expected value: 0xEA
00199      *
00200      * Returns:
00201      * - true → Read successful
00202      * - false → Operation failed
00203      */
00204     bool readWhoAmI(uint8_t &whoAmI);
00205
00206     /**
00207      * selectBank
00208      *
00209      * - Select USER BANK (0-3)
00210      * - Used to access different register groups inside ICM20948
00211      *
00212      * Returns:
00213      * - true → Bank switch successful
00214      * - false → Write failed
00215      */
00216     bool selectBank(uint8_t bank);
00217
00218     /**
00219      * softReset
00220      *
00221      * - Perform software reset of ICM20948
00222      * - Resets all internal registers to default state
00223      *
00224      * Flow:
00225      * - Select USER BANK 0
00226      * - Set DEVICE_RESET bit
00227      * - Wait for device to restart
00228      *
00229      * Returns:
00230      * - true → Reset successful
00231      * - false → Operation failed
00232      */
00233     bool softReset();
00234
00235     /**
00236      * sleep
00237      *
00238      * - Enable or disable sleep mode
00239      * - Maintains clock source (CLKSEL = 1)
00240      *
00241      * Flow:
00242      * - Select USER BANK 0
00243      * - Set or clear SLEEP bit
00244      *
00245      * Returns:
00246      * - true → Operation successful
00247      * - false → Operation failed
00248      */
00249     bool sleep(bool en);
00250
00251     /**
00252      * applyBasicDefaults
00253      *
00254      * - Apply basic sensor configuration
00255      * - Configure power, filters, ranges, and sampling rates
00256      */
00257

```



```

00258     * Returns:
00259     * - true  → Configuration successful
00260     * - false → Any register write failed
00261     */
00262     bool applyBasicDefaults();
00263
00264     /**
00265     * readAccel
00266     *
00267     * - Read accelerometer data (X, Y, Z)
00268     * - Convert raw values to g using LSB scale
00269     *
00270     * Returns:
00271     * - true  → Read successful
00272     * - false → Read failed
00273     */
00274     bool readAccel(float &x, float &y, float &z);
00275
00276     /**
00277     * readGyro
00278     *
00279     * - Read gyroscope data (X, Y, Z)
00280     * - Convert raw values to dps using LSB scale
00281     *
00282     * Returns:
00283     * - true  → Read successful
00284     * - false → Read failed
00285     */
00286     bool readGyro(float &x, float &y, float &z);
00287
00288     /**
00289     * readTemperature
00290     *
00291     * - Read temperature sensor
00292     * - Convert raw values to °C
00293     *
00294     * Returns:
00295     * - true  → Read successful
00296     * - false → Read failed
00297     */
00298     bool readTemperature(float &temperature);
00299
00300     /**
00301     * readMag
00302     *
00303     * - Read magnetometer data via internal I2C master
00304     * - Data comes from EXT_SLV_SENS_DATA registers
00305     * - Convert raw values to µT
00306     *
00307     * Returns:
00308     * - true  → Read successful
00309     * - false → Read failed
00310     */
00311     bool readMag(float &x, float &y, float &z);
00312
00313     /**
00314     * initMag
00315     *
00316     * - Initialize AK09916 magnetometer using internal I2C master
00317     * - Verifies WHO_AM_I and enables continuous mode
00318     * - Configures SLV0 for automatic data read
00319     *
00320     * Returns:
00321     * - true  → Initialization successful
00322     * - false → Operation failed
00323     */
00324     bool initMag();
00325
00326     /**
00327     * setMagOpMode
00328     *
00329     * - Set magnetometer operation mode
00330     *
00331     * Returns:
00332     * - true  → Mode set successfully
00333     * - false → Write failed
00334     */
00335     bool setMagOpMode(ICM20948_Op_Mode opMode);
00336
00337     /**
00338     * setLowPower
00339     *
00340     * - Enable or disable low power mode
00341     * - Controls LP_EN bit in PWR_MGMT_1
00342     *
00343     * Returns:
00344     * - true  → Operation successful

```

```

00345     * - false → Operation failed
00346     */
00347     bool setLowPower(bool enable);
00348
00349     /**
00350     * getLowPower
00351     *
00352     * - Read low power mode status
00353     * - Returns state of LP_EN bit
00354     *
00355     * Returns:
00356     * - true → Read successful
00357     * - false → Operation failed
00358     */
00359     bool getLowPower(bool &enable);
00360
00361     /**
00362     * setClock
00363     *
00364     * - Set clock source (CLKSEL [2:0])
00365     * - Selects internal clock for sensor operation
00366     *
00367     * Returns:
00368     * - true → Operation successful
00369     * - false → Operation failed
00370     */
00371     bool setClock(ICM20948_Clock_Source clock);
00372
00373     /**
00374     * getClock
00375     *
00376     * - Read current clock source (CLKSEL [2:0])
00377     *
00378     * Returns:
00379     * - true → Read successful
00380     * - false → Operation failed
00381     */
00382     bool getClock(uint8_t &clock);
00383
00384     /**
00385     * setGyroSampleRate
00386     *
00387     * - Set gyroscope sample rate
00388     * - Uses internal divider (SRD) based on 1100 Hz base rate
00389     *
00390     * Returns:
00391     * - true → Operation successful
00392     * - false → Invalid input or write failed
00393     */
00394     bool setGyroSampleRate(float sampleRate);
00395
00396     /**
00397     * getGyroSampleRate
00398     *
00399     * - Get current gyroscope sample rate
00400     * - Computes value from divider register
00401     *
00402     * Returns:
00403     * - true → Read successful
00404     * - false → Operation failed
00405     */
00406     bool getGyroSampleRate(float &sampleRate);
00407     bool setGyroDivRate(uint8_t divisor);
00408
00409     /**
00410     * setDLPF
00411     *
00412     * - Configure gyroscope digital low-pass filter (DLPF)
00413     * - Option to bypass filter (full bandwidth)
00414     *
00415     * Returns:
00416     * - true → Operation successful
00417     * - false → Operation failed
00418     */
00419     bool setDLPF(ICM20948_Gyro_DLPF dlpf, bool bypass);
00420
00421     /**
00422     * getDLPF
00423     *
00424     * - Read gyroscope DLPF configuration
00425     * - Returns filter setting and bypass state
00426     *
00427     * Returns:
00428     * - true → Read successful
00429     * - false → Operation failed
00430     */
00431     bool getDLPF(uint8_t &dlpf, bool &bypass);

```

```

00432  /**
00433   * setGyroScale
00434   *
00435   * - Set gyroscope full-scale range (FS_SEL)
00436   * - Controls sensitivity (dps range)
00437   *
00438   * Flow:
00439   * - Validate bus pointer
00440   * - Select USER BANK 2
00441   * - Write FS_SEL bits [2:1]
00442   *
00443   * Returns:
00444   * - true → Operation successful
00445   * - false → Operation failed
00446   */
00447  bool setGyroScale(ICM20948_Gyro_FullScale fullScale);
00448  bool setGyroAveraging(ICM20948_Gyro_Average avg);
00449  /**
00450   * getGyroScale
00451   *
00452   * - Get current gyroscope full-scale range (FS_SEL)
00453   *
00454   * Returns:
00455   * - true → Read successful
00456   * - false → Operation failed
00457   */
00458  bool getGyroScale(uint8_t &fullScale);
00459
00460  /**
00461   * selfTestGyro
00462   *
00463   * - Enable or disable gyroscope self-test per axis
00464   *
00465   * Returns:
00466   * - true → Operation successful
00467   * - false → Operation failed
00468   */
00469  bool selfTestGyro(bool x, bool y, bool z);
00470
00471  bool setAccelDivRate(uint16_t divisor);
00472  /**
00473   * setAccelSampleRate
00474   *
00475   * - Set accelerometer output data rate (ODR)
00476   * - Uses 1125 Hz base rate with 12-bit divider
00477   *
00478   * Returns:
00479   * - true → Operation successful
00480   * - false → Operation failed
00481   */
00482  bool setAccelSampleRate(uint16_t sampleRate);
00483
00484  /**
00485   * getAccelSampleRate
00486   *
00487   * - Get accelerometer output data rate (ODR)
00488   * - Computes value from divider registers
00489   *
00490   * Returns:
00491   * - true → Read successful
00492   * - false → Operation failed
00493   */
00494  bool getAccelSampleRate(float &sampleRate);
00495
00496  /**
00497   * selfTestAccel
00498   *
00499   * - Enable or disable accelerometer self-test per axis
00500   *
00501   * Returns:
00502   * - true → Operation successful
00503   * - false → Operation failed
00504   */
00505  bool selfTestAccel(bool x, bool y, bool z);
00506
00507  /**
00508   * setAccelScale
00509   *
00510   * - Set accelerometer full-scale range (FS_SEL)
00511   * - Controls measurement range (±2g, ±4g, ±8g, ±16g)
00512   *
00513   * Returns:
00514   * - true → Operation successful
00515   * - false → Operation failed
00516   */
00517  bool setAccelScale(ICM20948_Accel_FullScale fullScale);
00518

```

```

00519  /**
00520  * getAccelScale
00521  *
00522  * - Get current accelerometer full-scale range (FS_SEL)
00523  *
00524  * Returns:
00525  * - true → Read successful
00526  * - false → Operation failed
00527  */
00528  bool getAccelScale(uint8_t &fullScale);
00529
00530  /**
00531  * setAccelDLPF
00532  *
00533  * - Configure accelerometer digital low-pass filter (DLPF)
00534  * - Option to bypass filter (full bandwidth)
00535  *
00536  * Flow:
00537  * - Validate bus pointer
00538  * - Select USER BANK 2
00539  * - Set or clear bypass bit
00540  * - Configure DLPF bits if not bypassed
00541  *
00542  * Returns:
00543  * - true → Operation successful
00544  * - false → Operation failed
00545  */
00546  bool setAccelDLPF(uint8_t dlpf, bool bypass);
00547
00548  /**
00549  * getAccelDLPF
00550  *
00551  * - Read accelerometer DLPF configuration
00552  * - Returns filter setting and bypass state
00553  *
00554  * Flow:
00555  * - Validate bus pointer
00556  * - Select USER BANK 2
00557  * - Read ACCEL_CONFIG register
00558  * - Extract bypass and DLPF bits
00559  *
00560  * Returns:
00561  * - true → Read successful
00562  * - false → Operation failed
00563  */
00564  bool getAccelDLPF(uint8_t &dlpf, bool &bypass);
00565
00566  /**
00567  * setAccelAveraging
00568  *
00569  * - Configure accelerometer averaging / decimation (DEC3)
00570  * - Controls internal averaging of accel samples
00571  *
00572  * Flow:
00573  * - Validate bus pointer
00574  * - Select USER BANK 2
00575  * - Write DEC3 bits [1:0]
00576  *
00577  * Returns:
00578  * - true → Operation successful
00579  * - false → Operation failed
00580  */
00581  bool setAccelAveraging(ICM20948_Accel_Average avg);
00582
00583  /**
00584  * getAccelAveraging
00585  *
00586  * - Read accelerometer averaging / decimation setting (DEC3)
00587  *
00588  * Flow:
00589  * - Validate bus pointer
00590  * - Select USER BANK 2
00591  * - Read DEC3 bits [1:0]
00592  *
00593  * Returns:
00594  * - true → Read successful
00595  * - false → Operation failed
00596  */
00597  bool getAccelAveraging(uint8_t &avg);
00598
00599  /**
00600  * setSensors
00601  *
00602  * - Enable or disable accel, gyro, and temperature sensor
00603  * - Controls power gating via PWR_MGMT_2 and TEMP_DIS
00604  *
00605  * Flow:

```

```

00606     * - Validate bus pointer
00607     * - Select USER BANK 0
00608     * - Build PWR_MGMT_2 mask
00609     * - Configure temperature enable/disable
00610     *
00611     * Returns:
00612     * - true → Operation successful
00613     * - false → Operation failed
00614     */
00615     bool setSensors(bool accel_on, bool gyro_on, bool temp_on);
00616
00617     /**
00618     * getSensors
00619     *
00620     * - Read accel, gyro, and temperature enable state
00621     *
00622     * Flow:
00623     * - Validate bus pointer
00624     * - Select USER BANK 0
00625     * - Read PWR_MGMT_2 register
00626     * - Read TEMP_DIS bit
00627     * - Decode enable states
00628     *
00629     * Returns:
00630     * - true → Read successful
00631     * - false → Operation failed
00632     */
00633     bool getSensors(bool &accel_on, bool &gyro_on, bool &temp_on);
00634
00635     /**
00636     * setGyroOffset
00637     *
00638     * - Set gyroscope offset values for X, Y, Z axes
00639     * - Writes offset registers in USER BANK 2
00640     *
00641     * Flow:
00642     * - Validate bus pointer
00643     * - Select USER BANK 2
00644     * - Pack offsets into byte array
00645     * - Write to offset registers
00646     *
00647     * Returns:
00648     * - true → Operation successful
00649     * - false → Operation failed
00650     */
00651     bool setGyroOffset(uint16_t offsetX, uint16_t offsetY, uint16_t offsetZ);
00652
00653     /**
00654     * getGyroOffset
00655     *
00656     * - Read gyroscope offset values for X, Y, Z axes
00657     * - Reads offset registers from USER BANK 2
00658     *
00659     * Flow:
00660     * - Validate bus pointer
00661     * - Select USER BANK 2
00662     * - Read offset registers
00663     * - Convert to signed values
00664     *
00665     * Returns:
00666     * - true → Read successful
00667     * - false → Operation failed
00668     */
00669     bool getGyroOffset(int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ);
00670
00671     /**
00672     * setAccelOffset
00673     *
00674     * - Set accelerometer offset values for X, Y, Z axes
00675     * - Writes offset registers in USER BANK 1
00676     *
00677     * Flow:
00678     * - Validate bus pointer
00679     * - Select USER BANK 1
00680     * - Pack offsets into byte array
00681     * - Write to offset registers
00682     *
00683     * Returns:
00684     * - true → Operation successful
00685     * - false → Operation failed
00686     */
00687     bool setAccelOffset(int16_t offsetX, int16_t offsetY, int16_t offsetZ);
00688
00689     /**
00690     * getAccelOffset
00691     *
00692     * - Read accelerometer offset values for X, Y, Z axes

```

```

00693     * - Reads offset registers from USER BANK 1
00694     *
00695     * Flow:
00696     * - Validate bus pointer
00697     * - Select USER BANK 1
00698     * - Read offset registers
00699     * - Convert to signed values
00700     *
00701     * Returns:
00702     * - true → Read successful
00703     * - false → Operation failed
00704     */
00705     bool getAccelOffset(int16_t &offsetX, int16_t &offsetY, int16_t &offsetZ);
00706
00707     /**
00708     * intInit
00709     *
00710     * - Configure the physical behavior of the INT pin (INT_PIN_CFG register, BANK 0)
00711     * - Sets active level, drive mode (push-pull / open-drain), latch vs pulse mode,
00712     *   clear condition, FSYNC level, FSYNC interrupt enable, and I2C bypass
00713     *
00714     * Parameters:
00715     * - cfg: ICM20948_IntPinConfig struct with the desired pin settings
00716     *
00717     * Returns:
00718     * - true → Configuration written successfully
00719     * - false → Operation failed
00720     */
00721     bool intInit(const ICM20948_IntPinConfig &cfg);
00722
00723     /**
00724     * intEnableConfig
00725     *
00726     * - Enable or disable individual interrupt sources routed to the INT pin
00727     * - Writes INT_ENABLE (0x10), INT_ENABLE_1 (0x11), INT_ENABLE_2 (0x12),
00728     *   and INT_ENABLE_3 (0x13) in BANK 0
00729     * - Must call intInit() first to configure the pin electrical behavior
00730     *
00731     * Parameters:
00732     * - cfg: ICM20948_IntEnableConfig struct with the desired interrupt sources enabled
00733     *
00734     * Returns:
00735     * - true → All four registers written successfully
00736     * - false → Operation failed
00737     */
00738     bool intEnableConfig(const ICM20948_IntEnableConfig &cfg);
00739
00740     /**
00741     * checkIntStatus
00742     *
00743     * - Read and parse all four interrupt status registers in a single burst read
00744     * - Covers INT_STATUS (0x19), INT_STATUS_1 (0x1A), INT_STATUS_2 (0x1B),
00745     *   INT_STATUS_3 (0x1C) from BANK 0
00746     * - Reading clears the interrupt flags when clearMode = 0 in ICM20948_IntPinConfig
00747     * - Typically called inside the INT pin ISR or polled in the main loop
00748     *
00749     * Parameters:
00750     * - status: ICM20948_IntStatus struct populated with the current interrupt flags
00751     *
00752     * Returns:
00753     * - true → Status read successfully
00754     * - false → Operation failed
00755     */
00756     bool checkIntStatus(ICM20948_IntStatus &status);
00757
00758     /**
00759     * auxMasterEnable
00760     *
00761     * - Enable the ICM20948 internal I2C master for auxiliary bus
00762     * - Clears BYPASS_EN so the aux bus is controlled by the ICM
00763     * - Sets I2C_MST_EN in USER_CTRL
00764     * - Configures auxiliary master clock frequency
00765     *
00766     * Parameters:
00767     * - clkFreq: clock divider value (e.g. 0x07 ≈ 345.6 kHz, 0x0D ≈ 400 kHz)
00768     *
00769     * Returns:
00770     * - true → Master enabled successfully
00771     * - false → Operation failed
00772     */
00773     bool auxMasterEnable(uint8_t clkFreq);
00774
00775     /**
00776     * auxWriteByte
00777     *
00778     * - Write a single byte to a register of an auxiliary I2C slave via SLV4
00779     * - Uses one-shot SLV4 transaction: polls SLV4_DONE, checks for NACK

```

```

00780 * - Suitable for configuration writes (not continuous streaming)
00781 *
00782 * Parameters:
00783 * - slaveAddr: 7-bit I2C address of the auxiliary slave
00784 * - reg:      target register address on the slave
00785 * - data:     byte to write
00786 *
00787 * Returns:
00788 * - true  → Write acknowledged by slave
00789 * - false → Timeout or NACK
00790 */
00791 bool auxWriteByte(uint8_t slaveAddr, uint8_t reg, uint8_t data);
00792
00793 /**
00794 * auxReadByte
00795 *
00796 * - Read a single byte from a register of an auxiliary I2C slave via SLV4
00797 * - Generates a combined write-then-read transaction (reg byte + repeated START)
00798 * - Suitable for slaves that follow standard I2C register-read protocol
00799 *
00800 * Note: slaves that need separate write/read transactions (e.g. command-response
00801 * firmware) should use auxWriteCommand + auxReadResponse instead.
00802 *
00803 * Parameters:
00804 * - slaveAddr: 7-bit I2C address of the auxiliary slave
00805 * - reg:      register address to read from
00806 * - data:     output byte received from slave
00807 *
00808 * Returns:
00809 * - true  → Read successful
00810 * - false → Timeout or NACK
00811 */
00812 bool auxReadByte(uint8_t slaveAddr, uint8_t reg, uint8_t &data);
00813
00814 /**
00815 * auxWriteCommand
00816 *
00817 * - Send a single command byte to an auxiliary I2C slave via SLV4
00818 * - Uses REG_DIS: generates [START, addr+W, cmd_byte, STOP] with no register prefix
00819 * - Designed for slaves that use a command-response protocol (no register addressing)
00820 *
00821 * Parameters:
00822 * - slaveAddr: 7-bit I2C address of the auxiliary slave
00823 * - cmd:      command byte to send
00824 *
00825 * Returns:
00826 * - true  → Command acknowledged by slave
00827 * - false → Timeout or NACK
00828 */
00829 bool auxWriteCommand(uint8_t slaveAddr, uint8_t cmd);
00830
00831 /**
00832 * auxConfigSlave
00833 *
00834 * - Configure SLV0 for automatic periodic reads from an auxiliary I2C slave
00835 * - The ICM20948 master will read numBytes starting at reg on every ODR cycle
00836 * - Data is deposited into EXT_SLV_SENS_DATA_00 and subsequent registers
00837 * - Call once during initialization; read results with auxReadSensorData
00838 *
00839 * Parameters:
00840 * - slaveAddr: 7-bit I2C address of the auxiliary slave
00841 * - reg:      starting register address to read from on the slave
00842 * - numBytes: number of bytes to read per cycle (1-15)
00843 *
00844 * Returns:
00845 * - true  → SLV0 configured successfully
00846 * - false → Operation failed
00847 */
00848 bool auxConfigSlave(uint8_t slaveAddr, uint8_t reg, uint8_t numBytes);
00849
00850 /**
00851 * auxReadSensorData
00852 *
00853 * - Read bytes from EXT_SLV_SENS_DATA registers (BANK 0)
00854 * - Returns data collected by the ICM20948 master from SLV0-SLV3 on the last cycle
00855 * - Must call auxConfigSlave first to set up the automatic read
00856 *
00857 * Parameters:
00858 * - buf: output buffer to store the received bytes
00859 * - len: number of bytes to read (must match numBytes configured in auxConfigSlave)
00860 *
00861 * Returns:
00862 * - true  → Read successful
00863 * - false → Bus error
00864 */
00865 bool auxReadSensorData(uint8_t *buf, uint8_t len);
00866

```

```

00867  /**
00868   * auxReadResponse
00869   *
00870   * - Read a single response byte from an auxiliary I2C slave via SLV4
00871   * - Generates a pure read transaction: [START, addr+R, 1 byte, STOP]
00872   * - No register byte is sent (REG_DIS); slave must already be ready to transmit
00873   * - Use after auxWriteCommand to complete a command-response exchange with slaves
00874   *   that require separate write and read transactions (e.g. PY32F0 firmware slaves)
00875   *
00876   * Parameters:
00877   * - slaveAddr: 7-bit I2C address of the auxiliary slave
00878   * - data:      output byte received from slave
00879   *
00880   * Returns:
00881   * - true  → Response received successfully
00882   * - false → Timeout or NACK
00883   */
00884 bool auxReadResponse(uint8_t slaveAddr, uint8_t &data);
00885
00886  /**
00887   * auxRead12bit
00888   *
00889   * - Read a 12-bit value previously configured via auxConfigSlave (SLV0, numBytes = 2)
00890   * - Assumes little-endian packing: first byte = LSB, second byte = MSB
00891   * - Result is masked to 12 bits (0-4095)
00892   *
00893   * Returns:
00894   * - true  → Read successful
00895   * - false → Read failed
00896   */
00897 bool auxRead12bit(uint16_t &raw);
00898 private:
00899 I2C_Interface i2c;
00900 SPI_Interface spi;
00901
00902 Interface_7Semi *iface = nullptr;
00903
00904 BusIO_7Semi<Interface_7Semi> *bus = nullptr;
00905
00906 float mg_per_lsb = 16384.0f; // LSB/g at ±16g
00907 float degree_per_second = 131.072f; // LSB/dps at ±2000 dps
00908
00909 bool writeSlave4(uint8_t reg, uint8_t value);
00910 bool readSlave4(uint8_t reg, uint8_t &value);
00911 };
00912
00913
00914 #endif /* ICM20948_DEVLAB_H */

```

## 6.38 src/ICM20948\_\_platform.h File Reference

### Classes

- struct `icm_plat_t`

## 6.39 ICM20948\_\_platform.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ICM20948_PLATFORM_H
00002 #define ICM20948_PLATFORM_H
00003
00004 // #include <stdint.h>
00005 // #include <stddef.h>
00006
00007 /**
00008  * 7Semi comment style
00009  * - Platform glue for bus IO, delays, and timing
00010  * - Implement these for your system (I2C or SPI)
00011  */
00012
00013 typedef struct {
00014     /**
00015      * - Bus read: read `len` bytes from `reg` into `buf`
00016      * - Return 0 on success, nonzero on error
00017      */
00018     int (*read)(uint8_t reg, uint8_t *buf, size_t len, void *user);
00019     /**
00020      * - Bus write: write `len` bytes from `buf` to `reg`
00021      * - Return 0 on success, nonzero on error
00022      */
00023     int (*write)(uint8_t reg, const uint8_t *buf, size_t len, void *user);

```



```

00024  /**
00025   * - Optional: SPI transfers may need register R/W bit mangling
00026   * - For I2C, leave as NULL
00027  */
00028  uint8_t (*spi_reg_rw_bits)(uint8_t reg, int is_write);
00029  /**
00030   * - Millisecond delay
00031  */
00032  void (*delay_ms)(uint32_t ms);
00033  /**
00034   * - User context pointer passed to read/write
00035  */
00036  void *user;
00037  /**
00038   * - If using SPI: set chip select active (1) or inactive (0)
00039   * - For I2C: leave as NULL
00040  */
00041  void (*spi_cs)(int level, void *user);
00042  /**
00043   * - Device address (I2C) or ignored for SPI if you wrap inside user
00044  */
00045  uint8_t i2c_addr;
00046 } icm_plat_t;
00047
00048 #endif

```

## 6.40 src/ICM20948\_regs.h File Reference

This graph shows which files directly or indirectly include this file:



## Macros

- #define WHO\_AM\_I 0x00 /\* WHO\_AM\_I[7:0] \*/
- #define WHO\_AM\_I\_VAL 0xEA
- #define USER\_CTRL 0x03 /\* DMP\_EN FIFO\_EN I2C\_MST\_EN I2C\_IF\_DIS DMP\_RST SRAM\_RST I2C\_MST\_RST - \*/
- #define USER\_CTRL\_DMP\_EN BIT(7)
- #define USER\_CTRL\_FIFO\_EN BIT(6)
- #define USER\_CTRL\_I2C\_MST\_EN BIT(5)
- #define USER\_CTRL\_I2C\_IF\_DIS BIT(4)
- #define USER\_CTRL\_DMP\_RST BIT(3)
- #define USER\_CTRL\_SRAM\_RST BIT(2)
- #define USER\_CTRL\_I2C\_MST\_RST BIT(1)
- #define LP\_CONFIG 0x05 /\* I2C\_MST\_CYCLE ACCEL\_CYCLE GYRO\_CYCLE - \*/
- #define LP\_I2C\_MST\_CYCLE BIT(6)
- #define LP\_ACCEL\_CYCLE BIT(5)
- #define LP\_GYRO\_CYCLE BIT(4)
- #define PWR\_MGMT\_1 0x06 /\* DEVICE\_RESET SLEEP LP\_EN - TEMP\_DIS CLKSEL[2:0] \*/
- #define PWR\_DEVICE\_RESET BIT(7)
- #define PWR\_SLEEP BIT(6)
- #define PWR\_LP\_EN BIT(5)
- #define PWR\_TEMP\_DIS BIT(3)
- #define PWR\_CLKSEL\_MASK 0x07
- #define PWR\_CLKSEL\_INT\_20MHZ 0x01
- #define PWR\_CLKSEL\_AUTO 0x01 /\* typical: auto selects best source \*/
- #define PWR\_MGMT\_2 0x07 /\* - DISABLE\_ACCEL DISABLE\_GYRO \*/
- #define PWR\_DISABLE\_ACCEL BIT(3)
- #define PWR\_DISABLE\_GYRO BIT(0)

- #define INT\_PIN\_CFG 0x0F /\* INT1\_ACTL INT1\_OPEN INT1\_LATCH\_INT\_EN  
INT\_ANYRD\_2CLEAR ACTL\_FSYNC FSYNC\_INT\_MODE\_EN BYPASS\_EN - \*/
- #define INT1\_ACTL BIT(7) /\* active low \*/
- #define INT1\_OPEN BIT(6) /\* open-drain \*/
- #define INT1\_LATCH\_INT\_EN BIT(5) /\* latch until status read \*/
- #define INT\_ANYRD\_2CLEAR BIT(4)
- #define INT\_ACTL\_FSYNC BIT(3)
- #define FSYNC\_INT\_MODE\_EN BIT(2)
- #define BYPASS\_EN BIT(1) /\* I2C bypass to aux devices \*/
- #define INT\_ENABLE 0x10 /\* REG\_WOF\_EN - WOM\_INT\_EN PLL\_RDY\_EN DMP\_↵  
INT1\_EN I2C\_MST\_INT\_EN \*/
- #define INT\_REG\_WOF\_EN BIT(7)
- #define INT\_WOM\_INT\_EN BIT(3)
- #define INT\_PLL\_RDY\_EN BIT(2)
- #define INT\_DMP\_INT1\_EN BIT(1)
- #define INT\_I2C\_MST\_INT\_EN BIT(0)
- #define INT\_ENABLE\_1 0x11 /\* RAW\_DATA\_0\_RDY\_EN \*/
- #define INT\_RAW\_DATA\_0\_RDY\_EN BIT(0)
- #define INT\_ENABLE\_2 0x12 /\* FIFO\_OVERFLOW\_EN[4:0] \*/
- #define INT\_ENABLE\_3 0x13 /\* FIFO\_WM\_EN[4:0] \*/
- #define I2C\_MST\_STATUS 0x17 /\* PASS\_THROUGH, \*\_DONE, \*\_NACK, LOST\_ARB \*/
- #define MST\_PASS\_THROUGH BIT(7)
- #define MST\_SLV4\_DONE BIT(6)
- #define MST\_LOST\_ARB BIT(5)
- #define MST\_SLV4\_NACK BIT(4)
- #define MST\_SLV3\_NACK BIT(3)
- #define MST\_SLV2\_NACK BIT(2)
- #define MST\_SLV1\_NACK BIT(1)
- #define MST\_SLV0\_NACK BIT(0)
- #define INT\_STATUS 0x19 /\* WOM\_INT PLL\_RDY\_INT DMP\_INT1 I2C\_MST\_INT \*/
- #define STS\_WOM\_INT BIT(3)
- #define STS\_PLL\_RDY\_INT BIT(2)
- #define STS\_DMP\_INT1 BIT(1)
- #define STS\_I2C\_MST\_INT BIT(0)
- #define INT\_STATUS\_1 0x1A /\* RAW\_DATA\_0\_RDY\_INT \*/
- #define STS\_RAW\_DATA\_0\_RDY\_INT BIT(0)
- #define INT\_STATUS\_2 0x1B /\* FIFO\_OVERFLOW\_INT[4:0] \*/
- #define INT\_STATUS\_3 0x1C /\* FIFO\_WM\_INT[4:0] \*/
- #define DELAY\_TIMEH 0x28
- #define DELAY\_TIMEL 0x29
- #define ACCEL\_XOUT\_H 0x2D
- #define ACCEL\_XOUT\_L 0x2E
- #define ACCEL\_YOUT\_H 0x2F
- #define ACCEL\_YOUT\_L 0x30
- #define ACCEL\_ZOUT\_H 0x31
- #define ACCEL\_ZOUT\_L 0x32
- #define GYRO\_XOUT\_H 0x33
- #define GYRO\_XOUT\_L 0x34
- #define GYRO\_YOUT\_H 0x35
- #define GYRO\_YOUT\_L 0x36
- #define GYRO\_ZOUT\_H 0x37
- #define GYRO\_ZOUT\_L 0x38
- #define TEMP\_OUT\_H 0x39
- #define TEMP\_OUT\_L 0x3A
- #define EXT\_SLV\_SENS\_DATA\_00 0x3B

- #define `EXT_SLV_SENS_DATA_23` 0x52 /\* contiguous range 0x3B..0x52 \*/
- #define `FIFO_EN_1` 0x66 /\* SLV3..SLV0 FIFO\_EN \*/
- #define `FIFO_EN_2` 0x67 /\* ACCEL GYRO\_Z/Y/X TEMP FIFO\_EN \*/
- #define `FIFO_RST` 0x68 /\* FIFO\_RESET[4:0] \*/
- #define `FIFO_MODE` 0x69 /\* FIFO\_MODE[4:0] \*/
- #define `FIFO_COUNTH` 0x70
- #define `FIFO_COUNTL` 0x71
- #define `FIFO_R_W` 0x72
- #define `FIFO_CFG` 0x76
- #define `REG_BANK_SEL` 0x7F /\* USER\_BANK[1:0] \*/
- #define `SELF_TEST_X_GYRO` 0x02 /\* XG\_ST\_DATA[7:0] \*/
- #define `SELF_TEST_Y_GYRO` 0x03 /\* YG\_ST\_DATA[7:0] \*/
- #define `SELF_TEST_Z_GYRO` 0x04 /\* ZG\_ST\_DATA[7:0] \*/
- #define `SELF_TEST_X_ACCEL` 0x0E /\* XA\_ST\_DATA[7:0] \*/
- #define `SELF_TEST_Y_ACCEL` 0x0F /\* YA\_ST\_DATA[7:0] \*/
- #define `SELF_TEST_Z_ACCEL` 0x10 /\* ZA\_ST\_DATA[7:0] \*/
- #define `XA_OFFS_H` 0x14 /\* XA\_OFFS[14:7] \*/
- #define `XA_OFFS_L` 0x15 /\* XA\_OFFS[6:0] \*/
- #define `YA_OFFS_H` 0x17 /\* YA\_OFFS[14:7] \*/
- #define `YA_OFFS_L` 0x18 /\* YA\_OFFS[6:0] \*/
- #define `ZA_OFFS_H` 0x1A /\* ZA\_OFFS[14:7] \*/
- #define `ZA_OFFS_L` 0x1B /\* ZA\_OFFS[6:0] \*/
- #define `TIMEBASE_CORRECTION_PLL` 0x28 /\* TBC\_PLL[7:0] \*/
- #define `BANK1_REG_BANK_SEL` 0x7F /\* mirror of `REG_BANK_SEL` \*/
- #define `GYRO_SMPLRT_DIV` 0x00 /\* GYRO\_SMPLRT\_DIV[7:0] \*/
- #define `GYRO_CONFIG_1` 0x01 /\* GYRO\_DLPFCFG[2:0] GYRO\_FS\_SEL[1:0] `GYRO_FCHOICE` \*/
- #define `GYRO_FCHOICE` BIT(0) /\* when 0: DLPF on, when 1: off (per datasheet) \*/
- #define `GYRO_FS_SEL_SHIFT` 1
- #define `GYRO_FS_SEL_MASK` (0x3u << `GYRO_FS_SEL_SHIFT`)
- #define `GYRO_FS_250DPS` (0u << `GYRO_FS_SEL_SHIFT`)
- #define `GYRO_FS_500DPS` (1u << `GYRO_FS_SEL_SHIFT`)
- #define `GYRO_FS_1000DPS` (2u << `GYRO_FS_SEL_SHIFT`)
- #define `GYRO_FS_2000DPS` (3u << `GYRO_FS_SEL_SHIFT`)
- #define `GYRO_DLPFCFG_SHIFT` 5
- #define `GYRO_DLPFCFG_MASK` (0x7u << `GYRO_DLPFCFG_SHIFT`)
- #define `GYRO_CONFIG_2` 0x02 /\* `XGYRO_CTEN` `YGYRO_CTEN` `ZGYRO_CTEN` `GYRO_AVGCFG`[2:0] \*/
- #define `XGYRO_CTEN` BIT(5)
- #define `YGYRO_CTEN` BIT(4)
- #define `ZGYRO_CTEN` BIT(3)
- #define `GYRO_AVGCFG_SHIFT` 0
- #define `GYRO_AVGCFG_MASK` (0x7u << `GYRO_AVGCFG_SHIFT`)
- #define `XG_OFFS_USRH` 0x03
- #define `XG_OFFS_USRL` 0x04
- #define `YG_OFFS_USRH` 0x05
- #define `YG_OFFS_USRL` 0x06
- #define `ZG_OFFS_USRH` 0x07
- #define `ZG_OFFS_USRL` 0x08
- #define `ODR_ALIGN_EN` 0x09 /\* ODR\_ALIGN\_EN \*/
- #define `ODR_ALIGN_EN_BIT` BIT(0)
- #define `ACCEL_SMPLRT_DIV_1` 0x10 /\* ACCEL\_SMPLRT\_DIV[11:8] \*/
- #define `ACCEL_SMPLRT_DIV_2` 0x11 /\* ACCEL\_SMPLRT\_DIV[7:0] \*/
- #define `ACCEL_INTEL_CTRL` 0x12 /\* `ACCEL_INTEL_EN` `ACCEL_INTEL_MODE_INT` \*/

- #define ACCEL\_INTEL\_EN BIT(7)
- #define ACCEL\_INTEL\_MODE\_INT BIT(6)
- #define ACCEL\_WOM\_THR 0x13 /\* WOM\_THRESHOLD[7:0] \*/
- #define ACCEL\_CONFIG 0x14 /\* ACCEL\_DLPFCFG[2:0] ACCEL\_FS\_SEL[1:0] ACCEL\_FCHOICE \*/
- #define ACCEL\_FCHOICE BIT(0)
- #define ACCEL\_FS\_SEL\_SHIFT 1
- #define ACCEL\_FS\_SEL\_MASK (0x3u << ACCEL\_FS\_SEL\_SHIFT)
- #define ACCEL\_FS\_2G (0u << ACCEL\_FS\_SEL\_SHIFT)
- #define ACCEL\_FS\_4G (1u << ACCEL\_FS\_SEL\_SHIFT)
- #define ACCEL\_FS\_8G (2u << ACCEL\_FS\_SEL\_SHIFT)
- #define ACCEL\_FS\_16G (3u << ACCEL\_FS\_SEL\_SHIFT)
- #define ACCEL\_DLPFCFG\_SHIFT 5
- #define ACCEL\_DLPFCFG\_MASK (0x7u << ACCEL\_DLPFCFG\_SHIFT)
- #define ACCEL\_CONFIG\_2 0x15 /\* AX\_ST\_EN\_REG AY\_ST\_EN\_REG AZ\_ST\_EN\_REG DEC3\_CFG[1:0] \*/
- #define AX\_ST\_EN\_REG BIT(7)
- #define AY\_ST\_EN\_REG BIT(6)
- #define AZ\_ST\_EN\_REG BIT(5)
- #define DEC3\_CFG\_SHIFT 0
- #define DEC3\_CFG\_MASK (0x3u << DEC3\_CFG\_SHIFT)
- #define FSYNC\_CONFIG 0x52 /\* DELAY\_TIME\_EN WOF\_DEGLITCH\_EN WOF\_EDGE\_INT EXT\_SYNC\_SET[3:0] \*/
- #define DELAY\_TIME\_EN BIT(7)
- #define WOF\_DEGLITCH\_EN BIT(6)
- #define WOF\_EDGE\_INT BIT(5)
- #define EXT\_SYNC\_SET\_MASK 0x0F
- #define TEMP\_CONFIG 0x53 /\* TEMP\_DLPFCFG[2:0] \*/
- #define TEMP\_DLPFCFG\_SHIFT 5
- #define TEMP\_DLPFCFG\_MASK (0x7u << TEMP\_DLPFCFG\_SHIFT)
- #define MOD\_CTRL\_USR 0x54 /\* REG\_LP\_DMP\_EN \*/
- #define REG\_LP\_DMP\_EN BIT(7)
- #define BANK2\_REG\_BANK\_SEL 0x7F /\* mirror of REG\_BANK\_SEL \*/
- #define I2C\_MST\_ODR\_CONFIG 0x00 /\* I2C\_MST\_ODR\_CONFIG[3:0] \*/
- #define MST\_ODR\_CFG\_MASK 0x0F
- #define I2C\_MST\_CTRL 0x01 /\* MULT\_MST\_EN - I2C\_MST\_P\_NSR I2C\_MST\_CLK[3:0] \*/
- #define MULT\_MST\_EN BIT(7)
- #define I2C\_MST\_P\_NSR BIT(4)
- #define I2C\_MST\_CLK\_MASK 0x0F
- #define I2C\_MST\_DELAY\_CTRL 0x02 /\* DELAY\_ES\_SHADOW + I2C\_SLVx\_DELAY\_↔ EN bits \*/
- #define DELAY\_ES\_SHADOW BIT(7)
- #define I2C\_SLV4\_DELAY\_EN BIT(4)
- #define I2C\_SLV3\_DELAY\_EN BIT(3)
- #define I2C\_SLV2\_DELAY\_EN BIT(2)
- #define I2C\_SLV1\_DELAY\_EN BIT(1)
- #define I2C\_SLV0\_DELAY\_EN BIT(0)
- #define I2C\_SLV0\_ADDR 0x03 /\* RNW + ID[6:0] \*/
- #define I2C\_SLVx\_RNW BIT(7)
- #define I2C\_SLV0\_REG 0x04
- #define I2C\_SLV0\_CTRL 0x05 /\* EN BYTE\_SW REG\_DIS GRP LENG[3:0] \*/
- #define I2C\_SLVx\_EN BIT(7)
- #define I2C\_SLVx\_BYTE\_SW BIT(6)
- #define I2C\_SLVx\_REG\_DIS BIT(5)

- #define I2C\_SLVx\_GRP BIT(4)
- #define I2C\_SLVx LENG\_MASK 0x0F
- #define I2C\_SLV0\_DO 0x06
- #define I2C\_SLV1\_ADDR 0x07
- #define I2C\_SLV1\_REG 0x08
- #define I2C\_SLV1\_CTRL 0x09
- #define I2C\_SLV1\_DO 0x0A
- #define I2C\_SLV2\_ADDR 0x0B
- #define I2C\_SLV2\_REG 0x0C
- #define I2C\_SLV2\_CTRL 0x0D
- #define I2C\_SLV2\_DO 0x0E
- #define I2C\_SLV3\_ADDR 0x0F
- #define I2C\_SLV3\_REG 0x10
- #define I2C\_SLV3\_CTRL 0x11
- #define I2C\_SLV3\_DO 0x12
- #define I2C\_SLV4\_ADDR 0x13
- #define I2C\_SLV4\_REG 0x14
- #define I2C\_SLV4\_CTRL 0x15 /\* EN BYTE\_SW REG\_DIS DLY[4:0] \*/
- #define I2C\_SLV4\_DLY\_MASK 0x1F
- #define I2C\_SLV4\_DO 0x16
- #define I2C\_SLV4\_DI 0x17
- #define BANK3\_REG\_BANK\_SEL 0x7F /\* mirror of REG\_BANK\_SEL \*/
- #define REG\_BANK\_SEL\_USER\_BANK\_SHIFT 4
- #define REG\_BANK\_SEL\_USER\_BANK\_MASK (0x3u << REG\_BANK\_SEL\_USER\_BANK\_SHIFT)
- #define USER\_BANK\_0 BANK0
- #define USER\_BANK\_1 BANK1
- #define USER\_BANK\_2 BANK2
- #define USER\_BANK\_3 BANK3
- #define AK09916\_I2C\_ADDR 0x0C
- #define AK\_WIA2 0x01
- #define AK\_ST1 0x10
- #define AK\_HXL 0x11
- #define AK\_ST2 0x18
- #define AK\_CNTL2 0x31
- #define AK\_CNTL3 0x32
- #define AK\_WIA2\_VAL 0x09 /\* AK09916C WIA2 expected \*/
- #define INTERNAL\_20MHZ 0x00 /\* 0: Internal 20 MHz RC \*/
- #define AUTO\_SEL 0x01 /\* 15: Auto/PLL preferred (use 1 as default) \*/
- #define CLK\_STOP 0x07 /\* 7: Stop clock / timing gen reset \*/
- #define g2 0
- #define g4 1
- #define g8 2
- #define g16 3
- #define ACCEL\_FCHOICE\_BYPASS 0
- #define ACCEL\_FCHOICE\_DLPF 1
- #define ACCEL\_DLPFCFG\_0 0
- #define ACCEL\_DLPFCFG\_1 1
- #define ACCEL\_DLPFCFG\_2 2
- #define ACCEL\_DLPFCFG\_3 3
- #define ACCEL\_DLPFCFG\_4 4
- #define ACCEL\_DLPFCFG\_5 5
- #define ACCEL\_DLPFCFG\_6 6
- #define ACCEL\_DLPFCFG\_7 7
- #define ACCEL\_DEC3\_AVG\_4 0u /\* averages 1 or 4 (depends on FCHOICE) \*/
- #define ACCEL\_DEC3\_AVG\_8 1u

- #define ACCEL\_DEC3\_AVG\_16 2u
- #define ACCEL\_DEC3\_AVG\_32 3u
- #define ACCEL\_SMPLRT\_DIV\_MIN 0u
- #define ACCEL\_SMPLRT\_DIV\_MAX 4095u
- #define ACCEL\_DLPF\_BASE\_HZ 1125.0f
- #define ACCEL\_DLPF\_ODR\_HZ(div) (ACCEL\_DLPF\_BASE\_HZ / (1.0f + (float)(div)))
- #define ACCEL\_BYPASS\_3DB\_BW\_HZ 1209.0f
- #define ACCEL\_BYPASS\_NBW\_HZ 1248.0f
- #define ACCEL\_BYPASS\_RATE\_HZ 4500.0f
- #define ACCEL\_DLPF0\_3DB\_BW\_HZ 246.0f
- #define ACCEL\_DLPF0\_NBW\_HZ 265.0f
- #define ACCEL\_DLPF1\_3DB\_BW\_HZ 246.0f
- #define ACCEL\_DLPF1\_NBW\_HZ 265.0f
- #define ACCEL\_DLPF2\_3DB\_BW\_HZ 111.4f
- #define ACCEL\_DLPF2\_NBW\_HZ 136.0f
- #define ACCEL\_DLPF3\_3DB\_BW\_HZ 50.4f
- #define ACCEL\_DLPF3\_NBW\_HZ 68.8f
- #define ACCEL\_DLPF4\_3DB\_BW\_HZ 23.9f
- #define ACCEL\_DLPF4\_NBW\_HZ 34.4f
- #define ACCEL\_DLPF5\_3DB\_BW\_HZ 11.5f
- #define ACCEL\_DLPF5\_NBW\_HZ 17.0f
- #define ACCEL\_DLPF6\_3DB\_BW\_HZ 5.7f
- #define ACCEL\_DLPF6\_NBW\_HZ 8.3f
- #define ACCEL\_DLPF7\_3DB\_BW\_HZ 473.0f
- #define ACCEL\_DLPF7\_NBW\_HZ 499.0f
- #define dps250 0
- #define dps500 1
- #define dps1000 2
- #define dps2000 3
- #define GYRO\_FCHOICE\_BYPASS 0
- #define GYRO\_FCHOICE\_DLPF 1
- #define GYRO\_DLPFCFG\_0 0
- #define GYRO\_DLPFCFG\_1 1
- #define GYRO\_DLPFCFG\_2 2
- #define GYRO\_DLPFCFG\_3 3
- #define GYRO\_DLPFCFG\_4 4
- #define GYRO\_DLPFCFG\_5 5
- #define GYRO\_DLPFCFG\_6 6
- #define GYRO\_DLPFCFG\_7 7
- #define GYRO\_SMPLRT\_MIN\_HZ 4.3f
- #define GYRO\_DLPF\_BASE\_HZ 1100.0f
- #define GYRO\_DLPF\_ODR\_HZ(rate\_request) (rate\_request) /\* symbolic; your driver computes DIV = round(1100/rate)-1 \*/
- #define GYRO\_BW\_ULTRA\_WIDE 0 /\* bypass path \*/
- #define GYRO\_BW\_WIDE 1 /\* e.g., CFG 0/1 \*/
- #define GYRO\_BW\_MEDIUM 2 /\* e.g., CFG 2/3 \*/
- #define GYRO\_BW\_NARROW 3 /\* e.g., CFG 4/5/6/7 \*/
- #define Hz2 0
- #define Hz6 1
- #define Hz8 2
- #define Hz10 3
- #define Hz15 4
- #define Hz20 5
- #define Hz25 6
- #define Hz30 7

- #define [LowPower](#) 0
- #define [Regular](#) 1
- #define [EnhancedRegular](#) 2
- #define [HighAccuracy](#) 3

## 6.40.1 Macro Definition Documentation

### 6.40.1.1 ACCEL\_BYPASS\_3DB\_BW\_HZ

```
#define ACCEL_BYPASS_3DB_BW_HZ 1209.0f
```

Definition at line [369](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.2 ACCEL\_BYPASS\_NBW\_HZ

```
#define ACCEL_BYPASS_NBW_HZ 1248.0f
```

Definition at line [370](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.3 ACCEL\_BYPASS\_RATE\_HZ

```
#define ACCEL_BYPASS_RATE_HZ 4500.0f
```

Definition at line [371](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.4 ACCEL\_CONFIG

```
#define ACCEL_CONFIG 0x14 /* ACCEL_DLPFCFG[2:0] ACCEL_FS_SEL[1:0] ACCEL\_FCHOICE */
```

Definition at line [209](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.5 ACCEL\_CONFIG\_2

```
#define ACCEL_CONFIG_2 0x15 /* AX\_ST\_EN\_REG AY\_ST\_EN\_REG AZ\_ST\_EN\_REG DEC3_CFG[1:0] */
```

Definition at line [220](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.6 ACCEL\_DEC3\_AVG\_16

```
#define ACCEL_DEC3_AVG_16 2u
```

Definition at line [356](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.7 ACCEL\_DEC3\_AVG\_32

```
#define ACCEL_DEC3_AVG_32 3u
```

Definition at line [357](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.8 ACCEL\_DEC3\_AVG\_4

```
#define ACCEL_DEC3_AVG_4 0u /* averages 1 or 4 (depends on FCHOICE) */
```

Definition at line [354](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.9 ACCEL\_DEC3\_AVG\_8

```
#define ACCEL_DEC3_AVG_8 1u
```

Definition at line [355](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.10 ACCEL\_DLPF0\_3DB\_BW\_HZ

```
#define ACCEL_DLPF0_3DB_BW_HZ 246.0f
```

Definition at line [376](#) of file [ICM20948\\_regs.h](#).

### 6.40.1.11 ACCEL\_DLPF0\_NBW\_HZ

```
#define ACCEL_DLPF0_NBW_HZ 265.0f
```

Definition at line [377](#) of file [ICM20948\\_regs.h](#).

#### 6.40.1.12 ACCEL\_DLPF1\_3DB\_BW\_HZ

```
#define ACCEL_DLPF1_3DB_BW_HZ 246.0f
```

Definition at line 378 of file [ICM20948\\_regs.h](#).

#### 6.40.1.13 ACCEL\_DLPF1\_NBW\_HZ

```
#define ACCEL_DLPF1_NBW_HZ 265.0f
```

Definition at line 379 of file [ICM20948\\_regs.h](#).

#### 6.40.1.14 ACCEL\_DLPF2\_3DB\_BW\_HZ

```
#define ACCEL_DLPF2_3DB_BW_HZ 111.4f
```

Definition at line 380 of file [ICM20948\\_regs.h](#).

#### 6.40.1.15 ACCEL\_DLPF2\_NBW\_HZ

```
#define ACCEL_DLPF2_NBW_HZ 136.0f
```

Definition at line 381 of file [ICM20948\\_regs.h](#).

#### 6.40.1.16 ACCEL\_DLPF3\_3DB\_BW\_HZ

```
#define ACCEL_DLPF3_3DB_BW_HZ 50.4f
```

Definition at line 382 of file [ICM20948\\_regs.h](#).

#### 6.40.1.17 ACCEL\_DLPF3\_NBW\_HZ

```
#define ACCEL_DLPF3_NBW_HZ 68.8f
```

Definition at line 383 of file [ICM20948\\_regs.h](#).

#### 6.40.1.18 ACCEL\_DLPF4\_3DB\_BW\_HZ

```
#define ACCEL_DLPF4_3DB_BW_HZ 23.9f
```

Definition at line 384 of file [ICM20948\\_regs.h](#).

#### 6.40.1.19 ACCEL\_DLPF4\_NBW\_HZ

```
#define ACCEL_DLPF4_NBW_HZ 34.4f
```

Definition at line 385 of file [ICM20948\\_regs.h](#).

#### 6.40.1.20 ACCEL\_DLPF5\_3DB\_BW\_HZ

```
#define ACCEL_DLPF5_3DB_BW_HZ 11.5f
```

Definition at line 386 of file [ICM20948\\_regs.h](#).

#### 6.40.1.21 ACCEL\_DLPF5\_NBW\_HZ

```
#define ACCEL_DLPF5_NBW_HZ 17.0f
```

Definition at line 387 of file [ICM20948\\_regs.h](#).

#### 6.40.1.22 ACCEL\_DLPF6\_3DB\_BW\_HZ

```
#define ACCEL_DLPF6_3DB_BW_HZ 5.7f
```

Definition at line 388 of file [ICM20948\\_regs.h](#).

#### 6.40.1.23 ACCEL\_DLPF6\_NBW\_HZ

```
#define ACCEL_DLPF6_NBW_HZ 8.3f
```

Definition at line 389 of file [ICM20948\\_regs.h](#).



## 6.40.1.24 ACCEL\_DLPF7\_3DB\_BW\_HZ

```
#define ACCEL_DLPF7_3DB_BW_HZ 473.0f
```

Definition at line 390 of file [ICM20948\\_regs.h](#).

## 6.40.1.25 ACCEL\_DLPF7\_NBW\_HZ

```
#define ACCEL_DLPF7_NBW_HZ 499.0f
```

Definition at line 391 of file [ICM20948\\_regs.h](#).

## 6.40.1.26 ACCEL\_DLPF\_BASE\_HZ

```
#define ACCEL_DLPF_BASE_HZ 1125.0f
```

Definition at line 362 of file [ICM20948\\_regs.h](#).

## 6.40.1.27 ACCEL\_DLPF\_ODR\_HZ

```
#define ACCEL_DLPF_ODR_HZ(  
    div ) (ACCEL_DLPF_BASE_HZ / (1.0f + (float)(div)))
```

Definition at line 363 of file [ICM20948\\_regs.h](#).

## 6.40.1.28 ACCEL\_DLPFCFG\_0

```
#define ACCEL_DLPFCFG_0 0
```

ACCEL\_DLPFCFG (0..7) choose cutoff/noise BW set (when DLPF is ON). Tip: start with ACCEL\_DLPFCFG\_3 for a good noise/latency tradeoff.

Definition at line 345 of file [ICM20948\\_regs.h](#).

## 6.40.1.29 ACCEL\_DLPFCFG\_1

```
#define ACCEL_DLPFCFG_1 1
```

Definition at line 346 of file [ICM20948\\_regs.h](#).

## 6.40.1.30 ACCEL\_DLPFCFG\_2

```
#define ACCEL_DLPFCFG_2 2
```

Definition at line 347 of file [ICM20948\\_regs.h](#).

## 6.40.1.31 ACCEL\_DLPFCFG\_3

```
#define ACCEL_DLPFCFG_3 3
```

Definition at line 348 of file [ICM20948\\_regs.h](#).

## 6.40.1.32 ACCEL\_DLPFCFG\_4

```
#define ACCEL_DLPFCFG_4 4
```

Definition at line 349 of file [ICM20948\\_regs.h](#).

## 6.40.1.33 ACCEL\_DLPFCFG\_5

```
#define ACCEL_DLPFCFG_5 5
```

Definition at line 350 of file [ICM20948\\_regs.h](#).

## 6.40.1.34 ACCEL\_DLPFCFG\_6

```
#define ACCEL_DLPFCFG_6 6
```

Definition at line 351 of file [ICM20948\\_regs.h](#).

## 6.40.1.35 ACCEL\_DLPFCFG\_7

```
#define ACCEL_DLPFCFG_7 7
```

Definition at line 352 of file [ICM20948\\_regs.h](#).

#### 6.40.1.36 ACCEL\_DLPFCFG\_MASK

```
#define ACCEL_DLPFCFG_MASK (0x7u << ACCEL_DLPFCFG_SHIFT)
```

Definition at line 218 of file [ICM20948\\_regs.h](#).

#### 6.40.1.37 ACCEL\_DLPFCFG\_SHIFT

```
#define ACCEL_DLPFCFG_SHIFT 5
```

Definition at line 217 of file [ICM20948\\_regs.h](#).

#### 6.40.1.38 ACCEL\_FCHOICE

```
#define ACCEL_FCHOICE BIT(0)
```

Definition at line 210 of file [ICM20948\\_regs.h](#).

#### 6.40.1.39 ACCEL\_FCHOICE\_BYPASS

```
#define ACCEL_FCHOICE_BYPASS 0
```

Path select for accel:

- ACCEL\_FCHOICE\_BYPASS : DLPF bypassed (very wide BW, fastest)
- ACCEL\_FCHOICE\_DLPF : DLPF enabled (use ACCEL\_DLPFCFG + SMPLRT\_DIV)

Definition at line 337 of file [ICM20948\\_regs.h](#).

#### 6.40.1.40 ACCEL\_FCHOICE\_DLPF

```
#define ACCEL_FCHOICE_DLPF 1
```

Definition at line 338 of file [ICM20948\\_regs.h](#).

#### 6.40.1.41 ACCEL\_FS\_16G

```
#define ACCEL_FS_16G (3u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 216 of file [ICM20948\\_regs.h](#).

#### 6.40.1.42 ACCEL\_FS\_2G

```
#define ACCEL_FS_2G (0u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 213 of file [ICM20948\\_regs.h](#).

#### 6.40.1.43 ACCEL\_FS\_4G

```
#define ACCEL_FS_4G (1u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 214 of file [ICM20948\\_regs.h](#).

#### 6.40.1.44 ACCEL\_FS\_8G

```
#define ACCEL_FS_8G (2u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 215 of file [ICM20948\\_regs.h](#).

#### 6.40.1.45 ACCEL\_FS\_SEL\_MASK

```
#define ACCEL_FS_SEL_MASK (0x3u << ACCEL_FS_SEL_SHIFT)
```

Definition at line 212 of file [ICM20948\\_regs.h](#).

#### 6.40.1.46 ACCEL\_FS\_SEL\_SHIFT

```
#define ACCEL_FS_SEL_SHIFT 1
```

Definition at line 211 of file [ICM20948\\_regs.h](#).

## 6.40.1.47 ACCEL\_INTEL\_CTRL

```
#define ACCEL_INTEL_CTRL 0x12 /* ACCEL_INTEL_EN ACCEL_INTEL_MODE_INT */
```

Definition at line 203 of file [ICM20948\\_regs.h](#).

## 6.40.1.48 ACCEL\_INTEL\_EN

```
#define ACCEL_INTEL_EN BIT(7)
```

Definition at line 204 of file [ICM20948\\_regs.h](#).

## 6.40.1.49 ACCEL\_INTEL\_MODE\_INT

```
#define ACCEL_INTEL_MODE_INT BIT(6)
```

Definition at line 205 of file [ICM20948\\_regs.h](#).

## 6.40.1.50 ACCEL\_SMPLRT\_DIV\_1

```
#define ACCEL_SMPLRT_DIV_1 0x10 /* ACCEL_SMPLRT_DIV[11:8] */
```

Definition at line 200 of file [ICM20948\\_regs.h](#).

## 6.40.1.51 ACCEL\_SMPLRT\_DIV\_2

```
#define ACCEL_SMPLRT_DIV_2 0x11 /* ACCEL_SMPLRT_DIV[7:0] */
```

Definition at line 201 of file [ICM20948\\_regs.h](#).

## 6.40.1.52 ACCEL\_SMPLRT\_DIV\_MAX

```
#define ACCEL_SMPLRT_DIV_MAX 4095u
```

Definition at line 361 of file [ICM20948\\_regs.h](#).

## 6.40.1.53 ACCEL\_SMPLRT\_DIV\_MIN

```
#define ACCEL_SMPLRT_DIV_MIN 0u
```

Definition at line 360 of file [ICM20948\\_regs.h](#).

## 6.40.1.54 ACCEL\_WOM\_THR

```
#define ACCEL_WOM_THR 0x13 /* WOM_THRESHOLD[7:0] */
```

Definition at line 207 of file [ICM20948\\_regs.h](#).

## 6.40.1.55 ACCEL\_XOUT\_H

```
#define ACCEL_XOUT_H 0x2D
```

Definition at line 111 of file [ICM20948\\_regs.h](#).

## 6.40.1.56 ACCEL\_XOUT\_L

```
#define ACCEL_XOUT_L 0x2E
```

Definition at line 112 of file [ICM20948\\_regs.h](#).

## 6.40.1.57 ACCEL\_YOUT\_H

```
#define ACCEL_YOUT_H 0x2F
```

Definition at line 113 of file [ICM20948\\_regs.h](#).

## 6.40.1.58 ACCEL\_YOUT\_L

```
#define ACCEL_YOUT_L 0x30
```

Definition at line 114 of file [ICM20948\\_regs.h](#).

#### 6.40.1.59 ACCEL\_ZOUT\_H

```
#define ACCEL_ZOUT_H 0x31
```

Definition at line 115 of file [ICM20948\\_regs.h](#).

#### 6.40.1.60 ACCEL\_ZOUT\_L

```
#define ACCEL_ZOUT_L 0x32
```

Definition at line 116 of file [ICM20948\\_regs.h](#).

#### 6.40.1.61 AK09916\_I2C\_ADDR

```
#define AK09916_I2C_ADDR 0x0C
```

Definition at line 310 of file [ICM20948\\_regs.h](#).

#### 6.40.1.62 AK\_CNTL2

```
#define AK_CNTL2 0x31
```

Definition at line 315 of file [ICM20948\\_regs.h](#).

#### 6.40.1.63 AK\_CNTL3

```
#define AK_CNTL3 0x32
```

Definition at line 316 of file [ICM20948\\_regs.h](#).

#### 6.40.1.64 AK\_HXL

```
#define AK_HXL 0x11
```

Definition at line 313 of file [ICM20948\\_regs.h](#).

#### 6.40.1.65 AK\_ST1

```
#define AK_ST1 0x10
```

Definition at line 312 of file [ICM20948\\_regs.h](#).

#### 6.40.1.66 AK\_ST2

```
#define AK_ST2 0x18
```

Definition at line 314 of file [ICM20948\\_regs.h](#).

#### 6.40.1.67 AK\_WIA2

```
#define AK_WIA2 0x01
```

Definition at line 311 of file [ICM20948\\_regs.h](#).

#### 6.40.1.68 AK\_WIA2\_VAL

```
#define AK_WIA2_VAL 0x09 /* AK09916C WIA2 expected */
```

Definition at line 317 of file [ICM20948\\_regs.h](#).

#### 6.40.1.69 AUTO\_SEL

```
#define AUTO_SEL 0x01 /* 1-5: Auto/PLL preferred (use 1 as default) */
```

Definition at line 320 of file [ICM20948\\_regs.h](#).

#### 6.40.1.70 AX\_ST\_EN\_REG

```
#define AX_ST_EN_REG BIT(7)
```

Definition at line 221 of file [ICM20948\\_regs.h](#).

## 6.40.1.71 AY\_ST\_EN\_REG

```
#define AY_ST_EN_REG BIT(6)
```

Definition at line 222 of file [ICM20948\\_regs.h](#).

## 6.40.1.72 AZ\_ST\_EN\_REG

```
#define AZ_ST_EN_REG BIT(5)
```

Definition at line 223 of file [ICM20948\\_regs.h](#).

## 6.40.1.73 BANK1\_REG\_BANK\_SEL

```
#define BANK1_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
```

Definition at line 163 of file [ICM20948\\_regs.h](#).

## 6.40.1.74 BANK2\_REG\_BANK\_SEL

```
#define BANK2_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
```

Definition at line 240 of file [ICM20948\\_regs.h](#).

## 6.40.1.75 BANK3\_REG\_BANK\_SEL

```
#define BANK3_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
```

Definition at line 296 of file [ICM20948\\_regs.h](#).

## 6.40.1.76 BYPASS\_EN

```
#define BYPASS_EN BIT(1) /* I2C bypass to aux devices */
```

Definition at line 71 of file [ICM20948\\_regs.h](#).

## 6.40.1.77 CLK\_STOP

```
#define CLK_STOP 0x07 /* 7: Stop clock / timing gen reset */
```

Definition at line 321 of file [ICM20948\\_regs.h](#).

## 6.40.1.78 DEC3\_CFG\_MASK

```
#define DEC3_CFG_MASK (0x3u << DEC3_CFG_SHIFT)
```

Definition at line 225 of file [ICM20948\\_regs.h](#).

## 6.40.1.79 DEC3\_CFG\_SHIFT

```
#define DEC3_CFG_SHIFT 0
```

Definition at line 224 of file [ICM20948\\_regs.h](#).

## 6.40.1.80 DELAY\_ES\_SHADOW

```
#define DELAY_ES_SHADOW BIT(7)
```

Definition at line 256 of file [ICM20948\\_regs.h](#).

## 6.40.1.81 DELAY\_TIME\_EN

```
#define DELAY_TIME_EN BIT(7)
```

Definition at line 228 of file [ICM20948\\_regs.h](#).

## 6.40.1.82 DELAY\_TIMEH

```
#define DELAY_TIMEH 0x28
```

Definition at line 107 of file [ICM20948\\_regs.h](#).

#### 6.40.1.83 DELAY\_TIMEL

```
#define DELAY_TIMEL 0x29
```

Definition at line 108 of file [ICM20948\\_regs.h](#).

#### 6.40.1.84 dps1000

```
#define dps1000 2
```

Definition at line 398 of file [ICM20948\\_regs.h](#).

#### 6.40.1.85 dps2000

```
#define dps2000 3
```

Definition at line 399 of file [ICM20948\\_regs.h](#).

#### 6.40.1.86 dps250

```
#define dps250 0
```

- $\$250/\$500/\$1000/\$2000$  dps (maps to FS\_SEL 0..3)

Definition at line 396 of file [ICM20948\\_regs.h](#).

#### 6.40.1.87 dps500

```
#define dps500 1
```

Definition at line 397 of file [ICM20948\\_regs.h](#).

#### 6.40.1.88 EnhancedRegular

```
#define EnhancedRegular 2
```

Definition at line 456 of file [ICM20948\\_regs.h](#).

#### 6.40.1.89 EXT\_SLV\_SENS\_DATA\_00

```
#define EXT_SLV_SENS_DATA_00 0x3B
```

Definition at line 127 of file [ICM20948\\_regs.h](#).

#### 6.40.1.90 EXT\_SLV\_SENS\_DATA\_23

```
#define EXT_SLV_SENS_DATA_23 0x52 /* contiguous range 0x3B..0x52 */
```

Definition at line 128 of file [ICM20948\\_regs.h](#).

#### 6.40.1.91 EXT\_SYNC\_SET\_MASK

```
#define EXT_SYNC_SET_MASK 0x0F
```

Definition at line 231 of file [ICM20948\\_regs.h](#).

#### 6.40.1.92 FIFO\_CFG

```
#define FIFO_CFG 0x76
```

Definition at line 138 of file [ICM20948\\_regs.h](#).

#### 6.40.1.93 FIFO\_COUNTH

```
#define FIFO_COUNTH 0x70
```

Definition at line 134 of file [ICM20948\\_regs.h](#).

#### 6.40.1.94 FIFO\_COUNTL

```
#define FIFO_COUNTL 0x71
```

Definition at line 135 of file [ICM20948\\_regs.h](#).

## 6.40.1.95 FIFO\_EN\_1

```
#define FIFO_EN_1 0x66 /* SLV3..SLV0 FIFO_EN */  
Definition at line 130 of file ICM20948_regs.h.
```

## 6.40.1.96 FIFO\_EN\_2

```
#define FIFO_EN_2 0x67 /* ACCEL_GYRO_Z/Y/X TEMP_FIFO_EN */  
Definition at line 131 of file ICM20948_regs.h.
```

## 6.40.1.97 FIFO\_MODE

```
#define FIFO_MODE 0x69 /* FIFO_MODE[4:0] */  
Definition at line 133 of file ICM20948_regs.h.
```

## 6.40.1.98 FIFO\_R\_W

```
#define FIFO_R_W 0x72  
Definition at line 136 of file ICM20948_regs.h.
```

## 6.40.1.99 FIFO\_RST

```
#define FIFO_RST 0x68 /* FIFO_RESET[4:0] */  
Definition at line 132 of file ICM20948_regs.h.
```

## 6.40.1.100 FSYNC\_CONFIG

```
#define FSYNC_CONFIG 0x52 /* DELAY_TIME_EN WOF_DEGLITCH_EN WOF_EDGE_INT EXT_SYNC_SET[3:0] */  
Definition at line 227 of file ICM20948_regs.h.
```

## 6.40.1.101 FSYNC\_INT\_MODE\_EN

```
#define FSYNC_INT_MODE_EN BIT(2)  
Definition at line 70 of file ICM20948_regs.h.
```

## 6.40.1.102 g16

```
#define g16 3  
Definition at line 329 of file ICM20948_regs.h.
```

## 6.40.1.103 g2

```
#define g2 0  
  
•  $\$2g$  /  $\$4g$  /  $\$8g$  /  $\$16g$  (maps to FS_SEL 0..3)
```

Definition at line 326 of file ICM20948\_regs.h.

## 6.40.1.104 g4

```
#define g4 1  
Definition at line 327 of file ICM20948_regs.h.
```

## 6.40.1.105 g8

```
#define g8 2  
Definition at line 328 of file ICM20948_regs.h.
```

## 6.40.1.106 GYRO\_AVGCFG\_MASK

```
#define GYRO_AVGCFG_MASK (0x7u << GYRO_AVGCFG_SHIFT)  
Definition at line 188 of file ICM20948_regs.h.
```

## 6.40.1.107 GYRO\_AVGCFG\_SHIFT

```
#define GYRO_AVGCFG_SHIFT 0
```

Definition at line 187 of file [ICM20948\\_regs.h](#).

## 6.40.1.108 GYRO\_BW\_MEDIUM

```
#define GYRO_BW_MEDIUM 2 /* e.g., CFG 2/3 */
```

Definition at line 435 of file [ICM20948\\_regs.h](#).

## 6.40.1.109 GYRO\_BW\_NARROW

```
#define GYRO_BW_NARROW 3 /* e.g., CFG 4/5/6/7 */
```

Definition at line 436 of file [ICM20948\\_regs.h](#).

## 6.40.1.110 GYRO\_BW\_ULTRA\_WIDE

```
#define GYRO_BW_ULTRA_WIDE 0 /* bypass path */
```

Definition at line 433 of file [ICM20948\\_regs.h](#).

## 6.40.1.111 GYRO\_BW\_WIDE

```
#define GYRO_BW_WIDE 1 /* e.g., CFG 0/1 */
```

Definition at line 434 of file [ICM20948\\_regs.h](#).

## 6.40.1.112 GYRO\_CONFIG\_1

```
#define GYRO_CONFIG_1 0x01 /* GYRO_DLPFCFG[2:0] GYRO_FS_SEL[1:0] GYRO_FCHOICE */
```

Definition at line 172 of file [ICM20948\\_regs.h](#).

## 6.40.1.113 GYRO\_CONFIG\_2

```
#define GYRO_CONFIG_2 0x02 /* XGYRO_CTEN YGYRO_CTEN ZGYRO_CTEN GYRO_AVGCFG[2:0] */
```

Definition at line 183 of file [ICM20948\\_regs.h](#).

## 6.40.1.114 GYRO\_DLPF\_BASE\_HZ

```
#define GYRO_DLPF_BASE_HZ 1100.0f
```

Definition at line 426 of file [ICM20948\\_regs.h](#).

## 6.40.1.115 GYRO\_DLPF\_ODR\_HZ

```
#define GYRO_DLPF_ODR_HZ(
    rate_request ) (rate_request) /* symbolic; your driver computes DIV = round(1100/rate)-1
*/
```

Definition at line 427 of file [ICM20948\\_regs.h](#).

## 6.40.1.116 GYRO\_DLPFCFG\_0

```
#define GYRO_DLPFCFG_0 0
```

GYRO\_DLPFCFG (0..7) choose cutoff/noise BW set (when DLPF is ON). Tip: start with GYRO\_↔  
DLPFCFG\_3 or \_2 for balanced noise/latency.

Definition at line 415 of file [ICM20948\\_regs.h](#).

## 6.40.1.117 GYRO\_DLPFCFG\_1

```
#define GYRO_DLPFCFG_1 1
```

Definition at line 416 of file [ICM20948\\_regs.h](#).

## 6.40.1.118 GYRO\_DLPFCFG\_2

```
#define GYRO_DLPFCFG_2 2
```

Definition at line 417 of file [ICM20948\\_regs.h](#).



## 6.40.1.119 GYRO\_DLPFCFG\_3

```
#define GYRO_DLPFCFG_3 3
```

Definition at line 418 of file [ICM20948\\_regs.h](#).

## 6.40.1.120 GYRO\_DLPFCFG\_4

```
#define GYRO_DLPFCFG_4 4
```

Definition at line 419 of file [ICM20948\\_regs.h](#).

## 6.40.1.121 GYRO\_DLPFCFG\_5

```
#define GYRO_DLPFCFG_5 5
```

Definition at line 420 of file [ICM20948\\_regs.h](#).

## 6.40.1.122 GYRO\_DLPFCFG\_6

```
#define GYRO_DLPFCFG_6 6
```

Definition at line 421 of file [ICM20948\\_regs.h](#).

## 6.40.1.123 GYRO\_DLPFCFG\_7

```
#define GYRO_DLPFCFG_7 7
```

Definition at line 422 of file [ICM20948\\_regs.h](#).

## 6.40.1.124 GYRO\_DLPFCFG\_MASK

```
#define GYRO_DLPFCFG_MASK (0x7u << GYRO_DLPFCFG_SHIFT)
```

Definition at line 181 of file [ICM20948\\_regs.h](#).

## 6.40.1.125 GYRO\_DLPFCFG\_SHIFT

```
#define GYRO_DLPFCFG_SHIFT 5
```

Definition at line 180 of file [ICM20948\\_regs.h](#).

## 6.40.1.126 GYRO\_FCHOICE

```
#define GYRO_FCHOICE BIT(0) /* when 0: DLPF on, when 1: off (per datasheet) */
```

Definition at line 173 of file [ICM20948\\_regs.h](#).

## 6.40.1.127 GYRO\_FCHOICE\_BYPASS

```
#define GYRO_FCHOICE_BYPASS 0
```

Path select for gyro:

- GYRO\_FCHOICE\_BYPASS : DLPF bypassed (widest BW, fastest)
- GYRO\_FCHOICE\_DLPF : DLPF enabled (use GYRO\_DLPFCFG + SMPLRT\_DIV)

Definition at line 407 of file [ICM20948\\_regs.h](#).

## 6.40.1.128 GYRO\_FCHOICE\_DLPF

```
#define GYRO_FCHOICE_DLPF 1
```

Definition at line 408 of file [ICM20948\\_regs.h](#).

## 6.40.1.129 GYRO\_FS\_1000DPS

```
#define GYRO_FS_1000DPS (2u << GYRO_FS_SEL_SHIFT)
```

Definition at line 178 of file [ICM20948\\_regs.h](#).

#### 6.40.1.130 GYRO\_FS\_2000DPS

```
#define GYRO_FS_2000DPS (3u << GYRO_FS_SEL_SHIFT)
```

Definition at line 179 of file [ICM20948\\_regs.h](#).

#### 6.40.1.131 GYRO\_FS\_250DPS

```
#define GYRO_FS_250DPS (0u << GYRO_FS_SEL_SHIFT)
```

Definition at line 176 of file [ICM20948\\_regs.h](#).

#### 6.40.1.132 GYRO\_FS\_500DPS

```
#define GYRO_FS_500DPS (1u << GYRO_FS_SEL_SHIFT)
```

Definition at line 177 of file [ICM20948\\_regs.h](#).

#### 6.40.1.133 GYRO\_FS\_SEL\_MASK

```
#define GYRO_FS_SEL_MASK (0x3u << GYRO_FS_SEL_SHIFT)
```

Definition at line 175 of file [ICM20948\\_regs.h](#).

#### 6.40.1.134 GYRO\_FS\_SEL\_SHIFT

```
#define GYRO_FS_SEL_SHIFT 1
```

Definition at line 174 of file [ICM20948\\_regs.h](#).

#### 6.40.1.135 GYRO\_SMPLRT\_DIV

```
#define GYRO_SMPLRT_DIV 0x00 /* GYRO_SMPLRT_DIV[7:0] */
```

- Gyro/Accel configuration, offsets, FSYNC, temperature filter

Definition at line 170 of file [ICM20948\\_regs.h](#).

#### 6.40.1.136 GYRO\_SMPLRT\_MIN\_HZ

```
#define GYRO_SMPLRT_MIN_HZ 4.3f
```

Definition at line 425 of file [ICM20948\\_regs.h](#).

#### 6.40.1.137 GYRO\_XOUT\_H

```
#define GYRO_XOUT_H 0x33
```

Definition at line 117 of file [ICM20948\\_regs.h](#).

#### 6.40.1.138 GYRO\_XOUT\_L

```
#define GYRO_XOUT_L 0x34
```

Definition at line 118 of file [ICM20948\\_regs.h](#).

#### 6.40.1.139 GYRO\_YOUT\_H

```
#define GYRO_YOUT_H 0x35
```

Definition at line 119 of file [ICM20948\\_regs.h](#).

#### 6.40.1.140 GYRO\_YOUT\_L

```
#define GYRO_YOUT_L 0x36
```

Definition at line 120 of file [ICM20948\\_regs.h](#).

#### 6.40.1.141 GYRO\_ZOUT\_H

```
#define GYRO_ZOUT_H 0x37
```

Definition at line 121 of file [ICM20948\\_regs.h](#).

## 6.40.1.142 GYRO\_ZOUT\_L

```
#define GYRO_ZOUT_L 0x38
```

Definition at line 122 of file [ICM20948\\_regs.h](#).

## 6.40.1.143 HighAccuracy

```
#define HighAccuracy 3
```

Definition at line 457 of file [ICM20948\\_regs.h](#).

## 6.40.1.144 Hz10

```
#define Hz10 3
```

Definition at line 443 of file [ICM20948\\_regs.h](#).

## 6.40.1.145 Hz15

```
#define Hz15 4
```

Definition at line 444 of file [ICM20948\\_regs.h](#).

## 6.40.1.146 Hz2

```
#define Hz2 0
```

- Friendly ODR presets you can map to dividers

Definition at line 440 of file [ICM20948\\_regs.h](#).

## 6.40.1.147 Hz20

```
#define Hz20 5
```

Definition at line 445 of file [ICM20948\\_regs.h](#).

## 6.40.1.148 Hz25

```
#define Hz25 6
```

Definition at line 446 of file [ICM20948\\_regs.h](#).

## 6.40.1.149 Hz30

```
#define Hz30 7
```

Definition at line 447 of file [ICM20948\\_regs.h](#).

## 6.40.1.150 Hz6

```
#define Hz6 1
```

Definition at line 441 of file [ICM20948\\_regs.h](#).

## 6.40.1.151 Hz8

```
#define Hz8 2
```

Definition at line 442 of file [ICM20948\\_regs.h](#).

## 6.40.1.152 I2C\_MST\_CLK\_MASK

```
#define I2C_MST_CLK_MASK 0x0F
```

Definition at line 253 of file [ICM20948\\_regs.h](#).

## 6.40.1.153 I2C\_MST\_CTRL

```
#define I2C_MST_CTRL 0x01 /* MULT_MST_EN - I2C_MST_P_NSR I2C_MST_CLK[3:0] */
```

Definition at line 250 of file [ICM20948\\_regs.h](#).

## 6.40.1.154 I2C\_MST\_DELAY\_CTRL

```
#define I2C_MST_DELAY_CTRL 0x02 /* DELAY_ES_SHADOW + I2C_SLVx_DELAY_EN bits */
```

Definition at line 255 of file [ICM20948\\_regs.h](#).

## 6.40.1.155 I2C\_MST\_ODR\_CONFIG

```
#define I2C_MST_ODR_CONFIG 0x00 /* I2C_MST_ODR_CONFIG[3:0] */
```

- I2C master (aux) interface and slave windows

Definition at line 247 of file [ICM20948\\_regs.h](#).

## 6.40.1.156 I2C\_MST\_P\_NSR

```
#define I2C_MST_P_NSR BIT(4)
```

Definition at line 252 of file [ICM20948\\_regs.h](#).

## 6.40.1.157 I2C\_MST\_STATUS

```
#define I2C_MST_STATUS 0x17 /* PASS_THROUGH, *_DONE, *_NACK, LOST_ARB */
```

Definition at line 86 of file [ICM20948\\_regs.h](#).

## 6.40.1.158 I2C\_SLV0\_ADDR

```
#define I2C_SLV0_ADDR 0x03 /* RNW + ID[6:0] */
```

Definition at line 263 of file [ICM20948\\_regs.h](#).

## 6.40.1.159 I2C\_SLV0\_CTRL

```
#define I2C_SLV0_CTRL 0x05 /* EN BYTE_SW REG_DIS GRP LENG[3:0] */
```

Definition at line 266 of file [ICM20948\\_regs.h](#).

## 6.40.1.160 I2C\_SLV0\_DELAY\_EN

```
#define I2C_SLV0_DELAY_EN BIT(0)
```

Definition at line 261 of file [ICM20948\\_regs.h](#).

## 6.40.1.161 I2C\_SLV0\_DO

```
#define I2C_SLV0_DO 0x06
```

Definition at line 272 of file [ICM20948\\_regs.h](#).

## 6.40.1.162 I2C\_SLV0\_REG

```
#define I2C_SLV0_REG 0x04
```

Definition at line 265 of file [ICM20948\\_regs.h](#).

## 6.40.1.163 I2C\_SLV1\_ADDR

```
#define I2C_SLV1_ADDR 0x07
```

Definition at line 274 of file [ICM20948\\_regs.h](#).

## 6.40.1.164 I2C\_SLV1\_CTRL

```
#define I2C_SLV1_CTRL 0x09
```

Definition at line 276 of file [ICM20948\\_regs.h](#).

## 6.40.1.165 I2C\_SLV1\_DELAY\_EN

```
#define I2C_SLV1_DELAY_EN BIT(1)
```

Definition at line 260 of file [ICM20948\\_regs.h](#).

## 6.40.1.166 I2C\_SLV1\_DO

```
#define I2C_SLV1_DO 0x0A
```

Definition at line 277 of file [ICM20948\\_regs.h](#).

## 6.40.1.167 I2C\_SLV1\_REG

```
#define I2C_SLV1_REG 0x08
```

Definition at line 275 of file [ICM20948\\_regs.h](#).

## 6.40.1.168 I2C\_SLV2\_ADDR

```
#define I2C_SLV2_ADDR 0x0B
```

Definition at line 279 of file [ICM20948\\_regs.h](#).

## 6.40.1.169 I2C\_SLV2\_CTRL

```
#define I2C_SLV2_CTRL 0x0D
```

Definition at line 281 of file [ICM20948\\_regs.h](#).

## 6.40.1.170 I2C\_SLV2\_DELAY\_EN

```
#define I2C_SLV2_DELAY_EN BIT(2)
```

Definition at line 259 of file [ICM20948\\_regs.h](#).

## 6.40.1.171 I2C\_SLV2\_DO

```
#define I2C_SLV2_DO 0x0E
```

Definition at line 282 of file [ICM20948\\_regs.h](#).

## 6.40.1.172 I2C\_SLV2\_REG

```
#define I2C_SLV2_REG 0x0C
```

Definition at line 280 of file [ICM20948\\_regs.h](#).

## 6.40.1.173 I2C\_SLV3\_ADDR

```
#define I2C_SLV3_ADDR 0x0F
```

Definition at line 284 of file [ICM20948\\_regs.h](#).

## 6.40.1.174 I2C\_SLV3\_CTRL

```
#define I2C_SLV3_CTRL 0x11
```

Definition at line 286 of file [ICM20948\\_regs.h](#).

## 6.40.1.175 I2C\_SLV3\_DELAY\_EN

```
#define I2C_SLV3_DELAY_EN BIT(3)
```

Definition at line 258 of file [ICM20948\\_regs.h](#).

## 6.40.1.176 I2C\_SLV3\_DO

```
#define I2C_SLV3_DO 0x12
```

Definition at line 287 of file [ICM20948\\_regs.h](#).

## 6.40.1.177 I2C\_SLV3\_REG

```
#define I2C_SLV3_REG 0x10
```

Definition at line 285 of file [ICM20948\\_regs.h](#).

## 6.40.1.178 I2C\_SLV4\_ADDR

```
#define I2C_SLV4_ADDR 0x13
```

Definition at line 289 of file [ICM20948\\_regs.h](#).

## 6.40.1.179 I2C\_SLV4\_CTRL

```
#define I2C_SLV4_CTRL 0x15 /* EN BYTE_SW REG_DIS DLY[4:0] */
```

Definition at line 291 of file [ICM20948\\_regs.h](#).

## 6.40.1.180 I2C\_SLV4\_DELAY\_EN

```
#define I2C_SLV4_DELAY_EN BIT(4)
```

Definition at line 257 of file [ICM20948\\_regs.h](#).

## 6.40.1.181 I2C\_SLV4\_DI

```
#define I2C_SLV4_DI 0x17
```

Definition at line 294 of file [ICM20948\\_regs.h](#).

## 6.40.1.182 I2C\_SLV4\_DLY\_MASK

```
#define I2C_SLV4_DLY_MASK 0x1F
```

Definition at line 292 of file [ICM20948\\_regs.h](#).

## 6.40.1.183 I2C\_SLV4\_DO

```
#define I2C_SLV4_DO 0x16
```

Definition at line 293 of file [ICM20948\\_regs.h](#).

## 6.40.1.184 I2C\_SLV4\_REG

```
#define I2C_SLV4_REG 0x14
```

Definition at line 290 of file [ICM20948\\_regs.h](#).

## 6.40.1.185 I2C\_SLVx\_BYTE\_SW

```
#define I2C_SLVx_BYTE_SW BIT(6)
```

Definition at line 268 of file [ICM20948\\_regs.h](#).

## 6.40.1.186 I2C\_SLVx\_EN

```
#define I2C_SLVx_EN BIT(7)
```

Definition at line 267 of file [ICM20948\\_regs.h](#).

## 6.40.1.187 I2C\_SLVx\_GRP

```
#define I2C_SLVx_GRP BIT(4)
```

Definition at line 270 of file [ICM20948\\_regs.h](#).

## 6.40.1.188 I2C\_SLVx LENG\_MASK

```
#define I2C_SLVx LENG_MASK 0x0F
```

Definition at line 271 of file [ICM20948\\_regs.h](#).

## 6.40.1.189 I2C\_SLVx\_REG\_DIS

```
#define I2C_SLVx_REG_DIS BIT(5)
```

Definition at line 269 of file [ICM20948\\_regs.h](#).

## 6.40.1.190 I2C\_SLVx\_RNW

```
#define I2C_SLVx_RNW BIT(7)
```

Definition at line 264 of file [ICM20948\\_regs.h](#).

## 6.40.1.191 INT1\_ACTL

```
#define INT1_ACTL BIT(7) /* active low */
```

Definition at line 65 of file [ICM20948\\_regs.h](#).

## 6.40.1.192 INT1\_LATCH\_INT\_EN

```
#define INT1_LATCH_INT_EN BIT(5) /* latch until status read */
```

Definition at line 67 of file [ICM20948\\_regs.h](#).

## 6.40.1.193 INT1\_OPEN

```
#define INT1_OPEN BIT(6) /* open-drain */
```

Definition at line 66 of file [ICM20948\\_regs.h](#).

## 6.40.1.194 INT\_ACTL\_FSYNC

```
#define INT_ACTL_FSYNC BIT(3)
```

Definition at line 69 of file [ICM20948\\_regs.h](#).

## 6.40.1.195 INT\_ANYRD\_2CLEAR

```
#define INT_ANYRD_2CLEAR BIT(4)
```

Definition at line 68 of file [ICM20948\\_regs.h](#).

## 6.40.1.196 INT\_DMP\_INT1\_EN

```
#define INT_DMP_INT1_EN BIT(1)
```

Definition at line 77 of file [ICM20948\\_regs.h](#).

## 6.40.1.197 INT\_ENABLE

```
#define INT_ENABLE 0x10 /* REG_WOF_EN - WOM_INT_EN PLL_RDY_EN DMP_INT1_EN I2C_MST_INT_EN */
```

Definition at line 73 of file [ICM20948\\_regs.h](#).

## 6.40.1.198 INT\_ENABLE\_1

```
#define INT_ENABLE_1 0x11 /* RAW_DATA_0_RDY_EN */
```

Definition at line 80 of file [ICM20948\\_regs.h](#).

## 6.40.1.199 INT\_ENABLE\_2

```
#define INT_ENABLE_2 0x12 /* FIFO_OVERFLOW_EN[4:0] */
```

Definition at line 83 of file [ICM20948\\_regs.h](#).

## 6.40.1.200 INT\_ENABLE\_3

```
#define INT_ENABLE_3 0x13 /* FIFO_WM_EN[4:0] */
```

Definition at line 84 of file [ICM20948\\_regs.h](#).

## 6.40.1.201 INT\_I2C\_MST\_INT\_EN

```
#define INT_I2C_MST_INT_EN BIT(0)
```

Definition at line 78 of file [ICM20948\\_regs.h](#).

## 6.40.1.202 INT\_PIN\_CFG

```
#define INT_PIN_CFG 0x0F /* INT1_ACTL INT1_OPEN INT1_LATCH_INT_EN INT_ANYRD_2CLEAR ACTL_FSYNC
FSYNC_INT_MODE_EN BYPASS_EN - */
```

Definition at line 64 of file [ICM20948\\_regs.h](#).

## 6.40.1.203 INT\_PLL\_RDY\_EN

```
#define INT_PLL_RDY_EN BIT(2)
```

Definition at line 76 of file [ICM20948\\_regs.h](#).

## 6.40.1.204 INT\_RAW\_DATA\_0\_RDY\_EN

```
#define INT_RAW_DATA_0_RDY_EN BIT(0)
```

Definition at line 81 of file [ICM20948\\_regs.h](#).

## 6.40.1.205 INT\_REG\_WOF\_EN

```
#define INT_REG_WOF_EN BIT(7)
```

Definition at line 74 of file [ICM20948\\_regs.h](#).

## 6.40.1.206 INT\_STATUS

```
#define INT_STATUS 0x19 /* WOM_INT PLL_RDY_INT DMP_INT1 I2C_MST_INT */
```

Definition at line 96 of file [ICM20948\\_regs.h](#).

## 6.40.1.207 INT\_STATUS\_1

```
#define INT_STATUS_1 0x1A /* RAW_DATA_0_RDY_INT */
```

Definition at line 102 of file [ICM20948\\_regs.h](#).

## 6.40.1.208 INT\_STATUS\_2

```
#define INT_STATUS_2 0x1B /* FIFO_OVERFLOW_INT[4:0] */
```

Definition at line 104 of file [ICM20948\\_regs.h](#).

## 6.40.1.209 INT\_STATUS\_3

```
#define INT_STATUS_3 0x1C /* FIFO_WM_INT[4:0] */
```

Definition at line 105 of file [ICM20948\\_regs.h](#).

## 6.40.1.210 INT\_WOM\_INT\_EN

```
#define INT_WOM_INT_EN BIT(3)
```

Definition at line 75 of file [ICM20948\\_regs.h](#).

## 6.40.1.211 INTERNAL\_20MHZ

```
#define INTERNAL_20MHZ 0x00 /* 0: Internal 20 MHz RC */
```

Definition at line 319 of file [ICM20948\\_regs.h](#).

## 6.40.1.212 LowPower

```
#define LowPower 0
```

Generic power or fusion quality modes (BNO-style). Map to sensor-specific sequences inside your driver.

Definition at line 454 of file [ICM20948\\_regs.h](#).

## 6.40.1.213 LP\_ACCEL\_CYCLE

```
#define LP_ACCEL_CYCLE BIT(5)
```

Definition at line 47 of file [ICM20948\\_regs.h](#).



## 6.40.1.214 LP\_CONFIG

```
#define LP_CONFIG 0x05 /* I2C_MST_CYCLE ACCEL_CYCLE GYRO_CYCLE - */
```

Definition at line 45 of file [ICM20948\\_regs.h](#).

## 6.40.1.215 LP\_GYRO\_CYCLE

```
#define LP_GYRO_CYCLE BIT(4)
```

Definition at line 48 of file [ICM20948\\_regs.h](#).

## 6.40.1.216 LP\_I2C\_MST\_CYCLE

```
#define LP_I2C_MST_CYCLE BIT(6)
```

Definition at line 46 of file [ICM20948\\_regs.h](#).

## 6.40.1.217 MOD\_CTRL\_USR

```
#define MOD_CTRL_USR 0x54 /* REG_LP_DMP_EN */
```

Definition at line 237 of file [ICM20948\\_regs.h](#).

## 6.40.1.218 MST\_LOST\_ARB

```
#define MST_LOST_ARB BIT(5)
```

Definition at line 89 of file [ICM20948\\_regs.h](#).

## 6.40.1.219 MST\_ODR\_CFG\_MASK

```
#define MST_ODR_CFG_MASK 0x0F
```

Definition at line 248 of file [ICM20948\\_regs.h](#).

## 6.40.1.220 MST\_PASS\_THROUGH

```
#define MST_PASS_THROUGH BIT(7)
```

Definition at line 87 of file [ICM20948\\_regs.h](#).

## 6.40.1.221 MST\_SLV0\_NACK

```
#define MST_SLV0_NACK BIT(0)
```

Definition at line 94 of file [ICM20948\\_regs.h](#).

## 6.40.1.222 MST\_SLV1\_NACK

```
#define MST_SLV1_NACK BIT(1)
```

Definition at line 93 of file [ICM20948\\_regs.h](#).

## 6.40.1.223 MST\_SLV2\_NACK

```
#define MST_SLV2_NACK BIT(2)
```

Definition at line 92 of file [ICM20948\\_regs.h](#).

## 6.40.1.224 MST\_SLV3\_NACK

```
#define MST_SLV3_NACK BIT(3)
```

Definition at line 91 of file [ICM20948\\_regs.h](#).

## 6.40.1.225 MST\_SLV4\_DONE

```
#define MST_SLV4_DONE BIT(6)
```

Definition at line 88 of file [ICM20948\\_regs.h](#).

#### 6.40.1.226 MST\_SLV4\_NACK

```
#define MST_SLV4_NACK BIT(4)
```

Definition at line 90 of file [ICM20948\\_regs.h](#).

#### 6.40.1.227 MULT\_MST\_EN

```
#define MULT_MST_EN BIT(7)
```

Definition at line 251 of file [ICM20948\\_regs.h](#).

#### 6.40.1.228 ODR\_ALIGN\_EN

```
#define ODR_ALIGN_EN 0x09 /* ODR_ALIGN_EN */
```

Definition at line 197 of file [ICM20948\\_regs.h](#).

#### 6.40.1.229 ODR\_ALIGN\_EN\_BIT

```
#define ODR_ALIGN_EN_BIT BIT(0)
```

Definition at line 198 of file [ICM20948\\_regs.h](#).

#### 6.40.1.230 PWR\_CLKSEL\_AUTO

```
#define PWR_CLKSEL_AUTO 0x01 /* typical: auto selects best source */
```

Definition at line 57 of file [ICM20948\\_regs.h](#).

#### 6.40.1.231 PWR\_CLKSEL\_INT\_20MHZ

```
#define PWR_CLKSEL_INT_20MHZ 0x01
```

Definition at line 56 of file [ICM20948\\_regs.h](#).

#### 6.40.1.232 PWR\_CLKSEL\_MASK

```
#define PWR_CLKSEL_MASK 0x07
```

Definition at line 55 of file [ICM20948\\_regs.h](#).

#### 6.40.1.233 PWR\_DEVICE\_RESET

```
#define PWR_DEVICE_RESET BIT(7)
```

Definition at line 51 of file [ICM20948\\_regs.h](#).

#### 6.40.1.234 PWR\_DISABLE\_ACCEL

```
#define PWR_DISABLE_ACCEL BIT(3)
```

Definition at line 60 of file [ICM20948\\_regs.h](#).

#### 6.40.1.235 PWR\_DISABLE\_GYRO

```
#define PWR_DISABLE_GYRO BIT(0)
```

Definition at line 61 of file [ICM20948\\_regs.h](#).

#### 6.40.1.236 PWR\_LP\_EN

```
#define PWR_LP_EN BIT(5)
```

Definition at line 53 of file [ICM20948\\_regs.h](#).

#### 6.40.1.237 PWR\_MGMT\_1

```
#define PWR_MGMT_1 0x06 /* DEVICE_RESET SLEEP LP_EN - TEMP_DIS CLKSEL[2:0] */
```

Definition at line 50 of file [ICM20948\\_regs.h](#).

## 6.40.1.238 PWR\_MGMT\_2

```
#define PWR_MGMT_2 0x07 /* - DISABLE_ACCEL DISABLE_GYRO */
```

Definition at line 59 of file [ICM20948\\_regs.h](#).

## 6.40.1.239 PWR\_SLEEP

```
#define PWR_SLEEP BIT(6)
```

Definition at line 52 of file [ICM20948\\_regs.h](#).

## 6.40.1.240 PWR\_TEMP\_DIS

```
#define PWR_TEMP_DIS BIT(3)
```

Definition at line 54 of file [ICM20948\\_regs.h](#).

## 6.40.1.241 REG\_BANK\_SEL

```
#define REG_BANK_SEL 0x7F /* USER_BANK[1:0] */
```

Definition at line 140 of file [ICM20948\\_regs.h](#).

## 6.40.1.242 REG\_BANK\_SEL\_USER\_BANK\_MASK

```
#define REG_BANK_SEL_USER_BANK_MASK (0x3u << REG_BANK_SEL_USER_BANK_SHIFT)
```

Definition at line 303 of file [ICM20948\\_regs.h](#).

## 6.40.1.243 REG\_BANK\_SEL\_USER\_BANK\_SHIFT

```
#define REG_BANK_SEL_USER_BANK_SHIFT 4
```

Definition at line 302 of file [ICM20948\\_regs.h](#).

## 6.40.1.244 REG\_LP\_DMP\_EN

```
#define REG_LP_DMP_EN BIT(7)
```

Definition at line 238 of file [ICM20948\\_regs.h](#).

## 6.40.1.245 Regular

```
#define Regular 1
```

Definition at line 455 of file [ICM20948\\_regs.h](#).

## 6.40.1.246 SELF\_TEST\_X\_ACCEL

```
#define SELF_TEST_X_ACCEL 0x0E /* XA_ST_DATA[7:0] */
```

Definition at line 150 of file [ICM20948\\_regs.h](#).

## 6.40.1.247 SELF\_TEST\_X\_GYRO

```
#define SELF_TEST_X_GYRO 0x02 /* XG_ST_DATA[7:0] */
```

- Self-test / accel offsets / timebase

Definition at line 147 of file [ICM20948\\_regs.h](#).

## 6.40.1.248 SELF\_TEST\_Y\_ACCEL

```
#define SELF_TEST_Y_ACCEL 0x0F /* YA_ST_DATA[7:0] */
```

Definition at line 151 of file [ICM20948\\_regs.h](#).

## 6.40.1.249 SELF\_TEST\_Y\_GYRO

```
#define SELF_TEST_Y_GYRO 0x03 /* YG_ST_DATA[7:0] */
```

Definition at line 148 of file [ICM20948\\_regs.h](#).

#### 6.40.1.250 SELF\_TEST\_Z\_ACCEL

```
#define SELF_TEST_Z_ACCEL 0x10 /* ZA_ST_DATA[7:0] */  
Definition at line 152 of file ICM20948\_regs.h.
```

#### 6.40.1.251 SELF\_TEST\_Z\_GYRO

```
#define SELF_TEST_Z_GYRO 0x04 /* ZG_ST_DATA[7:0] */  
Definition at line 149 of file ICM20948\_regs.h.
```

#### 6.40.1.252 STS\_DMP\_INT1

```
#define STS_DMP_INT1 BIT(1)  
Definition at line 99 of file ICM20948\_regs.h.
```

#### 6.40.1.253 STS\_I2C\_MST\_INT

```
#define STS_I2C_MST_INT BIT(0)  
Definition at line 100 of file ICM20948\_regs.h.
```

#### 6.40.1.254 STS\_PLL\_RDY\_INT

```
#define STS_PLL_RDY_INT BIT(2)  
Definition at line 98 of file ICM20948\_regs.h.
```

#### 6.40.1.255 STS\_RAW\_DATA\_0\_RDY\_INT

```
#define STS_RAW_DATA_0_RDY_INT BIT(0)  
Definition at line 103 of file ICM20948\_regs.h.
```

#### 6.40.1.256 STS\_WOM\_INT

```
#define STS_WOM_INT BIT(3)  
Definition at line 97 of file ICM20948\_regs.h.
```

#### 6.40.1.257 TEMP\_CONFIG

```
#define TEMP_CONFIG 0x53 /* TEMP_DLPFCFG[2:0] */  
Definition at line 233 of file ICM20948\_regs.h.
```

#### 6.40.1.258 TEMP\_DLPFCFG\_MASK

```
#define TEMP_DLPFCFG_MASK (0x7u << TEMP_DLPFCFG_SHIFT)  
Definition at line 235 of file ICM20948\_regs.h.
```

#### 6.40.1.259 TEMP\_DLPFCFG\_SHIFT

```
#define TEMP_DLPFCFG_SHIFT 5  
Definition at line 234 of file ICM20948\_regs.h.
```

#### 6.40.1.260 TEMP\_OUT\_H

```
#define TEMP_OUT_H 0x39  
Definition at line 123 of file ICM20948\_regs.h.
```

#### 6.40.1.261 TEMP\_OUT\_L

```
#define TEMP_OUT_L 0x3A  
Definition at line 124 of file ICM20948\_regs.h.
```

## 6.40.1.262 TIMEBASE\_CORRECTION\_PLL

```
#define TIMEBASE_CORRECTION_PLL 0x28 /* TBC_PLL[7:0] */
```

Definition at line 161 of file [ICM20948\\_regs.h](#).

## 6.40.1.263 USER\_BANK\_0

```
#define USER_BANK_0 BANK0
```

Definition at line 304 of file [ICM20948\\_regs.h](#).

## 6.40.1.264 USER\_BANK\_1

```
#define USER_BANK_1 BANK1
```

Definition at line 305 of file [ICM20948\\_regs.h](#).

## 6.40.1.265 USER\_BANK\_2

```
#define USER_BANK_2 BANK2
```

Definition at line 306 of file [ICM20948\\_regs.h](#).

## 6.40.1.266 USER\_BANK\_3

```
#define USER_BANK_3 BANK3
```

Definition at line 307 of file [ICM20948\\_regs.h](#).

## 6.40.1.267 USER\_CTRL

```
#define USER_CTRL 0x03 /* DMP_EN FIFO_EN I2C_MST_EN I2C_IF_DIS DMP_RST SRAM_RST I2C_MST_RST - */
```

Definition at line 35 of file [ICM20948\\_regs.h](#).

## 6.40.1.268 USER\_CTRL\_DMP\_EN

```
#define USER_CTRL_DMP_EN BIT(7)
```

Definition at line 37 of file [ICM20948\\_regs.h](#).

## 6.40.1.269 USER\_CTRL\_DMP\_RST

```
#define USER_CTRL_DMP_RST BIT(3)
```

Definition at line 41 of file [ICM20948\\_regs.h](#).

## 6.40.1.270 USER\_CTRL\_FIFO\_EN

```
#define USER_CTRL_FIFO_EN BIT(6)
```

Definition at line 38 of file [ICM20948\\_regs.h](#).

## 6.40.1.271 USER\_CTRL\_I2C\_IF\_DIS

```
#define USER_CTRL_I2C_IF_DIS BIT(4)
```

Definition at line 40 of file [ICM20948\\_regs.h](#).

## 6.40.1.272 USER\_CTRL\_I2C\_MST\_EN

```
#define USER_CTRL_I2C_MST_EN BIT(5)
```

Definition at line 39 of file [ICM20948\\_regs.h](#).

## 6.40.1.273 USER\_CTRL\_I2C\_MST\_RST

```
#define USER_CTRL_I2C_MST_RST BIT(1)
```

Definition at line 43 of file [ICM20948\\_regs.h](#).

## 6.40.1.274 USER\_CTRL\_SRAM\_RST

```
#define USER_CTRL_SRAM_RST BIT(2)
```

Definition at line 42 of file [ICM20948\\_regs.h](#).

## 6.40.1.275 WHO\_AM\_I

```
#define WHO_AM_I 0x00 /* WHO_AM_I[7:0] */
```

7Semi comment style

- Full ICM-20948 register map (Banks 03) + key bit fields
- Simple, portable #defines for firmware use
- Datasheet naming kept where practical; fields grouped by bank

## Notes

- WHO\_AM\_I expected value: 0xEA
- Select register bank via REG\_BANK\_SEL in any bank
- Use BANK(n) helper or write raw values 0x00/0x10/0x20/0x30
- Identity / power / interrupts / sensor data

Definition at line 32 of file [ICM20948\\_regs.h](#).

## 6.40.1.276 WHO\_AM\_I\_VAL

```
#define WHO_AM_I_VAL 0xEA
```

Definition at line 33 of file [ICM20948\\_regs.h](#).

## 6.40.1.277 WOF\_DEGLITCH\_EN

```
#define WOF_DEGLITCH_EN BIT(6)
```

Definition at line 229 of file [ICM20948\\_regs.h](#).

## 6.40.1.278 WOF\_EDGE\_INT

```
#define WOF_EDGE_INT BIT(5)
```

Definition at line 230 of file [ICM20948\\_regs.h](#).

## 6.40.1.279 XA\_OFFS\_H

```
#define XA_OFFS_H 0x14 /* XA_OFFS[14:7] */
```

Definition at line 154 of file [ICM20948\\_regs.h](#).

## 6.40.1.280 XA\_OFFS\_L

```
#define XA_OFFS_L 0x15 /* XA_OFFS[6:0] */
```

Definition at line 155 of file [ICM20948\\_regs.h](#).

## 6.40.1.281 XG\_OFFS\_USRH

```
#define XG_OFFS_USRH 0x03
```

Definition at line 190 of file [ICM20948\\_regs.h](#).

## 6.40.1.282 XG\_OFFS\_USRL

```
#define XG_OFFS_USRL 0x04
```

Definition at line 191 of file [ICM20948\\_regs.h](#).

## 6.40.1.283 XGYRO\_CTEN

```
#define XGYRO_CTEN BIT(5)
```

Definition at line 184 of file [ICM20948\\_regs.h](#).

## 6.40.1.284 YA\_OFFS\_H

```
#define YA_OFFS_H 0x17 /* YA_OFFS[14:7] */
```

Definition at line 156 of file [ICM20948\\_regs.h](#).

## 6.40.1.285 YA\_OFFS\_L

```
#define YA_OFFS_L 0x18 /* YA_OFFS[6:0] */
```

Definition at line 157 of file [ICM20948\\_regs.h](#).

## 6.40.1.286 YG\_OFFS\_USRH

```
#define YG_OFFS_USRH 0x05
```

Definition at line 192 of file [ICM20948\\_regs.h](#).

## 6.40.1.287 YG\_OFFS\_USRL

```
#define YG_OFFS_USRL 0x06
```

Definition at line 193 of file [ICM20948\\_regs.h](#).

## 6.40.1.288 YGYRO\_CTEN

```
#define YGYRO_CTEN BIT(4)
```

Definition at line 185 of file [ICM20948\\_regs.h](#).

## 6.40.1.289 ZA\_OFFS\_H

```
#define ZA_OFFS_H 0x1A /* ZA_OFFS[14:7] */
```

Definition at line 158 of file [ICM20948\\_regs.h](#).

## 6.40.1.290 ZA\_OFFS\_L

```
#define ZA_OFFS_L 0x1B /* ZA_OFFS[6:0] */
```

Definition at line 159 of file [ICM20948\\_regs.h](#).

## 6.40.1.291 ZG\_OFFS\_USRH

```
#define ZG_OFFS_USRH 0x07
```

Definition at line 194 of file [ICM20948\\_regs.h](#).

## 6.40.1.292 ZG\_OFFS\_USRL

```
#define ZG_OFFS_USRL 0x08
```

Definition at line 195 of file [ICM20948\\_regs.h](#).

## 6.40.1.293 ZGYRO\_CTEN

```
#define ZGYRO_CTEN BIT(3)
```

Definition at line 186 of file [ICM20948\\_regs.h](#).

## 6.41 ICM20948\_regs.h

[Go to the documentation of this file.](#)

```
00001 #ifndef ICM20948_REGS_H
00002 #define ICM20948_REGS_H
00003
00004 /**
00005  * 7Semi comment style
```

```

00006  * - Full ICM-20948 register map (Banks 0-3) + key bit fields
00007  * - Simple, portable #defines for firmware use
00008  * - Datasheet naming kept where practical; fields grouped by bank
00009  *
00010  * Notes
00011  * - WHO_AM_I expected value: 0xEA
00012  * - Select register bank via REG_BANK_SEL in any bank
00013  * - Use BANK(n) helper or write raw values 0x00/0x10/0x20/0x30
00014  */
00015
00016 /* ----- Common helpers ----- */
00017 #ifndef BIT
00018 #define BIT(n) (1u << (n))
00019 #endif
00020
00021 // #define BANK(n) ((uint8_t)((n) << 4))
00022 // #define BANK0 BANK(0)
00023 // #define BANK1 BANK(1)
00024 // #define BANK2 BANK(2)
00025 // #define BANK3 BANK(3)
00026
00027 /* ===== */
00028 /*                                BANK 0                                */
00029 /* ===== */
00030
00031 /** - Identity / power / interrupts / sensor data */
00032 #define WHO_AM_I 0x00 /* WHO_AM_I[7:0] */
00033 #define WHO_AM_I_VAL 0xEA
00034
00035 #define USER_CTRL 0x03 /* DMP_EN FIFO_EN I2C_MST_EN I2C_IF_DIS DMP_RST SRAM_RST I2C_MST_RST - */
00036 /* bits */
00037 #define USER_CTRL_DMP_EN BIT(7)
00038 #define USER_CTRL_FIFO_EN BIT(6)
00039 #define USER_CTRL_I2C_MST_EN BIT(5)
00040 #define USER_CTRL_I2C_IF_DIS BIT(4)
00041 #define USER_CTRL_DMP_RST BIT(3)
00042 #define USER_CTRL_SRAM_RST BIT(2)
00043 #define USER_CTRL_I2C_MST_RST BIT(1)
00044
00045 #define LP_CONFIG 0x05 /* I2C_MST_CYCLE ACCEL_CYCLE GYRO_CYCLE - */
00046 #define LP_I2C_MST_CYCLE BIT(6)
00047 #define LP_ACCEL_CYCLE BIT(5)
00048 #define LP_GYRO_CYCLE BIT(4)
00049
00050 #define PWR_MGMT_1 0x06 /* DEVICE_RESET SLEEP LP_EN - TEMP_DIS CLKSEL[2:0] */
00051 #define PWR_DEVICE_RESET BIT(7)
00052 #define PWR_SLEEP BIT(6)
00053 #define PWR_LP_EN BIT(5)
00054 #define PWR_TEMP_DIS BIT(3)
00055 #define PWR_CLKSEL_MASK 0x07
00056 #define PWR_CLKSEL_INT_20MHZ 0x01
00057 #define PWR_CLKSEL_AUTO 0x01 /* typical: auto selects best source */
00058
00059 #define PWR_MGMT_2 0x07 /* - DISABLE_ACCEL DISABLE_GYRO */
00060 #define PWR_DISABLE_ACCEL BIT(3)
00061 #define PWR_DISABLE_GYRO BIT(0)
00062
00063 //Interrupt Configs register
00064 #define INT_PIN_CFG 0x0F /* INT1_ACTL INT1_OPEN INT1_LATCH_INT_EN INT_ANYRD_2CLEAR ACTL_FSYNC
FSYNC_INT_MODE_EN BYPASS_EN - */
00065 #define INT1_ACTL BIT(7) /* active low */
00066 #define INT1_OPEN BIT(6) /* open-drain */
00067 #define INT1_LATCH_INT_EN BIT(5) /* latch until status read */
00068 #define INT_ANYRD_2CLEAR BIT(4)
00069 #define INT_ACTL_FSYNC BIT(3)
00070 #define FSYNC_INT_MODE_EN BIT(2)
00071 #define BYPASS_EN BIT(1) /* I2C bypass to aux devices */
00072
00073 #define INT_ENABLE 0x10 /* REG_WOF_EN - WOM_INT_EN PLL_RDY_EN DMP_INT1_EN I2C_MST_INT_EN */
00074 #define INT_REG_WOF_EN BIT(7)
00075 #define INT_WOM_INT_EN BIT(3)
00076 #define INT_PLL_RDY_EN BIT(2)
00077 #define INT_DMP_INT1_EN BIT(1)
00078 #define INT_I2C_MST_INT_EN BIT(0)
00079
00080 #define INT_ENABLE_1 0x11 /* RAW_DATA_0_RDY_EN */
00081 #define INT_RAW_DATA_0_RDY_EN BIT(0)
00082
00083 #define INT_ENABLE_2 0x12 /* FIFO_OVERFLOW_EN[4:0] */
00084 #define INT_ENABLE_3 0x13 /* FIFO_WM_EN[4:0] */
00085
00086 #define I2C_MST_STATUS 0x17 /* PASS_THROUGH, *_DONE, *_NACK, LOST_ARB */
00087 #define MST_PASS_THROUGH BIT(7)
00088 #define MST_SLV4_DONE BIT(6)
00089 #define MST_LOST_ARB BIT(5)
00090 #define MST_SLV4_NACK BIT(4)
00091 #define MST_SLV3_NACK BIT(3)

```



```

00092 #define MST_SLV2_NACK BIT(2)
00093 #define MST_SLV1_NACK BIT(1)
00094 #define MST_SLV0_NACK BIT(0)
00095
00096 #define INT_STATUS 0x19 /* WOM_INT PLL_RDY_INT DMP_INT1 I2C_MST_INT */
00097 #define STS_WOM_INT BIT(3)
00098 #define STS_PLL_RDY_INT BIT(2)
00099 #define STS_DMP_INT1 BIT(1)
00100 #define STS_I2C_MST_INT BIT(0)
00101
00102 #define INT_STATUS_1 0x1A /* RAW_DATA_0_RDY_INT */
00103 #define STS_RAW_DATA_0_RDY_INT BIT(0)
00104 #define INT_STATUS_2 0x1B /* FIFO_OVERFLOW_INT[4:0] */
00105 #define INT_STATUS_3 0x1C /* FIFO_WM_INT[4:0] */
00106
00107 #define DELAY_TIMEH 0x28
00108 #define DELAY_TIMEL 0x29
00109
00110 /* - Sensor outputs */
00111 #define ACCEL_XOUT_H 0x2D
00112 #define ACCEL_XOUT_L 0x2E
00113 #define ACCEL_YOUT_H 0x2F
00114 #define ACCEL_YOUT_L 0x30
00115 #define ACCEL_ZOUT_H 0x31
00116 #define ACCEL_ZOUT_L 0x32
00117 #define GYRO_XOUT_H 0x33
00118 #define GYRO_XOUT_L 0x34
00119 #define GYRO_YOUT_H 0x35
00120 #define GYRO_YOUT_L 0x36
00121 #define GYRO_ZOUT_H 0x37
00122 #define GYRO_ZOUT_L 0x38
00123 #define TEMP_OUT_H 0x39
00124 #define TEMP_OUT_L 0x3A
00125
00126 /* - External sensor shadow data (aux I2C) */
00127 #define EXT_SLV_SENS_DATA_00 0x3B
00128 #define EXT_SLV_SENS_DATA_23 0x52 /* contiguous range 0x3B..0x52 */
00129
00130 #define FIFO_EN_1 0x66 /* SLV3..SLV0 FIFO_EN */
00131 #define FIFO_EN_2 0x67 /* ACCEL GYRO_Z/Y/X TEMP FIFO_EN */
00132 #define FIFO_RST 0x68 /* FIFO_RESET[4:0] */
00133 #define FIFO_MODE 0x69 /* FIFO_MODE[4:0] */
00134 #define FIFO_COUNTH 0x70
00135 #define FIFO_COUNTL 0x71
00136 #define FIFO_R_W 0x72
00137 // #define DATA_RDY_STATUS 0x74 /* WOF_STATUS RAW_DATA_RDY[3:0] */
00138 #define FIFO_CFG 0x76
00139
00140 #define REG_BANK_SEL 0x7F /* USER_BANK[1:0] */
00141
00142 /* ===== */
00143 /* BANK 1 */
00144 /* ===== */
00145
00146 /** - Self-test / accel offsets / timebase */
00147 #define SELF_TEST_X_GYRO 0x02 /* XG_ST_DATA[7:0] */
00148 #define SELF_TEST_Y_GYRO 0x03 /* YG_ST_DATA[7:0] */
00149 #define SELF_TEST_Z_GYRO 0x04 /* ZG_ST_DATA[7:0] */
00150 #define SELF_TEST_X_ACCEL 0x0E /* XA_ST_DATA[7:0] */
00151 #define SELF_TEST_Y_ACCEL 0x0F /* YA_ST_DATA[7:0] */
00152 #define SELF_TEST_Z_ACCEL 0x10 /* ZA_ST_DATA[7:0] */
00153
00154 #define XA_OFFS_H 0x14 /* XA_OFFS[14:7] */
00155 #define XA_OFFS_L 0x15 /* XA_OFFS[6:0] */
00156 #define YA_OFFS_H 0x17 /* YA_OFFS[14:7] */
00157 #define YA_OFFS_L 0x18 /* YA_OFFS[6:0] */
00158 #define ZA_OFFS_H 0x1A /* ZA_OFFS[14:7] */
00159 #define ZA_OFFS_L 0x1B /* ZA_OFFS[6:0] */
00160
00161 #define TIMEBASE_CORRECTION_PLL 0x28 /* TBC_PLL[7:0] */
00162
00163 #define BANK1_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
00164
00165 /* ===== */
00166 /* BANK 2 */
00167 /* ===== */
00168
00169 /** - Gyro/Accel configuration, offsets, FSYNC, temperature filter */
00170 #define GYRO_SMPLRT_DIV 0x00 /* GYRO_SMPLRT_DIV[7:0] */
00171
00172 #define GYRO_CONFIG_1 0x01 /* GYRO_DLPFCFG[2:0] GYRO_FS_SEL[1:0] GYRO_FCHOICE */
00173 #define GYRO_FCHOICE BIT(0) /* when 0: DLPF on, when 1: off (per datasheet) */
00174 #define GYRO_FS_SEL_SHIFT 1
00175 #define GYRO_FS_SEL_MASK (0x3u << GYRO_FS_SEL_SHIFT)
00176 #define GYRO_FS_250DPS (0u << GYRO_FS_SEL_SHIFT)
00177 #define GYRO_FS_500DPS (1u << GYRO_FS_SEL_SHIFT)
00178 #define GYRO_FS_1000DPS (2u << GYRO_FS_SEL_SHIFT)

```

```

00179 #define GYRO_FS_2000DPS (3u << GYRO_FS_SEL_SHIFT)
00180 #define GYRO_DLPFCFG_SHIFT 5
00181 #define GYRO_DLPFCFG_MASK (0x7u << GYRO_DLPFCFG_SHIFT)
00182
00183 #define GYRO_CONFIG_2 0x02 /* XGYRO_CTEN YGYRO_CTEN ZGYRO_CTEN GYRO_AVGCFG[2:0] */
00184 #define XGYRO_CTEN BIT(5)
00185 #define YGYRO_CTEN BIT(4)
00186 #define ZGYRO_CTEN BIT(3)
00187 #define GYRO_AVGCFG_SHIFT 0
00188 #define GYRO_AVGCFG_MASK (0x7u << GYRO_AVGCFG_SHIFT)
00189
00190 #define XG_OFFS_USRH 0x03
00191 #define XG_OFFS_USRL 0x04
00192 #define YG_OFFS_USRH 0x05
00193 #define YG_OFFS_USRL 0x06
00194 #define ZG_OFFS_USRH 0x07
00195 #define ZG_OFFS_USRL 0x08
00196
00197 #define ODR_ALIGN_EN 0x09 /* ODR_ALIGN_EN */
00198 #define ODR_ALIGN_EN_BIT BIT(0)
00199
00200 #define ACCEL_SMPLRT_DIV_1 0x10 /* ACCEL_SMPLRT_DIV[11:8] */
00201 #define ACCEL_SMPLRT_DIV_2 0x11 /* ACCEL_SMPLRT_DIV[7:0] */
00202
00203 #define ACCEL_INTEL_CTRL 0x12 /* ACCEL_INTEL_EN ACCEL_INTEL_MODE_INT */
00204 #define ACCEL_INTEL_EN BIT(7)
00205 #define ACCEL_INTEL_MODE_INT BIT(6)
00206
00207 #define ACCEL_WOM_THR 0x13 /* WOM_THRESHOLD[7:0] */
00208
00209 #define ACCEL_CONFIG 0x14 /* ACCEL_DLPFCFG[2:0] ACCEL_FS_SEL[1:0] ACCEL_FCHOICE */
00210 #define ACCEL_FCHOICE BIT(0)
00211 #define ACCEL_FS_SEL_SHIFT 1
00212 #define ACCEL_FS_SEL_MASK (0x3u << ACCEL_FS_SEL_SHIFT)
00213 #define ACCEL_FS_2G (0u << ACCEL_FS_SEL_SHIFT)
00214 #define ACCEL_FS_4G (1u << ACCEL_FS_SEL_SHIFT)
00215 #define ACCEL_FS_8G (2u << ACCEL_FS_SEL_SHIFT)
00216 #define ACCEL_FS_16G (3u << ACCEL_FS_SEL_SHIFT)
00217 #define ACCEL_DLPFCFG_SHIFT 5
00218 #define ACCEL_DLPFCFG_MASK (0x7u << ACCEL_DLPFCFG_SHIFT)
00219
00220 #define ACCEL_CONFIG_2 0x15 /* AX_ST_EN_REG AY_ST_EN_REG AZ_ST_EN_REG DEC3_CFG[1:0] */
00221 #define AX_ST_EN_REG BIT(7)
00222 #define AY_ST_EN_REG BIT(6)
00223 #define AZ_ST_EN_REG BIT(5)
00224 #define DEC3_CFG_SHIFT 0
00225 #define DEC3_CFG_MASK (0x3u << DEC3_CFG_SHIFT)
00226
00227 #define FSYNC_CONFIG 0x52 /* DELAY_TIME_EN WOF DEGLITCH_EN WOF_EDGE_INT EXT_SYNC_SET[3:0] */
00228 #define DELAY_TIME_EN BIT(7)
00229 #define WOF DEGLITCH_EN BIT(6)
00230 #define WOF_EDGE_INT BIT(5)
00231 #define EXT_SYNC_SET_MASK 0x0F
00232
00233 #define TEMP_CONFIG 0x53 /* TEMP_DLPFCFG[2:0] */
00234 #define TEMP_DLPFCFG_SHIFT 5
00235 #define TEMP_DLPFCFG_MASK (0x7u << TEMP_DLPFCFG_SHIFT)
00236
00237 #define MOD_CTRL_USR 0x54 /* REG_LP_DMP_EN */
00238 #define REG_LP_DMP_EN BIT(7)
00239
00240 #define BANK2_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
00241
00242 /* ===== */
00243 /* BANK 3 */
00244 /* ===== */
00245
00246 /** - I2C master (aux) interface and slave windows */
00247 #define I2C_MST_ODR_CONFIG 0x00 /* I2C_MST_ODR_CONFIG[3:0] */
00248 #define MST_ODR_CFG_MASK 0x0F
00249
00250 #define I2C_MST_CTRL 0x01 /* MULT_MST_EN - I2C_MST_P_NSR I2C_MST_CLK[3:0] */
00251 #define MULT_MST_EN BIT(7)
00252 #define I2C_MST_P_NSR BIT(4)
00253 #define I2C_MST_CLK_MASK 0x0F
00254
00255 #define I2C_MST_DELAY_CTRL 0x02 /* DELAY_ES_SHADOW + I2C_SLVx_DELAY_EN bits */
00256 #define DELAY_ES_SHADOW BIT(7)
00257 #define I2C_SLV4_DELAY_EN BIT(4)
00258 #define I2C_SLV3_DELAY_EN BIT(3)
00259 #define I2C_SLV2_DELAY_EN BIT(2)
00260 #define I2C_SLV1_DELAY_EN BIT(1)
00261 #define I2C_SLV0_DELAY_EN BIT(0)
00262
00263 #define I2C_SLV0_ADDR 0x03 /* RNW + ID[6:0] */
00264 #define I2C_SLVx_RNW BIT(7)
00265 #define I2C_SLV0_REG 0x04

```

```

00266 #define I2C_SLV0_CTRL 0x05 /* EN BYTE_SW REG_DIS GRP LENG[3:0] */
00267 #define I2C_SLVx_EN BIT(7)
00268 #define I2C_SLVx_BYTE_SW BIT(6)
00269 #define I2C_SLVx_REG_DIS BIT(5)
00270 #define I2C_SLVx_GRP BIT(4)
00271 #define I2C_SLVx_LENG_MASK 0x0F
00272 #define I2C_SLV0_DO 0x06
00273
00274 #define I2C_SLV1_ADDR 0x07
00275 #define I2C_SLV1_REG 0x08
00276 #define I2C_SLV1_CTRL 0x09
00277 #define I2C_SLV1_DO 0x0A
00278
00279 #define I2C_SLV2_ADDR 0x0B
00280 #define I2C_SLV2_REG 0x0C
00281 #define I2C_SLV2_CTRL 0x0D
00282 #define I2C_SLV2_DO 0x0E
00283
00284 #define I2C_SLV3_ADDR 0x0F
00285 #define I2C_SLV3_REG 0x10
00286 #define I2C_SLV3_CTRL 0x11
00287 #define I2C_SLV3_DO 0x12
00288
00289 #define I2C_SLV4_ADDR 0x13
00290 #define I2C_SLV4_REG 0x14
00291 #define I2C_SLV4_CTRL 0x15 /* EN BYTE_SW REG_DIS DLY[4:0] */
00292 #define I2C_SLV4_DLY_MASK 0x1F
00293 #define I2C_SLV4_DO 0x16
00294 #define I2C_SLV4_DI 0x17
00295
00296 #define BANK3_REG_BANK_SEL 0x7F /* mirror of REG_BANK_SEL */
00297
00298 /* ===== */
00299 /* REG_BANK_SEL (all banks) */
00300 /* ===== */
00301
00302 #define REG_BANK_SEL_USER_BANK_SHIFT 4
00303 #define REG_BANK_SEL_USER_BANK_MASK (0x3u << REG_BANK_SEL_USER_BANK_SHIFT)
00304 #define USER_BANK_0 BANK0
00305 #define USER_BANK_1 BANK1
00306 #define USER_BANK_2 BANK2
00307 #define USER_BANK_3 BANK3
00308
00309 /* AK09916 magnetometer registers (connected internally) */
00310 #define AK09916_I2C_ADDR 0x0C
00311 #define AK_WIA2 0x01
00312 #define AK_ST1 0x10
00313 #define AK_HXL 0x11
00314 #define AK_ST2 0x18
00315 #define AK_CNTL2 0x31
00316 #define AK_CNTL3 0x32
00317 #define AK_WIA2_VAL 0x09 /* AK09916C WIA2 expected */
00318
00319 #define INTERNAL_20MHZ 0x00 /* 0: Internal 20 MHz RC */
00320 #define AUTO_SEL 0x01 /* 1-5: Auto/PLL preferred (use 1 as default) */
00321 #define CLK_STOP 0x07 /* 7: Stop clock / timing gen reset */
00322
00323
00324 /* ----- Accelerometer Range ----- */
00325 /** - ±2g / ±4g / ±8g / ±16g (maps to FS_SEL 0..3) */
00326 #define g2 0
00327 #define g4 1
00328 #define g8 2
00329 #define g16 3
00330
00331 /* ----- Accelerometer Filter Path (FCHOICE) ----- */
00332 /**
00333  * Path select for accel:
00334  * - ACCEL_FCHOICE_BYPASS : DLPF bypassed (very wide BW, fastest)
00335  * - ACCEL_FCHOICE_DLPF : DLPF enabled (use ACCEL_DLPFCFG + SMPLRT_DIV)
00336  */
00337 #define ACCEL_FCHOICE_BYPASS 0
00338 #define ACCEL_FCHOICE_DLPF 1
00339
00340 /* ----- Accelerometer DLPF Config (CFG) ----- */
00341 /**
00342  * ACCEL_DLPFCFG (0..7) – choose cutoff/noise BW set (when DLPF is ON).
00343  * Tip: start with ACCEL_DLPFCFG_3 for a good noise/latency tradeoff.
00344  */
00345 #define ACCEL_DLPFCFG_0 0
00346 #define ACCEL_DLPFCFG_1 1
00347 #define ACCEL_DLPFCFG_2 2
00348 #define ACCEL_DLPFCFG_3 3
00349 #define ACCEL_DLPFCFG_4 4
00350 #define ACCEL_DLPFCFG_5 5
00351 #define ACCEL_DLPFCFG_6 6
00352 #define ACCEL_DLPFCFG_7 7

```

```

00353
00354 #define ACCEL_DEC3_AVG_4      0u  /* averages 1 or 4 (depends on FCHOICE) */
00355 #define ACCEL_DEC3_AVG_8      1u
00356 #define ACCEL_DEC3_AVG_16     2u
00357 #define ACCEL_DEC3_AVG_32     3u
00358
00359 /* ---- Accel DLPF ODR helper (Hz) : base 1125 Hz, DIV 0..4095 ---- */
00360 #define ACCEL_SMPLRT_DIV_MIN   0u
00361 #define ACCEL_SMPLRT_DIV_MAX   4095u
00362 #define ACCEL_DLPF_BASE_HZ     1125.0f
00363 #define ACCEL_DLPF_ODR_HZ(div) (ACCEL_DLPF_BASE_HZ / (1.0f + (float)(div)))
00364
00365 /* ---- Accel BYPASS quick reference (FCHOICE=0) ----
00366 * 3dB ≈ 1209 Hz, NBW ≈ 1248 Hz, output rate ≈ 4500 Hz
00367 * (No divider/ODR control in bypass path.)
00368 */
00369 #define ACCEL_BYPASS_3DB_BW_HZ 1209.0f
00370 #define ACCEL_BYPASS_NBW_HZ    1248.0f
00371 #define ACCEL_BYPASS_RATE_HZ   4500.0f
00372
00373 /* (Optional) Accel DLPF nominal bandwidth references (FCHOICE=1)
00374 * Keep here for quick docs; exact values depend on datasheet rev.
00375 */
00376 #define ACCEL_DLPF0_3DB_BW_HZ  246.0f
00377 #define ACCEL_DLPF0_NBW_HZ     265.0f
00378 #define ACCEL_DLPF1_3DB_BW_HZ  246.0f
00379 #define ACCEL_DLPF1_NBW_HZ     265.0f
00380 #define ACCEL_DLPF2_3DB_BW_HZ  111.4f
00381 #define ACCEL_DLPF2_NBW_HZ     136.0f
00382 #define ACCEL_DLPF3_3DB_BW_HZ  50.4f
00383 #define ACCEL_DLPF3_NBW_HZ     68.8f
00384 #define ACCEL_DLPF4_3DB_BW_HZ  23.9f
00385 #define ACCEL_DLPF4_NBW_HZ     34.4f
00386 #define ACCEL_DLPF5_3DB_BW_HZ  11.5f
00387 #define ACCEL_DLPF5_NBW_HZ     17.0f
00388 #define ACCEL_DLPF6_3DB_BW_HZ  5.7f
00389 #define ACCEL_DLPF6_NBW_HZ     8.3f
00390 #define ACCEL_DLPF7_3DB_BW_HZ  473.0f
00391 #define ACCEL_DLPF7_NBW_HZ     499.0f
00392
00393
00394 /* ----- Gyroscope Range ----- */
00395 /** - ±250/±500/±1000/±2000 dps (maps to FS_SEL 0..3) */
00396 #define dps250    0
00397 #define dps500    1
00398 #define dps1000   2
00399 #define dps2000   3
00400
00401 /* ----- Gyroscope Filter Path (FCHOICE) ----- */
00402 /**
00403  * Path select for gyro:
00404  * - GYRO_FCHOICE_BYPASS : DLPF bypassed (widest BW, fastest)
00405  * - GYRO_FCHOICE_DLPF   : DLPF enabled (use GYRO_DLPFCFG + SMPLRT_DIV)
00406  */
00407 #define GYRO_FCHOICE_BYPASS  0
00408 #define GYRO_FCHOICE_DLPF    1
00409
00410 /* ----- Gyroscope DLPF Config (CFG) ----- */
00411 /**
00412  * GYRO_DLPFCFG (0..7) – choose cutoff/noise BW set (when DLPF is ON).
00413  * Tip: start with GYRO_DLPFCFG_3 or _2 for balanced noise/latency.
00414  */
00415 #define GYRO_DLPFCFG_0      0
00416 #define GYRO_DLPFCFG_1      1
00417 #define GYRO_DLPFCFG_2      2
00418 #define GYRO_DLPFCFG_3      3
00419 #define GYRO_DLPFCFG_4      4
00420 #define GYRO_DLPFCFG_5      5
00421 #define GYRO_DLPFCFG_6      6
00422 #define GYRO_DLPFCFG_7      7
00423
00424 /* ---- Gyro DLPF ODR helper (Hz) : base 1100 Hz, guard ≥~4.3 Hz ---- */
00425 #define GYRO_SMPLRT_MIN_HZ   4.3f
00426 #define GYRO_DLPF_BASE_HZ    1100.0f
00427 #define GYRO_DLPF_ODR_HZ(rate_request) (rate_request) /* symbolic; your driver computes DIV =
round(1100/rate)-1 */
00428
00429 /* (Optional) Gyro BW “buckets” to tag configs in UI/logs (symbolic)
00430 * Use these if you want a quick human-readable label for NBW tiers.
00431 * Exact Hz depend on datasheet table; map per your implementation.
00432 */
00433 #define GYRO_BW_ULTRA_WIDE  0  /* bypass path */
00434 #define GYRO_BW_WIDE        1  /* e.g., CFG 0/1 */
00435 #define GYRO_BW_MEDIUM      2  /* e.g., CFG 2/3 */
00436 #define GYRO_BW_NARROW      3  /* e.g., CFG 4/5/6/7 */
00437
00438 /* ----- Convenience: presets (symbolic) ----- */

```

```
00439 /** - Friendly ODR presets you can map to dividers */
00440 #define Hz2      0
00441 #define Hz6      1
00442 #define Hz8      2
00443 #define Hz10     3
00444 #define Hz15     4
00445 #define Hz20     5
00446 #define Hz25     6
00447 #define Hz30     7
00448
00449 /* ----- Power / Fusion Modes ----- */
00450 /**
00451  * Generic power or fusion quality modes (BNO-style).
00452  * Map to sensor-specific sequences inside your driver.
00453  */
00454 #define LowPower      0
00455 #define Regular       1
00456 #define EnhancedRegular 2
00457 #define HighAccuracy  3
00458
00459 #endif /* ICM20948_REGS_H */
```



# Index

~Interface\_7Semi  
    Interface\_7Semi, [64](#)

ACCEL\_BYPASS\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [179](#)

ACCEL\_BYPASS\_NBW\_HZ  
    ICM20948\_regs.h, [179](#)

ACCEL\_BYPASS\_RATE\_HZ  
    ICM20948\_regs.h, [179](#)

ACCEL\_CONFIG  
    ICM20948\_regs.h, [179](#)

ACCEL\_CONFIG\_2  
    ICM20948\_regs.h, [179](#)

ACCEL\_DEC3\_AVG\_16  
    ICM20948\_regs.h, [179](#)

ACCEL\_DEC3\_AVG\_32  
    ICM20948\_regs.h, [179](#)

ACCEL\_DEC3\_AVG\_4  
    ICM20948\_regs.h, [179](#)

ACCEL\_DEC3\_AVG\_8  
    ICM20948\_regs.h, [179](#)

ACCEL\_DLPF0\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [179](#)

ACCEL\_DLPF0\_NBW\_HZ  
    ICM20948\_regs.h, [179](#)

ACCEL\_DLPF1\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [179](#)

ACCEL\_DLPF1\_NBW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF2\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF2\_NBW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF3\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF3\_NBW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF4\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF4\_NBW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF5\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF5\_NBW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF6\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF6\_NBW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF7\_3DB\_BW\_HZ  
    ICM20948\_regs.h, [180](#)

ACCEL\_DLPF7\_NBW\_HZ  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPF\_BASE\_HZ  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPF\_ODR\_HZ  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_0  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_1  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_2  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_3  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_4  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_5  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_6  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_7  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_MASK  
    ICM20948\_regs.h, [181](#)

ACCEL\_DLPFCFG\_SHIFT  
    ICM20948\_regs.h, [182](#)

ACCEL\_FCHOICE  
    ICM20948\_regs.h, [182](#)

ACCEL\_FCHOICE\_BYPASS  
    ICM20948\_regs.h, [182](#)

ACCEL\_FCHOICE\_DLPF  
    ICM20948\_regs.h, [182](#)

ACCEL\_FS\_16G  
    ICM20948\_regs.h, [182](#)

ACCEL\_FS\_2G  
    ICM20948\_regs.h, [182](#)

ACCEL\_FS\_4G  
    ICM20948\_regs.h, [182](#)

ACCEL\_FS\_8G  
    ICM20948\_regs.h, [182](#)

ACCEL\_FS\_SEL\_MASK  
    ICM20948\_regs.h, [182](#)

ACCEL\_FS\_SEL\_SHIFT  
    ICM20948\_regs.h, [182](#)

ACCEL\_INTEL\_CTRL  
    ICM20948\_regs.h, [182](#)

ACCEL\_INTEL\_EN  
    ICM20948\_regs.h, [183](#)

ACCEL\_INTEL\_MODE\_INT  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_SMPLRT\_DIV\_1  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_SMPLRT\_DIV\_2  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_SMPLRT\_DIV\_MAX  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_SMPLRT\_DIV\_MIN  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_WOM\_THR  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_XOUT\_H  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_XOUT\_L  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_YOUT\_H  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_YOUT\_L  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_ZOUT\_H  
     ICM20948\_regs.h, [183](#)  
 ACCEL\_ZOUT\_L  
     ICM20948\_regs.h, [184](#)  
 accelAVG  
     test.ino, [117](#)  
     test2.ino, [127](#)  
 accelCfg  
     test.ino, [117](#)  
     test2.ino, [127](#)  
 accelDLPF  
     test.ino, [117](#)  
     test2.ino, [127](#)  
 accelSR  
     test.ino, [118](#)  
     test2.ino, [127](#)  
 activeLevel  
     ICM20948\_IntPinConfig, [60](#)  
 address  
     I2C\_Interface, [58](#)  
 AK09916\_I2C\_ADDR  
     ICM20948\_regs.h, [184](#)  
 AK\_CNTL2  
     ICM20948\_regs.h, [184](#)  
 AK\_CNTL3  
     ICM20948\_regs.h, [184](#)  
 AK\_HXL  
     ICM20948\_regs.h, [184](#)  
 AK\_ST1  
     ICM20948\_regs.h, [184](#)  
 AK\_ST2  
     ICM20948\_regs.h, [184](#)  
 AK\_WIA2  
     ICM20948\_regs.h, [184](#)  
 AK\_WIA2\_VAL  
     ICM20948\_regs.h, [184](#)  
 applyBasicDefaults  
     DevLab\_ICM20948, [23](#)  
 applySettings  
     gyr\_test.ino, [71](#)  
     mag\_test.ino, [99](#)  
     test.ino, [114](#)  
     test2.ino, [124](#)  
 AUTO\_PLL\_1  
     DevLab\_ICM20948.h, [160](#)  
 AUTO\_PLL\_2  
     DevLab\_ICM20948.h, [160](#)  
 AUTO\_PLL\_3  
     DevLab\_ICM20948.h, [160](#)  
 AUTO\_PLL\_4  
     DevLab\_ICM20948.h, [160](#)  
 AUTO\_PLL\_5  
     DevLab\_ICM20948.h, [160](#)  
 AUTO\_SEL  
     ICM20948\_regs.h, [184](#)  
 auxConfigSlave  
     DevLab\_ICM20948, [23](#)  
 auxMasterEnable  
     DevLab\_ICM20948, [24](#)  
 auxRead12bit  
     DevLab\_ICM20948, [24](#)  
 auxReadByte  
     DevLab\_ICM20948, [25](#)  
 auxReadResponse  
     DevLab\_ICM20948, [25](#)  
 auxReadSensorData  
     DevLab\_ICM20948, [26](#)  
 auxWriteByte  
     DevLab\_ICM20948, [27](#)  
 auxWriteCommand  
     DevLab\_ICM20948, [27](#)  
 AVG\_1  
     DevLab\_ICM20948.h, [160](#)  
 AVG\_128  
     DevLab\_ICM20948.h, [161](#)  
 AVG\_16  
     DevLab\_ICM20948.h, [159](#), [160](#)  
 AVG\_1\_OR\_4  
     DevLab\_ICM20948.h, [159](#)  
 AVG\_2  
     DevLab\_ICM20948.h, [160](#)  
 AVG\_32  
     DevLab\_ICM20948.h, [159](#), [160](#)  
 AVG\_4  
     DevLab\_ICM20948.h, [160](#)  
 AVG\_64  
     DevLab\_ICM20948.h, [160](#)  
 AVG\_8  
     DevLab\_ICM20948.h, [159](#), [160](#)  
 AX\_ST\_EN\_REG  
     ICM20948\_regs.h, [184](#)  
 AY\_ST\_EN\_REG  
     ICM20948\_regs.h, [184](#)  
 AZ\_ST\_EN\_REG  
     ICM20948\_regs.h, [185](#)  
 BANK1\_REG\_BANK\_SEL



- ICM20948\_\_regs.h, [185](#)
- BANK2\_REG\_BANK\_SEL
  - ICM20948\_\_regs.h, [185](#)
- BANK3\_REG\_BANK\_SEL
  - ICM20948\_\_regs.h, [185](#)
- beginI2C
  - DevLab\_ICM20948, [28](#)
  - I2C\_Interface, [57](#)
  - Interface\_7Semi, [64](#)
  - SPI\_Interface, [67](#)
- beginSPI
  - DevLab\_ICM20948, [29](#)
  - I2C\_Interface, [57](#)
  - Interface\_7Semi, [64](#)
  - SPI\_Interface, [67](#)
- BusIO\_7Semi
  - BusIO\_7Semi< Bus >, [11](#)
- BusIO\_7Semi< Bus >, [11](#)
  - BusIO\_7Semi, [11](#)
  - read, [12](#), [13](#)
  - readBit, [13](#), [14](#)
  - readBits, [14](#), [15](#)
  - write, [15](#), [16](#)
  - writeBit, [17](#), [18](#)
  - writeBits, [18](#), [19](#)
- BYPASS\_EN
  - ICM20948\_\_regs.h, [185](#)
- bypassEn
  - ICM20948\_\_IntPinConfig, [60](#)
- checkIntStatus
  - DevLab\_ICM20948, [30](#)
- clearMode
  - ICM20948\_\_IntPinConfig, [60](#)
- CLK\_STOP
  - ICM20948\_\_regs.h, [185](#)
- configDLPF, [20](#)
  - div, [20](#)
  - dlpf\_idx, [21](#)
  - nbw\_hz, [21](#)
  - nombre, [21](#)
  - odr\_hz, [21](#)
- configsDLPF
  - gyr\_test.ino, [73](#)
  - test.ino, [118](#)
  - test2.ino, [127](#)
- cs
  - SPI\_Interface, [68](#)
- DEC3\_CFG\_MASK
  - ICM20948\_\_regs.h, [185](#)
- DEC3\_CFG\_SHIFT
  - ICM20948\_\_regs.h, [185](#)
- DELAY\_ES\_SHADOW
  - ICM20948\_\_regs.h, [185](#)
- delay\_ms
  - icm\_plat\_t, [62](#)
- DELAY\_TIME\_EN
  - ICM20948\_\_regs.h, [185](#)
- DELAY\_TIMEH
  - ICM20948\_\_regs.h, [185](#)
- DELAY\_TIMEL
  - ICM20948\_\_regs.h, [185](#)
- delay\_us
  - Interface\_7Semi, [64](#)
- DevLab ICM20948 Arduino Library, [1](#)
- DevLab\_ICM20948, [21](#)
  - applyBasicDefaults, [23](#)
  - auxConfigSlave, [23](#)
  - auxMasterEnable, [24](#)
  - auxRead12bit, [24](#)
  - auxReadByte, [25](#)
  - auxReadResponse, [25](#)
  - auxReadSensorData, [26](#)
  - auxWriteByte, [27](#)
  - auxWriteCommand, [27](#)
  - beginI2C, [28](#)
  - beginSPI, [29](#)
  - checkIntStatus, [30](#)
  - DevLab\_ICM20948, [22](#)
  - getAccelAveraging, [30](#)
  - getAccelDLPF, [31](#)
  - getAccelOffset, [31](#)
  - getAccelSampleRate, [32](#)
  - getAccelScale, [32](#)
  - getClock, [33](#)
  - getDLPF, [33](#)
  - getGyroOffset, [34](#)
  - getGyroSampleRate, [34](#)
  - getGyroScale, [35](#)
  - getLowPower, [35](#)
  - getSensors, [36](#)
  - initMag, [36](#)
  - intEnableConfig, [37](#)
  - intInit, [38](#)
  - readAccel, [39](#)
  - readGyro, [39](#)
  - readMag, [40](#)
  - readTemperature, [41](#)
  - readWhoAmI, [42](#)
  - selectBank, [42](#)
  - selfTestAccel, [43](#)
  - selfTestGyro, [43](#)
  - setAccelAveraging, [44](#)
  - setAccelDivRate, [45](#)
  - setAccelDLPF, [45](#)
  - setAccelOffset, [46](#)
  - setAccelSampleRate, [47](#)
  - setAccelScale, [47](#)
  - setClock, [48](#)
  - setDLPF, [49](#)
  - setGyroAveraging, [49](#)
  - setGyroDivRate, [50](#)
  - setGyroOffset, [50](#)
  - setGyroSampleRate, [51](#)
  - setGyroScale, [51](#)
  - setLowPower, [52](#)

- setMagOpMode, [52](#)
- setSensors, [53](#)
- sleep, [54](#)
- softReset, [54](#)
- DevLab\_ICM20948.h
  - AUTO\_PLL\_1, [160](#)
  - AUTO\_PLL\_2, [160](#)
  - AUTO\_PLL\_3, [160](#)
  - AUTO\_PLL\_4, [160](#)
  - AUTO\_PLL\_5, [160](#)
  - AVG\_1, [160](#)
  - AVG\_128, [161](#)
  - AVG\_16, [159](#), [160](#)
  - AVG\_1\_OR\_4, [159](#)
  - AVG\_2, [160](#)
  - AVG\_32, [159](#), [160](#)
  - AVG\_4, [160](#)
  - AVG\_64, [160](#)
  - AVG\_8, [159](#), [160](#)
  - DLPF0\_246HZ, [159](#)
  - DLPF\_111HZ, [159](#)
  - DLPF\_119HZ, [161](#)
  - DLPF\_11HZ, [161](#)
  - DLPF\_12HZ, [160](#)
  - DLPF\_151HZ, [161](#)
  - DLPF\_196HZ, [161](#)
  - DLPF\_23HZ, [161](#)
  - DLPF\_246HZ, [159](#)
  - DLPF\_24HZ, [159](#)
  - DLPF\_361HZ, [161](#)
  - DLPF\_473HZ, [160](#)
  - DLPF\_50HZ, [159](#)
  - DLPF\_51HZ, [161](#)
  - DLPF\_5HZ, [161](#)
  - DLPF\_6HZ, [160](#)
  - DPS\_1000, [161](#)
  - DPS\_2000, [161](#)
  - DPS\_250, [161](#)
  - DPS\_500, [161](#)
  - G\_16, [160](#)
  - G\_2, [160](#)
  - G\_4, [160](#)
  - G\_8, [160](#)
  - ICM20948\_Accel\_Average, [159](#)
  - ICM20948\_Accel\_DLPCFG, [159](#)
  - ICM20948\_Accel\_FullScale, [160](#)
  - ICM20948\_Clock\_Source, [160](#)
  - ICM20948\_Gyro\_Average, [160](#)
  - ICM20948\_Gyro\_DLPF, [161](#)
  - ICM20948\_Gyro\_FullScale, [161](#)
  - ICM20948\_Op\_Mode, [161](#)
  - INTERNAL\_20MHZ\_0, [160](#)
  - INTERNAL\_20MHZ\_6, [160](#)
  - MODE\_CONTINUOUS\_1, [161](#)
  - MODE\_CONTINUOUS\_2, [161](#)
  - MODE\_CONTINUOUS\_3, [161](#)
  - MODE\_CONTINUOUS\_4, [161](#)
  - MODE\_POWER\_DOWN, [161](#)
  - MODE\_SELF\_TEST, [161](#)
  - MODE\_SINGLE\_MEASUREMENT, [161](#)
  - STOP\_CLOCK, [160](#)
- div
  - configDLPF, [20](#)
- DLPF0\_246HZ
  - DevLab\_ICM20948.h, [159](#)
- DLPF\_111HZ
  - DevLab\_ICM20948.h, [159](#)
- DLPF\_119HZ
  - DevLab\_ICM20948.h, [161](#)
- DLPF\_11HZ
  - DevLab\_ICM20948.h, [161](#)
- DLPF\_12HZ
  - DevLab\_ICM20948.h, [160](#)
- DLPF\_151HZ
  - DevLab\_ICM20948.h, [161](#)
- DLPF\_196HZ
  - DevLab\_ICM20948.h, [161](#)
- DLPF\_23HZ
  - DevLab\_ICM20948.h, [161](#)
- DLPF\_246HZ
  - DevLab\_ICM20948.h, [159](#)
- DLPF\_24HZ
  - DevLab\_ICM20948.h, [159](#)
- DLPF\_361HZ
  - DevLab\_ICM20948.h, [161](#)
- DLPF\_473HZ
  - DevLab\_ICM20948.h, [160](#)
- DLPF\_50HZ
  - DevLab\_ICM20948.h, [159](#)
- DLPF\_51HZ
  - DevLab\_ICM20948.h, [161](#)
- DLPF\_5HZ
  - DevLab\_ICM20948.h, [161](#)
- DLPF\_6HZ
  - DevLab\_ICM20948.h, [160](#)
- dlpf\_idx
  - configDLPF, [21](#)
- dmpInt1
  - ICM20948\_IntStatus, [61](#)
- dmpInt1En
  - ICM20948\_IntEnableConfig, [59](#)
- dps1000
  - ICM20948\_regs.h, [186](#)
- dps2000
  - ICM20948\_regs.h, [186](#)
- dps250
  - ICM20948\_regs.h, [186](#)
- dps500
  - ICM20948\_regs.h, [186](#)
- DPS\_1000
  - DevLab\_ICM20948.h, [161](#)
- DPS\_2000
  - DevLab\_ICM20948.h, [161](#)
- DPS\_250
  - DevLab\_ICM20948.h, [161](#)
- DPS\_500

- DevLab\_ICM20948.h, [161](#)
- driveMode
  - ICM20948\_IntPinConfig, [60](#)
- EnhancedRegular
  - ICM20948\_regs.h, [186](#)
- etiquetasAccelAVG
  - test.ino, [118](#)
  - test2.ino, [127](#)
- etiquetasAccelCfg
  - test.ino, [118](#)
  - test2.ino, [128](#)
- etiquetasAccelDLPFCFG
  - test.ino, [118](#)
- etiquetasGyroAVGCfg
  - gyr\_test.ino, [74](#)
- etiquetasGyroFSCfg
  - gyr\_test.ino, [74](#)
- etiquetasOpModeCfg
  - mag\_test.ino, [100](#)
- etiquetasSR
  - test.ino, [118](#)
  - test2.ino, [128](#)
- examples/gyr\_test/gyr\_test.ino, [69](#), [75](#)
- examples/i2c\_accel/i2c\_accel.ino, [78](#), [80](#)
- examples/i2c\_basic/i2c\_basic.ino, [82](#), [84](#)
- examples/i2c\_gyro/i2c\_gyro.ino, [86](#), [88](#)
- examples/i2c\_magneto/i2c\_magneto.ino, [89](#), [91](#)
- examples/interrupt/interrupt.ino, [93](#), [96](#)
- examples/mag\_test/mag\_test.ino, [97](#), [101](#)
- examples/spi\_accel/spi\_accel.ino, [102](#), [104](#)
- examples/spi\_basic/spi\_basic.ino, [106](#), [107](#)
- examples/spi\_gyro/spi\_gyro.ino, [109](#), [110](#)
- examples/test/test.ino, [112](#), [119](#)
- examples/test2/test2.ino, [122](#), [129](#)
- EXT\_SLV\_SENS\_DATA\_00
  - ICM20948\_regs.h, [186](#)
- EXT\_SLV\_SENS\_DATA\_23
  - ICM20948\_regs.h, [186](#)
- EXT\_SYNC\_SET\_MASK
  - ICM20948\_regs.h, [186](#)
- FIFO\_CFG
  - ICM20948\_regs.h, [186](#)
- FIFO\_COUNTH
  - ICM20948\_regs.h, [186](#)
- FIFO\_COUNTL
  - ICM20948\_regs.h, [186](#)
- FIFO\_EN\_1
  - ICM20948\_regs.h, [186](#)
- FIFO\_EN\_2
  - ICM20948\_regs.h, [187](#)
- FIFO\_MODE
  - ICM20948\_regs.h, [187](#)
- FIFO\_R\_W
  - ICM20948\_regs.h, [187](#)
- FIFO\_RST
  - ICM20948\_regs.h, [187](#)
- fifoOvf
  - ICM20948\_IntStatus, [61](#)
- fifoOvfEn
  - ICM20948\_IntEnableConfig, [59](#)
- fifoWm
  - ICM20948\_IntStatus, [61](#)
- fifoWmEn
  - ICM20948\_IntEnableConfig, [59](#)
- firstStage
  - gyr\_test.ino, [71](#)
  - test.ino, [115](#)
  - test2.ino, [124](#)
- FSYNC\_CONFIG
  - ICM20948\_regs.h, [187](#)
- FSYNC\_INT\_MODE\_EN
  - ICM20948\_regs.h, [187](#)
- fsyncActLevel
  - ICM20948\_IntPinConfig, [60](#)
- fsyncIntEn
  - ICM20948\_IntPinConfig, [60](#)
- g16
  - ICM20948\_regs.h, [187](#)
- g2
  - ICM20948\_regs.h, [187](#)
- g4
  - ICM20948\_regs.h, [187](#)
- g8
  - ICM20948\_regs.h, [187](#)
- G\_16
  - DevLab\_ICM20948.h, [160](#)
- G\_2
  - DevLab\_ICM20948.h, [160](#)
- G\_4
  - DevLab\_ICM20948.h, [160](#)
- G\_8
  - DevLab\_ICM20948.h, [160](#)
- getAccelAveraging
  - DevLab\_ICM20948, [30](#)
- getAccelDLPF
  - DevLab\_ICM20948, [31](#)
- getAccelOffset
  - DevLab\_ICM20948, [31](#)
- getAccelSampleRate
  - DevLab\_ICM20948, [32](#)
- getAccelScale
  - DevLab\_ICM20948, [32](#)
- getClock
  - DevLab\_ICM20948, [33](#)
- getDLPF
  - DevLab\_ICM20948, [33](#)
- getGyroOffset
  - DevLab\_ICM20948, [34](#)
- getGyroSampleRate
  - DevLab\_ICM20948, [34](#)
- getGyroScale
  - DevLab\_ICM20948, [35](#)
- getLowPower
  - DevLab\_ICM20948, [35](#)
- getSensors

- DevLab\_ICM20948, [36](#)
- gyr\_test.ino
  - applySettings, [71](#)
  - configsDLPF, [73](#)
  - etiquetasGyroAVGCfg, [74](#)
  - etiquetasGyroFSCfg, [74](#)
  - firstStage, [71](#)
  - gyroAVGCfg, [74](#)
  - gyroDLPF, [74](#)
  - gyroFSCfg, [74](#)
  - I2C\_FREQ, [70](#)
  - ICM\_ADDR, [70](#)
  - imu, [74](#)
  - loop, [72](#)
  - N\_AVG, [75](#)
  - N\_DLPF, [75](#)
  - N\_FS, [75](#)
  - printDobleLinea, [72](#)
  - printLinea, [72](#)
  - runSweep, [72](#)
  - SCL\_PIN, [70](#)
  - SDA\_PIN, [71](#)
  - setup, [73](#)
- GYRO\_AVGC\_CFG\_MASK
  - ICM20948\_regs.h, [187](#)
- GYRO\_AVGC\_CFG\_SHIFT
  - ICM20948\_regs.h, [187](#)
- GYRO\_BW\_MEDIUM
  - ICM20948\_regs.h, [188](#)
- GYRO\_BW\_NARROW
  - ICM20948\_regs.h, [188](#)
- GYRO\_BW\_ULTRA\_WIDE
  - ICM20948\_regs.h, [188](#)
- GYRO\_BW\_WIDE
  - ICM20948\_regs.h, [188](#)
- GYRO\_CONFIG\_1
  - ICM20948\_regs.h, [188](#)
- GYRO\_CONFIG\_2
  - ICM20948\_regs.h, [188](#)
- GYRO\_DLPF\_BASE\_HZ
  - ICM20948\_regs.h, [188](#)
- GYRO\_DLPF\_ODR\_HZ
  - ICM20948\_regs.h, [188](#)
- GYRO\_DLPFCFG\_0
  - ICM20948\_regs.h, [188](#)
- GYRO\_DLPFCFG\_1
  - ICM20948\_regs.h, [188](#)
- GYRO\_DLPFCFG\_2
  - ICM20948\_regs.h, [188](#)
- GYRO\_DLPFCFG\_3
  - ICM20948\_regs.h, [188](#)
- GYRO\_DLPFCFG\_4
  - ICM20948\_regs.h, [189](#)
- GYRO\_DLPFCFG\_5
  - ICM20948\_regs.h, [189](#)
- GYRO\_DLPFCFG\_6
  - ICM20948\_regs.h, [189](#)
- GYRO\_DLPFCFG\_7
  - ICM20948\_regs.h, [189](#)
- GYRO\_DLPFCFG\_MASK
  - ICM20948\_regs.h, [189](#)
- GYRO\_DLPFCFG\_SHIFT
  - ICM20948\_regs.h, [189](#)
- GYRO\_FCHOICE
  - ICM20948\_regs.h, [189](#)
- GYRO\_FCHOICE\_BYPASS
  - ICM20948\_regs.h, [189](#)
- GYRO\_FCHOICE\_DLPF
  - ICM20948\_regs.h, [189](#)
- GYRO\_FS\_1000DPS
  - ICM20948\_regs.h, [189](#)
- GYRO\_FS\_2000DPS
  - ICM20948\_regs.h, [189](#)
- GYRO\_FS\_250DPS
  - ICM20948\_regs.h, [190](#)
- GYRO\_FS\_500DPS
  - ICM20948\_regs.h, [190](#)
- GYRO\_FS\_SEL\_MASK
  - ICM20948\_regs.h, [190](#)
- GYRO\_FS\_SEL\_SHIFT
  - ICM20948\_regs.h, [190](#)
- GYRO\_SMPLRT\_DIV
  - ICM20948\_regs.h, [190](#)
- GYRO\_SMPLRT\_MIN\_HZ
  - ICM20948\_regs.h, [190](#)
- GYRO\_XOUT\_H
  - ICM20948\_regs.h, [190](#)
- GYRO\_XOUT\_L
  - ICM20948\_regs.h, [190](#)
- GYRO\_YOUT\_H
  - ICM20948\_regs.h, [190](#)
- GYRO\_YOUT\_L
  - ICM20948\_regs.h, [190](#)
- GYRO\_ZOUT\_H
  - ICM20948\_regs.h, [190](#)
- GYRO\_ZOUT\_L
  - ICM20948\_regs.h, [190](#)
- gyroAVGCfg
  - gyr\_test.ino, [74](#)
- gyroDLPF
  - gyr\_test.ino, [74](#)
- gyroFSCfg
  - gyr\_test.ino, [74](#)
- HighAccuracy
  - ICM20948\_regs.h, [191](#)
- Hz10
  - ICM20948\_regs.h, [191](#)
- Hz15
  - ICM20948\_regs.h, [191](#)
- Hz2
  - ICM20948\_regs.h, [191](#)
- Hz20
  - ICM20948\_regs.h, [191](#)
- Hz25
  - ICM20948\_regs.h, [191](#)
- Hz30
  - ICM20948\_regs.h, [191](#)

- ICM20948\_regs.h, [191](#)
- Hz6
  - ICM20948\_regs.h, [191](#)
- Hz8
  - ICM20948\_regs.h, [191](#)
- i2c
  - I2C\_Interface, [58](#)
- i2c\_accel.ino
  - I2C\_FREQ, [78](#)
  - ICM\_ADDR, [78](#)
  - imu, [80](#)
  - loop, [79](#)
  - SCL\_PIN, [79](#)
  - SDA\_PIN, [79](#)
  - setup, [79](#)
- i2c\_addr
  - icm\_plat\_t, [62](#)
- i2c\_basic.ino
  - I2C\_FREQ, [82](#)
  - ICM\_ADDR, [82](#)
  - imu, [84](#)
  - loop, [83](#)
  - SCL\_PIN, [82](#)
  - SDA\_PIN, [83](#)
  - setup, [83](#)
- I2C\_FREQ
  - gyr\_test.ino, [70](#)
  - i2c\_accel.ino, [78](#)
  - i2c\_basic.ino, [82](#)
  - i2c\_gyro.ino, [86](#)
  - i2c\_magneto.ino, [90](#)
  - interrupt.ino, [94](#)
  - mag\_test.ino, [98](#)
  - test.ino, [114](#)
  - test2.ino, [124](#)
- i2c\_gyro.ino
  - I2C\_FREQ, [86](#)
  - ICM\_ADDR, [86](#)
  - imu, [87](#)
  - loop, [87](#)
  - SCL\_PIN, [86](#)
  - SDA\_PIN, [87](#)
  - setup, [87](#)
- I2C\_Interface, [55](#)
  - address, [58](#)
  - beginI2C, [57](#)
  - beginSPI, [57](#)
  - i2c, [58](#)
  - read, [57](#)
  - write, [58](#)
- i2c\_magneto.ino
  - I2C\_FREQ, [90](#)
  - ICM\_ADDR, [90](#)
  - imu, [91](#)
  - loop, [90](#)
  - SCL\_PIN, [90](#)
  - SDA\_PIN, [90](#)
  - setup, [90](#)
- I2C\_MST\_CLK\_MASK
  - ICM20948\_regs.h, [191](#)
- I2C\_MST\_CTRL
  - ICM20948\_regs.h, [191](#)
- I2C\_MST\_DELAY\_CTRL
  - ICM20948\_regs.h, [191](#)
- I2C\_MST\_ODR\_CONFIG
  - ICM20948\_regs.h, [192](#)
- I2C\_MST\_P\_NSR
  - ICM20948\_regs.h, [192](#)
- I2C\_MST\_STATUS
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV0\_ADDR
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV0\_CTRL
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV0\_DELAY\_EN
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV0\_DO
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV0\_REG
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV1\_ADDR
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV1\_CTRL
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV1\_DELAY\_EN
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV1\_DO
  - ICM20948\_regs.h, [192](#)
- I2C\_SLV1\_REG
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV2\_ADDR
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV2\_CTRL
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV2\_DELAY\_EN
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV2\_DO
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV2\_REG
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV3\_ADDR
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV3\_CTRL
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV3\_DELAY\_EN
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV3\_DO
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV3\_REG
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV4\_ADDR
  - ICM20948\_regs.h, [193](#)
- I2C\_SLV4\_CTRL
  - ICM20948\_regs.h, [194](#)
- I2C\_SLV4\_DELAY\_EN
  - ICM20948\_regs.h, [194](#)

- I2C\_SLV4\_DI
  - ICM20948\_regs.h, [194](#)
- I2C\_SLV4\_DLY\_MASK
  - ICM20948\_regs.h, [194](#)
- I2C\_SLV4\_DO
  - ICM20948\_regs.h, [194](#)
- I2C\_SLV4\_REG
  - ICM20948\_regs.h, [194](#)
- I2C\_SLVx\_BYTE\_SW
  - ICM20948\_regs.h, [194](#)
- I2C\_SLVx\_EN
  - ICM20948\_regs.h, [194](#)
- I2C\_SLVx\_GRP
  - ICM20948\_regs.h, [194](#)
- I2C\_SLVx LENG\_MASK
  - ICM20948\_regs.h, [194](#)
- I2C\_SLVx\_REG\_DIS
  - ICM20948\_regs.h, [194](#)
- I2C\_SLVx\_RNW
  - ICM20948\_regs.h, [194](#)
- i2cMstInt
  - ICM20948\_IntStatus, [61](#)
- i2cMstIntEn
  - ICM20948\_IntEnableConfig, [59](#)
- ICM20948\_Accel\_Average
  - DevLab\_ICM20948.h, [159](#)
- ICM20948\_Accel\_DLPFCFG
  - DevLab\_ICM20948.h, [159](#)
- ICM20948\_Accel\_FullScale
  - DevLab\_ICM20948.h, [160](#)
- ICM20948\_Clock\_Source
  - DevLab\_ICM20948.h, [160](#)
- ICM20948\_Gyro\_Average
  - DevLab\_ICM20948.h, [160](#)
- ICM20948\_Gyro\_DLPF
  - DevLab\_ICM20948.h, [161](#)
- ICM20948\_Gyro\_FullScale
  - DevLab\_ICM20948.h, [161](#)
- ICM20948\_IntEnableConfig, [58](#)
  - dmpInt1En, [59](#)
  - fifoOvfEn, [59](#)
  - fifoWmEn, [59](#)
  - i2cMstIntEn, [59](#)
  - pllRdyEn, [59](#)
  - rawDataRdyEn, [59](#)
  - wofEn, [59](#)
  - womIntEn, [59](#)
- ICM20948\_IntPinConfig, [59](#)
  - activeLevel, [60](#)
  - bypassEn, [60](#)
  - clearMode, [60](#)
  - driveMode, [60](#)
  - fsyncActLevel, [60](#)
  - fsyncIntEn, [60](#)
  - latchMode, [60](#)
- ICM20948\_IntStatus, [61](#)
  - dmpInt1, [61](#)
  - fifoOvf, [61](#)
  - fifoWm, [61](#)
  - i2cMstInt, [61](#)
  - pllRdyInt, [61](#)
  - rawDataRdy, [61](#)
  - womInt, [61](#)
- ICM20948\_Op\_Mode
  - DevLab\_ICM20948.h, [161](#)
- ICM20948\_regs.h
  - ACCEL\_BYPASS\_3DB\_BW\_HZ, [179](#)
  - ACCEL\_BYPASS\_NBW\_HZ, [179](#)
  - ACCEL\_BYPASS\_RATE\_HZ, [179](#)
  - ACCEL\_CONFIG, [179](#)
  - ACCEL\_CONFIG\_2, [179](#)
  - ACCEL\_DEC3\_AVG\_16, [179](#)
  - ACCEL\_DEC3\_AVG\_32, [179](#)
  - ACCEL\_DEC3\_AVG\_4, [179](#)
  - ACCEL\_DEC3\_AVG\_8, [179](#)
  - ACCEL\_DLFPF0\_3DB\_BW\_HZ, [179](#)
  - ACCEL\_DLFPF0\_NBW\_HZ, [179](#)
  - ACCEL\_DLFPF1\_3DB\_BW\_HZ, [179](#)
  - ACCEL\_DLFPF1\_NBW\_HZ, [180](#)
  - ACCEL\_DLFPF2\_3DB\_BW\_HZ, [180](#)
  - ACCEL\_DLFPF2\_NBW\_HZ, [180](#)
  - ACCEL\_DLFPF3\_3DB\_BW\_HZ, [180](#)
  - ACCEL\_DLFPF3\_NBW\_HZ, [180](#)
  - ACCEL\_DLFPF4\_3DB\_BW\_HZ, [180](#)
  - ACCEL\_DLFPF4\_NBW\_HZ, [180](#)
  - ACCEL\_DLFPF5\_3DB\_BW\_HZ, [180](#)
  - ACCEL\_DLFPF5\_NBW\_HZ, [180](#)
  - ACCEL\_DLFPF6\_3DB\_BW\_HZ, [180](#)
  - ACCEL\_DLFPF6\_NBW\_HZ, [180](#)
  - ACCEL\_DLFPF7\_3DB\_BW\_HZ, [180](#)
  - ACCEL\_DLFPF7\_NBW\_HZ, [181](#)
  - ACCEL\_DLFPF\_BASE\_HZ, [181](#)
  - ACCEL\_DLFPF\_ODR\_HZ, [181](#)
  - ACCEL\_DLPFCFG\_0, [181](#)
  - ACCEL\_DLPFCFG\_1, [181](#)
  - ACCEL\_DLPFCFG\_2, [181](#)
  - ACCEL\_DLPFCFG\_3, [181](#)
  - ACCEL\_DLPFCFG\_4, [181](#)
  - ACCEL\_DLPFCFG\_5, [181](#)
  - ACCEL\_DLPFCFG\_6, [181](#)
  - ACCEL\_DLPFCFG\_7, [181](#)
  - ACCEL\_DLPFCFG\_MASK, [181](#)
  - ACCEL\_DLPFCFG\_SHIFT, [182](#)
  - ACCEL\_FCHOICE, [182](#)
  - ACCEL\_FCHOICE\_BYPASS, [182](#)
  - ACCEL\_FCHOICE\_DLPF, [182](#)
  - ACCEL\_FS\_16G, [182](#)
  - ACCEL\_FS\_2G, [182](#)
  - ACCEL\_FS\_4G, [182](#)
  - ACCEL\_FS\_8G, [182](#)
  - ACCEL\_FS\_SEL\_MASK, [182](#)
  - ACCEL\_FS\_SEL\_SHIFT, [182](#)
  - ACCEL\_INTEL\_CTRL, [182](#)
  - ACCEL\_INTEL\_EN, [183](#)
  - ACCEL\_INTEL\_MODE\_INT, [183](#)
  - ACCEL\_SMPLRT\_DIV\_1, [183](#)

ACCEL\_SMPLRT\_DIV\_2, 183  
ACCEL\_SMPLRT\_DIV\_MAX, 183  
ACCEL\_SMPLRT\_DIV\_MIN, 183  
ACCEL\_WOM\_THR, 183  
ACCEL\_XOUT\_H, 183  
ACCEL\_XOUT\_L, 183  
ACCEL\_YOUT\_H, 183  
ACCEL\_YOUT\_L, 183  
ACCEL\_ZOUT\_H, 183  
ACCEL\_ZOUT\_L, 184  
AK09916\_I2C\_ADDR, 184  
AK\_CNTL2, 184  
AK\_CNTL3, 184  
AK\_HXL, 184  
AK\_ST1, 184  
AK\_ST2, 184  
AK\_WIA2, 184  
AK\_WIA2\_VAL, 184  
AUTO\_SEL, 184  
AX\_ST\_EN\_REG, 184  
AY\_ST\_EN\_REG, 184  
AZ\_ST\_EN\_REG, 185  
BANK1\_REG\_BANK\_SEL, 185  
BANK2\_REG\_BANK\_SEL, 185  
BANK3\_REG\_BANK\_SEL, 185  
BYPASS\_EN, 185  
CLK\_STOP, 185  
DEC3\_CFG\_MASK, 185  
DEC3\_CFG\_SHIFT, 185  
DELAY\_ES\_SHADOW, 185  
DELAY\_TIME\_EN, 185  
DELAY\_TIMEH, 185  
DELAY\_TIMEL, 185  
dps1000, 186  
dps2000, 186  
dps250, 186  
dps500, 186  
EnhancedRegular, 186  
EXT\_SLV\_SENS\_DATA\_00, 186  
EXT\_SLV\_SENS\_DATA\_23, 186  
EXT\_SYNC\_SET\_MASK, 186  
FIFO\_CFG, 186  
FIFO\_COUNTH, 186  
FIFO\_COUNTL, 186  
FIFO\_EN\_1, 186  
FIFO\_EN\_2, 187  
FIFO\_MODE, 187  
FIFO\_R\_W, 187  
FIFO\_RST, 187  
FSYNC\_CONFIG, 187  
FSYNC\_INT\_MODE\_EN, 187  
g16, 187  
g2, 187  
g4, 187  
g8, 187  
GYRO\_AVGCFG\_MASK, 187  
GYRO\_AVGCFG\_SHIFT, 187  
GYRO\_BW\_MEDIUM, 188  
GYRO\_BW\_NARROW, 188  
GYRO\_BW\_ULTRA\_WIDE, 188  
GYRO\_BW\_WIDE, 188  
GYRO\_CONFIG\_1, 188  
GYRO\_CONFIG\_2, 188  
GYRO\_DLPF\_BASE\_HZ, 188  
GYRO\_DLPF\_ODR\_HZ, 188  
GYRO\_DLPFCFG\_0, 188  
GYRO\_DLPFCFG\_1, 188  
GYRO\_DLPFCFG\_2, 188  
GYRO\_DLPFCFG\_3, 188  
GYRO\_DLPFCFG\_4, 189  
GYRO\_DLPFCFG\_5, 189  
GYRO\_DLPFCFG\_6, 189  
GYRO\_DLPFCFG\_7, 189  
GYRO\_DLPFCFG\_MASK, 189  
GYRO\_DLPFCFG\_SHIFT, 189  
GYRO\_FCHOICE, 189  
GYRO\_FCHOICE\_BYPASS, 189  
GYRO\_FCHOICE\_DLPF, 189  
GYRO\_FS\_1000DPS, 189  
GYRO\_FS\_2000DPS, 189  
GYRO\_FS\_250DPS, 190  
GYRO\_FS\_500DPS, 190  
GYRO\_FS\_SEL\_MASK, 190  
GYRO\_FS\_SEL\_SHIFT, 190  
GYRO\_SMPLRT\_DIV, 190  
GYRO\_SMPLRT\_MIN\_HZ, 190  
GYRO\_XOUT\_H, 190  
GYRO\_XOUT\_L, 190  
GYRO\_YOUT\_H, 190  
GYRO\_YOUT\_L, 190  
GYRO\_ZOUT\_H, 190  
GYRO\_ZOUT\_L, 190  
HighAccuracy, 191  
Hz10, 191  
Hz15, 191  
Hz2, 191  
Hz20, 191  
Hz25, 191  
Hz30, 191  
Hz6, 191  
Hz8, 191  
I2C\_MST\_CLK\_MASK, 191  
I2C\_MST\_CTRL, 191  
I2C\_MST\_DELAY\_CTRL, 191  
I2C\_MST\_ODR\_CONFIG, 192  
I2C\_MST\_P\_NSR, 192  
I2C\_MST\_STATUS, 192  
I2C\_SLV0\_ADDR, 192  
I2C\_SLV0\_CTRL, 192  
I2C\_SLV0\_DELAY\_EN, 192  
I2C\_SLV0\_DO, 192  
I2C\_SLV0\_REG, 192  
I2C\_SLV1\_ADDR, 192  
I2C\_SLV1\_CTRL, 192  
I2C\_SLV1\_DELAY\_EN, 192  
I2C\_SLV1\_DO, 192

I2C\_SLV1\_REG, 193  
 I2C\_SLV2\_ADDR, 193  
 I2C\_SLV2\_CTRL, 193  
 I2C\_SLV2\_DELAY\_EN, 193  
 I2C\_SLV2\_DO, 193  
 I2C\_SLV2\_REG, 193  
 I2C\_SLV3\_ADDR, 193  
 I2C\_SLV3\_CTRL, 193  
 I2C\_SLV3\_DELAY\_EN, 193  
 I2C\_SLV3\_DO, 193  
 I2C\_SLV3\_REG, 193  
 I2C\_SLV4\_ADDR, 193  
 I2C\_SLV4\_CTRL, 194  
 I2C\_SLV4\_DELAY\_EN, 194  
 I2C\_SLV4\_DI, 194  
 I2C\_SLV4\_DLY\_MASK, 194  
 I2C\_SLV4\_DO, 194  
 I2C\_SLV4\_REG, 194  
 I2C\_SLVx\_BYTE\_SW, 194  
 I2C\_SLVx\_EN, 194  
 I2C\_SLVx\_GRP, 194  
 I2C\_SLVx LENG\_MASK, 194  
 I2C\_SLVx\_REG\_DIS, 194  
 I2C\_SLVx\_RNW, 194  
 INT1\_ACTL, 195  
 INT1\_LATCH\_INT\_EN, 195  
 INT1\_OPEN, 195  
 INT\_ACTL\_FSYNC, 195  
 INT\_ANYRD\_2CLEAR, 195  
 INT\_DMP\_INT1\_EN, 195  
 INT\_ENABLE, 195  
 INT\_ENABLE\_1, 195  
 INT\_ENABLE\_2, 195  
 INT\_ENABLE\_3, 195  
 INT\_I2C\_MST\_INT\_EN, 195  
 INT\_PIN\_CFG, 195  
 INT\_PLL\_RDY\_EN, 196  
 INT\_RAW\_DATA\_0\_RDY\_EN, 196  
 INT\_REG\_WOF\_EN, 196  
 INT\_STATUS, 196  
 INT\_STATUS\_1, 196  
 INT\_STATUS\_2, 196  
 INT\_STATUS\_3, 196  
 INT\_WOM\_INT\_EN, 196  
 INTERNAL\_20MHZ, 196  
 LowPower, 196  
 LP\_ACCEL\_CYCLE, 196  
 LP\_CONFIG, 196  
 LP\_GYRO\_CYCLE, 197  
 LP\_I2C\_MST\_CYCLE, 197  
 MOD\_CTRL\_USR, 197  
 MST\_LOST\_ARB, 197  
 MST\_ODR\_CFG\_MASK, 197  
 MST\_PASS\_THROUGH, 197  
 MST\_SLV0\_NACK, 197  
 MST\_SLV1\_NACK, 197  
 MST\_SLV2\_NACK, 197  
 MST\_SLV3\_NACK, 197  
 MST\_SLV4\_DONE, 197  
 MST\_SLV4\_NACK, 197  
 MULT\_MST\_EN, 198  
 ODR\_ALIGN\_EN, 198  
 ODR\_ALIGN\_EN\_BIT, 198  
 PWR\_CLKSEL\_AUTO, 198  
 PWR\_CLKSEL\_INT\_20MHZ, 198  
 PWR\_CLKSEL\_MASK, 198  
 PWR\_DEVICE\_RESET, 198  
 PWR\_DISABLE\_ACCEL, 198  
 PWR\_DISABLE\_GYRO, 198  
 PWR\_LP\_EN, 198  
 PWR\_MGMT\_1, 198  
 PWR\_MGMT\_2, 198  
 PWR\_SLEEP, 199  
 PWR\_TEMP\_DIS, 199  
 REG\_BANK\_SEL, 199  
 REG\_BANK\_SEL\_USER\_BANK\_MASK, 199  
 REG\_BANK\_SEL\_USER\_BANK\_SHIFT, 199  
 REG\_LP\_DMP\_EN, 199  
 Regular, 199  
 SELF\_TEST\_X\_ACCEL, 199  
 SELF\_TEST\_X\_GYRO, 199  
 SELF\_TEST\_Y\_ACCEL, 199  
 SELF\_TEST\_Y\_GYRO, 199  
 SELF\_TEST\_Z\_ACCEL, 199  
 SELF\_TEST\_Z\_GYRO, 200  
 STS\_DMP\_INT1, 200  
 STS\_I2C\_MST\_INT, 200  
 STS\_PLL\_RDY\_INT, 200  
 STS\_RAW\_DATA\_0\_RDY\_INT, 200  
 STS\_WOM\_INT, 200  
 TEMP\_CONFIG, 200  
 TEMP\_DLPFCFG\_MASK, 200  
 TEMP\_DLPFCFG\_SHIFT, 200  
 TEMP\_OUT\_H, 200  
 TEMP\_OUT\_L, 200  
 TIMEBASE\_CORRECTION\_PLL, 200  
 USER\_BANK\_0, 201  
 USER\_BANK\_1, 201  
 USER\_BANK\_2, 201  
 USER\_BANK\_3, 201  
 USER\_CTRL, 201  
 USER\_CTRL\_DMP\_EN, 201  
 USER\_CTRL\_DMP\_RST, 201  
 USER\_CTRL\_FIFO\_EN, 201  
 USER\_CTRL\_I2C\_IF\_DIS, 201  
 USER\_CTRL\_I2C\_MST\_EN, 201  
 USER\_CTRL\_I2C\_MST\_RST, 201  
 USER\_CTRL\_SRAM\_RST, 201  
 WHO\_AM\_I, 202  
 WHO\_AM\_I\_VAL, 202  
 WOF\_DEGLITCH\_EN, 202  
 WOF\_EDGE\_INT, 202  
 XA\_OFFS\_H, 202  
 XA\_OFFS\_L, 202



- XG\_OFFS\_USRH, [202](#)
- XG\_OFFS\_USRL, [202](#)
- XGYRO\_CTEN, [202](#)
- YA\_OFFS\_H, [203](#)
- YA\_OFFS\_L, [203](#)
- YG\_OFFS\_USRH, [203](#)
- YG\_OFFS\_USRL, [203](#)
- YGYRO\_CTEN, [203](#)
- ZA\_OFFS\_H, [203](#)
- ZA\_OFFS\_L, [203](#)
- ZG\_OFFS\_USRH, [203](#)
- ZG\_OFFS\_USRL, [203](#)
- ZGYRO\_CTEN, [203](#)
- ICM\_ADDR
  - gyr\_test.ino, [70](#)
  - i2c\_accel.ino, [78](#)
  - i2c\_basic.ino, [82](#)
  - i2c\_gyro.ino, [86](#)
  - i2c\_magneto.ino, [90](#)
  - interrupt.ino, [94](#)
  - mag\_test.ino, [98](#)
  - test.ino, [114](#)
  - test2.ino, [124](#)
- icm\_plat\_t, [62](#)
  - delay\_ms, [62](#)
  - i2c\_addr, [62](#)
  - read, [62](#)
  - spi\_cs, [62](#)
  - spi\_reg\_rw\_bits, [63](#)
  - user, [63](#)
  - write, [63](#)
- imu
  - gyr\_test.ino, [74](#)
  - i2c\_accel.ino, [80](#)
  - i2c\_basic.ino, [84](#)
  - i2c\_gyro.ino, [87](#)
  - i2c\_magneto.ino, [91](#)
  - interrupt.ino, [95](#)
  - mag\_test.ino, [100](#)
  - spi\_accel.ino, [104](#)
  - spi\_basic.ino, [107](#)
  - spi\_gyro.ino, [110](#)
  - test.ino, [118](#)
  - test2.ino, [128](#)
- initMag
  - DevLab\_ICM20948, [36](#)
- INT1\_ACTL
  - ICM20948\_regs.h, [195](#)
- INT1\_LATCH\_INT\_EN
  - ICM20948\_regs.h, [195](#)
- INT1\_OPEN
  - ICM20948\_regs.h, [195](#)
- INT\_ACTL\_FSYNC
  - ICM20948\_regs.h, [195](#)
- INT\_ANYRD\_2CLEAR
  - ICM20948\_regs.h, [195](#)
- INT\_DMP\_INT1\_EN
  - ICM20948\_regs.h, [195](#)
- INT\_ENABLE
  - ICM20948\_regs.h, [195](#)
- INT\_ENABLE\_1
  - ICM20948\_regs.h, [195](#)
- INT\_ENABLE\_2
  - ICM20948\_regs.h, [195](#)
- INT\_ENABLE\_3
  - ICM20948\_regs.h, [195](#)
- INT\_I2C\_MST\_INT\_EN
  - ICM20948\_regs.h, [195](#)
- INT\_PIN\_CFG
  - ICM20948\_regs.h, [195](#)
- INT\_PLL\_RDY\_EN
  - ICM20948\_regs.h, [196](#)
- INT\_RAW\_DATA\_0\_RDY\_EN
  - ICM20948\_regs.h, [196](#)
- INT\_REG\_WOF\_EN
  - ICM20948\_regs.h, [196](#)
- INT\_STATUS
  - ICM20948\_regs.h, [196](#)
- INT\_STATUS\_1
  - ICM20948\_regs.h, [196](#)
- INT\_STATUS\_2
  - ICM20948\_regs.h, [196](#)
- INT\_STATUS\_3
  - ICM20948\_regs.h, [196](#)
- INT\_WOM\_INT\_EN
  - ICM20948\_regs.h, [196](#)
- intEn
  - interrupt.ino, [95](#)
- intEnableConfig
  - DevLab\_ICM20948, [37](#)
- Interface\_7Semi, [63](#)
  - ~Interface\_7Semi, [64](#)
  - beginI2C, [64](#)
  - beginSPI, [64](#)
  - delay\_us, [64](#)
  - read, [65](#)
  - write, [65](#)
- INTERNAL\_20MHZ
  - ICM20948\_regs.h, [196](#)
- INTERNAL\_20MHZ\_0
  - DevLab\_ICM20948.h, [160](#)
- INTERNAL\_20MHZ\_6
  - DevLab\_ICM20948.h, [160](#)
- interrupt.ino
  - I2C\_FREQ, [94](#)
  - ICM\_ADDR, [94](#)
  - imu, [95](#)
  - intEn, [95](#)
  - isrCount, [95](#)
  - isrHandler, [94](#)
  - loop, [95](#)
  - PIN\_INT, [94](#)
  - pinCfg, [95](#)
  - printDobleLinea, [95](#)
  - printLinea, [95](#)
  - SCL\_PIN, [94](#)

- SDA\_PIN, [94](#)
- setup, [95](#)
- intInit
  - DevLab\_ICM20948, [38](#)
- isrCount
  - interrupt.ino, [95](#)
- isrHandler
  - interrupt.ino, [94](#)
- latchMode
  - ICM20948\_IntPinConfig, [60](#)
- loop
  - gyr\_test.ino, [72](#)
  - i2c\_accel.ino, [79](#)
  - i2c\_basic.ino, [83](#)
  - i2c\_gyro.ino, [87](#)
  - i2c\_magneto.ino, [90](#)
  - interrupt.ino, [95](#)
  - mag\_test.ino, [99](#)
  - spi\_accel.ino, [103](#)
  - spi\_basic.ino, [106](#)
  - spi\_gyro.ino, [109](#)
  - test.ino, [115](#)
  - test2.ino, [125](#)
- LowPower
  - ICM20948\_regs.h, [196](#)
- LP\_ACCEL\_CYCLE
  - ICM20948\_regs.h, [196](#)
- LP\_CONFIG
  - ICM20948\_regs.h, [196](#)
- LP\_GYRO\_CYCLE
  - ICM20948\_regs.h, [197](#)
- LP\_I2C\_MST\_CYCLE
  - ICM20948\_regs.h, [197](#)
- mag\_test.ino
  - applySettings, [99](#)
  - etiquetasOpModeCfg, [100](#)
  - I2C\_FREQ, [98](#)
  - ICM\_ADDR, [98](#)
  - imu, [100](#)
  - loop, [99](#)
  - N\_OPM, [100](#)
  - opMode, [100](#)
  - printDobleLinea, [99](#)
  - printLinea, [99](#)
  - SCL\_PIN, [98](#)
  - SDA\_PIN, [98](#)
  - setup, [99](#)
- MOD\_CTRL\_USR
  - ICM20948\_regs.h, [197](#)
- MODE\_CONTINUOUS\_1
  - DevLab\_ICM20948.h, [161](#)
- MODE\_CONTINUOUS\_2
  - DevLab\_ICM20948.h, [161](#)
- MODE\_CONTINUOUS\_3
  - DevLab\_ICM20948.h, [161](#)
- MODE\_CONTINUOUS\_4
  - DevLab\_ICM20948.h, [161](#)
- MODE\_POWER\_DOWN
  - DevLab\_ICM20948.h, [161](#)
- MODE\_SELF\_TEST
  - DevLab\_ICM20948.h, [161](#)
- MODE\_SINGLE\_MEASUREMENT
  - DevLab\_ICM20948.h, [161](#)
- MST\_LOST\_ARB
  - ICM20948\_regs.h, [197](#)
- MST\_ODR\_CFG\_MASK
  - ICM20948\_regs.h, [197](#)
- MST\_PASS\_THROUGH
  - ICM20948\_regs.h, [197](#)
- MST\_SLV0\_NACK
  - ICM20948\_regs.h, [197](#)
- MST\_SLV1\_NACK
  - ICM20948\_regs.h, [197](#)
- MST\_SLV2\_NACK
  - ICM20948\_regs.h, [197](#)
- MST\_SLV3\_NACK
  - ICM20948\_regs.h, [197](#)
- MST\_SLV4\_DONE
  - ICM20948\_regs.h, [197](#)
- MST\_SLV4\_NACK
  - ICM20948\_regs.h, [197](#)
- MULT\_MST\_EN
  - ICM20948\_regs.h, [198](#)
- N\_AVG
  - gyr\_test.ino, [75](#)
  - test2.ino, [128](#)
- N\_DLPF
  - gyr\_test.ino, [75](#)
  - test.ino, [118](#)
  - test2.ino, [128](#)
- N\_FS
  - gyr\_test.ino, [75](#)
  - test.ino, [118](#)
  - test2.ino, [128](#)
- N\_OPM
  - mag\_test.ino, [100](#)
- nbw\_hz
  - configDLPF, [21](#)
- nombre
  - configDLPF, [21](#)
- ODR\_ALIGN\_EN
  - ICM20948\_regs.h, [198](#)
- ODR\_ALIGN\_EN\_BIT
  - ICM20948\_regs.h, [198](#)
- odr\_hz
  - configDLPF, [21](#)
- opMode
  - mag\_test.ino, [100](#)
- PIN\_INT
  - interrupt.ino, [94](#)
- pinCfg
  - interrupt.ino, [95](#)
- pllRdyEn

- ICM20948\_IntEnableConfig, [59](#)
- pllRdyInt
  - ICM20948\_IntStatus, [61](#)
- printDobleLinea
  - gyr\_test.ino, [72](#)
  - interrupt.ino, [95](#)
  - mag\_test.ino, [99](#)
  - test.ino, [115](#)
  - test2.ino, [125](#)
- printLinea
  - gyr\_test.ino, [72](#)
  - interrupt.ino, [95](#)
  - mag\_test.ino, [99](#)
  - test.ino, [116](#)
  - test2.ino, [125](#)
- PWR\_CLKSEL\_AUTO
  - ICM20948\_regs.h, [198](#)
- PWR\_CLKSEL\_INT\_20MHZ
  - ICM20948\_regs.h, [198](#)
- PWR\_CLKSEL\_MASK
  - ICM20948\_regs.h, [198](#)
- PWR\_DEVICE\_RESET
  - ICM20948\_regs.h, [198](#)
- PWR\_DISABLE\_ACCEL
  - ICM20948\_regs.h, [198](#)
- PWR\_DISABLE\_GYRO
  - ICM20948\_regs.h, [198](#)
- PWR\_LP\_EN
  - ICM20948\_regs.h, [198](#)
- PWR\_MGMT\_1
  - ICM20948\_regs.h, [198](#)
- PWR\_MGMT\_2
  - ICM20948\_regs.h, [198](#)
- PWR\_SLEEP
  - ICM20948\_regs.h, [199](#)
- PWR\_TEMP\_DIS
  - ICM20948\_regs.h, [199](#)
- rawDataRdy
  - ICM20948\_IntStatus, [61](#)
- rawDataRdyEn
  - ICM20948\_IntEnableConfig, [59](#)
- read
  - BusIO\_7Semi< Bus >, [12](#), [13](#)
  - I2C\_Interface, [57](#)
  - icm\_plat\_t, [62](#)
  - Interface\_7Semi, [65](#)
  - SPI\_Interface, [67](#)
- readAccel
  - DevLab\_ICM20948, [39](#)
- readBit
  - BusIO\_7Semi< Bus >, [13](#), [14](#)
- readBits
  - BusIO\_7Semi< Bus >, [14](#), [15](#)
- readGyro
  - DevLab\_ICM20948, [39](#)
- readMag
  - DevLab\_ICM20948, [40](#)
- README.md, [132](#)
- readTemperature
  - DevLab\_ICM20948, [41](#)
- readWhoAmI
  - DevLab\_ICM20948, [42](#)
- REG\_BANK\_SEL
  - ICM20948\_regs.h, [199](#)
- REG\_BANK\_SEL\_USER\_BANK\_MASK
  - ICM20948\_regs.h, [199](#)
- REG\_BANK\_SEL\_USER\_BANK\_SHIFT
  - ICM20948\_regs.h, [199](#)
- REG\_LP\_DMP\_EN
  - ICM20948\_regs.h, [199](#)
- Regular
  - ICM20948\_regs.h, [199](#)
- runSweep
  - gyr\_test.ino, [72](#)
  - test.ino, [116](#)
  - test2.ino, [125](#)
- SCL\_PIN
  - gyr\_test.ino, [70](#)
  - i2c\_accel.ino, [79](#)
  - i2c\_basic.ino, [82](#)
  - i2c\_gyro.ino, [86](#)
  - i2c\_magneto.ino, [90](#)
  - interrupt.ino, [94](#)
  - mag\_test.ino, [98](#)
  - test.ino, [114](#)
  - test2.ino, [124](#)
- SDA\_PIN
  - gyr\_test.ino, [71](#)
  - i2c\_accel.ino, [79](#)
  - i2c\_basic.ino, [83](#)
  - i2c\_gyro.ino, [87](#)
  - i2c\_magneto.ino, [90](#)
  - interrupt.ino, [94](#)
  - mag\_test.ino, [98](#)
  - test.ino, [114](#)
  - test2.ino, [124](#)
- selectBank
  - DevLab\_ICM20948, [42](#)
- SELF\_TEST\_X\_ACCEL
  - ICM20948\_regs.h, [199](#)
- SELF\_TEST\_X\_GYRO
  - ICM20948\_regs.h, [199](#)
- SELF\_TEST\_Y\_ACCEL
  - ICM20948\_regs.h, [199](#)
- SELF\_TEST\_Y\_GYRO
  - ICM20948\_regs.h, [199](#)
- SELF\_TEST\_Z\_ACCEL
  - ICM20948\_regs.h, [199](#)
- SELF\_TEST\_Z\_GYRO
  - ICM20948\_regs.h, [200](#)
- selfTestAccel
  - DevLab\_ICM20948, [43](#)
- selfTestGyro
  - DevLab\_ICM20948, [43](#)
- setAccelAveraging
  - DevLab\_ICM20948, [44](#)

- setAccelDivRate
  - DevLab\_ICM20948, [45](#)
- setAccelDLPF
  - DevLab\_ICM20948, [45](#)
- setAccelOffset
  - DevLab\_ICM20948, [46](#)
- setAccelSampleRate
  - DevLab\_ICM20948, [47](#)
- setAccelScale
  - DevLab\_ICM20948, [47](#)
- setClock
  - DevLab\_ICM20948, [48](#)
- setDLPF
  - DevLab\_ICM20948, [49](#)
- setGyroAveraging
  - DevLab\_ICM20948, [49](#)
- setGyroDivRate
  - DevLab\_ICM20948, [50](#)
- setGyroOffset
  - DevLab\_ICM20948, [50](#)
- setGyroSampleRate
  - DevLab\_ICM20948, [51](#)
- setGyroScale
  - DevLab\_ICM20948, [51](#)
- setLowPower
  - DevLab\_ICM20948, [52](#)
- setMagOpMode
  - DevLab\_ICM20948, [52](#)
- setSensors
  - DevLab\_ICM20948, [53](#)
- setup
  - gyr\_test.ino, [73](#)
  - i2c\_accel.ino, [79](#)
  - i2c\_basic.ino, [83](#)
  - i2c\_gyro.ino, [87](#)
  - i2c\_magneto.ino, [90](#)
  - interrupt.ino, [95](#)
  - mag\_test.ino, [99](#)
  - spi\_accel.ino, [103](#)
  - spi\_basic.ino, [106](#)
  - spi\_gyro.ino, [109](#)
  - test.ino, [117](#)
  - test2.ino, [126](#)
- sleep
  - DevLab\_ICM20948, [54](#)
- softReset
  - DevLab\_ICM20948, [54](#)
- speed
  - SPI\_Interface, [68](#)
- spi
  - SPI\_Interface, [68](#)
- spi\_accel.ino
  - imu, [104](#)
  - loop, [103](#)
  - setup, [103](#)
  - spi\_bus, [103](#)
  - SPI\_FAST\_SPEED, [103](#)
- spi\_basic.ino
  - imu, [107](#)
  - loop, [106](#)
  - setup, [106](#)
  - spi\_bus, [107](#)
  - SPI\_FAST\_SPEED, [106](#)
- spi\_bus
  - spi\_accel.ino, [103](#)
  - spi\_basic.ino, [107](#)
  - spi\_gyro.ino, [110](#)
- spi\_cs
  - icm\_plat\_t, [62](#)
- SPI\_FAST\_SPEED
  - spi\_accel.ino, [103](#)
  - spi\_basic.ino, [106](#)
  - spi\_gyro.ino, [109](#)
- spi\_gyro.ino
  - imu, [110](#)
  - loop, [109](#)
  - setup, [109](#)
  - spi\_bus, [110](#)
  - SPI\_FAST\_SPEED, [109](#)
- SPI\_Interface, [65](#)
  - beginI2C, [67](#)
  - beginSPI, [67](#)
  - cs, [68](#)
  - read, [67](#)
  - speed, [68](#)
  - spi, [68](#)
  - write, [68](#)
- spi\_reg\_rw\_bits
  - icm\_plat\_t, [63](#)
- src/7Semi\_I2C\_Interface.h, [132](#), [133](#)
- src/7Semi\_Interface.h, [134](#), [135](#)
- src/7Semi\_SPI\_Interface.h, [136](#), [137](#)
- src/BusIO\_7Semi.h, [139](#)
- src/DevLab\_ICM20948.cpp, [141](#)
- src/DevLab\_ICM20948.h, [158](#), [162](#)
- src/ICM20948\_platform.h, [172](#)
- src/ICM20948\_regs.h, [173](#), [203](#)
- STOP\_CLOCK
  - DevLab\_ICM20948.h, [160](#)
- STS\_DMP\_INT1
  - ICM20948\_regs.h, [200](#)
- STS\_I2C\_MST\_INT
  - ICM20948\_regs.h, [200](#)
- STS\_PLL\_RDY\_INT
  - ICM20948\_regs.h, [200](#)
- STS\_RAW\_DATA\_0\_RDY\_INT
  - ICM20948\_regs.h, [200](#)
- STS\_WOM\_INT
  - ICM20948\_regs.h, [200](#)
- TEMP\_CONFIG
  - ICM20948\_regs.h, [200](#)
- TEMP\_DLPFCFG\_MASK
  - ICM20948\_regs.h, [200](#)
- TEMP\_DLPFCFG\_SHIFT
  - ICM20948\_regs.h, [200](#)
- TEMP\_OUT\_H

- ICM20948\_\_regs.h, [200](#)
- TEMP\_OUT\_L
  - ICM20948\_\_regs.h, [200](#)
- test.ino
  - accelAVG, [117](#)
  - accelCfg, [117](#)
  - accelDLPF, [117](#)
  - accelSR, [118](#)
  - applySettings, [114](#)
  - configsDLPF, [118](#)
  - etiquetasAccelAVG, [118](#)
  - etiquetasAccelCfg, [118](#)
  - etiquetasAccelDLPFCFG, [118](#)
  - etiquetasSR, [118](#)
  - firstStage, [115](#)
  - I2C\_FREQ, [114](#)
  - ICM\_ADDR, [114](#)
  - imu, [118](#)
  - loop, [115](#)
  - N\_DLPF, [118](#)
  - N\_FS, [118](#)
  - printDobleLinea, [115](#)
  - printLinea, [116](#)
  - runSweep, [116](#)
  - SCL\_PIN, [114](#)
  - SDA\_PIN, [114](#)
  - setup, [117](#)
  - total\_fail, [119](#)
  - total\_pass, [119](#)
  - total\_warn, [119](#)
- test2.ino
  - accelAVG, [127](#)
  - accelCfg, [127](#)
  - accelDLPF, [127](#)
  - accelSR, [127](#)
  - applySettings, [124](#)
  - configsDLPF, [127](#)
  - etiquetasAccelAVG, [127](#)
  - etiquetasAccelCfg, [128](#)
  - etiquetasSR, [128](#)
  - firstStage, [124](#)
  - I2C\_FREQ, [124](#)
  - ICM\_ADDR, [124](#)
  - imu, [128](#)
  - loop, [125](#)
  - N\_AVG, [128](#)
  - N\_DLPF, [128](#)
  - N\_FS, [128](#)
  - printDobleLinea, [125](#)
  - printLinea, [125](#)
  - runSweep, [125](#)
  - SCL\_PIN, [124](#)
  - SDA\_PIN, [124](#)
  - setup, [126](#)
  - total\_fail, [128](#)
  - total\_pass, [128](#)
  - total\_warn, [128](#)
- TIMEBASE\_CORRECTION\_PLL
  - ICM20948\_\_regs.h, [200](#)
- total\_fail
  - test.ino, [119](#)
  - test2.ino, [128](#)
- total\_pass
  - test.ino, [119](#)
  - test2.ino, [128](#)
- total\_warn
  - test.ino, [119](#)
  - test2.ino, [128](#)
- user
  - icm\_plat\_t, [63](#)
- USER\_BANK\_0
  - ICM20948\_\_regs.h, [201](#)
- USER\_BANK\_1
  - ICM20948\_\_regs.h, [201](#)
- USER\_BANK\_2
  - ICM20948\_\_regs.h, [201](#)
- USER\_BANK\_3
  - ICM20948\_\_regs.h, [201](#)
- USER\_CTRL
  - ICM20948\_\_regs.h, [201](#)
- USER\_CTRL\_DMP\_EN
  - ICM20948\_\_regs.h, [201](#)
- USER\_CTRL\_DMP\_RST
  - ICM20948\_\_regs.h, [201](#)
- USER\_CTRL\_FIFO\_EN
  - ICM20948\_\_regs.h, [201](#)
- USER\_CTRL\_I2C\_IF\_DIS
  - ICM20948\_\_regs.h, [201](#)
- USER\_CTRL\_I2C\_MST\_EN
  - ICM20948\_\_regs.h, [201](#)
- USER\_CTRL\_I2C\_MST\_RST
  - ICM20948\_\_regs.h, [201](#)
- USER\_CTRL\_SRAM\_RST
  - ICM20948\_\_regs.h, [201](#)
- WHO\_AM\_I
  - ICM20948\_\_regs.h, [202](#)
- WHO\_AM\_I\_VAL
  - ICM20948\_\_regs.h, [202](#)
- WOF DEGLITCH\_EN
  - ICM20948\_\_regs.h, [202](#)
- WOF\_EDGE\_INT
  - ICM20948\_\_regs.h, [202](#)
- wofEn
  - ICM20948\_\_IntEnableConfig, [59](#)
- womInt
  - ICM20948\_\_IntStatus, [61](#)
- womIntEn
  - ICM20948\_\_IntEnableConfig, [59](#)
- write
  - BusIO\_7Semi< Bus >, [15](#), [16](#)
  - I2C\_Interface, [58](#)
  - icm\_plat\_t, [63](#)
  - Interface\_7Semi, [65](#)
  - SPI\_Interface, [68](#)
- writeBit

BusIO\_7Semi< Bus >, [17](#), [18](#)  
writeBits  
    BusIO\_7Semi< Bus >, [18](#), [19](#)  
  
XA\_OFFS\_H  
    ICM20948\_regs.h, [202](#)  
XA\_OFFS\_L  
    ICM20948\_regs.h, [202](#)  
XG\_OFFS\_USRH  
    ICM20948\_regs.h, [202](#)  
XG\_OFFS\_USRL  
    ICM20948\_regs.h, [202](#)  
XGYRO\_CTEN  
    ICM20948\_regs.h, [202](#)  
  
YA\_OFFS\_H  
    ICM20948\_regs.h, [203](#)  
YA\_OFFS\_L  
    ICM20948\_regs.h, [203](#)  
YG\_OFFS\_USRH  
    ICM20948\_regs.h, [203](#)  
YG\_OFFS\_USRL  
    ICM20948\_regs.h, [203](#)  
YGYRO\_CTEN  
    ICM20948\_regs.h, [203](#)  
  
ZA\_OFFS\_H  
    ICM20948\_regs.h, [203](#)  
ZA\_OFFS\_L  
    ICM20948\_regs.h, [203](#)  
ZG\_OFFS\_USRH  
    ICM20948\_regs.h, [203](#)  
ZG\_OFFS\_USRL  
    ICM20948\_regs.h, [203](#)  
ZGYRO\_CTEN  
    ICM20948\_regs.h, [203](#)